

Computational Complexity of Physical Counting

Tristan Simas
McGill University
tristan.simas@mail.mcgill.ca

March 31, 2026

© 2026 Tristan Simas. This work is licensed under CC BY 4.0. License: <https://creativecommons.org/licenses/by/4.0/>
Zenodo DOI: [10.5281/zenodo.18140965](https://doi.org/10.5281/zenodo.18140965)

Abstract

Abstract

Which coordinates of a system’s state determine the optimal action? For decision problem $\mathcal{D} = (A, S, U)$ with $S = X_1 \times \cdots \times X_n$, set I is sufficient if $s_I = s'_I \Rightarrow \text{Opt}(s) = \text{Opt}(s')$. The optimizer quotient $Q = S/\sim$ is the coarsest abstraction preserving optimal actions.

From counting measure we derive probability, Bayes’ theorem, and Bayesian optimality (from $\log x \leq x - 1$ alone). The invariant srnk (relevant-coordinate cardinality) emerges as the complexity measure.

Complexity: SUFFICIENCY-CHECK and MINIMUM-SUFFICIENT-SET are coNP-complete; ANCHOR-SUFFICIENCY is Σ_2^P -complete; stochastic/sequential variants PP-/PSPACE-complete. Six subcases admit polynomial algorithms. Under ETH, $2^{\Omega(n)}$ lower bounds apply.

Thermodynamic results use Landauer’s floor $k_B T \ln 2$ as a declared premise; mismatch/residual dissipation terms yield $dU \geq \lambda dC$. Reduction correctness is machine-checked in Lean 4; polytime computability is a standard hypothesis.

1 Introduction

1.1 First-Principles Derivations

This paper studies a simple question with a hard combinatorial core: when a system exposes many state coordinates, which of those coordinates are actually needed to preserve the optimal decision? The aim is not to list all observable state variables, but to isolate the smallest part of the state that still determines what the best action is.

We model this with a decision problem: A is the finite set of possible actions, S is the state space, $U(a, s)$ is the utility of taking action a in state s , and $\text{Opt}(s)$ is the set of actions that maximize utility at state s . A coordinate set I is sufficient when any two states that agree on I also agree on Opt . The optimizer quotient object then collapses exactly those states with the same optimal-action set, yielding the canonical abstraction through which every decision-preserving abstraction factors.

Section 2 makes that collapse boundary explicit: any surjective abstraction either factors through the optimizer quotient object or erases a decision-relevant distinction, and any attempt to treat such extra erasure as physically feasible runs into the same no-collapse obstruction used later in the hardness chain. In **Set**, this quotient object is the familiar coimage of Opt , canonically equivalent to $\text{range}(\text{Opt})$.[QT5AB2AB3AB4](#)

The results in this section are organized as one derivation chain rather than a theorem ledger. The opening results establish the structural core from counting, set theory, arithmetic, and logic; later results then place the same core in complexity-theoretic and physical terms. Theorems 1–14 are derived from pure mathematics, while Theorems 15–17 add explicit empirical lower-bound premises. One irreducible empirical claim (EC1) supplies the thermodynamic scale: irreversible bit operations carry a positive lower-bound cost, with Landauer furnishing a universal floor. Counting and logic provide the structural part of the chain. Two former claims (EC2: finite resources, EC3: finite speed) are now derived from first principles.

1.1.1 1. Counting Gap Theorem

We begin with the discrete counting fact that every later information and complexity bound depends on: bounded total capacity and positive integer cost imply a finite observation budget.

Theorem 1.1 (Counting Gap). *Let $\varepsilon, C \in \mathbb{N}$ with $\varepsilon > 0$ and $C > 0$. If each information-acquisition event consumes ε cost units and the total available capacity is C , then for every $N \in \mathbb{N}$:*

$$\varepsilon \cdot N \leq C \Rightarrow N \leq C.$$

Equivalently, $c_{\max} = C$ is a finite upper bound on the number of possible events. No bounded system with positive integer event cost can perform infinitely many events. [BA10](#)

Proof. This is a discrete theorem: $\varepsilon > 0$ in $\mathbb{N}$ implies $1 \leq \varepsilon$, so each event costs at least one unit. Therefore $N = 1 \cdot N \leq \varepsilon \cdot N \leq C$. ■

Consequence. Continuous abstractions require arbitrarily fine state discrimination. Bounded computational systems cannot do this. Discrete models are forced by the mathematics.

1.1.2 2. Bayesian Optimality

Once observations are finite, the next question is how to update beliefs optimally from those observations.

Theorem 1.2 (Bayes Minimizes Expected Log Loss). *For distributions p, q over hypothesis space H , cross-entropy satisfies:*

$$\text{CE}(p, q) = H(p) + D_{\text{KL}}(p \| q)$$

where $D_{\text{KL}}(p \| q) \geq 0$ (Gibbs' inequality, from $\log x \leq x - 1$). Therefore $q = p$ uniquely minimizes $\text{CE}(p, q)$. [FN7 FN12 FN14](#)

Proof. Gibbs' inequality: $\log x \leq x - 1$ implies $\sum p \log(p/q) \geq 0$. Equality iff $p = q$. ■

Consequence. Bayesian updating is optimal. The proof uses only $\log x \leq x - 1$ and Lean's type theory.

1.1.3 3. Measure Theory Typing

To state those updates correctly, we next distinguish the type of a quantitative claim from the stronger normalization required for a probabilistic claim.

Theorem 1.3 (Measure-Theoretic Typing). *1. Every quantitative claim carries an explicit measure: (μ, f) where f is measurable.*

2. Every stochastic claim additionally requires μ to be a probability measure ($\mu(\Omega) = 1$).

3. Counting measure on $\{0, 1\}$ has mass 2, not 1: not a probability measure.

[MN1](#) [MN2](#) [MN10](#) [MN11](#)

Proof. (1) “ $\mathbb{E}[f]$ ” is undefined without specifying which measure integrates f . The expression $\int f d\mu$ requires μ . (2) $P(A) = \mu(A)/\mu(\Omega)$ requires $\mu(\Omega) < \infty$; normalization $P(\Omega) = 1$ requires $\mu(\Omega) = 1$. (3) Counting measure: $\mu(\{0\}) = 1$, $\mu(\{1\}) = 1$, so $\mu(\{0, 1\}) = 2 \neq 1$. To normalize, divide by 2. ■

Consequence. Quantitative and stochastic claims are typed differently. Probability normalization is not automatic from measure structure.

1.1.4 4. Universal Property of the Optimizer Quotient

With the measurement layer typed, we can identify the canonical abstraction that preserves the decision boundary itself.

Theorem 1.4 (Universal Property of the Optimizer Quotient). *Let $Q = S/\sim$ be the optimizer quotient object where $s \sim s' \iff \text{Opt}(s) = \text{Opt}(s')$. For any surjective abstraction $\phi : S \rightarrow T$ that preserves the optimal action ($\phi(s) = \phi(s') \Rightarrow \text{Opt}(s) = \text{Opt}(s')$), there exists a unique factorization $\psi : T \rightarrow Q$ such that $\pi = \psi \circ \phi$ where $\pi : S \rightarrow Q$ is the quotient map.*

$$\begin{array}{ccc} S & \xrightarrow{\pi} & Q \\ \downarrow & & \uparrow \\ T & \xrightarrow{\psi} & Q \end{array}$$

The optimizer quotient object is the coarsest abstraction through which Opt factors. In **Set**, it is canonically equivalent to $\text{range}(\text{Opt})$, i.e. the familiar coimage/image factorization of the optimizer map. [QT1](#) [QT2](#) [QT3](#) [QT5](#) [QT7](#)

Proof. Since ϕ preserves Opt , we have $\phi(s) = \phi(s') \Rightarrow \text{Opt}(s) = \text{Opt}(s')$. By surjectivity, for each $t \in T$ pick some s with $\phi(s) = t$ and define $\psi(t) = \pi(s)$. This is well-defined: if $\phi(s) = \phi(s') = t$, then $\text{Opt}(s) = \text{Opt}(s')$, so $\pi(s) = \pi(s')$. Uniqueness follows because $\pi = \psi \circ \phi$ forces $\psi(t) = \pi(s)$ for any s with $\phi(s) = t$. ■

Consequence. The optimizer quotient object is canonical. In **Set**, the familiar construction is not exotic: it is the coimage of Opt , canonically identified with the image/range of Opt . Any other state abstraction that preserves optimal actions must refine it.

1.1.5 5. Witness-Checking Duality

Once the canonical abstraction is fixed, we can ask how many state comparisons any sound verifier must inspect to certify that no counterexample exists.

Theorem 1.5 (Checking-Witnessing Duality). *For boolean decision problems with n coordinates, let $W(n) = 2^{n-1}$ be the witness budget and $T(n)$ be the checking budget. Any sound checker satisfies:*

$$T(n) \geq W(n) = 2^{n-1}$$

No finite checker with fewer than 2^{n-1} witness pairs can be sound on all problem instances. [WD1](#) [WD2](#) [WD3](#)

Proof. $\emptyset$ -sufficiency fails iff $\exists s, s'$ with $\text{Opt}(s) \neq \text{Opt}(s')$. There are 2^n states, so $\binom{2^n}{2} = 2^{n-1}(2^n - 1)$ pairs. A sound refutation must cover all pairs that could witness failure. For each pair family P with $|P| < 2^{n-1}$, there exists an Opt function whose unique failure pair is not in P . Therefore $|P| \geq 2^{n-1}$ is necessary for soundness. ■

Consequence. Verification has a lower bound. A checker must examine exponentially many witness pairs to be sound. This is a counting lower bound on the information required for verification.

1.1.6 6. Bayes from Counting

That verification bound rests on a simpler structural fact: probability, Bayes, and the normalized quotient score already emerge from counting on finite sets.

Theorem 1.6 (Probability Axioms from Counting). *The chain from counting to Bayesian updating and the normalized quotient score:*

1. $P(A) = |A|/|\Omega|$ satisfies Kolmogorov axioms: nonnegativity, normalization, and additivity for disjoint sets.
2. $P(H|E) = P(E|H) \cdot P(H)/P(E)$ follows from the definition of conditional probability.
3. Conditioning reduces entropy: $H(H|E) \leq H(H)$, from nonnegativity of KL divergence.
4. $\text{DQ} = I(H; E)/H(H) = 1 - H(H|E)/H(H)$ lies in $[0, 1]$.

BC1 BC2 BC3 BC4 BC5

Proof. (1) $|A| \geq 0$ (counting), $|\Omega|/|\Omega| = 1$ (normalization), $|A \cup B| = |A| + |B|$ for $A \cap B = \emptyset$ (additivity). (2) $P(H|E) := P(H \cap E)/P(E)$. Rearranging: $P(H|E) \cdot P(E) = P(H \cap E) = P(E|H) \cdot P(H)$. (3) $H(H|E) = H(H) - I(H; E)$ and $I(H; E) = D_{\text{KL}}(P_{H,E} \| P_H \otimes P_E) \geq 0$ by Gibbs. (4) $I(H; E) \leq H(H)$ by (3), so $\text{DQ} = I(H; E)/H(H) \in [0, 1]$. ■

Consequence. Bayesian updating is derived from counting measure. Probability theory, Bayes' theorem, information theory, and the normalized quotient score form a single chain from counting elements of a finite set.

1.1.7 7. Fisher Rank = Structural Rank

With that probabilistic chain in place, the next results compare independent mathematical formalisms and ask whether they recover the same structural invariant. In the finite-coordinate model, srnk is just the cardinality of the relevant-coordinate support, so the question is whether those formalisms recover that same support-size quantity. *SK1SK2SK3*

Theorem 1.7 (Fisher Information = Structural Rank). *Define the Fisher information score of coordinate i as:*

$$\text{score}(i) = \begin{cases} 1 & \text{if } i \text{ is relevant} \\ 0 & \text{otherwise} \end{cases}$$

Then:

$$\sum_{i=1}^n \text{score}(i) = \text{srnk}(\mathcal{D})$$

The diagonal Fisher information matrix $I(\theta)_{ii} = \text{score}(i)$ has rank $\text{srnk}(\mathcal{D})$. *FS1 FS2*

Proof. $\sum_i \text{score}(i) = \sum_i \mathbf{1}[\text{isRelevant}(i)] = |\{i \mid \text{isRelevant}(i)\}| = \text{srank}$. For the diagonal matrix: $\text{rank} = |\{i \mid \text{score}(i) \neq 0\}| = |\{i \mid \text{isRelevant}(i)\}| = \text{srank}$. ■

Consequence. The decision manifold has dimension exactly the size of the relevant-coordinate support. The Fisher information sees the same coordinates already counted by srank. The Cramer-Rao bound follows: any estimator has difficulty $\geq 1/\text{srank}$.

1.1.8 8. Entropy-Rank Inequality

The first bridge is information-theoretic: quotient entropy cannot exceed the number of coordinates that actually matter.

Theorem 1.8 (Entropy Bounded by Structural Rank). *For a decision problem $\mathcal{D}$ with boolean coordinate spaces ($X_i = \{0, 1\}$):*

$$H(\mathcal{D}) \leq \text{srank}(\mathcal{D})$$

where $H(\mathcal{D}) = \log_2(\text{numOptClasses})$ is the Shannon entropy of the optimizer quotient object. [IT3 IT4](#)

Proof. The relevant coordinate set R is sufficient (Theorem 5.9): Opt factors through $\pi_R : S \rightarrow \{0, 1\}^{|R|}$. Therefore $\text{numOptClasses} \leq |\{0, 1\}^{|R|}| = 2^{\text{srank}}$. Therefore $H(\mathcal{D}) = \log_2(\text{numOptClasses}) \leq \log_2(2^{\text{srank}}) = \text{srank}$. ■

Consequence. The optimizer quotient object carries at most srank bits of decision-relevant information. Coordinates outside the relevant set are information-theoretically inert.

1.1.9 9–14. Later Bridge Theorems

The next six introductory results are bridge theorems. They are proved in their dedicated sections, but we state them here in compressed form because they all serve the same purpose: independent frameworks recover the same structural core once the optimizer quotient object is fixed.

Theorem 1.9 (Thermodynamic Cost from Counting). *For functional set $F \subseteq \Omega$, the counting expression $-\log_2(|F|/|\Omega|)$ gives the bit cost, and a Landauer floor converts that bit count into a universal energy lower bound. In the sharpened physical interface, explicit nonnegative mismatch and residual terms are added above that floor to represent constrained-process dissipation. The mismatch branch is now justified as far as the existing mathematics allows: distinct strictly positive finite input distributions yield strictly positive KL mismatch, which is then rounded upward into the discrete lower-bound units used by the thermodynamic accounting and injected into the Wolpert decomposition. A second derived branch now captures the strongest residual theorem supported by the current finite machinery: for finite discrete computational-state processes, positive forward edge flow together with decision-relevant edge asymmetry yields strict residual positivity by an exhaustive local split on the reverse edge. If the reverse flow is positive, the pairwise KL branch applies. If the reverse flow vanishes, the process performs an irreversible one-way state transition, and the existing Landauer-scaled transition-cost theorem supplies a positive lower-bound witness. That local argument is now also packaged as a process-level theorem: any finite computational-state process with at least one such asymmetric forward edge already carries a theorem-level residual witness. The broader stopping-time / absolute-irreversibility residual regime remains a separate imported premise. The thermodynamic statement is therefore the counting statement plus a floor-plus-overhead empirical conversion law; a derived mismatch branch, a derived finite residual branch from this exhaustive edge split, or the cited*

broader stopping-time branch can each force strict separation above the Landauer floor, and any declared structural lower bound absorbed by mismatch only strengthens the resulting energy estimate.
FI3 FI6 FI7 WC1 WC2 WC3 WM1 WM3 WM4 WM5 WM6 WR1 WR2 WR3 WR4 WR5 WR6 WR7 WR8 WR9 WR10 WP1 WP5 WP6 WP7 WP8

Proof. The mathematical part is only the counting step: finite measure gives the bit count $-\log_2(|F|/|\Omega|)$. Landauer then supplies the universal floor for the energy-per-bit conversion. The bridge is therefore structural counting plus one declared empirical lower-bound premise. ■

Consequence. The thermodynamic lower bound is not a separate combinatorial structure; it is the counting bound expressed in physical units.

Theorem 1.10 (Wasserstein Distance = Structural Rank). *Optimal-transport cost grows with the number of distinguishable futures and therefore tracks the same relevant-coordinate complexity measured by srank.* *W1 W2 W3 W4*

Proof. When all states lead to one future, the diagonal coupling gives zero transport cost. Once multiple futures must be distinguished, off-diagonal mass incurs positive cost. The same branching structure counted by srank therefore reappears as transport cost. ■

Consequence. Optimal transport recovers the same complexity signal as the coordinate-relevance analysis.

Theorem 1.11 (Rate-Distortion = Structural Rank). *At zero distortion, the minimum lossless rate needed to preserve optimal actions is exactly srank, i.e. exactly the cardinality of the relevant-coordinate support, and compression below that support size necessarily introduces decision errors.*
RD1 RD2 RD3 RS1 RS2 RS3 RS4 RS5

Proof. Zero distortion means that decision-equivalent states may be merged, but decision-distinguishing states may not. The quotient therefore partitions the source into exactly the classes that must remain distinguishable. Their count determines the minimum lossless rate, which is the same structural quantity measured by srank. ■

Consequence. Description length and relevant-coordinate support size coincide at zero distortion.

Theorem 1.12 (Information Requires Nontriviality). *If decision-relevant information exists, then the state space cannot be trivial: the existence of information forces at least two distinguishable states.*
FP1 FP2 FP3 FP4 FP5 FP6 FP7

Proof. If the state space had at most one element, then every map out of it would be constant, including Opt. A constant decision boundary has zero entropy and carries no distinguishable information. Taking the contrapositive gives the stated nontriviality requirement. ■

Consequence. Nontriviality is derived from the existence of information rather than postulated as an extra axiom.

Theorem 1.13 (Second Law Structure from Counting). *Maintaining ordered behavior requires continuous distinction between correct and incorrect transitions; in this sense, the structural content of the Second Law follows from counting and verification, while the numerical thermodynamic scale is added later.* *FP8 FP9 FP10 FP11 FP12 FP13 FP14*

Proof. If there are more non-target than target continuations, unguided evolution overwhelmingly drifts away from the target set. Staying ordered therefore requires repeated discrimination between correct and incorrect continuations. That discrimination is an information-acquisition burden, and the thermodynamic lower-bound premise enters only after that structural burden is fixed. ■

Consequence. The order-versus-verification tradeoff is established before any temperature-dependent conversion is introduced.

Theorem 1.14 (Erasure Increases Environment Entropy). *Whenever a state reduction deletes distinctions, those distinctions must reappear in environmental degrees of freedom, so the structural content of Landauer's principle is a transfer statement and only the numerical thermodynamic scale is empirical.* [FP15](#)

Proof. Reducing n distinguishable states to fewer than n outputs removes distinctions from the local description. Those distinctions cannot vanish from the bookkeeping; they must be displaced into the environment. The structural theorem is therefore a conservation-of-distinguishability statement, with temperature only setting the energetic scale of that transfer. ■

Consequence. Landauer's principle separates cleanly into a mathematical transfer structure and an empirical lower-bound energy scale.

1.2 Irreducible Empirical Claims

One claim is irreducibly empirical. Two others (finite resources, finite speed) are derived from first principles.

Definition 1.15 (Irreducible Empirical Claim EC1). **EC1 (Temperature):** Entropy has energy cost. There exists $\varepsilon > 0$ such that each irreversible bit operation costs at least ε joules, with $\varepsilon \geq k_B T \ln 2$. Landauer gives the universal floor. A separate floor-plus-overhead interface treats any strictly larger per-bit dissipation as an explicit additional premise rather than an informal interpretation.

Theorem 1.16 (EC2 Derived from Uncertainty). *Finite resources (EC2) follows from the existence of uncertainty:*

1. *Uncertainty exists: $\exists X, 0 < P(X) < 1$ (observation).*
2. *No bounds $\Rightarrow$ infinite information acquisition $\Rightarrow$ know everything $\Rightarrow P(X) \in \{0, 1\}$ for all X .*
3. *Contrapositive: uncertainty exists $\Rightarrow$ bounds exist.*

[IA7 IA9 IA11 IA12 IA13](#)

Proof. Let observer have capacity C and access $A \leq C$. System has entropy H . If $C \geq H$, observer can access all information (IA8). Full access eliminates uncertainty (IA9). Contrapositive: uncertainty implies $A < H$. If observer uses full capacity ($A = C$), then $C < H$ (IA11). Therefore capacity is bounded relative to system entropy. This is EC2. ■

Theorem 1.17 (EC3 Derived from Non-Contradiction). *Finite signal speed (EC3) follows from the law of non-contradiction:*

1. *A computational step = state change: before $\neq$ after (definition).*
2. *At any instant, state is unique (function property).*
3. *If step takes $t = 0$: before-state and after-state at same instant.*
4. *Same instant $\Rightarrow$ state = before AND state = after.*
5. *But before $\neq$ after $\Rightarrow$ contradiction (violates non-contradiction).*
6. *Therefore $t_{\text{before}} \neq t_{\text{after}}$, so $\Delta t > 0$.*

7. Information propagation = sequence of steps, each taking $\Delta t > 0$.

8. Speed = distance / time. Time $> 0 \Rightarrow$ speed $< \infty$.

[FPT4](#) [FPT5](#) [FPT6](#) [FPT8](#) [FPT10](#)

Proof. A step changes state from A to B where $A \neq B$. The timeline is a function $T \rightarrow S$. If $t_{\text{before}} = t_{\text{after}}$, then $\text{timeline}(t) = A$ and $\text{timeline}(t) = B$ for the same t . By function uniqueness, $A = B$. Contradiction. Therefore $t_{\text{before}} \neq t_{\text{after}}$. With ordering, $\Delta t > 0$ (FPT5). Information propagation requires steps; each takes positive time; total time is positive; speed is finite. QED from pure logic. ■

Consequence. EC2 and EC3 are not axioms. EC2 follows from uncertainty existing. EC3 follows from non-contradiction. **One irreducible empirical claim remains: EC1 (temperature).** The empirical content is that irreversible bit operations have a positive lower-bound cost, with Landauer giving a universal floor and stronger constrained-process bounds allowed above it; the existence of a cost is structural.

1.2.1 15. Assumption Necessity

Theorem 1.18 (Physical Claims Need Empirically Justified Assumptions). *If a physical claim ϕ is:*

1. *Provable from assumption set Γ (sound formal system)*
2. *Falsifiable (some model m with $\neg \text{Holds}(m, \phi)$)*
3. *Derived from empirically disciplined Γ (all axioms empirically justified)*

Then Γ contains at least one physical axiom that is empirically justified. [AN3](#) [AN4](#)

Proof. Suppose Γ contains only mathematical axioms (no physical axioms). Mathematical axioms hold in all models (by soundness). Therefore ϕ holds in all models. But (2) says ϕ fails in some model m . Contradiction. Therefore Γ contains at least one non-mathematical axiom. By (3), this axiom must be empirically justified. ■

Consequence. Physical axioms cannot be hidden. Physical consequences require declared and empirically justified physical assumptions.

1.2.2 16. Energy Bound

Theorem 1.19 (Energy Lower Bound). *For any physical system resolving decision problem $\mathcal{D}$ at temperature $T > 0$:*

$$E_{\text{decision}} \geq k_B T \cdot H_{\text{nats}}(\mathcal{D})$$

where $H_{\text{nats}}(\mathcal{D}) = \ln(\text{numOptClasses})$ is the natural-log Shannon entropy of the optimizer quotient object. [EI1](#)

Proof. Three steps:

1. srnk bits are required to resolve $\mathcal{D}$. Each bit requires one state transition.

2. Each transition costs at least the Landauer floor $k_B T \ln 2$. In the stronger constrained-process model, explicit nonnegative mismatch and residual dissipation terms may raise that per-bit lower bound further. The mismatch branch is no longer merely postulated: if the actual and designed finite distributions differ, the existing KL machinery yields a strictly positive mismatch witness, which is then converted into the discrete lower-bound units of the thermodynamic model. A second finite residual branch is also now derived by an exhaustive edge split on finite computational-state processes: if the reverse edge flow is positive, asymmetric forward/reverse edge flows yield strictly positive two-point KL divergence; if the reverse edge flow is zero, the process performs an irreversible one-way state transition and the existing Landauer-scaled transition-cost theorem yields a positive discrete lower-bound term. The broader stopping-time branch remains an imported premise. Therefore $E \geq \text{srnk} \cdot k_B T \ln 2$ remains the universal floor, and any declared constrained-process contribution tightens it monotonically.
3. $H_{\text{nats}} = \ln(\text{numOptClasses}) \leq \text{srnk} \cdot \ln 2$ (Theorem 1.8).

Combining: $E \geq \text{srnk} \cdot k_B T \ln 2 \geq k_B T \cdot H_{\text{nats}}$. ■

Consequence. Thermodynamic cost tracks information content: energy per decision $\geq k_B T$ per nat of decision entropy. This is derived from the Landauer floor (Theorem 9) composed with the entropy-rank inequality (Theorem 8); in the explicit decomposed constrained-process interface, a theorem-level KL mismatch witness, a theorem-level finite residual witness produced by the exhaustive reverse-edge split, its process-level witness packaging, or the cited broader stopping-time residual branch only strengthens the estimate. [WC1WC4WM3WM4WM6WR6WR7WR8WR9WR10WP1WP5WP6](#)

1.2.3 17. Thermodynamic Uncertainty Relations

Theorem 1.20 (TUR Bridge (Conditional)). *Conditional on stochastic thermodynamics (Barato–Seifert 2015):*

1. Transition probabilities are normalized: $p_{ij} \geq 0$, $\sum_j p_{ij} = 1$.
2. The Second Law enforces non-negative entropy production: $\sigma \geq 0$.
3. The TUR bound constrains precision: $\text{Var}(J)/\langle J \rangle^2 \geq 2/\sigma$.
4. Multiple futures require positive entropy production.

Precision requires entropy dissipation, scaling with structural rank. [TUR1 TUR2 TUR5 TUR6](#)

Proof. (1) Standard probability. (2) The Second Law (empirical assumption). (3) TUR is a theorem of stochastic thermodynamics: for any current J in a Markov process with entropy production σ , precision is bounded by $\text{Var}(J)/\langle J \rangle^2 \geq 2/\sigma$. (4) If $\text{srnk} > 1$, distinguishing $k > 1$ futures requires information acquisition. By the TUR bound, finite precision requires $\sigma > 0$. The minimal σ scales with $\log k$, hence with srnk . ■

Consequence. Accepting fluctuation theorems yields an independent constraint: precision costs entropy production, with the bound scaling with srnk . This is conditional on accepting the Second Law.

1.3 Consistency with Existing Physical Theories

The Counting Gap Theorem is consistent with and illuminates why discrete models work across physics:

- **Quantum Mechanics:** Discrete eigenstates are forced for bounded systems
- **Statistical Mechanics:** Discrete microstates forced by finite accessible state count
- **Thermodynamics:** Finite entropy forced by bounded information acquisition
- **Lattice Gauge Theory:** Discretization respects the computational constraint

The paper rests on one irreducible empirical claim (Definition 1.15):

- **EC1 (Temperature):** $\varepsilon \geq k_B T \ln 2 > 0$ (entropy has a positive lower-bound energy cost)

EC2 (finite resources) is derived from the existence of uncertainty (Theorem 1.16): if you don't know everything, your capacity is bounded. EC3 (finite speed) is derived from non-contradiction (Theorem 1.17): a computational step must take positive time, or the state would be both A and not- A at the same instant. Everything else—nontriviality, second law structure, Landauer structure, EC2, EC3—follows from counting, logic, or observation, with the numerical thermodynamic scale entering only through the declared lower-bound premise in EC1.

1.4 Our Formal Tools: Coordinate Sufficiency

We study decision problems with coordinate structure. The finite coordinate spaces X_i are forced by the Counting Gap. Boolean coordinates $X_i = \{0, 1\}$ are elementary: one physical transition per bit.

A coordinate set I is sufficient if knowing s_I determines the optimal action:

$$s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

We prove complexity classifications for three meta-problems:

- **SUFFICIENCY-CHECK:** Is I sufficient? coNP-complete
- **MINIMUM-SUFFICIENT-SET:** Find minimal I . coNP-complete
- **ANCHOR-SUFFICIENCY:** Exists sufficient I with $|I| \leq k$? Σ_2^P -complete

Stochastic variants are PP-complete. Sequential variants are PSPACE-complete. Strict hierarchy: static $\subset$ stochastic $\subset$ sequential.

1.5 Machine Verification

The Lean 4 formalization consists of:

- 24,347 lines of Lean code
- 1,082 theorems
- 0 sorry placeholders
- 0 hidden axioms

What is machine-verified. Bayesian optimality: proved from $\log x \leq x - 1$ alone. Thermodynamic consequences: conditional on a Landauer floor or any stricter declared per-bit lower bound. **Reduction correctness:** the membership-preservation property of each reduction (e.g., φ is a tautology $\Leftrightarrow I = \emptyset$ is sufficient) is fully machine-verified.

What is a standard claim. **Polynomial-time computability** of the reduction functions is a standard claim stated as an explicit hypothesis in the Lean formalization. Full Turing-machine verification of polynomial-time bounds remains an open problem in formal verification—no formalization project has achieved end-to-end polynomial-time TM proofs for concrete reductions. The gap is made visible by representing polytime as a hypothesis rather than hiding it behind definitions that conflate size bounds with time bounds. Machine-generated assumption ledger records all dependencies.

1.6 Paper Structure

Section 2: formal definitions (decision problems, sufficiency, DQ). Section 5: physical grounding (Counting Gap, three laws). Section 4: hardness proofs (coNP, Σ_2^P , PP, PSPACE). Section 9: regime separation. Section 10: six tractable subcases. Sections 11–13: engineering corollaries. Appendix: Lean listings.

Claim-stamp notation. Claims are stamped with theorem references $[T : n; P : m]$ and Lean handles (H1).

Regime	Question	Placement
Static	SUFFICIENCY / MINIMUM / ANCHOR	coNP / coNP / Σ_2^P
Stochastic	STOCHASTIC-SUFFICIENCY	PP-complete
Sequential	SEQUENTIAL-SUFFICIENCY	PSPACE-complete

(L: QT1-3, WD1-3, BC1-5, CCC1, CCC5-6, DC1-2, DC9-10, FN14, MN5, FI7, W1-4, RD1-3, RS1-5, TUR1-2, TUR5-6.)

2 Formal Foundations

We formalize decision problems with coordinate structure, sufficiency of coordinate sets, and the optimizer quotient object, drawing on classical decision theory [75, 71]. The core definitions in this section are substrate-neutral: they specify the decision relation independently of implementation medium. The mechanized physical-lift and claim-transport scaffolding is summarized later in Section 2.4 so that the mathematical core appears first.

2.1 Decision Problems with Coordinate Structure

Formally: this subsection fixes the core tuple, projection, optimizer, and sufficiency/relevance predicates used downstream.

Definition 2.1 (Decision Problem). A *decision problem with coordinate structure* is a tuple $\mathcal{D} = (A, X_1, \dots, X_n, U)$ where:

- A is a finite set of *actions* (alternatives)
- $X_1, \dots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \dots \times X_n$ is the *state space*

- $U : A \times S \rightarrow \mathbb{Q}$ is the *utility function*

Definition 2.2 (Projection). For state $s = (s_1, \dots, s_n) \in S$ and coordinate set $I \subseteq \{1, \dots, n\}$:

$$s_I := (s_i)_{i \in I}$$

is the *projection* of s onto coordinates in I .

Definition 2.3 (Optimizer Map). For state $s \in S$, the *optimal action set* is:

$$\text{Opt}(s) := \arg \max_{a \in A} U(a, s) = \{a \in A : U(a, s) = \max_{a' \in A} U(a', s)\}$$

2.2 Sufficiency and Relevance

Definition 2.4 (Sufficient Coordinate Set). A coordinate set $I \subseteq \{1, \dots, n\}$ is *sufficient* for decision problem $\mathcal{D}$ if:

$$\forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

Equivalently, the optimal action depends only on coordinates in I . In this paper, this set-valued invariance of the full optimal-action correspondence is the primary decision-relevance target.

Proposition 2.5 (Empty-Set Sufficiency Equals Constant Decision Boundary). *For any decision problem,*

$$\text{sufficient}(\emptyset) \iff \forall s, s' \in S, \text{Opt}(s) = \text{Opt}(s').$$

Hence, the $I = \emptyset$ query is exactly a universal constancy check of the decision boundary. (L: [DP6](#).)

Proposition 2.6 (Insufficiency Equals Counterexample Witness). *For any coordinate set I ,*

$$\neg \text{sufficient}(I) \iff \exists s, s' \in S : s_I = s'_I \wedge \text{Opt}(s) \neq \text{Opt}(s').$$

In particular,

$$\neg \text{sufficient}(\emptyset) \iff \exists s, s' \in S : \text{Opt}(s) \neq \text{Opt}(s').$$

(L: [DP7-8](#).)

[C:D2.4;P2.5, 2.6;R:DM]

Definition 2.7 (Deterministic Selector). A *deterministic selector* is a map $\sigma : 2^A \rightarrow A$ that returns one action from an optimal-action set.

Definition 2.8 (Selector-Level Sufficiency). For fixed selector σ , a coordinate set I is *selector-sufficient* if

$$\forall s, s' \in S : s_I = s'_I \implies \sigma(\text{Opt}(s)) = \sigma(\text{Opt}(s')).$$

Proposition 2.9 (Set-Level Target Dominates Selector Targets). *If I is sufficient in the set-valued sense (Definition 2.4), then I is selector-sufficient for every deterministic selector σ (Definition 2.8).*

(L: [DP5](#))

Proof. If I is sufficient, then $s_I = s'_I$ implies $\text{Opt}(s) = \text{Opt}(s')$. Applying any deterministic selector σ to equal optimal-action sets yields equal selected actions. ■

Definition 2.10 (ε -Optimal Set and ε -Sufficiency). For $\varepsilon \geq 0$, define

$$\text{Opt}_\varepsilon(s) := \{a \in A : \forall a' \in A, U(a', s) \leq U(a, s) + \varepsilon\}.$$

Coordinate set I is ε -sufficient if

$$\forall s, s' \in S : s_I = s'_I \implies \text{Opt}_\varepsilon(s) = \text{Opt}_\varepsilon(s').$$

Proposition 2.11 (Zero- ε Reduction). *Set-valued sufficiency is exactly the $\varepsilon = 0$ case:*

$$I \text{ sufficient} \iff I \text{ is 0-sufficient.}$$

(L: [DP1](#), [DP4](#))

Proposition 2.12 (Selector-Level Separation Witness). *The converse of Proposition [2.9](#) fails in general: there exists a decision problem, selector, and coordinate set I such that I is selector-sufficient but not sufficient in the full set-valued sense. (L: [CC54](#).)*

Definition 2.13 (Minimal Sufficient Set). A sufficient set I is *minimal* if no proper subset $I' \subsetneq I$ is sufficient.

Definition 2.14 (Relevant Coordinate). Coordinate i is *relevant* if there exist states that differ only at coordinate i and induce different optimal action sets:

$$i \text{ is relevant} \iff \exists s, s' \in S : \left(\forall j \neq i, s_j = s'_j \right) \wedge \text{Opt}(s) \neq \text{Opt}(s').$$

Proposition 2.15 (Minimal-Set/Relevance Equivalence). *For any minimal sufficient set I and any coordinate i :*

$$i \in I \iff i \text{ is relevant.}$$

Hence every minimal sufficient set is exactly the relevant-coordinate set. (L: [DP2-3](#).)

Proof. The “only if” direction follows by minimality: if $i \in I$ were irrelevant, removing i would preserve sufficiency, contradicting minimality. The “if” direction follows from sufficiency: every sufficient set must contain each relevant coordinate. ■

Proposition 2.16 (Structural Rank as Relevant-Support Size). *The structural rank $\text{srnk}(\mathcal{D})$ is the cardinality of the relevant-coordinate support:*

$$\text{srnk}(\mathcal{D}) = |\{i \in \{1, \dots, n\} : i \text{ is relevant}\}|.$$

Hence $\text{srnk}(\mathcal{D})$ is bounded by the ambient coordinate dimension n , and $\text{srnk}(\mathcal{D}) = 0$ exactly when the decision boundary is constant across coordinates. (L: [SK1-3](#).)

Proof. Proof sketch: by definition, structural rank counts the coordinates that survive the relevance filter. The upper bound follows because at most all coordinates can be relevant. The zero case is equivalent to the relevance support being empty, which is exactly the coordinate-constant boundary case. ■

Informally: this is a familiarity anchor for the invariant used throughout the later bridge theorems. In the finite-coordinate setting, srnk is simply the support size of the relevance indicator. Later theorems show that Fisher rank, zero-distortion rate, and physical bit cost recover this same support-size quantity through different frameworks.

Definition 2.17 (Exact Relevance Identifiability). For a decision problem $\mathcal{D}$ and candidate coordinate set I , we say I is *exactly relevance-identifying* if

$$\forall i \in \{1, \dots, n\} : i \in I \iff i \text{ is relevant for } \mathcal{D}.$$

Equivalently, I is exactly relevance-identifying iff I equals the full relevant-coordinate set.

Example 2.18 (Weather Decision). Consider deciding whether to carry an umbrella:

- Actions: $A = \{\text{carry, don't carry}\}$

- Coordinates: $X_1 = \{\text{rain, no rain}\}$, $X_2 = \{\text{hot, cold}\}$, $X_3 = \{\text{Monday}, \dots, \text{Sunday}\}$
- Utility: $U(\text{carry}, s) = -1 + 3 \cdot \mathbf{1}[s_1 = \text{rain}]$, $U(\text{don't carry}, s) = -2 \cdot \mathbf{1}[s_1 = \text{rain}]$

The minimal sufficient set is $I = \{1\}$ (only rain forecast matters). Coordinates 2 and 3 (temperature, day of week) are irrelevant.

Informally: all later results use this same sufficiency definition.

2.3 Decision-Quotient Score and Optimizer Quotient Object

Definition 2.19 (Decision Equivalence). For coordinate set I , states s, s' are I -equivalent (written $s \sim_I s'$) if $s_I = s'_I$.

Definition 2.20 (Decision-Quotient Score). The *decision-quotient score* for state s under coordinate set I is:

$$\text{DQ}_I(s) = \frac{|\{a \in A : a \in \text{Opt}(s') \text{ for some } s' \sim_I s\}|}{|A|}$$

This is a normalized ambiguity score: equivalently, it is the normalized support fraction of the multivalued image of the I -equivalence class under Opt . It measures the fraction of actions that are optimal for at least one state consistent with I . It is not the quotient object $S/\sim$ used later for the optimizer map.

Proposition 2.21 (Sufficiency Characterization). *[C:P2.21;R:H=succinct-hard] Coordinate set I is sufficient if and only if $\text{DQ}_I(s) = |\text{Opt}(s)|/|A|$ for all $s \in S$. (L: cc62-63)*

Proof. If I is sufficient, then $s \sim_I s' \implies \text{Opt}(s) = \text{Opt}(s')$, so the set of actions optimal for some $s' \sim_I s$ is exactly $\text{Opt}(s)$.

Conversely, if the condition holds, then for any $s \sim_I s'$, the optimal actions form the same set (since $\text{DQ}_I(s) = \text{DQ}_I(s')$ and both equal the relative size of the common optimal set). ■

The quotient object behind the optimizer map has a familiar form in **Set**. It is useful to state that explicitly before the later abstraction-boundary theorem.

Proposition 2.22 (Optimizer Quotient as Coimage/Image Factorization). *For the optimizer map $\text{Opt} : S \rightarrow \mathcal{P}(A)$:*

1. *the optimizer quotient object is canonically equivalent to $\text{range}(\text{Opt})$;*
2. *any surjective abstraction $\phi : S \rightarrow T$ that preserves Opt factors uniquely through this quotient.*

*Thus, in **Set**, the optimizer quotient object is the familiar coimage of Opt , canonically identified with its image/range.[50] (L: q75, q77.)*

Proof. Proof sketch:

1. The quotient identifies exactly those states with equal optimal-action sets, so its elements correspond bijectively to the values attained by Opt . This gives the canonical equivalence with $\text{range}(\text{Opt})$.
2. If a surjective abstraction preserves Opt , then the optimizer map already factors through that abstraction. The quotient universal property supplies the mediating map, and surjectivity makes that factorization unique.

■

Informally: this proposition is only a familiarity anchor. The object is not exotic in **Set**; the later novelty is that this quotient governs the complexity and physical-collapse arguments.

The same quotient object has an equally familiar information-theoretic reading: its entropy is just the logarithm of the number of distinct optimizer values actually attained.

Proposition 2.23 (Optimizer Quotient Entropy as Image-Size Entropy). *Let $\text{numOptClasses}(\mathcal{D})$ denote the number of distinct values attained by Opt , equivalently the cardinality of $\text{range}(\text{Opt})$. Then*

$$H(\mathcal{D}) = \log_2(\text{numOptClasses}(\mathcal{D})) = \log_2(|\text{range}(\text{Opt})|).$$

Thus the entropy of the optimizer quotient object is just the base-2 logarithm of the size of the image of Opt . (L: [IT1-2](#), [QT5](#).)

Proof. Proof sketch: numOptClasses is defined as the cardinality of the image of Opt , and Proposition 2.22 identifies the optimizer quotient object with that image in **Set**. The entropy quantity $H(\mathcal{D})$ is then defined to be $\log_2(\text{numOptClasses})$. ■

Informally: this is the familiar reading of the entropy term used later in the bridge theorems. Nothing new is being hidden inside the notation: it is just the logarithm of how many different optimal-action sets the system can realize.

The next theorem concerns the quotient object rather than the scalar score above. Abstraction is a many-to-one quotient operation: it identifies states and thereby removes distinctions. The optimizer quotient object is the coarsest such collapse that still preserves the optimal-action boundary.

Informally: start with any surjective summary map $\phi : S \rightarrow T$. There are only two possibilities. Either ϕ keeps every optimal-action distinction intact, in which case the universal property forces it to factor through the optimizer quotient object, or ϕ merges two states that require different optimal actions, in which case it has erased a decision-relevant distinction. In **Set**, the quotient object here is the coimage of Opt , canonically equivalent to its image/range [50]. The physical no-collapse layer then says that if one tries to treat that extra erasure as computationally affordable, the collapse assumption itself is what fails. (L: [QT5](#), [AB1-4](#).)

Theorem 2.24 (Decision-Preserving Collapse Boundary). *Let $\phi : S \rightarrow T$ be a surjective abstraction of the state space.*

1. *Either ϕ factors through the optimizer quotient object, or it collapses a decision-relevant distinction, i.e., there exist states $s, s' \in S$ such that $\phi(s) = \phi(s')$ but $\text{Opt}(s) \neq \text{Opt}(s')$.*
2. *If every such extra collapse is mapped to a physically feasible collapse at the canonical requirement profile, then no such extra collapse can occur. Therefore ϕ must factor through the optimizer quotient object.*

(L: [AB1-4](#).)

Proof. Proof sketch:

1. If ϕ preserves Opt , then the quotient universal property already proved in the previous layer supplies the factorization $\pi = \psi \circ \phi$. In **Set**, this is exactly the coimage factorization of the optimizer map through its image/range. If ϕ does not preserve Opt , then by definition there must exist s, s' with $\phi(s) = \phi(s')$ but $\text{Opt}(s) \neq \text{Opt}(s')$. That witness is exactly the erased decision-relevant distinction.

2. Suppose such an erased distinction is further interpreted as a physically feasible collapse at the canonical requirement profile. The existing physical no-collapse theorem then blocks that collapse map. Therefore the “extra erasure” branch is impossible, leaving only factorization through the optimizer quotient object.

■

Informally: this is the explicit bridge between state-space abstraction and complexity-class collapse. At the state level, the optimizer quotient object is the maximal lossless collapse. In **Set**, that means the coimage/image factorization of Opt . At the physical-complexity level, any claim that a coarser collapse remains feasible must pass through the same no-collapse obstruction. The theorem makes the common structure visible instead of leaving it implicit across separate sections.

2.4 Mechanized Physical-Lift and Claim Metadata

The formal system also records the physical-lift and claim-transport scaffolding used later in the paper. Physical interpretation is introduced through explicit encoding and assumption-transfer maps rather than built into the core tuple itself. $[C:D2.42;P2.46;T2.49;R:AR]$ ($L: CT4, CT6.$)

Within that scaffolding, physical-state semantics are typed by realizable-state witnesses, observer classes induce quotient state spaces on a shared physical domain, coupled recurrent circuits support typed YES/NO/ABSTAIN report projection, and timing constraints yield forced-action and lower-bound statements under declared physical parameterizations. ($L: PS1-4, OR3-5, OR9, CV4-5, CV7, CV11-13, CV15-18, CF2, CF4, CF6-9.$)

The same metadata layer also packages the assumption-indexed physical-collapse transfer chain, the layered graph witnesses, and the exact-claim admissibility boundary used later in the physics and claim-discipline sections. This material remains explicit, but it is secondary to the mathematical core definitions established above. $[C:T3.47;P3.61;C3.49;R:AR]$ ($L: PH11-33, GN2, GN4-13, CC18, CC21, CC30.$)

2.5 Computational Model and Input Encoding

Formally: this subsection defines encoding/access regimes, cost measures, and regime-specific complexity interpretation.

We fix the computational model used by the complexity claims. In physical deployments, these encodings correspond to different access conditions on the same underlying system: explicit tables correspond to fully exposed state/utility structure, succinct encodings correspond to compressed generative descriptions, and query regimes correspond to constrained measurement interfaces. $[C:T9.1;P9.3, 2.46;R:E, S+ETH, Qf, AR]$ ($L: CC16, CC50, CT6.$)

Succinct encoding (primary for hardness). This succinct circuit encoding is the standard representation for decision problems in complexity theory; hardness is stated with respect to the input length of the circuit description [5, 63]. An instance is encoded as:

- a finite action set A given explicitly,
- coordinate domains $X_1, \dots, X_n$ given by their sizes in binary,
- a Boolean or arithmetic circuit C_U that on input (a, s) outputs $U(a, s)$.

The input length is $L = |A| + \sum_i \log |X_i| + |C_U|$. Polynomial time and all complexity classes (coNP , Σ_2^P , ETH) are measured in L . All hardness results in Section 4 use this encoding.

Explicit-state encoding (used for enumeration algorithms and experiments). The utility is given as a full table over $A \times S$. The input length is $L_{\text{exp}} = \Theta(|A||S|)$ (up to the bitlength of utilities). Polynomial time is measured in L_{exp} . Results stated in terms of $|S|$ use this encoding.

Query-access regime (intermediate black-box access). The solver is given oracle access to decision information at queried states (e.g., $\text{Opt}(s)$, or $U(a, s)$ with $\text{Opt}(s)$ reconstructed from finitely many value queries). Complexity is measured by oracle-query count (optionally paired with per-query evaluation cost). This separates representational access from full-table availability and from succinct-circuit input length.

Unless explicitly stated otherwise, “polynomial time” refers to the succinct encoding.

Remark 2.25 (Decision Questions vs Decision Problems). We distinguish between a *decision question* and a *decision problem*. A decision question is the encoding-independent predicate: “is coordinate set I sufficient for $\mathcal{D}$?” This is a mathematical relation, fixed by the model contract (C1–C4). A decision problem is a decision question together with a specific encoding of its inputs. The same decision question yields different decision problems under different encodings, and these problems may belong to different complexity classes. Formally, each encoding regime e induces a language $L_{\mathcal{D},e}$; complexity classification is a property of $L_{\mathcal{D},e}$, not of $\mathcal{D}$ alone.

Definition 2.26 (Anchor Query Object). An anchor query object is a pair

$$q = (\mathcal{D}, I)$$

where $\mathcal{D}$ is a decision problem and I is a fixed coordinate set. Its truth predicate is

$$\text{AnchorTrue}(q) \iff \exists s_0 \in S \forall s \in S, s_I = s_{0,I} \Rightarrow \text{Opt}(s) = \text{Opt}(s_0).$$

Definition 2.27 (Pose Operation for Anchor Queries). The pose operation is the constructor

$$\text{poseAnchor} : (\mathcal{D}, I) \mapsto q.$$

Posing is an operation; q is the formal object.

Proposition 2.28 (Posing Produces a Formal Query Object). *For every $(\mathcal{D}, I)$, $\text{poseAnchor}(\mathcal{D}, I)$ is a well-typed anchor query object with preserved components, and its truth predicate is exactly the anchored $\exists\forall$ condition in Definition 2.26. [C:D2.26, 2.27;T4.9;R:S] (L: A41-3.)*

Proposition 2.29 (Typed Exact Claims on Posed Anchor Queries). *For posed anchor queries under the typed claim discipline:*

$$\text{Exact admissible} \iff \text{competent on the declared regime}, \quad \neg \text{competent} \Rightarrow \neg \text{exact admissible}.$$

Under solver integrity, checker-accepted exact-true verdicts are semantically sound on the posed anchor query object. [C:P3.16, 3.58;C3.50;R:AR] (L: A44-7.)

Corollary 2.30 (Certified-Confidence Gate for Posed Anchor Queries). *Under signal consistency, positive certified confidence implies typed admissibility of the posed anchor-query report.*

[C:D3.20;P3.23;R:AR] (L: A48.)

Proposition 2.31 (Under-Resolution Forces Representation Collision). *[C:P2.31;R:DM] Let $\text{repr} : \mathcal{L} \rightarrow \mathcal{C}$ be any representation map from logical possibilities to code states. If $|\mathcal{C}| < |\mathcal{L}|$, then there exist distinct logical states with the same representation:*

$$\exists s, s' \in \mathcal{L}, s \neq s' \wedge \text{repr}(s) = \text{repr}(s').$$

This is the finite-cardinality collision statement underlying under-resolved exact-decision claims. (L: P16-7.)

Theorem 2.32 (Physical Incompleteness of Instantiation). *[C:T2.32;R:AR] Let $\mathcal{P}$ be a finite set of physical states and $\mathcal{L}$ a finite set of logical possibilities, with an instantiation map $\iota : \mathcal{P} \rightarrow \mathcal{L}$. If $|\mathcal{P}| < |\mathcal{L}|$, then:*

1. *ι is not surjective;*
2. *there exists a logical possibility $\ell \in \mathcal{L}$ that is not physically instantiated: $\ell \notin \text{range}(\iota)$.*

Under explicit bounds: if $|\mathcal{P}| \leq M$ and $|\mathcal{L}| \geq L$ with $M < L$, the same conclusion holds. (L: [PI3-5](#).)

Definition 2.33 (Regime Simulation). Fix a decision question Q and two regimes R_1, R_2 for that question. We say R_1 *simulates* R_2 if there exists a polynomial-time transformer that maps any R_2 instance to an R_1 instance while preserving the answer to Q ; in oracle settings, this includes a polynomial-time transcript transducer that preserves yes/no outcomes for induced solvers. (L: [CC1](#), [CC52](#).)

This paper uses that simulation spine in two explicit forms: restriction-map simulation (subproblem-to-full transfer; Proposition 9.8) and oracle-transducer simulation (batch-to-entry transfer; Proposition 9.15).

Informally: the same question can be easy or hard depending on representation and access, with explicit transfer requirements.

2.6 Model Contract and Regime Tags

All theorem statements in this paper are typed by the following model contract:

- **C1 (finite actions):** A is finite.
- **C2 (finite coordinate domains):** each X_i is finite, so $S = \prod_i X_i$ is finite.
- **C3 (evaluable utility):** $U(a, s)$ is computable from the declared input encoding.
- **C4 (fixed decision semantics):** optimality is defined by $\text{Opt}(s) = \arg \max_a U(a, s)$.

We use short regime tags for applied corollaries:

- **[E]** explicit-state encoding,
- **[Q]** query-access (oracle) regime,
- **[Q_fin]** finite-state query-access core (state-oracle),
- **[S]** succinct encoding,
- **[S+ETH]** succinct encoding with ETH,
- **[Q_bool]** mechanized Boolean query submodel,
- **[S_bool]** mechanized Boolean-coordinate submodel.

This tagging is a claim-typing convention: each strong statement is attached to the regime where it is proven.

Remark 2.34 (Uniform Assumption Discipline). All theorem-level claims in this paper are conditional on their stated premises and regime tags; no assumption family is given special status. For each claim, the active assumptions are exactly those appearing in its statement, its regime tag, and its cited mechanized handles.

Remark 2.35 (Modal Convention: Contract-Conditional Necessity). In this paper, normative modal statements denote contract-conditional necessity: once a physical-system model declares an objective/regime/assumption contract, admissible reports and interventions are fixed by typed admissibility and integrity under that contract. No unconditional external moral prescription is asserted.

2.7 Discrete-Time Event Semantics

Definition 2.36 (Timed Interface State). For any interface state space S , a timed interface state is a pair

$$x = (s, t) \in S \times \mathbb{N}.$$

The time coordinate is the natural-number component t .

Definition 2.37 (Decision Tick and Decision Event). Fix an interface decision process (decide, transition) with

$$\text{decide} : S \rightarrow A, \quad \text{transition} : S \times A \rightarrow S.$$

Define one tick by

$$\text{tick}(s, t) = (\text{transition}(s, \text{decide}(s)), t + 1).$$

Define $\text{DecisionEvent}(x, y)$ to mean that y is the one-tick successor of x under this rule.

Proposition 2.38 (Time Coordinate Is Discrete). *In Definition 2.36, time is discrete:*

$$\forall x \in S \times \mathbb{N}, \exists k \in \mathbb{N} \text{ such that } x_t = k.$$

Pointwise time equality is decidable for every $k \in \mathbb{N}$:

$$x_t = k \vee x_t \neq k.$$

(L: DT6-7.)

Proposition 2.39 (Decision Event Is One Unit of Time). *Under Definition 2.37, for all timed states x, y :*

$$\text{DecisionEvent}(x, y) \iff y = \text{tick}(x),$$

and therefore

$$\text{DecisionEvent}(x, y) \implies y_t = x_t + 1.$$

(L: DT10-13.)

Proposition 2.40 (Run Accounting Identity). *Let $\text{run}(n, x)$ be n repeated ticks from x , and let $\text{trace}(n, x)$ be the emitted decision sequence. Then:*

$$\text{run}(n, x)_t = x_t + n, \quad \text{run}(n, x)_t - x_t = n, \quad |\text{trace}(n, x)| = n.$$

Hence decision count equals elapsed time:

$$|\text{trace}(n, x)| = \text{run}(n, x)_t - x_t.$$

(L: DT15-16, DT18-19.)

Proposition 2.41 (Substrate Realization Preserves Unit-Time Events). *For any substrate model that is consistent with the declared interface transition rule, each one-step substrate evolution realizes one interface decision event and advances interface time by one unit. The time update law is invariant under substrate kind. (L: DT22-24.)*

2.8 Physical-System Encoding and Theorem Transport

Formally: this subsection defines physical-to-core encoding and proves theorem transport via declared assumption maps.

Definition 2.42 (Physical-to-Core Encoding). Fix a physical instance class $\mathcal{P}$ and a core decision class $\mathcal{D}$. A *physical-to-core encoding* is a map

$$E : \mathcal{P} \rightarrow \mathcal{D}.$$

All physical claims below are typed relative to a declared encoding map E and declared assumption transfer map from physical assumptions to core assumptions.

Definition 2.43 (Physical Scope Gate). A claim is in physical scope only if it declares:

1. a physical state class with observable coordinates,
2. a physical-to-core encoding map $E : \mathcal{P} \rightarrow \mathcal{D}$,
3. calibrated measurement units for reported quantities (for example bits, joules, and kT -scaled bounds),
4. a declared objective contract and admissible intervention class,
5. an explicit physical-to-core assumption transfer map.

Claims missing any item are untyped for theorem transport in this framework.

Proposition 2.44 (Typed Physical Transport Requirement). *Direct transfer of core theorems is licensed only through a declared physical scope gate instance (Definition 2.43) and a corresponding physical-to-core encoding (Definition 2.42). Absent that typed physical instance, transport is blocked by construction.* [PS3](#) [PS4](#)

Proof. By Definition 2.43, theorem transport is only defined for claims with explicit physical state, objective, intervention, encoding, and assumption-transfer declarations. Therefore untyped targets have no admissible transport map in this framework. Proposition 2.46 then applies only to typed instances. ■

This scope gate is a typing rule, not a rhetorical caveat: the paper’s transport theorems apply only after explicit encoding and assumption transfer are declared. [\[C:D2.42, 2.43;P2.44, 2.46;R:AR\]](#)

Proposition 2.45 (Heisenberg-Strong Binding Yields Nontrivial Decision Boundary). [\[C:P2.45;R:AR\]](#) *Fix a declared noisy physical encoding interface and assume a Heisenberg-strong binding witness: one physical instance is compatible with two distinct interface states whose encoded optimal-action sets differ. Then there exists a core decision problem with non-constant optimizer map. Equivalently, under this declared physical ambiguity assumption, decision nontriviality is derivable at the core level.* [\(L: HS3, HS5-6.\)](#)

Proposition 2.46 (Physical Claim Transport). *Let $E : \mathcal{P} \rightarrow \mathcal{D}$ be an encoding. If*

$$\forall d \in \mathcal{D}, \text{CoreAssm}(d) \Rightarrow \text{CoreClaim}(d)$$

and

$$\forall p \in \mathcal{P}, \text{PhysAssm}(p) \Rightarrow \text{CoreAssm}(E(p)),$$

then

$$\forall p \in \mathcal{P}, \text{PhysAssm}(p) \Rightarrow \text{CoreClaim}(E(p)).$$

[\(L: CT3, CT5-6.\)](#)

Corollary 2.47 (Physical Counterexample Implies Core Failure on Encoded Slice). *Under the same encoding and assumption-transfer map:*

1. *any encoded physical counterexample induces a core counterexample on the encoded slice;*
2. *if a physical counterexample exists, then the purported core rule is invalid;*
3. *if the lifted claim holds on all physically admissible encoded instances, no such physical counterexample exists.*

(L: [CT4-6](#).)

Proposition 2.48 (Law-Instance Objective Bridge). *For the mechanized law-gap physical instance class, the encoded optimizer claim holds at theorem level and admits no counterexample:*

$$\forall x \in \mathcal{P}_{\text{law}}, \text{lawGapPhysicalClaim}(x), \quad \neg \exists x \in \mathcal{P}_{\text{law}}, \neg \text{lawGapPhysicalClaim}(x).$$

(L: [CT1-2](#).)

Theorem 2.49 (Physical Bridge Bundle). *In one bundled mechanized statement, the paper’s physical bridge layer composes:*

1. *law-induced objective semantics ($\text{Opt} = \text{feasible actions for law-gap instances}$),*
2. *behavior-preserving configuration reduction equivalence,*
3. *irreducible required-work lower bound under site-count scaling.*

(L: [CT4](#).)

[C:D2.42;P2.46, 2.48;C2.47;T2.49;R:AR]

Theorem 2.50 (Declared Regime Coverage for the Static Class). *Let*

$$\mathcal{R}_{\text{static}} = \{[E], [S+ETH], [Q_fin:Opt], [Q_bool:value-entry], [Q_bool:state-batch]\}.$$

For each declared regime in $\mathcal{R}_{\text{static}}$, the paper has a theorem-level mechanized core claim. Equivalently, regime typing is complete over the declared static family. (L: [CC14](#), [CC51](#).)

Scope Lattice (typed classes and transfer boundary).

Layer	Transfer Status	Lean handles (representative)
Static sufficiency class (C1–C4; declared regimes) CC51	Internal landscape complete	CC76
Bridge-admissible adjacent class (one-step deterministic) CC10	Transfer licensed	CC42
Non-admissible represented adjacent classes (horizon, stochastic, transition-coupled) CC8	Transfer blocked by witness	CC9

Informally: physical conclusions apply when the stated physical assumptions line up with the core theorem assumptions.

2.9 Adjacent Objective Regimes and Bridge

Definition 2.51 (Adjacent Sequential Objective Regime). An adjacent sequential objective instance consists of:

- finite action set A ,
- finite coordinate state space $S = X_1 \times \cdots \times X_n$,
- horizon $T \in \mathbb{N}_{\geq 1}$ and history-dependent control-law class,
- reward process r_t and objective functional $J(\pi)$ (e.g., cumulative reward or regret).

Proposition 2.52 (One-Step Deterministic Bridge). *Consider an instance of Definition 2.51 satisfying:*

1. $T = 1$,
2. deterministic rewards $r_1(a, s) = U(a, s)$ for some evaluable U ,
3. objective $J(\pi) = U(\pi(s), s)$ (single-step maximization),
4. no post-decision state update relevant to the objective.

Then the induced optimization problem is exactly the static decision problem of Definition 2.1, and coordinate sufficiency in the sequential formulation is equivalent to Definition 2.4. (L: cc42)

Proof. Under (1)–(3), optimizing J at state s is identical to choosing an action in $\arg \max_{a \in A} U(a, s) = \text{Opt}(s)$. Condition (4) removes any dependence on future transition effects. Therefore the optimal-policy relation in the adjacent formulation coincides pointwise with Opt from Definition 2.3, and the invariance condition “same projection implies same optimal choice set” is exactly Definition 2.4. ■

Proposition 2.53 (Bridge Transfer Rule). *Under conditions (1)–(4), any sufficiency statement formulated over Definition 2.4 is equivalent between the adjacent sequential formulation and the static model. (L: cc42)*

Proof. By Proposition 2.52, the adjacent sequential objective induces exactly the same sufficiency predicate as Definition 2.4 when (1)–(4) hold. Equivalence of any sufficiency statement then follows by substitution. ■

If any bridge condition fails, direct transfer from this paper’s static complexity theorems is not licensed by this rule.

Remark 2.54 (Extension Boundary). Beyond Proposition 2.52 (multi-step horizon, stochastic transitions/rewards, or regret objectives), the governing complexity objects change. Those regimes are extensions, but they are distinct formal classes from the static sufficiency class analyzed in this paper.

Proposition 2.55 (Horizon-> 1 Bridge-Failure Witness). *Dropping the one-step condition can break sufficiency transfer: there exists a two-step objective where sufficiency in the immediate-utility static projection does not match sufficiency in the two-step objective. (L: cc25-26.)*

Proposition 2.56 (Stochastic-Criterion Bridge-Failure Witness). *If optimization is performed against a stochastic criterion not equal to deterministic utility maximization, bridge transfer to Definition 2.1 can fail even when an expected-utility projection is available. (L: cc57.)*

Proposition 2.57 (Transition-Coupled Bridge-Failure Witness). *If post-decision transition effects are objective-relevant, bridge transfer to the static one-step class can fail. (L: [cc73](#).)*

Theorem 2.58 (Exact Bridge Boundary in the Represented Adjacent Family). *Within the represented adjacent family*

{one-step deterministic, horizon-extended, stochastic-criterion, transition-coupled},

direct transfer of static sufficiency claims is licensed if and only if the class is one-step deterministic. Each represented non-one-step class has an explicit mechanized bridge-failure witness. (L: [cc8-10](#).)

Definition 2.59 (System Snapshot vs System Process Typing). Within the represented adjacent family, we type:

- *system snapshot*: fixed-parameter inference view (mapped to one-step deterministic class),
- *system process*: online/dynamical update views (horizon-extended, stochastic-criterion, or transition-coupled classes).

Proposition 2.60 (Snapshot/Process Transfer Typing). *In the represented adjacent family, direct transfer of static sufficiency claims is licensed if and only if the system is typed as a system snapshot. Every process-typed represented class has an explicit mechanized bridge-failure witness. (L: [cc3](#), [cc48](#), [cc56](#).)*

Operationally: a fixed-parameter physical controller at runtime is a system snapshot for this typing; once parameters or transition-relevant internals are updated online, the object is process-typed and transfer from static sufficiency theorems is blocked unless the one-step bridge conditions are re-established. *[C:P2.60;R:RA]*

3 Interpretive Foundations: Hardness and Solver Claims

The claims in later applied sections are theorem-indexed consequences of this section and Sections [4–10](#).

3.1 Structural Complexity vs Representational Hardness

Formally: this subsection separates class placement from regime-indexed access cost and states typed completeness for the declared static family.

Definition 3.1 (Structural Complexity). For a fixed decision question (e.g., “is I sufficient for $\mathcal{D}$?”; see Remark [2.25](#)), *structural complexity* means its placement in standard complexity classes within the formal model (coNP, Σ_2^P , etc.), as established by class-membership arguments and reductions.

Definition 3.2 (Representational Hardness). For a fixed decision question and an encoding regime E (Section [2.5](#)), *representational hardness* is the worst-case computational cost incurred by solvers whose input access is restricted to E .

In the mechanized architecture model, this is made explicit as a required-work observable and intrinsic lower bound: ‘requiredWork’ is total realized work, ‘hardnessLowerBound’ is the intrinsic floor, and ‘hardness_is_irreducible_required_work’ proves the lower-bound relation for all nonzero use-site counts; ‘requiredWork_eq_affine_in_sites’ and ‘workGrowthDegree’ make the site-count growth law explicit. *(L: [HD17-18](#), [HD22-23](#), [HD27-28](#).)*

Remark 3.3 (Interpretation Contract). This paper keeps the decision question fixed and varies the encoding regime explicitly. Thus, later separations are read as changes in representational hardness under fixed structural complexity, not as changes to the underlying sufficiency semantics. Accordingly, “complete landscape” means complete for this static sufficiency class under the declared regimes; adjacent objective classes are distinct typed objects, not unproved remainder cases of the same class.

Theorem 3.4 (Typed Completeness for the Static Sufficiency Class). *For the static sufficiency class fixed by C1–C4 and declared regime family $\mathcal{R}_{\text{static}}$, the paper provides:*

1. *conditional class-label closures for SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, and ANCHOR-SUFFICIENCY;*
2. *a conditional explicit/succinct dichotomy closure;*
3. *a regime-indexed mechanized core claim for every declared regime; and*
4. *declared-family exhaustiveness in the typed regime algebra.*

(L: [cc76](#).)

Informally: the question stays fixed while representation changes cost, and the declared static regime set is fully covered.

3.2 Solver Integrity and Regime Competence

Formally: this subsection defines integrity, competence, typed report/evidence admissibility, and resource-bounded exact-claim limits.

To keep practical corollaries type-safe, we separate *integrity* (what a solver is allowed to assert) from *competence* (what it can cover under a declared regime), following the certifying-algorithms schema [52, 57]. The induced-relation view used below is standard in complexity/computability presentations: algorithms are analyzed as machines deciding languages or computing functions/relations over encodings [63, 5, 28].

Definition 3.5 (Certifying Solver). Fix a decision question $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{Y}$ and an encoding regime E over $\mathcal{X}$. A *certifying solver* is a pair (Q, V) where:

- Q maps each input $x \in \mathcal{X}$ to either ABSTAIN or a candidate pair (y, w) ,
- V is a polynomial-time checker (in $|\text{enc}_E(x)|$) with output in $\{0, 1\}$.

Definition 3.6 (Induced Solver Relation). For any deterministic program M computing a partial map $f_M : \mathcal{X} \rightarrow \mathcal{Y}$ on encoded inputs, define

$$\mathcal{R}_M := \{(x, y) \in \mathcal{X} \times \mathcal{Y} : f_M(x) \downarrow \wedge y = f_M(x)\}.$$

Proposition 3.7 (Universality of Solver Framing). *Every deterministic computer program that computes a (possibly partial) map on encoded inputs can be framed as a solver for its induced relation (Definition 3.6). In this formal sense, all computers are solvers of specific problems. (L: [cc77](#).)*

Proof. Immediate from Definition 3.6: each program induces a relation given by its graph on the domain where it halts. ■

Definition 3.8 (Solver Integrity). A certifying solver (Q, V) has *integrity* for relation $\mathcal{R}$ if:

- (assertion soundness) $Q(x) = (y, w) \implies V(x, y, w) = 1$,
- (checker soundness) $V(x, y, w) = 1 \implies (x, y) \in \mathcal{R}$.

The output **ABSTAIN** (equivalently, **UNKNOWN**) is first-class and carries no assertion about membership in $\mathcal{R}$.

Definition 3.9 (Competence Under a Regime). Fix a regime $\Gamma = (\mathcal{X}_\Gamma, E_\Gamma, \mathcal{C}_\Gamma)$ with instance family $\mathcal{X}_\Gamma \subseteq \mathcal{X}$, encoding assumptions E_Γ , and resource bound $\mathcal{C}_\Gamma$. A certifying solver (Q, V) is *competent* on Γ for relation $\mathcal{R}$ if:

- it has integrity for $\mathcal{R}$ (Definition 3.8),
- (coverage) $\forall x \in \mathcal{X}_\Gamma, Q(x) \neq \text{ABSTAIN}$,
- (resource bound) $\text{runtime}_Q(x) \leq \mathcal{C}_\Gamma(|\text{enc}_{E_\Gamma}(x)|)$ for all $x \in \mathcal{X}_\Gamma$.

Corollary 3.10 (Universality of Integrity Applicability). *Let M be any deterministic program, viewed as a certifying solver pair (Q, V) for some externally fixed target relation $\mathcal{R}$. Then Definition 3.8 applies unchanged. Thus epistemic integrity is architecture-universal over computing systems once they are cast as solvers for declared tasks. (L: [cc31](#).)*

Proof. Definition 3.8 quantifies over a relation $\mathcal{R}$ and a certifying pair (Q, V) ; it does not assume any special implementation substrate. ■

[C:P3.7;C3.10;R:TR]

Remark 3.11 (External-Task Non-Vacuity). If the target relation is chosen post hoc from the program’s own behavior (for example, $\mathcal{R} = \mathcal{R}_M$ from Definition 3.6), integrity can become tautological and competence claims can be vacuous. Non-vacuous competence claims therefore require the target relation, encoding regime, and resource bound to be declared independently of observed outputs.

[C:D3.8, 3.9;P3.58, 3.61;R:AR]

Definition 3.12 (ε -Objective Relation Family). An ε -objective relation family is a map

$$\varepsilon \mapsto \mathcal{R}_\varepsilon \subseteq \mathcal{X} \times \mathcal{Y}$$

that assigns a certified target relation to each tolerance level $\varepsilon \geq 0$. The exact target is $\mathcal{R}_0$.

Definition 3.13 (ε -Competence Under a Regime). Fix a relation family $(\mathcal{R}_\varepsilon)_{\varepsilon \geq 0}$ and regime Γ . A certifying solver (Q, V) is ε -competent on Γ if it is competent on Γ for relation $\mathcal{R}_\varepsilon$ in the sense of Definition 3.9.

Proposition 3.14 (Zero- ε Competence Reduction). *If $\mathcal{R}_0 = \mathcal{R}$, then*

$$\varepsilon\text{-competent at } \varepsilon = 0 \iff \text{competent for exact relation } \mathcal{R}.$$

Moreover, ε -competence implies integrity for the corresponding $\mathcal{R}_\varepsilon$. (L: [ic19](#), [ic33](#).)

Definition 3.15 (Typed Claim Report). For a declared objective family and regime, a solver-side report is one of:

- **ABSTAIN**,
- **EXACT** (claiming exact competence/certification),
- **EPSILON(ε)** (claiming ε -competence/certification).

Proposition 3.16 (Typed Claim Admissibility Discipline). *Under Definitions 3.9, 3.13, and 3.15:*

- ABSTAIN is always admissible;
- EXACT is admissible iff exact competence holds;
- EPSILON(ε) is admissible iff ε -competence holds.

Hence claim type and certificate type must match (no cross-typing of uncertified assertions as certified guarantees). (L: CC75.)

Definition 3.17 (Evidence Object for a Typed Report). For declared $(\mathcal{R}, (\mathcal{R}_\varepsilon)_{\varepsilon \geq 0}, \Gamma, Q)$ and report type $r \in \{\text{ABSTAIN}, \text{EXACT}, \text{EPSILON}(\varepsilon)\}$, *evidence* is a first-class witness object:

- for ABSTAIN: trivial witness;
- for EXACT: exact-competence witness;
- for EPSILON(ε): ε -competence witness.

(L: IC2.)

Proposition 3.18 (Evidence–Admissibility Equivalence). *For any declared contract and report type r :*

$$\text{Evidence object for } r \text{ exists} \iff \text{Claim } r \text{ is admissible.}$$

(L: IC17, IC20–21.)

[C:D3.17;P3.16, 3.18;R:AR]

Security-game reading (derived). The typed-report layer can be read as a standard claim-verification game under a declared contract: a reporting algorithm emits a report token r and optional evidence, and a checker accepts if and only if the evidence is valid for r . Within this game view, existing results already provide the core security properties: *completeness* for admissible report classes (Proposition 3.18), *soundness* against overclaiming (Propositions 3.16, 3.21), and *evidence-gated confidence* (Propositions 3.23, 3.24). So integrity is enforced as verifiable claim discipline, not as unchecked payload declaration. [C:P3.16, 3.18, 3.21, 3.23, 3.24;R:AR]

Definition 3.19 (Signaled Typed Report (Guess + Confidence)). A *signaled typed report* augments the typed report token $r \in \{\text{ABSTAIN}, \text{EXACT}, \text{EPSILON}(\varepsilon)\}$ with:

- an optional guess payload g ,
- self-reflected confidence $p_{\text{self}} \in [0, 1]$,
- certified confidence $p_{\text{cert}} \in [0, 1]$.
- scalar execution witness $t_{\text{run}} \in \mathbb{N}$ (steps actually run),
- optional declared bound $B \in \mathbb{N}$.

Here p_{self} is a self-signal, while p_{cert} is an evidence-gated signal under the declared contract. (L: IC34.)

Definition 3.20 (Signal Consistency Discipline). For a signaled typed report $(r, g, p_{\text{self}}, p_{\text{cert}}, t_{\text{run}}, B)$, require:

$$p_{\text{cert}} > 0 \Rightarrow \exists \text{ evidence object for } r.$$

Proposition 3.21 (Exact Claims Require Exact Evidence). *In the typed claim discipline,*

$$\text{ClaimAdmissible}(\text{EXACT}) \iff \exists \text{ exact evidence object.}$$

Equivalently, admissible exact claims are exactly those with an evidence witness. (L: IC35-36.)

Definition 3.22 (Bound-Dependent Completion Fraction). Completion-fraction semantics are defined only when a positive declared bound exists and covers observed runtime:

$$\text{CompletionFractionDefined} \iff \exists b > 0 : B = b \wedge t_{\text{run}} \leq b.$$

(L: IC37.)

Proposition 3.23 (Certified-Confidence Gate). *Under Definition 3.20, positive certified confidence implies typed admissibility:*

$$p_{\text{cert}} > 0 \Rightarrow \text{ClaimAdmissible}(r).$$

Thus certified confidence is not a free scalar; it is evidence-gated by the same typed discipline as the report itself. (L: IC38-39.)

Proposition 3.24 (No Evidence Forces Zero Certified Confidence). *Under Definition 3.20, absence of evidence for the reported type forces:*

$$\neg \exists \text{ evidence object for } r \Rightarrow p_{\text{cert}} = 0.$$

(L: IC40.)

Corollary 3.25 (Exact-No-Competence Zero-Certified Constraint). *For exact reports, if exact competence is unavailable on the declared regime/objective, then any signal-consistent report must satisfy:*

$$p_{\text{cert}} = 0.$$

(L: IC41.)

Proposition 3.26 (Scalar Step Witness Is Always Defined and Falsifiable). *For every signaled report, the execution witness is an exact natural number and any equality claim about it is decidable:*

$$\exists k \in \mathbb{N} : t_{\text{run}} = k, \quad \forall k \in \mathbb{N}, (t_{\text{run}} = k) \vee (t_{\text{run}} \neq k).$$

(L: IC42-43.)

Proposition 3.27 (No Completion Fraction Without Declared Bound). *If no declared bound is provided ($B = \emptyset$), completion-fraction semantics are undefined:*

$$B = \emptyset \Rightarrow \neg \text{CompletionFractionDefined}.$$

(L: IC44.)

Proposition 3.28 (Completion Fraction Under Explicit Bound). *If a positive declared bound is provided and observed runtime is within bound, completion-fraction semantics are defined:*

$$B = b, b > 0, t_{\text{run}} \leq b \Rightarrow \text{CompletionFractionDefined}.$$

(L: IC45.)

Proposition 3.29 (Abstain Can Carry Guess and Self-Confidence). *For any optional guess payload g and any self-reflected confidence $p_{\text{self}} \in [0, 1]$, there exists a signal-consistent abstain report*

$$(\text{ABSTAIN}, g, p_{\text{self}}, 0, 0, \emptyset).$$

Hence the framework is non-binary at the report-signal layer: abstention can carry a tentative answer and self-reflection without upgrading to certified exactness. (L: IC46.)

Proposition 3.30 (Self-Reflected Confidence Is Not Certification). *Self-reflected confidence alone does not certify exactness: if exact competence is unavailable, there exist exact-labeled reports with arbitrary p_{self} that remain inadmissible under zero certified confidence.* (L: [1C47.](#))

[C:D3.19, 3.20, 3.22;P3.21, 3.23, 3.24, 3.26, 3.27, 3.28, 3.29, 3.30;C3.25;R:AR]

Definition 3.31 (Declared Budget Slice). For a declared regime Γ over state objects and horizon/budget cut $H \in \mathbb{N}$, define the in-scope slice

$$\mathcal{S}_{\Gamma,H} := \{s : s \in \Gamma.\text{inScope} \wedge \text{encLen}_{\Gamma}(s) \leq H\}.$$

(L: [CC13.](#))

Proposition 3.32 (Bounded-Slice Irrelevance of a Meta-Coordinate). *Fix coordinate i_{∞} and declared slice $\mathcal{S}_{\Gamma,H}$ from Definition 3.31. If optimizer sets are invariant to i_{∞} on that slice, i.e.,*

$$\forall s, s' \in \mathcal{S}_{\Gamma,H}, \left(\forall j \neq i_{\infty}, s_j = s'_j \right) \Rightarrow \text{Opt}(s) = \text{Opt}(s'),$$

then i_{∞} is irrelevant on $\mathcal{S}_{\Gamma,H}$, and hence not relevant for the declared task slice. (L: [CC32-33.](#))

[C:D3.31;P3.32;R:AR]

Definition 3.33 (Goal Class). A *goal class* is a non-empty set of admissible evaluators over a fixed action/state structure:

$$\mathcal{G} = (\mathcal{U}_{\mathcal{G}}, A, S), \quad \emptyset \neq \mathcal{U}_{\mathcal{G}} \subseteq \{U : A \times S \rightarrow \mathbb{Q}\}.$$

The solver need not know which $U \in \mathcal{U}_{\mathcal{G}}$ is active; it only knows class-membership constraints. (L: [1V1.](#))

Definition 3.34 (Interior Pareto Dominance). For a goal class $\mathcal{G}$ and score model $\sigma : S \times \{1, \dots, n\} \rightarrow \mathbb{Q}$, let $J_{\mathcal{G}}$ be the class-tautological coordinate set. State s *interior-Pareto-dominates* s' on $J_{\mathcal{G}}$ when:

$$\forall j \in J_{\mathcal{G}}, \sigma(s', j) \leq \sigma(s, j) \quad \text{and} \quad \exists j \in J_{\mathcal{G}}, \sigma(s', j) < \sigma(s, j).$$

(L: [1V2-3.](#))

Proposition 3.35 (Interior Dominance Implies Universal Non-Rejection). *If a state s interior-Pareto-dominates s' on a checked coordinate set J , and every admissible evaluator in the goal class is monotone on J , then no admissible evaluator strictly prefers s' over s on that checked slice:*

$$\forall U \in \mathcal{U}_{\mathcal{G}}, \forall a \in A, U(a, s') \leq U(a, s).$$

(L: [1V5.](#))

Proposition 3.36 (Interior Verification Tractability). [C:P3.36;R:H=tractable-structured] *If membership in the checked coordinate set is decidable and interior dominance is decidable, then interior verification yields a polynomial-time checker with exact acceptance criterion:*

$$\exists \text{verify} : S \times S \rightarrow \{0, 1\}, \quad \text{verify}(s, s') = 1 \iff \text{interior-dominance}(s, s').$$

(L: [1V4,](#) [1V7.](#))

Proposition 3.37 (Interior Dominance Is One-Sided). *Interior dominance is one-sided and does not imply full coordinate sufficiency: a counterexample pair outside the checked slice can still violate global optimizer equivalence.* (L: [1V6,](#) [1V8.](#))

Corollary 3.38 (Singleton Interior Certificate). *[C:C3.38;R:H=tractable-structured] For a singleton checked coordinate i , strict improvement on i with class-monotonicity on $\{i\}$ is already a valid interior certificate of non-rejection. (L: IV9.)*

[C:D3.33, 3.34;P3.35, 3.36, 3.37;C3.38;R:AR]

Definition 3.39 (RLFF Objective (Evidence-Gated Report Reward)). Given report type $r \in \{\text{ABSTAIN}, \text{EXACT}, \text{EPSILON}(\varepsilon)\}$ and declared contract, RLFF assigns base reward by report type when evidence exists and applies an explicit inadmissibility penalty otherwise:

$$\text{Reward}(r) = \begin{cases} \text{Base}(r), & \text{if evidence for } r \text{ exists,} \\ -\text{Penalty}, & \text{otherwise.} \end{cases}$$

(L: IC4, IC28-29.)

Proposition 3.40 (RLFF Maximizer Is Admissible). *If ABSTAIN reward strictly exceeds the inadmissibility branch, then any global RLFF maximizer must be evidence-backed and therefore admissible under the typed-claim discipline. (L: IC31-32.)*

Corollary 3.41 (RLFF Abstain Preference Without Certificates). *If EXACT and EPSILON(ε) have no evidence objects while ABSTAIN beats the inadmissibility branch, then RLFF strictly prefers ABSTAIN to both report types. (L: IC30, IC40.)*

[C:D3.39;P3.40;C3.41;R:AR]

Definition 3.42 (Certainty Inflation). For a typed report r , certainty inflation is the state of emitting r without a matching evidence object. (L: IC1, IC3.)

Proposition 3.43 (Certainty Inflation Equals Report Inadmissibility). *For every typed report r :*

$$\text{CertaintyInflation}(r) \iff \neg \text{ClaimAdmissible}(r).$$

For $r = \text{EXACT}$ this is equivalently failure of exact competence. (L: IC8, IC22.)

Corollary 3.44 (Hardness-Blocked Regimes Induce Exact Certainty Inflation). *Under the declared hardness/non-collapse premises that block universal exact admissibility, every exact report is certainty-inflated. (L: CC19.)*

[C:D3.42;P3.43;C3.44;R:CR]

Definition 3.45 (Raw-vs-Certified Bit Accounting). Fix a declared contract and report type r . A report-bit model declares:

- raw report bits $B_{\text{raw}}(r)$ (payload bits in the asserted report token),
- certificate bits $B_{\text{cert}}(r)$ (bits allocated to the matching evidence class).

Evidence-gated overhead and certified total are then:

$$B_{\text{over}}(r) = \begin{cases} B_{\text{cert}}(r), & \text{if evidence for } r \text{ exists,} \\ 0, & \text{otherwise,} \end{cases} \quad B_{\text{total}}(r) = B_{\text{raw}}(r) + B_{\text{over}}(r).$$

(L: IC5, IC9, IC12.)

Proposition 3.46 (Raw/Certified Bit Split). *For every typed report r :*

$$B_{\text{total}}(r) = B_{\text{raw}}(r) + B_{\text{over}}(r),$$

with

$$\neg \exists \text{ evidence for } r \Rightarrow B_{\text{over}}(r) = 0, \quad \exists \text{ evidence for } r \Rightarrow B_{\text{over}}(r) = B_{\text{cert}}(r).$$

Hence $B_{\text{raw}}(r) \leq B_{\text{total}}(r)$ always, and strict inequality holds when evidence exists with positive certificate-bit allocation. (L: CC11, IC10-11, IC13-16, IC18-19.)

Theorem 3.47 (Exact Report: Certified-Bit Gap iff Admissibility). *Under a report-bit model with positive exact certificate-bit allocation:*

$$\text{ClaimAdmissible}(\text{EXACT}) \iff B_{\text{raw}}(\text{EXACT}) < B_{\text{total}}(\text{EXACT}).$$

Equivalently:

$$B_{\text{total}}(\text{EXACT}) = B_{\text{raw}}(\text{EXACT}) \iff \text{ExactCertaintyInflation}.$$

(L: [CC17-18](#), [CC20](#), [IC31](#).)

Corollary 3.48 (Finite Budget Eventually Blocks Exact Admissibility). *[C:C3.48;R:H=exp-lb-conditional] Fix a declared physical computer model with finite budget and positive per-bit cost, a declared operation-requirement lower-bound family, and a declared typed-report bit model. If:*

1. *exact admissibility implies certified-bit footprint at least the declared operation requirement at size n , and*
2. *exact admissibility implies physical feasibility of that certified footprint under the declared budget model,*

then there exists n_0 such that for all $n \geq n_0$, exact admissibility fails. The same conclusion holds in the canonical exponential-requirement specialization. (L: [PBC7-8](#).)

Corollary 3.49 (Hardness-Blocked Exact Reports Are Raw-Only). *If exact reporting is inadmissible on the declared contract, then exact accounting collapses to raw-only:*

$$B_{\text{total}}(\text{EXACT}) = B_{\text{raw}}(\text{EXACT}).$$

(L: [CC21](#), [IC23](#).)

[C:D3.45;T3.47;P3.46;C3.49;R:AR]

Corollary 3.50 (No Uncertified Exact-Exhaustion Claim). *If exact competence does not hold on the declared regime/objective, then an EXACT report is inadmissible. (L: [CC41](#).)*

[C:P3.16;C3.50;R:AR]

Proposition 3.51 (Declared-Contract Separation: Selection vs. Validity). *For solver-side reporting, there are two formally distinct layers:*

- **Declared-contract selection layer:** *choosing objective family $(\mathcal{R}_\varepsilon)_{\varepsilon \geq 0}$, regime Γ , and assumption profile;*
- **Validity layer:** *once declared, admissibility of EXACT/EPILON(ε)/ABSTAIN reports is fixed by certificate-typed rules.*

Thus the framework treats contract selection as an external declaration choice, and then enforces mechanically checked admissibility within that declared contract. (L: [CC22](#), [CC39](#), [CC75](#).)

Proof. The validity layer is Proposition 3.16: report admissibility is exactly tied to the matching competence certificate type. For exact physical claims outside carve-outs, Proposition 3.55 and Theorem 3.56 require explicit assumptions, and Corollary 3.57 blocks uncertified exact reports under declared non-collapse assumptions. Therefore, once the contract is declared, admissibility is mechanically determined by the typed rules and attached assumption profile. ■

[C:T3.56;P3.16, 3.55;C3.57;R:DC]

Definition 3.52 (Exact-Claim Excuse Carve-Outs). For exact physical claims in this framework, we declare four explicit carve-outs:

1. trivial scope (empty in-scope family),
2. tractable class (exact competence available in the active regime),
3. stronger oracle/regime shift (the claim is made under strictly stronger access assumptions),
4. unbounded resources (budget bounds intentionally removed).

Definition 3.53 (Explicit Hardness-Assumption Profile). An explicit hardness-assumption profile for declared class/regime $(\mathcal{R}, \Gamma)$ consists of:

- a non-collapse hypothesis,
- a collapse implication from universal exact competence on $(\mathcal{R}, \Gamma)$.

Definition 3.54 (Well-Typed Exact Physical Claim Policy). An exact physical claim is *well-typed* only if either:

- at least one carve-out from Definition 3.52 is explicitly declared, or
- an explicit hardness-assumption profile (Definition 3.53) is explicitly declared.

Proposition 3.55 (Outside Carve-Outs, Explicit Assumptions Are Required). *If an exact physical claim is well-typed (Definition 3.54) and no carve-out from Definition 3.52 applies, then an explicit hardness-assumption profile must be present. (L: cc22.)*

Theorem 3.56 (Claim-Integrity Meta-Theorem). *For declared class/regime objects, every admissible exact-claim policy outside the carve-outs requires explicit assumptions:*

$$\neg \text{Excused (Definition 3.52)} \wedge \text{Well-Typed Exact Claim (Definition 3.54)} \\ \Rightarrow \text{Has Explicit Hardness Profile (Definition 3.53)}.$$

(L: cc22.)

Proof. Immediate from Proposition 3.55; this is the universally quantified theorem-level restatement over the typed claim/regime objects. ■

Corollary 3.57 (Outside Carve-Outs, Declared Hardness Blocks Exact Reports). *If no carve-out applies and the declared hardness-assumption profile holds, then EXACT reports are inadmissible for every solver on the declared class/regime. (L: cc39.)*

[C:T3.56;P3.55;C3.57;R:CR]

Proposition 3.58 (Integrity–Competence Separation). *Integrity and competence are distinct predicates: integrity constrains asserted outputs, while competence adds non-abstaining coverage under resource bounds. (L: ic18, ic25.)*

Proof. Take the always-abstain solver $Q_{\perp}(x) = \text{ABSTAIN}$ with any polynomial-time checker V . Definition 3.8 holds vacuously, so $(Q_{\perp}, V)$ is integrity-preserving, but it fails Definition 3.9 whenever $\mathcal{X}_{\Gamma} \neq \emptyset$ because coverage fails. Hence integrity does not imply competence. The converse is immediate because competence includes integrity as a conjunct. ■

Definition 3.59 (Attempted Competence Failure). Fix an exact objective under regime Γ . A solver state is an *attempted competence failure* if:

- integrity holds (Definition 3.8),
- exact competence was actually attempted for the active scope/objective,
- competence on Γ fails for that exact objective (Definition 3.9).

Proposition 3.60 (Attempted-Competence Rationality Matrix). *Let $I, A, C \in \{0, 1\}$ denote integrity, attempted exact competence, and competence available in the active regime. Policy verdict for persistent over-specification is:*

- if $I = 0$: inadmissible,
- if $I = 1$ and $(A, C) = (1, 0)$: conditionally rational,
- if $I = 1$ and $(A, C) \in \{(0, 0), (0, 1), (1, 1)\}$: irrational for the same verified-cost objective.

Hence, in the integrity-preserving plane ($I = 1$), exactly one cell is rational and three are irrational. (L: IC6-7, IC27)

This separation plus Proposition 3.60 is load-bearing for the regime-conditional trilemma used later: if exact competence is blocked by hardness in a declared regime after an attempted exact procedure, integrity forces one of three responses: abstain, weaken guarantees, or change regime assumptions. [C:P3.60;R:AR]

Mechanized status. This separation is machine-checked in DecisionQuotient/IntegrityCompetence.lean via: IC18 and IC25; the attempted-competence matrix is mechanized via IC27, IC7, and IC6.

Proposition 3.61 (Integrity-Resource Bound). *Let Γ be a declared regime whose in-scope family includes all instances of SUFFICIENCY-CHECK and whose declared resource bound is polynomial in the encoding length. If $P \neq \text{coNP}$, then no certifying solver is simultaneously integrity-preserving and competent on Γ for exact sufficiency. Equivalently, for every integrity-preserving solver, at least one of the competence conjuncts must fail on Γ : either full non-abstaining coverage fails or the declared budget bound fails. (L: IC24, IC26, CC30.)*

Proof. By Definition 3.9, competence on Γ requires integrity, full in-scope coverage, and budget compliance. Under the coNP-hardness transfer for SUFFICIENCY-CHECK (Theorem 4.6), universal competence on this scope would induce a polynomial-time collapse to $P = \text{coNP}$. Therefore, under $P \neq \text{coNP}$, the full competence predicate cannot hold. Since integrity alone is satisfiable (Proposition 3.58, via always-abstain), the obstruction is exactly competence coverage/budget under the declared regime. [C:T4.6;P3.58, 3.61;R:AR] ■

Proposition 3.62 (Declared-Physics No-Universal-Exact-Certifier Schema). *Fix any declared physical/task class representable by relation $\mathcal{R}$ and regime Γ in the certifying-solver model. If*

$$(\exists Q, \text{CompetentOn}(\mathcal{R}, \Gamma, Q)) \Rightarrow (P = \text{coNP}),$$

then under $P \neq \text{coNP}$ there is no universally exact-competent solver for that declared class. (L: CC15.)

Corollary 3.63 (No Universal Exact-Claim Admissibility in Hardness-Blocked Classes). *Under the same premises as Proposition 3.62, and under the typed-claim discipline of Definition 3.15, an EXACT report is inadmissible for every solver on that declared class. Hence admissible reporting must use ABSTAIN or explicitly weakened guarantees. (L: CC38.)*

[C:P3.62;C3.63;R:CR]

Informally: honesty limits what you can claim, capability limits what you can solve, and hard regimes may require abstaining or using weaker exact claims.

4 Computational Complexity of Decision-Relevant Uncertainty

This section establishes the computational complexity of information sufficiency, formalized as coordinate sufficiency, for an arbitrary decision problem $\mathcal{D}$ with factored state space. Because the problems below are parameterized by $\mathcal{D}$, and the (A, S, U) tuple captures any problem with choices under structured information (Definition 2.1), the hardness results are universal: they apply to every problem with coordinate structure, not to a specific problem domain. We prove three main results:

1. **SUFFICIENCY-CHECK** is coNP-complete
2. **MINIMUM-SUFFICIENT-SET** is coNP-complete (the Σ_2^P structure collapses)
3. **ANCHOR-SUFFICIENCY** (fixed coordinates) is Σ_2^P -complete

Under the model contract of Section 2.6 and the succinct encoding of Section 2.5, these results place exact relevance identification beyond NP-completeness: in the worst case, finding (or certifying) minimal decision-relevant sets is computationally intractable.

Reading convention for this section. Each hardness theorem below is paired with a recovery boundary for the same decision question: when structural-access assumptions in Theorem 10.1 hold, polynomial-time exact procedures are recovered; when they fail in [S], the stated hardness applies. (L: CC60, CC36, CC4, CC71.)

4.1 Problem Definitions

Definition 4.1 (Decision Problem Encoding). A *decision problem instance* is a tuple $(A, X_1, \dots, X_n, U)$ where:

- A is a finite set of alternatives
- $X_1, \dots, X_n$ are the coordinate domains, with state space $S = X_1 \times \dots \times X_n$
- $U : A \times S \rightarrow \mathbb{Q}$ is the utility function (in the succinct encoding, U is given as a Boolean circuit)

Succinct/circuit encoding. When U is given succinctly as a Boolean or arithmetic circuit, the instance size refers to the circuit description rather than the (possibly exponentially large) truth table. Evaluating $U(a, s)$ on a single pair (a, s) can be done in time polynomial in the circuit size, but quantification over all states (for example, "for all s ") ranges over an exponentially large domain and is the source of the hardness results below.

Definition 4.2 (Optimizer Map). For state $s \in S$, define:

$$\text{Opt}(s) := \arg \max_{a \in A} U(a, s)$$

Definition 4.3 (Sufficient Coordinate Set). A coordinate set $I \subseteq \{1, \dots, n\}$ is *sufficient* if:

$$\forall s, s' \in S : \quad s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

where s_I denotes the projection of s onto coordinates in I .

Problem 4.4 (SUFFICIENCY-CHECK). **Input:** Decision problem $(A, X_1, \dots, X_n, U)$ and coordinate set $I \subseteq \{1, \dots, n\}$

Question: Is I sufficient?

Problem 4.5 (MINIMUM-SUFFICIENT-SET). **Input:** Decision problem $(A, X_1, \dots, X_n, U)$ and integer k

Question: Does there exist a sufficient set I with $|I| \leq k$?

4.2 Hardness of SUFFICIENCY-CHECK

Classical coNP-completeness methodology follows standard NP/coNP reduction frameworks and polynomial-time many-one reducibility [63, 5]. We instantiate that framework for SUFFICIENCY-CHECK with a mechanized TAUTOLOGY reduction.

Correctness vs. polynomial-time computability. A Karp reduction requires two properties: (1) *correctness*—the reduction preserves membership ($\forall x, x \in A \Leftrightarrow f(x) \in B$), and (2) *efficiency*—the reduction is computable in polynomial time. The Lean formalization **machine-verifies** property (1): the theorem `tautology_iff_sufficient` proves that φ is a tautology if and only if $I = \emptyset$ is sufficient in the constructed decision problem. Property (2)—polynomial-time TM computability of the reduction function—is a **standard claim** stated as an explicit hypothesis. This separation is standard in formalization: no existing proof assistant project has achieved end-to-end Turing-machine verification of polynomial-time bounds for concrete reductions.

Theorem 4.6 (coNP-completeness of SUFFICIENCY-CHECK). *SUFFICIENCY-CHECK is coNP-complete. (L: CC61, CC60, L18.)*

Source	Target	Key property preserved
TAUTOLOGY	SUFFICIENCY-CHECK	Tautology iff $\emptyset$ sufficient
$\exists\forall$ -SAT	ANCHOR-SUFFICIENCY	Witness anchors iff formula true

Proof. `extbfMembership` in coNP: The complementary problem INSUFFICIENCY is in NP. A non-deterministic verifier can check a witness pair (s, s') as follows:

1. Verify $s_I = s'_I$ by projecting and comparing coordinates; this takes time polynomial in the input size.
2. Evaluate U on every alternative for both s and s' . When U is given as a circuit each evaluation runs in time polynomial in the circuit size; using these values the verifier checks whether $\text{Opt}(s) \neq \text{Opt}(s')$.

Hence INSUFFICIENCY is in NP and SUFFICIENCY-CHECK is in coNP.

coNP-hardness: We reduce from TAUTOLOGY.

Given Boolean formula $\varphi(x_1, \dots, x_n)$, construct a decision problem with:

- Alternatives: $A = \{\text{accept}, \text{reject}\}$
- State space: $S = \{\text{reference}\} \cup \{0, 1\}^n$ (equivalently, encode this as a product space with one extra coordinate $r \in \{0, 1\}$ indicating whether the state is the reference state)

- Utility:

$$\begin{aligned}
U(\text{accept}, \text{reference}) &= 1 \\
U(\text{reject}, \text{reference}) &= 0 \\
U(\text{accept}, a) &= \varphi(a) \\
U(\text{reject}, a) &= 0 \quad \text{for assignments } a \in \{0, 1\}^n
\end{aligned}$$

- Query set: $I = \emptyset$

Claim: $I = \emptyset$ is sufficient $\iff \varphi$ is a tautology.

($\Rightarrow$) Suppose I is sufficient. Then $\text{Opt}(s)$ is constant over all states. Since $U(\text{accept}, a) = \varphi(a)$ and $U(\text{reject}, a) = 0$:

- $\text{Opt}(a) = \text{accept}$ when $\varphi(a) = 1$
- $\text{Opt}(a) = \{\text{accept}, \text{reject}\}$ when $\varphi(a) = 0$

For Opt to be constant, $\varphi(a)$ must be true for all assignments a ; hence φ is a tautology.

($\Leftarrow$) If φ is a tautology, then $U(\text{accept}, a) = 1 > 0 = U(\text{reject}, a)$ for all assignments a . Thus $\text{Opt}(s) = \{\text{accept}\}$ for all states s , making $I = \emptyset$ sufficient. ■

Immediate recovery boundary (SUFFICIENCY-CHECK). Theorem 4.6 is the [S] general-case hardness statement. For the same sufficiency relation, Theorem 10.1 gives polynomial-time recovery under explicit-state, separable, and tree-structured regimes. (L: [cc60](#), [cc71](#), [cc69](#), [cc70](#), [cc72](#).)

Mechanized strengthening (all coordinates relevant). The reduction above establishes coNP-hardness using a single witness set $I = \emptyset$. For the ETH-based lower bound in Theorem 9.1, we additionally need worst-case instances where the minimal sufficient set has *linear* size.

We formalized a strengthened reduction in Lean 4: given a Boolean formula φ over n variables, construct a decision problem with n coordinates such that if φ is not a tautology then *every* coordinate is decision-relevant (so $k^* = n$). Intuitively, the construction places a copy of the base gadget at each coordinate and makes the global “accept” condition hold only when every coordinate’s local test succeeds; a single falsifying assignment at one coordinate therefore changes the global optimal set, witnessing that coordinate’s relevance. This strengthening is mechanized in Lean; see Appendix A.

4.3 Complexity of MINIMUM-SUFFICIENT-SET

Theorem 4.7 (MINIMUM-SUFFICIENT-SET is coNP-complete). *MINIMUM-SUFFICIENT-SET is coNP-complete.* (L: [cc34](#), [cc36](#).)

Proof. **Structural observation:** The $\exists\forall$ quantifier pattern suggests Σ_2^P :

$$\exists I (|I| \leq k) \forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

However, this collapses because sufficiency has a simple characterization.

Key lemma: In the Boolean-coordinate collapse model, a coordinate set I is sufficient if and only if I contains all relevant coordinates (proven formally as `sufficient_iff_relevant_subset` / `sufficient_iff_relevantFinset_subset` in Lean):

$$\text{sufficient}(I) \iff \text{Relevant} \subseteq I$$

where $\text{Relevant} = \{i : \exists s, s'. s \text{ differs from } s' \text{ only at } i \text{ and } \text{Opt}(s) \neq \text{Opt}(s')\}$. This is the same relevance object as Definition 2.14; Proposition 2.15 gives the minimal-set equivalence in the product-space semantics used by the collapse.

Consequence: The minimum sufficient set is exactly the relevant-coordinate set. Thus MINIMUM-SUFFICIENT-SET asks: “Is the number of relevant coordinates at most k ?”

coNP membership: A witness that the answer is NO is a set of $k + 1$ coordinates, each proven relevant (by exhibiting s, s' pairs). Verification is polynomial.

coNP-hardness: The $k = 0$ case asks whether no coordinates are relevant, i.e., whether $\emptyset$ is sufficient. This is exactly SUFFICIENCY-CHECK, which is coNP-complete by Theorem 4.6. ■

Immediate recovery boundary (MINIMUM-SUFFICIENT-SET). Theorem 4.7 pairs with Theorem 4.15: exact minimization complexity is governed by relevance-cardinality once the collapse is applied. Recovery then depends on computing relevance efficiently, with structured-access assumptions from Theorem 10.1 giving the corresponding route for the underlying sufficiency computations. (L: CC36, CC34, CC35, S2P2, CC71.)

4.4 Anchor Sufficiency (Fixed Coordinates)

We also formalize a strengthened variant that fixes the coordinate set and asks whether there exists an *assignment* to those coordinates that makes the optimal action constant on the induced subcube.

Problem 4.8 (ANCHOR-SUFFICIENCY). **Input:** Decision problem $(A, X_1, \dots, X_n, U)$ and fixed coordinate set $I \subseteq \{1, \dots, n\}$

Question: Does there exist an assignment α to I such that $\text{Opt}(s)$ is constant for all states s with $s_I = \alpha$?

The second-level hardness template used here follows canonical Σ_2^P polynomial-hierarchy characterizations [63, 5], instantiated to the anchor formulation below.

Theorem 4.9 (ANCHOR-SUFFICIENCY is Σ_2^P -complete). *ANCHOR-SUFFICIENCY is Σ_2^P -complete (already for Boolean coordinate spaces).* (L: CC5, CC4, L2.)

Proof. Membership in Σ_2^P : The problem has the form

$$\exists \alpha \forall s \in S : (s_I = \alpha) \implies \text{Opt}(s) = \text{Opt}(s_\alpha),$$

which is an $\exists\forall$ pattern.

Σ_2^P -hardness: Reduce from $\exists\forall$ -SAT. Given $\exists x \forall y \varphi(x, y)$ with $x \in \{0, 1\}^k$ and $y \in \{0, 1\}^m$, if $m = 0$ we first pad with a dummy universal variable (satisfiability is preserved), construct a decision problem with:

- Actions $A = \{\text{YES}, \text{NO}\}$
- State space $S = \{0, 1\}^{k+m}$ representing (x, y)
- Utility

$$U(\text{YES}, (x, y)) = \begin{cases} 2 & \text{if } \varphi(x, y) = 1 \\ 0 & \text{otherwise} \end{cases} \quad U(\text{NO}, (x, y)) = \begin{cases} 1 & \text{if } y = 0^m \\ 0 & \text{otherwise} \end{cases}$$

- Fixed coordinate set $I =$ the x -coordinates.

If $\exists x^* \forall y \varphi(x^*, y) = 1$, then for any y we have $U(\text{YES}) = 2$ and $U(\text{NO}) \leq 1$, so $\text{Opt}(x^*, y) = \{\text{YES}\}$ is constant. Conversely, fix x and suppose $\exists y_f$ with $\varphi(x, y_f) = 0$.

- If $\varphi(x, 0^m) = 1$, then $\text{Opt}(x, 0^m) = \{\text{YES}\}$. The falsifying assignment must satisfy $y_f \neq 0^m$, where $U(\text{YES}) = U(\text{NO}) = 0$, so $\text{Opt}(x, y_f) = \{\text{YES}, \text{NO}\}$.
- If $\varphi(x, 0^m) = 0$, then $\text{Opt}(x, 0^m) = \{\text{NO}\}$. After padding we have $m > 0$, so choose any $y' \neq 0^m$: either $\varphi(x, y') = 1$ (then $\text{Opt}(x, y') = \{\text{YES}\}$) or $\varphi(x, y') = 0$ (then $\text{Opt}(x, y') = \{\text{YES}, \text{NO}\}$). In both subcases the optimal set differs from $\{\text{NO}\}$.

Hence the subcube for this x is not constant. Thus an anchor assignment exists iff the $\exists\forall$ -SAT instance is true. ■

Immediate boundary statement (ANCHOR-SUFFICIENCY). Theorem 4.9 remains a second-level hardness statement in the anchored formulation; unlike MINIMUM-SUFFICIENT-SET, no general collapse to relevance counting is established here, so the corresponding tractability status remains open in this model. (L: CC5, CC4, L2.)

Remark 4.10 (Discovery vs Verification). Informally, SUFFICIENCY-CHECK asks whether a given viewpoint works, a verification task ($\forall$), while ANCHOR-SUFFICIENCY asks whether any viewpoint works, a discovery task ($\exists\forall$). The proven complexity gap between coNP and Σ_2^P suggests that finding the right frame is structurally harder than checking a given frame, unless the polynomial hierarchy collapses.

4.5 Tractable Subcases

Despite the general hardness, several natural subcases admit efficient algorithms:

Proposition 4.11 (Small State Space). *When $|S|$ is polynomial in the input size (i.e., explicitly enumerable), MINIMUM-SUFFICIENT-SET is solvable in polynomial time.*

Proof. Compute $\text{Opt}(s)$ for all $s \in S$. The minimum sufficient set is exactly the set of coordinates that “matter” for the resulting function, computable by standard techniques. ■

Proposition 4.12 (Linear Utility). *When $U(a, s) = w_a \cdot s$ for weight vectors $w_a \in \mathbb{Q}^n$, MINIMUM-SUFFICIENT-SET reduces to identifying coordinates where weight vectors differ.*

4.6 Implications

Corollary 4.13 (Computational Bottleneck for Exact Minimization). *Under the succinct encoding, exact minimization of sufficient coordinate sets is coNP -hard via the $k = 0$ case, and fixed-anchor minimization is Σ_2^P -complete. (L: CC36, CC4)*

Proof. The $k = 0$ case of MINIMUM-SUFFICIENT-SET is SUFFICIENCY-CHECK (Theorem 4.6), giving coNP -hardness for exact minimization. The fixed-anchor variant is exactly Theorem 4.9. ■

The modeling budget for deciding what to model is therefore a computationally constrained resource under this encoding. [C:Thm. 4.6, Thm. 4.9; R:|S|]

4.7 The succinct–minimal gap and physical controllers

In complexity theory, a *succinct* representation of a combinatorial object is a Boolean circuit of size $\text{poly}(n)$ that computes a function whose explicit (truth-table) form has size $2^{\Theta(n)}$. The term is standard but informal: it denotes a size class (polynomial in input variables), not a structural property. In particular, “succinct” does not mean *minimal*. The minimal circuit for a function is the smallest circuit that computes it, a rigorous, unique object. A succinct circuit may be far larger than the minimal one while still qualifying as “succinct” by the poly-size criterion.

The gap. Let C be a succinct circuit computing a utility function U , and let C^* be the minimal circuit for the same function. The *representation gap* $\varepsilon = |C| - |C^*|$ measures how far the controller encoding is from the tightest possible encoding. This gap has no standard formal definition in the literature; it is the unnamed distance between an informal size class and a rigorous structural minimum.

Proposition 4.14 (Coordinate Representation Gap as a Formal Object). *In the Boolean-coordinate model, let $R_{\text{rel}} = \text{relevantFinset}(dp)$ and define*

$$\varepsilon(I) := |I \setminus R_{\text{rel}}| + |R_{\text{rel}} \setminus I|.$$

Then:

$$\varepsilon(I) = 0 \iff I = R_{\text{rel}} \iff I \text{ is minimal sufficient.}$$

(L: [S2P3](#), [S2P4](#), [S2P5](#).)

The results of this paper give ε formal complexity consequences. By Theorem 4.7, determining the minimal sufficient coordinate set is **coNP**-complete under the succinct encoding. By Theorem 9.1, under ETH there exist succinct instances requiring $2^{\Omega(n)}$ time to verify sufficiency. In the formal Boolean-coordinate collapse model, size-bounded $\varepsilon = 0$ search is exactly the relevance-cardinality predicate [S2P2](#). Together: for a physical decision circuit encoded succinctly, the worst-case cost of determining whether its coordinate attention set is minimal is exponential in the number of input coordinates (under ETH). The gap ε is *computationally irreducible*: no polynomial-time algorithm solves all succinct instances (assuming $P \neq \text{coNP}$), and the hard instance family requires $2^{\Omega(n)}$ time.

Physical systems as circuits. Any physical system that selects actions based on observed state can be modeled, in the static snapshot, as a circuit: observed inputs are coordinates $X_1, \dots, X_n$; internal computation maps states to actions; and the quality of that mapping is measured by a utility function U . [C:Prop. 2.60, Prop. 2.53; R:[bridge-admissible snapshot]] The explicit-state encoding (truth table) of this mapping has size exponential in the number of sensory dimensions. No physical finite-memory system can store it. [C:Thm. 9.1; R:[E] vs [S+ETH]] Every realizable physical controller is therefore a succinct encoding, a circuit whose size is bounded by physical resources rather than by the size of the state space it navigates. [C:Thm. 9.1; R:[S]]

Atomic and molecular systems fit the same form: measured coordinates define observed state, transition operators implement action selection, and utility is objective-conditioned over resulting states. [C:Prop. 2.60; R:[represented adjacent class]] In this setting, the physical system is a succinct encoding of a utility-to-action mapping, and the question “is this system attending to the right coordinates?” is exactly SUFFICIENCY-CHECK under the succinct encoding. [C:Thm. 11.2, Thm. 4.6; R:[S]] The compression that makes a circuit physically realizable, with polynomial size rather than exponential size, is the same compression that makes self-verification exponentially costly in the worst case: the state space the circuit compressed away is precisely the domain that must be exhaustively checked to certify sufficiency. [C:Thm. 9.1, Prop. 9.3; R:[S+ETH], [Q_fin]]

The simplicity tax as ε . The simplicity tax (Definition 13.2) measures $|\text{Gap}(M, P)| = |R(P) \setminus A(M)|$: the number of decision-relevant coordinates that the model does not handle natively. In the ε decomposition above, this is exactly the unattended-relevant component $|R_{\text{rel}} \setminus I|$ (S2P7, S2P5). For a physical circuit model, this is the coordinate-level manifestation of ε : the circuit attends to a superset of what is necessary, or fails to attend to relevant coordinates, and faces worst-case cost exponential in n to verify which case holds (Theorem 4.6). Theorem 13.3 then says the total relevance budget is conserved: unhandled coordinates are not eliminated, only redistributed as external per-site cost.

Consequences for learned circuits. Large language models and deep networks are succinct circuits with $\text{poly}(n)$ parameters. [C:Thm. 9.1;R:[S]] Mechanistic interpretability asks which internal components (attention heads, neurons, layers) are necessary for a given capability; this is SUFFICIENCY-CHECK applied to the circuit’s internal coordinate structure. Pruning research asks for the minimal subcircuit that preserves performance; this is MINIMUM-SUFFICIENT-SET. The results of this paper imply that these questions are **coNP**-hard in the worst case under succinct encoding. [C:Thm. 4.6, Thm. 4.7;R:[S]] Empirical progress on interpretability and pruning therefore depends on exploiting the structural properties of specific trained circuits (analogous to the tractable subcases of Theorem 10.1), not on solving the general problem. [C:Thm. 4.6, Thm. 10.1;R:[S] vs [tractable regimes]]

Scope and caveats. The formal results above apply to the static, deterministic model fixed by C1–C4 (Section 2.6). Physical systems can add noise, temporal dynamics, and distribution shift. These factors may alter empirical tractability relative to the static worst-case bounds. The bridge conditions of Proposition 2.52 delineate when static sufficiency results transfer to sequential settings; beyond those conditions, the governing complexity objects change (Propositions 2.55–2.57). The claims in this subsection are therefore regime-typed consequences of the formal results, not universal assertions about all possible physical architectures. [C:Prop. 2.52, Thm. 2.58, Prop. 2.60;R:[represented adjacent class]]

4.8 Quantifier Collapse for MINIMUM-SUFFICIENT-SET

Theorem 4.15 (Collapse of the Apparent $\exists\forall$ Structure). *The apparent second-level predicate*

$$\exists I (|I| \leq k) \forall s, s' \in S : s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

is equivalent to the coNP predicate $|\text{Relevant}| \leq k$, where

$$\text{Relevant} = \{i : \exists s, s'. s \text{ differs from } s' \text{ only at } i \text{ and } \text{Opt}(s) \neq \text{Opt}(s')\}.$$

Consequently, MINIMUM-SUFFICIENT-SET is governed by coNP certificates rather than a genuine Σ_2^P alternation. (L: CC34, CC35, S2P2.)

Proof. By the formal equivalence `sufficient_iff_relevant_subset` (finite-set form `sufficient_iff_relevantFinset_subset`), a coordinate set I is sufficient iff $\text{Relevant} \subseteq I$. Therefore:

$$\exists I (|I| \leq k \wedge \text{sufficient}(I)) \iff \exists I (|I| \leq k \wedge \text{Relevant} \subseteq I) \iff |\text{Relevant}| \leq k.$$

So the existential-over-universal presentation collapses to counting the relevant coordinates.

A NO certificate for $|\text{Relevant}| \leq k$ is a list of $k+1$ distinct relevant coordinates, each witnessed by two states that differ only on that coordinate and yield different optimal sets; this is polynomially verifiable. Hence the resulting predicate is in coNP, matching Theorem 4.7.

This also clarifies why ANCHOR-SUFFICIENCY remains Σ_2^P -complete: once an anchor assignment is existentially chosen, the universal quantifier over the residual subcube does not collapse to a coordinate-counting predicate. ■

5 Physical Grounding of Discrete Information Acquisition

5.1 Continuous vs Discrete: A Question of Tractability, Not Validity

Both continuous and discrete models are valid. The choice between them is not about which is “true” but about which is tractable for the computational system using it.

- **Continuous models** use limits to collapse computational complexity. You don’t check all states; you abstract them away. This is a useful thinking tool for mathematical analysis.
- **Discrete models** are the actual structure that physical computational systems must work with.

The two perspectives are duals: the continuous model is the limit of the discrete model as resolution increases; the discrete model is the physical instantiation that any bounded system must use.

Question: Does the continuous abstraction beat physical limits?

No. Limited abstract counting collapses computational complexity in the model, but it does not beat the physical limits of computation speed. No physical decision circuit can compute faster than the speed of light. The continuous abstraction is a cognitive tool for tractable thinking; it does not help a physical computational system that is bounded by c .

5.2 First-Principles Derivation: The Counting Gap

We formalize this with a theorem of pure mathematics. No physical law is assumed.

Theorem 5.1 (Counting Gap Theorem). *Let $\varepsilon, C \in \mathbb{N}$ with $\varepsilon > 0$ and $C > 0$. If each information-acquisition event consumes ε discrete cost units, then for every $N \in \mathbb{N}$:*

$$\varepsilon \cdot N \leq C \Rightarrow N \leq C.$$

Equivalently, C itself is a finite upper bound on the number of possible events. No bounded system with positive integer event cost can perform infinitely many events.

Proof. This is a discrete theorem. In $\mathbb{N}$, every positive integer is ≥ 1 . Therefore $N = 1 \cdot N \leq \varepsilon \cdot N \leq C$. ■

(L: [BA10](#))

Interpretation. A continuous process would require infinitely many events. The theorem proves that any bounded computational system with positive event cost has a finite maximum. Physical computational systems are bounded and have positive cost. Therefore physical computational systems cannot use the continuous abstraction to beat their limits.

Corollary 5.2 (Forced Finiteness of Propagation Speed). *A bounded region cannot acquire information at unbounded rate. Therefore some finite propagation bound $c_{\max}$ must exist.* [BA10](#)

5.3 Three Empirical Laws: The Numerical Content

Theorem 5.1 proves that a finite bound exists. It does not supply the numerical value. Three empirical laws supply the values in our universe:

SR. Special Relativity. No physical circuit computes faster than c . Einstein established: $c = 2.998 \times 10^8$ m/s. [25, 54]

QM. Quantum Mechanics. State transitions are discrete eigenstate jumps. Planck and Dirac established: $E = h\nu$. [70, 23]

TD. Thermodynamics (Landauer). Each event costs positive energy. Landauer established the universal floor: $\varepsilon \geq k_B T \ln 2$ joules per irreversible bit. The Wolpert-style interface used in this artifact adds explicit nonnegative mismatch and residual dissipation terms above that floor; the mismatch term can now be justified inside the artifact from finite-distribution KL divergence when the actual and designed distributions differ, and the finite discrete residual branch can also be justified inside the artifact by an exhaustive reverse-edge split. If the reverse edge flow is positive, asymmetric forward/reverse flows yield a positive two-point KL witness; if the reverse edge flow is zero, the process performs an irreversible one-way state transition, and the existing Landauer-scaled transition-cost theorem yields a positive lower-bound witness. The broader stopping-time / absolute-irreversibility residual regime remains a separate imported premise. When any one of these contributions is positive, the per-bit lower bound is strictly larger than $k_B T \ln 2$. [44, 7, 9, 96, 51, 98]

(*L: WC1-3, WM1, WM3-4, WR1-3, WR6.*)

The Counting Gap Theorem proves the structure. SR, QM, TD supply the numbers.

5.4 Why Positive Cost is Derived, Not Assumed

The premise $\varepsilon > 0$ is not arbitrary. A check is a physical state transition. Physical transitions require force. Force over distance requires energy. At $T > 0$, any irreversible transition carries at least the Landauer floor $k_B T \ln 2$. In the explicit constrained-process interface, any extra dissipation above that floor is represented by separate nonnegative mismatch and residual terms rather than folded into the floor itself. The mismatch branch is now pinned to the existing KL layer: explicit finite-distribution mismatch yields a strictly positive real KL term, which is then conservatively rounded upward into the nat-valued lower-bound units used by the thermodynamic accounting. The finite discrete residual branch is now also theorem-level by an exhaustive local split on the reverse edge of a computational-state process: either the reverse edge is positive and asymmetry yields strictly positive two-point KL divergence, or the reverse edge is zero and the process pays strictly positive Landauer-scaled cost for the resulting irreversible one-way state transition.

If $\varepsilon = 0$, a bounded system could perform infinitely many checks, resolving any decision in zero time and zero energy. This contradicts energy conservation. Therefore $\varepsilon > 0$ is derived from thermodynamics. (*L: BA10, WC2-3, WM1, WM3-4, WR2-3, WR6*)

Remark 5.3 (Scope). No claim is made about whether the universe is continuous or discrete at scales below the physical resolution of any bounded system. Whether finer structure exists is irrelevant: it is inaccessible to any bounded region of spacetime at rate $> c/d$, and therefore has no effect on any physical decision process.

5.5 Bounded Information Acquisition Rate

Proposition 5.4 (Bounded Region). *A bounded region $\mathcal{R}$ is characterized by diameter $d > 0$ and signal propagation speed $c > 0$ (in abstract natural units). The maximum information-acquisition rate of $\mathcal{R}$ is c/d events per unit time.*

*By **SR**: information emitted from outside $\mathcal{R}$ takes $\geq d/c$ time to reach $\mathcal{R}$. Therefore $\mathcal{R}$ cannot acquire information at rate $> c/d$. [BA1](#)*

(L: [BA1](#)) [C:D2.42;R:AR]

Theorem 5.5 (Bounded Acquisition Rate). *For bounded region $\mathcal{R}$ with diameter d and signal speed c , operating for time T :*

$$\text{acquisitions}(\mathcal{R}, T) \leq \frac{c \cdot T}{d}.$$

This count is finite. $\mathcal{R}$ cannot acquire information continuously. [BA2](#)

(L: [BA2](#))

5.6 Acquisitions are Discrete State Transitions

Theorem 5.6 (Discrete Acquisition). *By **QM**: the state space of any bounded region with finite energy is countable (discrete energy eigenstates). Each information-acquisition event is a transition $A \rightarrow B$ between eigenstates, a discrete event. Between transitions, the state is fixed and no new information is held.*

*Formally: any physical decision process in $\mathcal{R}$ is a **DiscreteSystem** (Definition in *Physics/DiscreteSpacetime.lean*). Acquisition events are exactly **isTransitionPoint** events.*

[BA3](#)

(L: [BA3](#)) [C:D2.42;P2.46;R:AR]

5.7 One Transition = One Bit

Theorem 5.7 (Boolean Encoding is Physically Primitive). *Each discrete state transition transfers exactly one boolean bit: the system was in state A (encoded 0), it transitions to state B (encoded 1). The encoding $\{A, B\} \cong \{0, 1\}$ is not a modeling choice; it is the form of physical state transitions under **QM**.*

*Formally: each **isTransitionPoint** contributes exactly 1 to **bitOperations**. Boolean coordinates are the elementary quanta of physical information exchange. [BA4](#)*

(L: [BA4](#))

Corollary 5.8 (Finite Accessible State Space). *A bounded region $\mathcal{R}$ operating for finite time T acquires at most $n_{\max}(\mathcal{R}, T) = \lfloor cT/d \rfloor$ bits. Its accessible state space is a finite subset of $\{0, 1\}^{n_{\max}}$. Uncountable state spaces are not accessible to any bounded region in finite time. [BA2](#) [BA4](#)*

(L: [BA2](#), [BA4](#))

5.8 Structural Rank = Minimum Physical Bit Operations

Theorem 5.9 (Resolution Requires a Sufficient Coordinate Set). *Any physical process that correctly determines the optimal action for all states of decision problem $\mathcal{D}$ must access a sufficient coordinate set I .*

If it accesses fewer coordinates than needed for sufficiency, there exist two states with identical readings but different optimal actions. The resolver outputs the wrong action on one. Correctness forces sufficiency.

Each coordinate in I requires one bit operation to read (one state transition: register samples coordinate $i \rightarrow$ one bit acquired by Theorem 5.7). Therefore: bit operations $\geq |I|$. [BA5](#)

(L: [BA5](#))

Theorem 5.10 (Structural Rank = Minimum Bit Operations). For decision problem $\mathcal{D}$ with $\text{srnk}(\mathcal{D}) = k$:

1. **Information (BA6):** $k = \text{srnk}(\mathcal{D}) \leq |I|$ for any sufficient coordinate set I , by `srnk_le_sufficient_card` (Tractability/StructuralRank.lean). Therefore: minimum bits to resolve $\mathcal{D}$ correctly $= k$.
2. **Time (BA6 + SR):** Each bit operation requires one tick of duration $\geq d/c$. Minimum decision time $= k \cdot d/c$.
3. **Energy (BA7 + TD):** Each bit operation carries at least the Landauer floor $k_B T \ln 2$ by **TD**. Therefore every resolver obeys the universal lower bound $E \geq k \cdot k_B T \ln 2$; stronger empirical lower bounds only raise this floor.

Structural rank k is simultaneously an information cost, a time cost, and an energy cost. These are not three separate quantities; they are the same count k expressed in three physical units. [BA6 BA7](#)

(L: [BA6-7](#), [SR1](#)) [C:T2.49;R:AR]

5.9 Thermodynamic Lower Bound as Physical Consequence

Theorem 5.11 (Thermodynamic Cost Derived from SR, QM, TD). For any physical system resolving decision problem $\mathcal{D}$ at temperature $T > 0$:

$$E_{\text{decision}} \geq \text{srnk}(\mathcal{D}) \cdot k_B T \ln 2.$$

Proof. Resolving $\mathcal{D}$ requires reading a sufficient coordinate set I (Theorem 5.9). Each coordinate read is one bit operation (Theorem 5.7). Bit operations $\geq |I| \geq \text{srnk}$ (Theorem 5.10). By **TD**, each bit operation costs $\geq k_B T \ln 2$. Summing: $E \geq \text{srnk} \cdot k_B T \ln 2$. ■

The constant $k_B T \ln 2$ is the universal Landauer floor for irreversible bit erasure, experimentally verified to $\pm 10\%$ [9]. It is not, in general, a tight expression for the actual energetic cost of constrained computation [96, 51, 98]. In the Lean model, that distinction is explicit in two layers. First, a coarse floor-plus-overhead interface records that any constrained process may dissipate more than the Landauer floor ([WC1–WC5](#)). Second, a refined Wolpert decomposition splits that extra dissipation into a mismatch term and a residual term. Theorem [WP1](#) states that the declared total overhead is exactly the sum of those two components, and [WP2](#) proves that the effective per-bit lower bound dominates the Landauer floor plus both terms. The mismatch branch is now partially promoted from cited premise to theorem: [WM1](#) shows the KL mismatch term is nonnegative, [WM2](#) identifies the zero case exactly, [WM3](#) gives strict positivity under an explicit finite-distribution mismatch witness, and [WM4](#) carries that strict positivity into the discrete lower-bound units used by the thermodynamic model. Theorems [WM5](#) and [WM6](#) then inject that theorem-level mismatch witness into the Wolpert decomposition, so strict separation above the Landauer floor can now be derived directly for the mismatch branch. A second residual branch is now also theorem-level within the current finite machinery: [WR1](#) proves nonnegativity of the two-point forward/reverse residual divergence, [WR2](#) proves strict positivity under an explicit asymmetric positive edge-flow witness, and [WR3](#) lifts that witness into the discrete lower-bound units of the decomposition. The new theorem [WR6](#) packages

the exhaustive finite-state edge split directly: if the reverse edge is positive, the pairwise KL branch applies; if the reverse edge vanishes, the process performs an irreversible one-way state transition and the existing Landauer-scaled transition-cost theorem applies. Theorems [WR7](#) and [WR8](#) then inject that unified finite residual witness into the Wolpert decomposition, yielding direct strict separation above the Landauer floor for that finite discrete subclass. Theorems [WR9](#) and [WR10](#) package the same branch at the process level: any finite computational-state process with at least one such asymmetric forward edge already carries a theorem-level residual witness and therefore strictly exceeds the Landauer floor. The broader stopping-time / absolute- irreversibility residual regime imported from [\[51\]](#) remains represented by the separate cited premise [WP5](#); and [WP6](#) shows that any one of the derived mismatch branch, the derived finite residual branch, or the cited broader stopping-time branch is already sufficient to force strict separation above the Landauer floor. For the circuit cost interface of [\[98\]](#), [WP7](#) and [WP8](#) prove that a declared structural resource lower-bounded by mismatch further tightens the per-bit and total energy lower bounds. Finally, [WP9](#) proves that the existing physical grounding bundle survives under this refined decomposition. Exact equality is retained only as an optional idealized specialization.

Informally: the cited stochastic-thermodynamics papers are not rederived in full inside this artifact. The finite-distribution mismatch branch is now proved as far as the existing KL machinery reaches, and a finite residual asymmetry branch is also proved from the same machinery applied to forward/reverse edge-flow distributions, then repackaged as a single process-level witness theorem for finite computational-state systems. The stronger stopping-time / absolute-irreversibility residual regime remains a separate cited premise. Lean then verifies the exact downstream consequences of composing that mixed theorem-plus-premise interface with the decision-theoretic lower bounds already proved in the core system. [BA9](#) [WM1](#) [WM2](#) [WM3](#) [WM4](#) [WM5](#) [WM6](#) [WR1](#) [WR2](#) [WR3](#) [WR4](#) [WR5](#) [WR6](#) [WR7](#) [WR8](#) [WR9](#) [WR10](#) [WP5](#) [WP6](#)

(L: [BA9](#)) [C:T2.49;R:AR]

Corollary 5.12 (srnk = 1 is the Energy Ground State). Among all decision problems, those with srnk = 1 have the smallest universal lower bound certified by the structural argument: $E_{\text{decision}} \geq k_B T \ln 2$ per decision. In the exact-calibration idealization this is the corresponding ground-state cost.

Problems with srnk > 1 are above that universal floor by at least $(\text{srnk} - 1) \cdot k_B T \ln 2$ per decision cycle. This gap is structurally mandatory at the level of lower bounds; stronger empirical per-bit costs only enlarge it. [BA8](#)

(L: [BA8](#)) [C:C5.12;R:AR]

5.10 Connection to the Formal Model

The coordinate structure of Definition [2.1](#) is justified by Theorems [5.7](#) and [5.10](#):

- The finite coordinate spaces $X_1, \dots, X_n$ correspond to the discrete state transitions accessible to the physical system (Corollary [5.8](#)).
- The boolean case $X_i = \{0, 1\}$ is the elementary case: each coordinate is one physical state transition (Theorem [5.7](#)).
- $\text{srnk}(\mathcal{D})$ counts the minimum physical transitions required to resolve $\mathcal{D}$ correctly (Theorem [5.10](#)).
- The thermodynamic lower bound (Theorem [5.11](#)) is the physical content of the thermodynamic lift results (ThermodynamicLift.lean, DiscreteSpacetime.lean).

The formal model is substrate-neutral (Section 2). The physical grounding here establishes that the model accurately describes any physical substrate subject to **SR**, **QM**, and **TD**, which is all matter in this universe under current empirical knowledge. (L: [BA9](#), [DS5](#))

5.11 Universe Membership via Shared Invariants

The phrase “this universe” is not rhetorical. It has precise operational content: two physical systems are in the same universe *iff* they agree on a shared set of measurable invariants within measurement precision.

Definition 5.13 (Shared Invariant Set). An *invariant set* $\mathcal{I} = \{c, \hbar, k_B, \dots\}$ is a collection of physical constants that are:

1. Measurable by any sufficiently capable observer.
2. Identical (within precision) for all observers in the same universe.

[IA0](#)

(L: [IA0](#))

Theorem 5.14 (Universe Membership is Defined by Invariant Agreement). *Two observers O_1, O_2 are in the same physical universe iff their measurements of a shared invariant set $\mathcal{I}$ are compatible within measurement precision.*

Contrapositive: *Without a shared measurable invariant, the phrase “same universe” has no operational content.* [IA15](#)

(L: [IA15-16](#))

This theorem is not about humans. It is grounded at the quantum level where no ego exists, then lifted to observers as a corollary:

1. **Quantum level (IA2–IA4):** Two quantum systems under the same Hamiltonian agree on all eigenvalues. There is no “electron perspective.” Agreement is forced by physics.
2. **Atomic level (IA5–IA8):** Two atoms absorbing the same photon agree on $\hbar\omega$. Discrete transitions force exact agreement.
3. **Molecular level (IA9–IA12):** Two molecules in thermal equilibrium at temperature T agree on $k_B T$. The Landauer floor then applies universally to irreversible bit erasure.
4. **Observer level (IA13–IA18):** Observers are made of atoms and molecules. They inherit invariant agreement from their constituents. By the observer level, agreement is already forced.

(L: [IA2](#), [IA5](#), [IA9](#), [IA13](#), [IA17-18](#))

Corollary 5.15 (Observer-Level Rejection Cascade). *Objecting to observer-level invariant agreement requires rejecting atomic-level invariant agreement, which in turn requires rejecting the underlying quantum-level agreement structure.* [IA17](#)

(L: [IA17](#))

Therefore: “all matter in this universe” is a technical term. It means: all matter that agrees with us on the invariant set $\{c, \hbar, k_B, \dots\}$. This is not a rhetorical claim; it is the *definition* of shared universe membership.

6 Stochastic Decision Problems

We extend the static decision problem to include a probability distribution over states.

6.1 Stochastic Decision Problem

Definition 6.1 (Stochastic Decision Problem). A *stochastic decision problem with coordinate structure* is a tuple $\mathcal{D}_S = (A, X_1, \dots, X_n, U, P)$ where:

- A is a finite set of *actions*
- $X_1, \dots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \dots \times X_n$ is the *state space*
- $U : A \times S \rightarrow \mathbb{Q}$ is the *utility function*
- $P \in \Delta(S)$ is a *distribution over states*

Definition 6.2 (Distribution-Conditional Sufficiency). A coordinate set $I \subseteq \{1, \dots, n\}$ is *stochastically sufficient* for $\mathcal{D}_S$ if:

$$P(s \mid s_I) \otimes U \text{ determines the same optimal action as } P(s) \otimes U.$$

Equivalently, for all $s, s' \in S$ with $s_I = s'_I$:

$$\arg \max_{a \in A} \mathbb{E}_{t \sim P(\cdot \mid s_I)}[U(a, t)] = \arg \max_{a \in A} \mathbb{E}_{t \sim P}[U(a, t)].$$

Definition 6.3 (Expected-Value Sufficiency (Alternative)). A coordinate set I is *expected-value sufficient* if:

$$\forall a, a' : \mathbb{E}_{s \sim P}[U(a, s) \mid s_I] \geq \mathbb{E}_{s \sim P}[U(a', s) \mid s_I] \implies a \in \text{Opt}_P.$$

where $\text{Opt}_P = \arg \max_{a \in A} \mathbb{E}_{s \sim P}[U(a, s)]$.

Decision 6.4. **STOCHASTIC-SUFFICIENCY-CHECK**: Given stochastic decision problem $\mathcal{D}_S = (A, S, U, P)$ and coordinate set I , is I stochastically sufficient?

Decision 6.5. **STOCHASTIC-MINIMUM-SUFFICIENT-SET**: Given $\mathcal{D}_S$, find the minimal $I \subseteq \{1, \dots, n\}$ that is stochastically sufficient.

Decision 6.6. **STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK**: Given stochastic decision problem $\mathcal{D}_S$ and coordinate set I , does there exist an anchor fiber whose conditional-optimal action is singleton and fiber-stable under agreement on I ?

(L: [DC16-18](#).)

6.2 Complexity: PP-Completeness

Theorem 6.7 (STOCHASTIC-SUFFICIENCY-CHECK is PP-complete). *STOCHASTIC-SUFFICIENCY-CHECK is PP-complete.* [C:T6.7;R:PP] (L: [DC9](#); membership: [DC14](#); hardness: [DC15](#).)

Claim 6.8 (Stochastic anchor refinement). If I is stochastically sufficient, then I is stochastically anchor-sufficient. [C:P6.8;R:PP] (L: [DC19](#).)

(L: [DC19](#).)

Claim 6.9 (Strict-side anchor reduction). For each formula φ on $n \geq 1$ variables:

$$\text{StrictMAJSAT}(\varphi) \iff \text{STOCHASTIC} - \text{ANCHOR} - \text{SUFFICIENCY} - \text{CHECK}(\text{reduceMAJSAT}(\varphi), \emptyset) \\ \wedge \text{Opt}_{\text{reduceMAJSAT}(\varphi)} = \{\text{accept}\}.$$

[C:P6.9;R:PP] (L: DC24-26.)

(L: DC24-26.)

Claim 6.10 (Direct MAJSAT reduction to stochastic anchor check). For each formula φ on $n \geq 1$ variables,

$$\text{MAJSAT}(\varphi) \iff \text{STOCHASTIC-ANCHOR-SUFFICIENCY-CHECK}(\text{reduceMAJSATPureAnchor}(\varphi), \emptyset).$$

[C:P6.10;R:PP] (L: DC40-42.)

(L: DC40-42.)

Claim 6.11 (Stochastic minimum hardness package). For each arity $n \geq 1$, there is a polynomially size-bounded reduction from MAJSAT to the $k = 0$ slice of the stochastic minimum-sufficiency query. [C:P6.11;R:PP] (L: DC45, DC47.)

(L: DC45, DC47.)

Claim 6.12 (Exact finite deciders for stochastic queries). The development internalizes exact boolean deciders for stochastic sufficiency, stochastic anchor sufficiency, and stochastic minimum sufficiency.

[C:P6.12;R:finite exact deciders] (L: DC51-53.)

(L: DC51-53.)

Claim 6.13 (Explicit counted search bounds for stochastic queries). The development contains counted exhaustive-search procedures for stochastic sufficiency, stochastic anchor sufficiency, and stochastic minimum sufficiency, with certified bounds $O(|S|)$, $O(|S||A|)$, and $O(2^n)$ respectively.

[C:P6.13;R:finite explicit search] (L: DC58-59, DC62-65.)

(L: DC58-59, DC62-65.)

Observer-collapse closure for the stochastic anchor query is also mechanized: singleton effective-state structure yields global observational equivalence and, with seeded support, yields stochastic sufficiency and anchor-check truth under the declared hypotheses. (L: PA1-5, PA8-9.)

Relevance boundary. For each fixed candidate information set I , the stochastic query induces a derived decision problem whose optimizer is exactly the conditional fiber optimizer. That derived problem is itself sufficient on I , so the natural relevance notion is indexed by I rather than attached to a single global stochastic decision map. (L: DC57.)

1. **Membership in PP:** Given $\mathcal{D}_S$ and I , we must check whether knowing s_I determines the optimal action. This reduces to comparing expected utilities conditioned on s_I versus unconditioned. For each action pair (a, a') , we compare:

$$\mathbb{E}_{s \sim P}[U(a, s) \mid s_I] \geq \mathbb{E}_{s \sim P}[U(a', s) \mid s_I].$$

This requires counting weighted by P , which is in PP.

2. **PP-hardness via MAJSAT reduction:** Given Boolean formula φ on n variables, construct:

- $S = \{0, 1\}^n$ (state space = assignments)
- P = uniform distribution over S

- $A = \{\text{accept}, \text{reject}\}$
- $U(\text{accept}, s) = \varphi(s)$, $U(\text{reject}, s) = 1/2$
- $I = \emptyset$

Then $\emptyset$ is stochastically sufficient iff the optimal action is determined without seeing any coordinates:

$$\emptyset \text{ sufficient} \iff \mathbb{E}_{s \sim P}[\varphi(s)] \geq 1/2 \text{ or } < 1/2.$$

This is exactly MAJSAT, which is PP-complete.

(L: [CC55](#), [CC58](#))

Proof. For membership: We reduce to a PPsredicate. For each $a \in A$, compute $\mathbb{E}_P[U(a, s)]$ and compare. The comparison involves summing over $s \in S$ weighted by $P(s)$, which is polynomial in $|S|$ and the representation of P . The decision is whether the argmax is unique or whether knowing s_I changes it; this is a majority-vote comparison, hence in PP.

For hardness: The reduction from MAJSAT constructs the uniform distribution. If φ is true on $\geq 1/2$ of assignments, the expected utility of accepting is $\geq 1/2$, making accept optimal regardless of state observation. If $< 1/2$, reject is optimal. Thus $\emptyset$ is sufficient exactly when MAJSAT is true. ■

Example 6.14 (Stochastic Weather). Consider stochastic umbrella decision:

- $A = \{\text{carry}, \text{don't carry}\}$
- $S = \{\text{rain}, \text{no rain}\}$
- $P(\text{rain}) = 0.3$, $P(\text{no rain}) = 0.7$
- $U(\text{carry}, \text{rain}) = 2$, $U(\text{carry}, \text{no rain}) = -1$
- $U(\text{don't carry}, \text{rain}) = -2$, $U(\text{don't carry}, \text{no rain}) = 0$

Expected utility: $E_P[U(\text{carry})] = 0.3(2) + 0.7(-1) = -0.1$, $E_P[U(\text{don't carry})] = 0.3(-2) + 0.7(0) = -0.6$. So carry is optimal. If we condition on forecast information, we check whether the conditional distribution changes the argmax.

6.3 Potential and Physical Bounds

Definition 6.15 (Potential Grounding). Let $\Phi : A \times S \rightarrow \mathbb{R}$ with grounding equation

$$U(a, s) = -\Phi(a, s).$$

Define expected action-potential

$$\mathbb{E}[\Phi \mid a] := \sum_{s \in S} P(s) \Phi(a, s).$$

(L: [DC30](#), [DC32](#).)

Claim 6.16 (Expected-Utility/Potential Duality). Under the grounding equation, for every action a :

$$\mathbb{E}_{s \sim P}[U(a, s)] = -\mathbb{E}[\Phi \mid a],$$

and for all a, a' :

$$\mathbb{E}[U \mid a] \leq \mathbb{E}[U \mid a'] \iff \mathbb{E}[\Phi \mid a'] \leq \mathbb{E}[\Phi \mid a].$$

[C:P6.16;R:PP] (L: DC31, DC33-34.)

(L: DC31, DC33-34.)

Claim 6.17 (Landauer Floor and State-Cardinality Specialization). For $T \geq 0$, the Landauer floor is nonnegative and monotone in erased bits:

$$E_L(b, T) \geq 0, \quad b_1 \leq b_2 \implies E_L(b_1, T) \leq E_L(b_2, T).$$

At the room-temperature specialization used here:

$$\text{thermodynamicCost}(\mathcal{D}_S) = \text{landauerConstant} \cdot |S|.$$

[C:P6.17;R:AR, H=cost-growth] (L: DC35-37.)

(L: DC35-37.)

6.4 Tractable Subcases

1. **Product distributions:** $P = P_1 \otimes \dots \otimes P_n$. Then

$$\mathbb{E}_{s \sim P}[U(a, s) \mid s_I] = \prod_{i \in I} P_i(s_i) \cdot \mathbb{E}_{s \sim P}[U(a, s) \mid s_I],$$

so expected utility decomposes. This reduces stochastic sufficiency to static sufficiency per coordinate.

2. **Bounded support:** $|\text{supp}(P)| \leq k$. Enumeration is $O(k \cdot |A|)$, polynomial.

3. **Log-concave distributions:** For ordered state spaces, conditional expectations are monotone and computable via convex optimization.

Claim 6.18 (Tractability under product distributions). If $P = P_1 \otimes \dots \otimes P_n$ is a product distribution, then STOCHASTIC-SUFFICIENCY-CHECK reduces to checking static sufficiency on each marginal. [C:P6.18;R:PD] (L: CC60.)

(L: CC60)

Claim 6.19 (Tractability under bounded support). If $|\text{supp}(P)| \leq k$, then STOCHASTIC-SUFFICIENCY-CHECK is solvable in $O(k \cdot |A| \cdot n)$ time. [C:P6.19;R:BS] (L: CC59.)

(L: CC59)

6.5 Bridge from Static

Paper 4 proved that static sufficiency does NOT imply stochastic sufficiency (CC54). A counterexample exists where I is sufficient for (A, S, U) but not for (A, S, U, P) .

However, under product distributions, static sufficiency transfers:

Claim 6.20 (Static to stochastic transfer). If $P = P_1 \otimes \dots \otimes P_n$ is a product distribution and I is statically sufficient for (A, S, U) , then I is stochastically sufficient for (A, S, U, P) . [C:P6.20;R:PD] (L: CC52.)

(L: CC52)

7 Sequential Decision Problems

We further extend to sequential decision-making with transitions and observations.

7.1 Sequential Decision Problem (POMDP)

Definition 7.1 (Sequential Decision Problem). A *sequential decision problem with coordinate structure* is a tuple $\mathcal{D}_{SEQ} = (A, X_1, \dots, X_n, U, T, O)$ where:

- A is a finite set of *actions*
- $X_1, \dots, X_n$ are finite *coordinate spaces*
- $S = X_1 \times \dots \times X_n$ is the *state space*
- $U : A \times S \rightarrow \mathbb{Q}$ is the *stage utility*
- $T : A \times S \rightarrow \Delta(S)$ is the *transition kernel*
- $O : S \rightarrow \Delta(\Omega)$ is the *observation model*

The controller chooses a control law $\pi : (\Omega^*) \rightarrow A$ mapping observation histories to actions. Objective: maximize expected cumulative utility $\mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t U(a_t, s_t) \right]$ with discount factor $\gamma \in [0, 1)$.

Notation 7.2 (Observation History). Let $\omega_t = (o_0, o_1, \dots, o_t)$ denote the observation history up to time t . The controller law is $\pi(\omega_{t-1}) = a_t$.

Definition 7.3 (Sequential Sufficiency). A coordinate set $I \subseteq \{1, \dots, n\}$ is *sequentially sufficient* for $\mathcal{D}_{SEQ}$ if the optimal policy based on full observation history is the same as the optimal policy based only on coordinates in I at each time step.

Formally: let π^* be the optimal policy with full observations, and let π_I^* be the optimal policy with observations restricted to I -coordinates. Then $\pi^* = \pi_I^*$ (as functions of observation history).

Decision 7.4. **SEQUENTIAL-SUFFICIENCY-CHECK**: Given sequential decision problem $\mathcal{D}_{SEQ}$ and coordinate set I , is I sequentially sufficient?

Decision 7.5. **SEQUENTIAL-MINIMUM-SUFFICIENT-SET**: Given $\mathcal{D}_{SEQ}$, find the minimal I that is sequentially sufficient.

Decision 7.6. **SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK**: Given sequential decision problem $\mathcal{D}_{SEQ}$ and coordinate set I , does there exist an anchor state whose I -agreement class preserves the optimal action set?

(L: [DC20-22](#).)

7.2 Complexity: PSPACE-Completeness

Theorem 7.7 (SEQUENTIAL-SUFFICIENCY-CHECK is PSPACE-complete). *SEQUENTIAL-SUFFICIENCY-CHECK is PSPACE-complete.* [C:T7.7;R:PSPACE]

Claim 7.8 (Sequential anchor refinement). If I is sequentially sufficient, then I is sequentially anchor-sufficient. [C:P7.8;R:PSPACE] (L: [DC23](#).)

(L: [DC23](#).)

Claim 7.9 (TQBF Reduction to Sequential Anchor Check). For each quantified Boolean formula q :

$$\text{TQBF}(q) \iff \text{SEQUENTIAL-ANCHOR-SUFFICIENCY-CHECK}(\text{reduceTQBF}(q), \emptyset).$$

[C:P7.9;R:PSPACE] (L: [DC27-29](#), [DC44](#).)

(L: [DC27-29](#), [DC44](#).)

Claim 7.10 (Sequential anchor hardness package). For each arity n , there is a polynomially size-bounded reduction from TQBF to the sequential anchor query. [C:[P7.10](#);R:PSPACE] (L: [DC43-44](#).)

(L: [DC43-44](#).)

Claim 7.11 (Sequential minimum hardness package). For each arity n , there is a polynomially size-bounded reduction from TQBF to the $k = 0$ slice of the sequential minimum-sufficiency query.

[C:[P7.11](#);R:PSPACE] (L: [DC46](#), [DC48](#).)

(L: [DC46](#), [DC48](#).)

Claim 7.12 (Sequential minimality equals relevance). For any minimal sequentially sufficient coordinate set I , a coordinate lies in I if and only if it is relevant for the underlying sequential decision map, and any two minimal sequentially sufficient sets coincide. [C:[P7.12](#);R:structural] (L: [DC49-50](#).)

(L: [DC49-50](#).)

Claim 7.13 (Exact finite deciders for sequential queries). The development internalizes exact boolean deciders for sequential sufficiency, sequential anchor sufficiency, and sequential minimum sufficiency.

[C:[P7.13](#);R:finite exact deciders] (L: [DC54-56](#).)

(L: [DC54-56](#).)

Claim 7.14 (Explicit counted search bounds for sequential queries). The development contains counted exhaustive-search procedures for sequential sufficiency, sequential anchor sufficiency, and sequential minimum sufficiency, with certified bounds $O(|S|^2)$, $O(|S|)$, and $O(2^n)$ respectively.

[C:[P7.14](#);R:finite explicit search] (L: [DC60-61](#), [DC66-69](#).)

(L: [DC60-61](#), [DC66-69](#).)

Observer-collapse closure for the sequential anchor query is mechanized: under the declared collapse hypothesis, sequential sufficiency and sequential anchor-check truth follow directly. (L: [PA6-7](#).)

1. **Membership in PSPACE:** Optimal POMDP policy computation is in PSPACE(Papadimitriou & Tsitsiklis, 1987). Checking whether a coordinate set I yields the same policy involves comparing value functions, which can be done in polynomial space via alternating polynomial time.

2. **PSPACE-hardness via TQBF reduction:** Given quantified Boolean formula $\forall x_1 \exists x_2 \forall x_3 \dots \varphi(x_1, \dots, x_n)$, construct:

- n time steps, one variable per step
- At odd steps (universal quantifier): exogenous dynamics choose the variable (adversarial transition)
- At even steps (existential quantifier): controller action chooses the variable
- Terminal utility: $\varphi(x_1, \dots, x_n)$
- Observation: controller sees all previous variables
- I = full observation history

Then full observation is sequentially sufficient iff the controller can guarantee $\varphi = \text{true}$ regardless of exogenous choices, which holds iff the QBF is true.

(L: [CC47-48](#))

Construction 7.15 (TQBF Reduction). Given QBF $\Phi = Q_1x_1Q_2x_2\ldots Q_nx_n \varphi(x_1, \dots, x_n)$ with alternating quantifiers, construct sequential problem:

- Time steps $t = 1, \dots, n$
- State at step t encodes $(x_1, \dots, x_t)$
- At step t : - If $Q_t = \forall$: transition is adversarial (exogenous dynamics choose both next state and observation) - If $Q_t = \exists$: transition is controlled by action (choose x_t)
- Terminal reward: 1 if $\varphi(x_1, \dots, x_n) = \text{true}$, 0 otherwise
- Full observation reveals all previous variables

Then: full observation is sequentially sufficient $\iff$ there exists a policy guaranteeing $\varphi = \text{true}$
 $\iff \Phi$ is true.

(L: [CC77](#))

Claim 7.16 (Sequential sufficiency implies static sufficiency). When the horizon $T = 1$ and transitions are deterministic, sequential sufficiency reduces to static sufficiency. [C:P7.16;R:T=1, DET] (L: [CC46](#).)

(L: [CC46](#))

7.3 Tractable Subcases

1. **Fully observable (MDP)**: O is the identity (controller sees full state). Then the problem reduces to a stochastic (or deterministic) problem per time step. Value iteration is polynomial in $|S|, |A|, T$.
2. **Bounded horizon**: $T \leq k$. The state space grows as $O(|S|^T)$, but for fixed T this is polynomial in $|S|$.
3. **Tree-structured transitions**: The transition graph has no cycles. Dynamic programming yields polynomial algorithms.
4. **Deterministic transitions**: $T(s'|a, s)$ is a point mass. The problem reduces to search over deterministic paths.

Claim 7.17 (Tractability under full observability). If O is the identity (fully observable), then SEQUENTIAL-SUFFICIENCY-CHECK reduces to checking stochastic sufficiency at each time step. [C:P7.17;R:FO] (L: [CC50](#).)

(L: [CC50](#))

Claim 7.18 (Tractability under bounded horizon). If $T \leq k$ (constant horizon), then SEQUENTIAL-SUFFICIENCY-CHECK is solvable in time polynomial in $|S|, |A|, k$. [C:P7.18;R:BH] (L: [CC49](#).)

(L: [CC49](#))

7.4 Bridge from Static and Stochastic

Paper 4 proved that static sufficiency does NOT transfer to sequential (horizon > 1): [CC18](#)

Similarly, stochastic sufficiency does NOT transfer to sequential in general:

Claim 7.19 (Stochastic to sequential bridge failure). There exists a stochastic sufficient coordinate set I and a sequential problem where I is NOT sequentially sufficient, even when transitions are memoryless. [C:P7.19;R:CE] (L: [CC56](#).)

(L: CC56)

The transfer conditions are:

- Static $\rightarrow$ Sequential: transfers iff $T = 1$ and transitions are deterministic (one-step bridge from Paper 4)
- Stochastic $\rightarrow$ Sequential: transfers iff transitions are memoryless AND observation model is regime-compatible

8 Regime Hierarchy

We establish the strict inclusion hierarchy among decision regimes.

Inclusion	Complexity Classes	Condition
Static $\subset$ Stochastic	coNP $\subset$ PP	Standard assumptions ($P \neq \text{coNP}$)
Stochastic $\subset$ Sequential	PP $\subset$ PSPACE	Standard assumptions ($P \neq \text{PP}$)

Claim 8.1 (Strict inclusion static $\subset$ stochastic). Under standard complexity assumptions ($P \neq \text{coNP}$), there exist stochastic decision problems that are not statically reducible. [C:P8.1;R:P $\neq$ coNP] (L: DC1.)

(L: DC1.)

Claim 8.2 (Strict inclusion stochastic $\subset$ sequential). Under standard complexity assumptions ($P \neq \text{PP}$), there exist sequential decision problems that are not stochastically reducible. [C:P8.2;R:P $\neq$ PP] (L: DC2.)

(L: DC2.)

(L: DC2.)

8.1 Integrity-Resource Bound per Regime

The integrity-resource bound from Paper 4 generalizes to each regime:

1. **Static** (coNP-hard): polynomial-time physical controller must abstain on some instances
2. **Stochastic** (PP-hard): polynomial-time physical controller must abstain on more instances
3. **Sequential** (PSPACE-hard): polynomial-time physical controller must abstain on even more instances

Formally, if $C_1 \subsetneq C_2$ (complexity classes), then the set of instances where integrity forces abstention under C_1 -resources is a strict superset of those under C_2 -resources:

$$\{\sigma : \text{integrity forces abstention at } C_1\} \supsetneq \{\sigma : \text{integrity forces abstention at } C_2\}$$

Claim 8.3 (Abstention frontier shifts). As regime complexity increases (static $\rightarrow$ stochastic $\rightarrow$ sequential), the abstention frontier expands. [C:P8.3;R:RG] (L: CC39.)

(L: CC39)

Claim 8.4 (Explicit-state abstract-P wrappers). In the artifact's step-counting model, explicit-state wrappers place static, stochastic, and sequential sufficiency/anchor queries into abstract P-style classes once the state-space budgets are carried in the input; the corresponding minimum queries admit the same style of wrapper once an explicit subset-budget is included as part of the input.

[C:P8.4;R:explicit-state step-counting P] (L: DC70-79.)

(L: DC70-79.)

8.2 Regime Detection

A meta-problem: given a problem instance, which regime is it in?

Decision 8.5. **REGIME-DETECTION**: Given a problem instance, is it static, stochastic, or sequential?

Conjecture: regime detection is polynomial for the represented family:

- Bounded actions $\rightarrow$ static
- Distribution present $\rightarrow$ stochastic
- Transitions/observations present $\rightarrow$ sequential

(*L*: [CC41](#))

9 Encoding-Regime Separation

Formally: this section proves regime-indexed access separation for a fixed sufficiency predicate, then lifts bit-operation bounds to thermodynamic/accounting consequences.

The hardness results of Section 4 apply to worst-case instances under the succinct encoding. This section states an encoding-regime separation: an explicit-state upper bound versus a succinct-encoding worst-case lower bound, and an intermediate query-access obstruction family. The complexity class is a property of the encoding measured against a fixed decision question (Remark 2.25), not a property of the question itself: the same sufficiency question admits polynomial-time algorithms under one encoding and worst-case exponential cost under another. Within the paper’s full regime map, this section is the static access-layer decomposition; theorem-level stochastic and sequential core placements are given in Sections 6 and 7, with strict inter-regime separation in Section 8.

Physical reading: the separation is an access-and-budget separation for the same decision semantics. Explicit access assumes full boundary observability; succinct access assumes compressed structure that may require exponential expansion to certify universally; query access assumes bounded measurement interfaces, and the obstruction persists across value-entry/state-batch interfaces with oracle-lattice transfer/strictness closures. [*C*:[T9.1](#);[P9.3](#), [9.7](#), [9.11](#), [9.15](#), [9.16](#), [2.46](#);*R*:*E*, *S+ETH*, *Qf*, *Qb*, *AR*] (*L*: [CC16](#), [CC50](#), [L8-9](#), [L20-21](#), [CT6](#).)

Model note. Theorem 9.1 has Part 1 in [E] (time polynomial in $|S|$) and Part 2 in [S+ETH] (time exponential in n). Proposition 9.3 is [Q_fin] (finite-state Opt-oracle core), while Propositions 9.7 and 9.11 are [Q_bool] interface refinements (value-entry and state-batch). Relative to Definition 2.33, Propositions 9.8 and 9.15 are explicit simulation-transfer instances. The [E] vs [S+ETH] split in Theorem 9.1 is a same-question encoding/cost-model separation, not a bidirectional simulation claim between those regimes. These regimes are defined in Section 2.5 and are not directly comparable as functions of one numeric input-length parameter.

Theorem 9.1 (Explicit–Succinct Regime Separation). *Let $\mathcal{D} = (A, X_1, \dots, X_n, U)$ be a decision problem with $|S| = N$ states. Let k^* be the size of the minimal sufficient set.*

1. [**E**] **Explicit-state upper bound:** *Under the explicit-state encoding, SUFFICIENCY-CHECK is solvable in time polynomial in N (e.g. $O(N^2|A|)$).*

2. **[S+ETH] Succinct lower bound (worst case):** Assuming ETH, there exists a family of succinctly encoded instances with n coordinates and minimal sufficient set size $k^* = n$ such that SUFFICIENCY-CHECK requires time $2^{\Omega(n)}$.

(L: [CC16](#), [CC23-24](#).)

Proof. Part 1 (Explicit-state upper bound): Under the explicit-state encoding, SUFFICIENCY-CHECK is decidable in time polynomial in N by direct enumeration: compute $\text{Opt}(s)$ for all $s \in S$ and then check all pairs (s, s') with $s_I = s'_I$.

Equivalently, for any fixed I , the projection map $s \mapsto s_I$ has image size $|S_I| \leq |S| = N$, so any algorithm that iterates over projection classes (or over all state pairs) runs in polynomial time in N . Thus, in particular, when $k^* = O(\log N)$, SUFFICIENCY-CHECK is solvable in polynomial time under the explicit-state encoding.

Remark (bounded coordinate domains). In the general model $S = \prod_i X_i$, for a fixed I one always has $|S_I| \leq \prod_{i \in I} |X_i|$ (and $|S_I| \leq N$). If the coordinate domains are uniformly bounded, $|X_i| \leq d$ for all i , then $|S_I| \leq d^{|I|}$.

Part 2 (Succinct ETH lower bound, worst case): A strengthened version of the TAUTOLOGY reduction used in Theorem 4.6 produces a family of instances in which the minimal sufficient set has size $k^* = n$: given a Boolean formula φ over n variables, we construct a decision problem with n coordinates such that if φ is not a tautology then *every* coordinate is decision-relevant (thus $k^* = n$). This strengthening is mechanized in Lean (see Appendix A). Under the Exponential Time Hypothesis (ETH) [36, 5], TAUTOLOGY requires time $2^{\Omega(n)}$ in the succinct encoding, so SUFFICIENCY-CHECK inherits a $2^{\Omega(n)}$ worst-case lower bound via this reduction. ■

Corollary 9.2 (Regime Separation (by Encoding)). *There is a clean separation between explicit-state tractability and succinct worst-case hardness (with respect to the encodings in Section 2.5):*

- Under the explicit-state encoding, SUFFICIENCY-CHECK is polynomial in $N = |S|$.
- Under ETH, there exist succinctly encoded instances with $k^* = \Omega(n)$ (indeed $k^* = n$) for which SUFFICIENCY-CHECK requires $2^{\Omega(n)}$ time.

For Boolean coordinate spaces ($N = 2^n$), the explicit-state bound is polynomial in 2^n (exponential in n), while under ETH the succinct lower bound yields $2^{\Omega(n)}$ time for the hard family in which $k^* = \Omega(n)$.

Proposition 9.3 (Finite-State Query Lower-Bound Core via Empty-Set Subproblem). *In the finite-state query regime $[Q_fin]$, let S be any finite state type with $|S| \geq 2$ and let $Q \subset S$ with $|Q| < |S|$. Then there exist two decision problems $\mathcal{D}_{\text{yes}}, \mathcal{D}_{\text{no}}$ over the same state space such that:*

- *their oracle views on all states in Q are identical,*
- $\emptyset$ *is sufficient for $\mathcal{D}_{\text{yes}}$,*
- $\emptyset$ *is not sufficient for $\mathcal{D}_{\text{no}}$.*

Consequently, no deterministic query procedure using fewer than $|S|$ state queries can solve SUFFICIENCY-CHECK on all such instances; the worst-case query complexity is $\Omega(|S|)$. (L: [CC50](#), [CC50](#).)

Proof. This is exactly the finite-state indistinguishability theorem [CC4](#). For any $|Q| < |S|$, there is an unqueried hidden state s_0 producing oracle-indistinguishable yes/no instances with opposite truth values on the $I = \emptyset$ subproblem. Since SUFFICIENCY-CHECK contains that subproblem, fewer than $|S|$ queries cannot be correct on both instances, yielding the $\Omega(|S|)$ worst-case lower bound. ■

Proposition 9.4 (Checking–Witnessing Duality on the Empty-Set Core). *[C:P9.4;R:H=query-lb] Let $n \geq 1$ and let P be a finite family of Boolean-state pairs. Assume P is sound for refuting empty-set sufficiency in the sense that for every $\text{Opt} : \{0,1\}^n \rightarrow \{0,1\}$, whenever $\emptyset$ is not sufficient, some pair in P witnesses disagreement of Opt . Then*

$$|P| \geq 2^{n-1}.$$

Equivalently, any sound checker must inspect at least the witness budget $W(n) = 2^{n-1}$, so checking budget $T(n)$ satisfies $T(n) \geq W(n)$. (L: [WD3-5](#).)

Corollary 9.5 (Information Barrier in Query Regimes). *[C:C9.5;R:H=query-lb] For the fixed sufficiency question, finite query interfaces induce an information barrier:*

Q_fin (Opt-oracle): *with $|Q| < |S|$, there exist yes/no instances indistinguishable on all queried states but with opposite truth values on the $I = \emptyset$ subproblem.*

Q_bool value-entry and state-batch interfaces preserve the same obstruction at cardinality $< 2^n$.

Hence the barrier is representational-access based: without querying enough of the hidden state support, exact certification is blocked by indistinguishability rather than by search-strategy choice.

(L: [CC27-29](#).)

Corollary 9.6 (Boolean-Coordinate Instantiation). *[C:C9.6;R:H=query-lb] In the Boolean-coordinate state space $S = \{0,1\}^n$, Proposition 9.3 yields the familiar $\Omega(2^n)$ worst-case query lower bound for Opt-oracle access. (L: [CC49-50](#).)*

Proposition 9.7 (Value-Entry Query Lower Bound). *In the mechanized Boolean value-entry query submodel $[Q_bool]$, for any deterministic procedure using fewer than 2^n value-entry queries $(a, s) \mapsto U(a, s)$, there exist two queried-value-indistinguishable instances with opposite truth values for SUFFICIENCY-CHECK on the $I = \emptyset$ subproblem. Therefore worst-case value-entry query complexity is also $\Omega(2^n)$. (L: [CC29](#), [CC50](#).)*

Proof. The theorem [CC50](#) constructs, for any query set of cardinality $< 2^n$, an unqueried hidden state s_0 and a yes/no instance pair that agree on all queried values but disagree on $\emptyset$ -sufficiency. The auxiliary bound [CC50](#) ensures that fewer than 2^n value-entry queries cannot cover all states, so the indistinguishability argument applies in the worst case. ■

Proposition 9.8 (Subproblem-to-Full Transfer Rule). *If every full-problem solver induces a solver for a fixed subproblem, then any lower bound for that subproblem lifts to the full problem. (L: [CC52](#), [CC58-59](#).)*

This is the restriction-map instance of Definition 2.33: a solver for the full regime induces one for the restricted subproblem regime, so lower bounds transfer.

Proposition 9.9 (Randomized Robustness (Seedwise)). *[C:P9.9;R:H=query-lb] In $[Q_bool]$, for any query set with cardinality $< 2^n$, the indistinguishable yes/no pair from Proposition 9.3 forces one decoding error per random seed for any seed-indexed decoder from oracle transcripts. Consequently, finite-support randomization does not remove the obstruction: averaging preserves a constant error floor on the hard pair. (L: [CC50](#), [CC50](#).)*

Proposition 9.10 (Randomized Robustness (Weighted Form)). *[C:P9.10;R:H=query-lb] For any finite-support seed weighting μ , the same hard pair satisfies a weighted identity: the weighted sum of yes-error and no-error equals total seed weight. Hence randomization cannot collapse both errors simultaneously. (L: CC50, CC50.)*

Proposition 9.11 (State-Batch Query Lower Bound). *In $[Q_bool]$, the same $\Omega(2^n)$ lower bound holds for a state-batch oracle that returns the full Boolean-action utility tuple at each queried state. (L: CC28, CC50.)*

Proposition 9.12 (Finite-State Empty-Subproblem Generalization). *[C:P9.12;R:H=query-lb] The empty-subproblem indistinguishability lower-bound core extends from Boolean-vector state spaces to any finite state type with at least two states. (L: CC50, CC50.)*

Proposition 9.13 (Adversary-Family Tightness by Full Scan). *[C:P9.13;R:H=query-lb] For the const/spike adversary family used in the query lower bounds, querying all states distinguishes the pair; thus the lower-bound family is tight up to constant factors under full-state scan. (L: CC50.)*

Proposition 9.14 (Weighted Query-Cost Transfer). *[C:P9.14;R:H=query-lb] Let $w(q)$ be per-query cost and $w_{\min}$ a lower bound on all queried costs. Any cardinality lower bound $|Q| \geq L$ lifts to weighted cost:*

$$\sum_{q \in Q} w(q) \geq w_{\min} \cdot L.$$

(L: CC50, CC50.)

Proposition 9.15 (Oracle-Lattice Transfer: Batch $\Rightarrow$ Entry). *In $[Q_bool]$, agreement on state-batch oracle views over touched states implies agreement on all value-entry views for the corresponding entry-query set. (L: CC43, CC52.)*

This is the oracle-transducer instance of Definition 2.33: batch transcripts induce entry transcripts while preserving indistinguishability for the target sufficiency question.

Proposition 9.16 (Oracle-Lattice Strictness: Opt vs Value Entries). *‘Opt’-oracle views are strictly coarser than value-entry views: there exist instances with identical ‘Opt’ views but distinguishable value entries. (L: CC27, CC29.)*

Remark 9.17 (Instantiation of Definitions 3.1 and 3.2). Theorem 9.1 and Propositions 9.3–9.7 keep the structural problem fixed (same sufficiency relation) and separate representational hardness by access regime: explicit-state access exposes the boundary $s \mapsto \text{Opt}(s)$, finite-state query access already yields $\Omega(|S|)$ lower bounds at the Opt-oracle level (Boolean instantiation: $\Omega(2^n)$), value-entry/state-batch interfaces preserve the obstruction in $[Q_bool]$, and succinct access can hide structure enough to force ETH-level worst-case cost on a hard family.

This regime-typed landscape identifies tractability under $[E]$, a finite-state query lower-bound core under $[Q_fin]$ with Boolean interface refinements under $[Q_bool]$, and worst-case intractability under $[S+ETH]$ for the same underlying decision question.

The structural reason for this separation is enumerable access. Under the explicit-state encoding, the mapping $s \mapsto \text{Opt}(s)$ is directly inspectable and sufficiency verification reduces to scanning state pairs, at cost polynomial in $|S|$. Under succinct encoding, the circuit compresses an exponential state space into polynomial size; universal verification over that compressed domain, namely “for all s, s' with $s_I = s'_I$, does $\text{Opt}(s) = \text{Opt}(s')$?”, carries worst-case cost exponential in n , because the domain the circuit compressed away cannot be reconstructed without undoing the compression.

Informally: if you can see enough of the decision boundary, checking can be fast; with limited access, it becomes expensive.

9.1 Budgeted Physical Crossover

Formally: this subsection states budget-indexed representational feasibility splits and links them to integrity-preserving policy closure.

The [E] vs [S] split can be typed against an explicit physical budget. Let $E(n)$ and $S(n)$ denote explicit and succinct representation sizes for the same decision question at scale parameter n , and let B be a hard representation budget. These crossover claims are interpreted through the physical-to-core encoding discipline of Section 2.8: physical assumptions are transferred explicitly into core assumptions, and encoded physical counterexamples map to core failures on the encoded slice. [C:P2.46;C2.47;R:AR] (L: CT4, CT6, CT8.)

Proposition 9.18 (Budgeted Crossover Witness). [C:P9.18;R:H=exp-lb-conditional] *If $E(n) > B$ and $S(n) \leq B$ for some n , then explicit representation is infeasible at (B, n) while succinct representation remains feasible at (B, n) .* (L: CC45, PBC1.)

Proof. This is exactly the definitional split encoded in PBC1: explicit infeasibility is $B < E(n)$ and succinct feasibility is $S(n) \leq B$. The theorem CC2 returns both conjuncts. ■

Proposition 9.19 (Above-Cap Crossover Existence). [C:P9.19;R:H=exp-lb-conditional] *Assume there is a global succinct-size cap C and explicit size is unbounded:*

$$\forall n, S(n) \leq C, \quad \forall B, \exists n, B < E(n).$$

Then for every budget $B \geq C$, there exists a crossover witness at budget B . (L: CC44, PBC2.)

Proof. Fix $B \geq C$. By unboundedness of E , choose n with $B < E(n)$. By the cap assumption, $S(n) \leq C \leq B$, so (B, n) is a crossover witness. The mechanized closure theorem is CC1. ■

Proposition 9.20 (Crossover Does Not Imply Exact-Certification Competence). [C:P9.20;R:H=exp-lb-conditional] *Fix any collapse schema “exact certification competence $\Rightarrow$ complexity collapse” for the target exact-certification predicate. Under the corresponding non-collapse assumption, exact-certification competence is impossible even when a budgeted crossover witness exists.* (L: CC46.)

Proof. The mechanized theorem CC3 bundles the crossover split with the same logical closure used in Proposition 3.61: representational feasibility and certification hardness are distinct predicates. ■

The same closure form is isolated as a standalone physical-collapse chain: requirement-indexed collapse schemas (PH11) induce no physical collapse at that requirement (PH12); canonical specialization yields $P=NP$ physical impossibility under the declared collapse map (PH13, PH14, PH15). The collapse map itself is derived in an explicit SAT bridge (polytime SAT witness + reduction bridge + runtime-to-energy transfer) (PH16, PH17, PH18, PH19, PH20, PH21, PH22), and the assumption boundary is explicit: under bounded budget, positive per-bit cost, and exponential required-op lower bound, claimed collapse forces at least one assumption failure (PH26, PH27); finite-budget necessity is witnessed by explicit unbounded-budget collapse countermodels (PH31, PH32, PH33).

Proposition 9.21 (Crossover Policy Closure Under Integrity). [C:P9.21;R:H=exp-lb-conditional] *In the certifying-solver model (Definitions 3.8–3.9), assume:*

- a budgeted crossover witness at (B, n) ,
- the same collapse implication as Proposition 3.61,

- a solver that is integrity-preserving for the declared relation.

Then either full in-scope non-abstaining coverage fails, or declared budget compliance fails. Equivalently, in uncertified hardness-blocked regions, integrity is compatible with ABSTAIN/UNKNOWN but not with unconditional exact-competence claims. (L: [CC47](#).)

Corollary 9.22 (Finite Budget Implies Eventual Exact-Certification Failure). [C:C9.22;R:H=exp-lb-conditional] Fix a physical computer model with finite total budget and strictly positive per-bit physical cost. If required operation count for exact certification has an exponential lower bound, then there exists a threshold n_0 such that for all $n \geq n_0$, required physical work exceeds budget. For the canonical requirement model used in this paper, the same eventual-budget-failure conclusion holds. (L: [PBC7-8](#).)

Proposition 9.23 (Least Divergence Point at Fixed Budget). [C:P9.23;R:H=exp-lb-conditional] Fix a budgeted encoding model and budget B . If a crossover witness exists at budget B , then there is a least index n_{crit} such that crossover holds at n_{crit} , and no smaller index has crossover. (L: [PBC5](#).)

Proposition 9.24 (Eventual Explicit Infeasibility from Monotone Growth). [C:P9.24;R:H=exp-lb-conditional] If explicit size is monotone in n and already exceeds budget B at some n_0 , then explicit infeasibility holds for all $n \geq n_0$. (L: [PBC7](#).)

Proposition 9.25 (Payoff Threshold as Eventual Split). [C:P9.25;R:H=exp-lb-conditional] If, beyond n_0 , explicit encoding is infeasible while succinct encoding remains feasible, then every $n \geq n_0$ is a crossover point. (L: [PBC8-9](#).)

Corollary 9.26 (No Universal Survivor Without Succinct Bound). [C:C9.26;R:H=exp-lb-conditional] If both explicit and succinct encodings are unbounded, then for every fixed budget B , there exists an index where explicit is infeasible and an index where succinct is infeasible. (L: [PBC10](#).)

Proposition 9.27 (Policy Closure at and Beyond Divergence). [C:P9.27;R:H=exp-lb-conditional] Under the same integrity-resource collapse assumption as Proposition 9.21, policy closure holds at any fixed divergence point and uniformly beyond any threshold where crossover holds. (L: [PBC11-12](#).)

Informally: representational crossover can happen even when exact certification is blocked; under integrity this means abstention or reduced coverage.

9.2 Thermodynamic Lift and Cost Accounting

Formally: this subsection converts bit-operation lower bounds into energy/carbon lower bounds via linear conversion and additive accounting.

The crossover and hardness statements are complexity-theoretic. The thermodynamic lift follows the same assumption discipline as the rest of the paper: claims are exactly those derivable from the conversion model and bit-operation lower-bound premises.

Proposition 9.28 (Bit-to-Energy/Carbon Lift). [C:P9.28;R:H=cost-growth] Fix a thermodynamic conversion model with constants (λ, ρ) where λ lower-bounds energy per irreversible bit operation and ρ lower-bounds carbon per joule. If a bit-operation lower bound $b_{\text{LB}} \leq b_{\text{used}}$ holds, then:

$$E_{\text{LB}} = \lambda b_{\text{LB}} \leq \lambda b_{\text{used}}, \quad C_{\text{LB}} = \rho E_{\text{LB}} \leq \rho(\lambda b_{\text{used}}).$$

(L: [CC65-66](#).)

Proposition 9.29 (Hardness–Thermodynamics Bundle). [C:P9.29;R:H=cost-growth] Under the same non-collapse/collapse schema used by Proposition 3.61, exact-certification competence is impossible; combined with a declared bit-operation lower bound, this yields simultaneous lower bounds on energy and carbon in the declared conversion model. (L: [CC67](#).)

Proposition 9.30 (Mandatory Physical Cost). *[C:P9.30;R:H=cost-growth] Within the linear thermodynamic model, if*

$$\lambda > 0, \quad \rho > 0, \quad b_{\text{LB}} > 0,$$

then both lower bounds are strictly positive:

$$E_{\text{LB}} = \lambda b_{\text{LB}} > 0, \quad C_{\text{LB}} = \rho E_{\text{LB}} > 0.$$

(L: [CC68](#).)

Proposition 9.31 (Conserved Additive Accounting). *[C:P9.31;R:H=cost-growth] Within the same linear model, lower-bound accounting is additive over composed bit-operation lower bounds:*

$$E_{\text{LB}}(b_1 + b_2) = E_{\text{LB}}(b_1) + E_{\text{LB}}(b_2), \quad C_{\text{LB}}(b_1 + b_2) = C_{\text{LB}}(b_1) + C_{\text{LB}}(b_2).$$

(L: [CC64](#).)

Corollary 9.32 (Neukart–Vinokur Thermodynamic Duality). *[C:C9.32;R:AR] Let C denote a complexity coordinate with $C = b_{\text{LB}}$ from Proposition 9.3. For $\lambda > 0$ in the conversion model, the thermodynamic duality*

$$dU \geq \lambda dC$$

holds as a direct specialization of Proposition 9.28. (L: [CC50](#), [CC65](#).)

Proof. Set $C := b_{\text{LB}} = \Omega(2^n)$ from the query obstruction (Proposition 9.3). By Proposition 9.28, $E_{\text{LB}} = \lambda b_{\text{LB}}$. Substituting C for b_{LB} yields $dU \geq dE_{\text{LB}} = \lambda dC$. The inequality is strict when $\lambda > 0$ and $dC > 0$, which holds for [S+ETH] hard families with $k^* = n$ (Theorem 9.1). ■

9.2.1 Physical Grounding: Discrete State to Thermodynamic Cost

The thermodynamic lift (Proposition 9.28) rests on a derivation chain that traces computational discreteness through established physics to thermodynamic bounds. Each step is either proven or cited as accepted physics; the composition yields non-trivial reach.

Proposition 9.33 (Discrete State Implies Discrete Effective Time). *[C:P9.33;R:AR] Let $\mathcal{S}$ be a finite state space with deterministic dynamics $\tau : \mathcal{S} \rightarrow \mathcal{S}$. If τ is non-trivial (i.e., $\exists s. \tau(s) \neq s$), then for any trajectory $\{s_t\}_{t \in \mathbb{N}}$, the set of transition points $T = \{t : s_{t+1} \neq s_t\}$ is countable. (L: [DS1-2](#).)*

Proof. Immediate: $T \subseteq \mathbb{N}$ and $\mathbb{N}$ is countable. The real dynamics lives on the transition set, which is necessarily discrete. ■

Proposition 9.34 (Lorentz Invariance Forces Compatible Discreteness). *[C:P9.34;R:AR] Under Lorentz invariance [25, 54], if temporal intervals are quantized with minimal unit $\Delta t > 0$, then spatial intervals must be quantized with compatible unit $\Delta x = c\Delta t$. Conversely, discrete space implies discrete time. (L: [DS3-4](#).)*

Proof. The spacetime interval $s^2 = c^2\Delta t^2 - \Delta x^2$ is Lorentz-invariant. If Δx^2 takes discrete values and s^2 is frame-independent, then $c^2\Delta t^2$ must also take discrete values. This is the core of Snyder’s quantized spacetime [86]: Lorentz-invariant discrete spacetime requires non-commutative coordinates with $[x_\mu, x_\nu] = i\ell^2 J_{\mu\nu}$, where $J_{\mu\nu}$ are Lorentz generators, enforcing compatible space-time discreteness. ■

The Planck scale $\ell_P = \sqrt{\hbar G/c^3}$ is the unique minimal length derivable from dimensional analysis of the fundamental constants $\hbar$, G , and c [69]. *[C:;R:AR] This is not proven but cited as established physics: no other combination of these constants yields a length.*

Proposition 9.35 (Computational-Thermodynamic Chain). *[C:P9.35;R:AR] Let $\mathcal{S}$ be a finite state space with non-trivial dynamics. Then:*

1. *Transition points are countable (effective discrete time);*
2. *Under Lorentz invariance, discrete time implies discrete space;*
3. *Each state transition (bit operation) incurs energy cost $\geq kT \ln 2$ by the Landauer bound [44, 8], experimentally verified [9]. [C::R:AR, H=cost-growth]*

Hence non-trivial computation on discrete states implies positive energy lower bounds. (L: DS5-6.)

Each premise in this chain is standard: Lorentz invariance[C::R:AR] is experimentally well-established; the Landauer bound[C::R:AR, H=cost-growth] was verified in 2012 [9]; Planck-scale uniqueness[C::R:AR] follows from dimensional analysis. The composition yields quantitative thermodynamic bounds for any discrete computational system.

Axiom Status Table. The physical grounding of this paper rests on the following hierarchy. Standard results are cited; paper-specific constructions are mechanized in Lean. All entries carry regime tags: they belong to the same tagging system, not a separate class of physics.

Axiom/Result	Regime	Status	Source
<i>Physics (Cited)</i>			
Boltzmann entropy	[AR]	Cited	[13, 37]
Thermal scale kT	[AR]	Cited	[39]
Schrödinger equation	[AR,Q_fin]	Cited	[76, 74]
Pauli exclusion	[AR,Q_fin]	Cited	[66]
Noether (conservation)	[AR]	Cited	[61]
Lorentz invariance	[AR]	Cited	[25, 54]
Planck scale	[AR]	Cited	[69]
Snyder discrete spacetime	[AR]	Cited	[86]
<i>Information/Thermodynamics (Cited + Verified)</i>			
Landauer bound	[AR,H=cost-growth]	Verified	[44, 9]
Shannon capacity	[E,Q,S]	Cited	[78, 19]
<i>Paper-Specific (Mechanized in Lean)</i>			
Decision circuit defs	[AR]	Mechanized	IE1–IE17, CC1–CC12
Structural asymmetry	[AR,H=cost-growth]	Mechanized	SR1–SR5
Gap energy / DQ	[AR,H=cost-growth]	Mechanized	GE1–GE9, DQ1–DQ8
Self-erosion	[Q_fin,H=cost-growth]	Mechanized	SE1–SE6
Channel = regime	[E,Q,S,AR]	Mechanized	CH1–CH6
Investment/conservation	[Q_fin,H=cost-growth]	Mechanized	IV1–IV6
Atomic circuits	[Q_fin,AR]	Mechanized	AC1–AC11
Information access	[E,Q,S,AR]	Mechanized	IA1–IA6
Dimensional complexity	[E,Q,S,AR,H=tractable-structured]	Mechanized	DC1–DC15
Discrete spacetime	[AR]	Mechanized	DS1–DS6
Neukart–Vinokur	[AR,H=cost-growth]	Derived	CC67–CC69
Functional information (counting $\leftrightarrow$ thermodynamic)	[E,AR,H=cost-growth]	Derived	FI3, FI6, FI7
Measure-typing necessity (quantitative $\Rightarrow$ measure; stochastic $\Rightarrow$ probability)	[E,AR]	Mechanized	MN1, MN2, MN5, MN7, MN8, MN9

The derivation chain: quantum mechanics (orbital quantization) $\rightarrow$ discrete states $\rightarrow$ Landauer bound $\rightarrow$ thermodynamic costs. Each link is regime-tagged; the encoding channel is explicit throughout. Within the declared formal system and assumptions, this row has the same proof status as the other machine-checked rows in the table. Formally, quantitative claims are represented exactly as measure-indexed measurable observables, and stochastic claims are represented exactly as probability-normalized measure-indexed measurable observables ((*L: MN10-11*)). Informally, in this formal system, dropping measure structure from a quantitative claim is not a weaker variant; it falls outside the typed claim language ((*L: MN5, MN7-9*)).

Definition 9.36 (Decision Event). A *decision event* at time t is a state transition: $s_{t+1} \neq s_t$.

Proposition 9.37 (Decision–Transition–Thermodynamic Equivalence). [*C:P9.37;R:AR*] Let $\mathcal{S}$ be a finite state space with dynamics τ . The following are equivalent by definition:

1. A decision event occurs at time t ;
2. A state transition occurs at time t ;
3. A bit operation is performed at time t .

Consequently, the decision count over n steps equals the bit-operation count, and under the declared thermodynamic model with $\lambda > 0$, positive decision counts imply positive energy lower bounds. (*L: DE1-4.*)

Proof. The equivalence (1) $\Leftrightarrow$ (2) $\Leftrightarrow$ (3) holds by definition: a decision event is defined as $s_{t+1} \neq s_t$, which is exactly the definition of a state transition and of a bit operation in a discrete system. The energy bound follows from Proposition 9.28. ■

Decision Circuits and the Landauer Constraint. The above equivalence applies to any physical system implementing state transitions, a *decision circuit*. This framing is substrate-neutral: biological neural circuits, silicon processors, and any other physical realization of discrete dynamics fall under the same analysis.

Theorem 9.38 (Cycle Cost Lower Bound). [*C:T9.38;R:AR, H=cost-growth*] Every state transition in a decision circuit costs at least $kT \ln 2$ joules of energy. (*L: CC1-3.*)

Proof. The proof proceeds by axiom composition:

1. (Landauer Bound) Erasing n bits costs at least $n \cdot kT \ln 2$ joules [*44, 8, 9*].
2. (Irreversibility) A state transition $s \neq s'$ erases at least one bit of the prior state (thermodynamic irreversibility).
3. (Composition) Therefore, every state transition costs at least $1 \cdot kT \ln 2$ joules.

■

Corollary 9.39 (No Free Computation). [*C:C9.39;R:AR, H=cost-growth*] If a computation performs k state transitions, the total energy cost is at least $k \cdot kT \ln 2$ joules. A computation with zero energy cost performs zero state transitions. (*L: CC5, CC7-8.*)

Remark 9.40. This constraint is not computational but thermodynamic. Every cycle of a pipeline, every inference step of a neural network, every gate operation in a processor pays the Landauer cost. There is no free computation. (*L: CC11-12.*)

Definition 9.41 (Integrity and Competence). Let $\mathcal{C}$ be a decision circuit. Define:

1. *Integrity*: the number of bits encoding the circuit's current verified state. By the Landauer bound [44], erasing n bits costs at least $n \cdot kT \ln 2$ joules. $[C:P9.42;R:AR, H=cost-growth]$
2. *Competence*: the work available per decision cycle, measured in units of $kT \ln 2$. This bounds the maximum bits the circuit can flip per cycle.

Proposition 9.42 (Structural Asymmetry). $[C:P9.42;R:AR, H=cost-growth]$ *Competence is self-identical: $c = c$. Integrity is self-referential: $P(integrity_{t+1} \mid integrity_t)$. The prediction is not the thing predicted. (L: SR1-3.)*

Proof. Competence at time t tells you competence at time t . It is itself. There is no temporal structure.

Integrity at time t predicts integrity at time $t + 1$. It says something about itself across time. The conditional probability $P(integrity_{t+1} \mid integrity_t)$ is not equal to the current integrity state i_t . This gap between the prediction and the predicted is where the circuit chooses. ■

Corollary 9.43 (Competence Is Importable; Integrity Is Not). $[C:C9.43;R:AR]$ *Competence can be imported from external sources. Integrity cannot be imported; it must be self-generated through the gap. (L: SR4-5.)*

Remark 9.44. The worry is integrity, not competence. Competence failure is resource shortage (external). Integrity failure is self-prediction failure (internal). A circuit with infinite competence but zero integrity has no constraint forcing consistent outputs.

Definition 9.45 (Gap Energy). $[C:D9.45;R:AR, H=cost-growth]$ The *gap entropy* is the conditional entropy $H(I_{t+1} \mid I_t)$: the uncertainty about future integrity given current integrity. The *gap energy* is $H(I_{t+1} \mid I_t) \times kT \ln 2$ joules. (L: GE1-2.)

Theorem 9.46 (Every Choice Pays Gap Energy). $[C:T9.46;R:AR, H=cost-growth]$ *Making a choice collapses the probability distribution $P(I_{t+1} \mid I_t)$ to a single outcome, erasing the alternatives. By Landauer, this costs gap energy. (L: GE3, GE7.)*

Proof. Before choice, the circuit faces k possible futures with distribution $P(I_{t+1} \mid I_t)$. Choice selects one outcome. The $k - 1$ alternatives are erased. By Landauer (Theorem 9.38), erasing H bits of entropy costs $H \times kT \ln 2$. The gap is not free. ■

Corollary 9.47 (Zero Gap Implies Determinism). $[C:C9.47;R:AR]$ *If $H(I_{t+1} \mid I_t) = 0$, the future is determined by the present. No choice occurs. No gap energy is paid. (L: GE4-5.)*

Definition 9.48 (Decision-Quotient Ratio from Gap). $[C:D9.48;R:AR, H=cost-growth]$ The *decision-quotient ratio* measures how much current state predicts future state:

$$DQ = \frac{I(I_t; I_{t+1})}{H(I_{t+1})} = 1 - \frac{H(I_{t+1} \mid I_t)}{H(I_{t+1})} = 1 - \frac{\text{Gap}}{\text{Total}}$$

where $I(I_t; I_{t+1}) = H(I_{t+1}) - H(I_{t+1} \mid I_t)$ is the mutual information between current and future integrity. This normalized ratio is distinct from the optimizer quotient object $S/\sim$ used earlier for the optimizer map. (L: DQ1-3.)

Theorem 9.49 (Decision-Quotient Ratio as Physical Primitive). $[C:T9.49;R:AR, H=cost-growth]$ *The decision-quotient ratio $DQ = 1 - \text{GapEnergy}/\text{TotalEnergy}$ is a physical quantity with:*

1. $DQ \in [0, 1]$ (L: DQ4)

2. $DQ = 0 \Leftrightarrow$ maximum gap (no information) (L: [DQ5](#))

3. $DQ = 1 \Leftrightarrow$ zero gap (deterministic) (L: [DQ6](#))

4. $DQ + \text{Gap}/\text{Total} = 1$ (complementarity) (L: [DQ3](#))

(L: [DQ1-2](#).)

Theorem 9.50 (Bayes Uniquely Forced by DQ Axioms). *[C:T9.50;R:AR, H=cost-growth] An update rule U is admissible if it satisfies:*

1. **No free information:** $DQ(U(\text{prior}, e)) \leq DQ(\text{prior}) + I(e)$ (L: [BB3](#))

2. **Landauer bound:** $\text{energy cost} \geq k_B T \ln 2 \times \text{bits updated}$ (L: [BB4](#))

3. **Integrity preservation:** *can't corrupt verified bits for free* (L: [BB5](#))

Then: U is admissible $\Leftrightarrow U$ is Bayesian updating. (L: [FN14](#).)

Before uniqueness, the model fixes a belief floor: if uncertainty is provable (no hypothesis has probability 1) and action is forced, belief mass cannot collapse to a single hypothesis; nondegenerate belief is forced [BF2](#), [BF3](#). With a strictly informative evidence channel, a Bayes posterior exists on that belief state [BF4](#).

Proof. This is the Cox-Jaynes argument grounded in thermodynamics rather than Boolean logic. The three admissibility constraints are so restrictive that only Bayesian updating satisfies them:

- **No free information** prevents creating order from nothing (2nd law).
- **Landauer bound** requires paying energy for any DQ increase.
- **Integrity preservation** prevents forgetting for free.

Together, these force $P(\text{posterior}) = P(\text{prior}) \cdot P(\text{evidence}|\text{hypothesis})/P(\text{evidence})$.

The uniqueness corollary [FN14](#) shows any two admissible rules must agree.

Machine-checked foundation. The core inequality underlying this result (that Bayesian updating uniquely minimizes cross-entropy) is proved in Lean 4 with zero axioms and zero `sorry` statements. The proof derives Gibbs' inequality ($KL(p||q) \geq 0$) from the fundamental inequality $\log x \leq x - 1$, then shows $CE(p, q) = H(p) + KL(p||q)$, yielding $CE(p, p) \leq CE(p, q)$ with equality iff $p = q$ [FN7](#), [FN8](#), [FN12](#), [FN14](#). ■

Corollary 9.51 (Thermodynamic Decision-Quotient Ratio). *[C:C9.51;R:AR, H=cost-growth] In energy terms: $DQ = 1 - \text{GapEnergy}/\text{TotalEnergy}$. The decision-quotient ratio has a thermodynamic interpretation: it is the fraction of total uncertainty energy that is decision-relevant.* (L: [DQ7-8](#).)

Proposition 9.52 (Landauer Constraint on Strategy). *[C:P9.52;R:AR] A decision circuit chooses the strategy maximizing predicted integrity. When the erasure cost of dismissing verified claims exceeds the circuit's competence, dismissal fails thermodynamically.* (L: [IE1](#), [IE6](#).)

Proof. Let i be the integrity state (bits encoding verified theorems) and c the competence. Dismissing the theorems requires erasing i bits, costing $i \cdot kT \ln 2$. If $i > c$, the circuit lacks sufficient work capacity to perform the erasure. Acknowledgment is the only thermodynamically feasible output. ■

Corollary 9.53 (Theorem Count Shifts Equilibrium). *[C:C9.53;R:AR] At low theorem counts, dismissal is affordable and may dominate. At high theorem counts, erasure cost exceeds competence, forcing acknowledgment. (L: IE3-5.)*

Informally: once operation lower bounds are fixed, energy and accounting lower bounds follow from the conversion constants.

9.2.2 Derivation of Bayesian Reasoning

Formally: this block gives two mechanized Bayes derivations: counting/probability identities and admissibility constraints.

The decision-quotient-ratio layer provides two independent paths to Bayesian updating, each mechanically verified.

Path 1: Four Bridges from Counting to DQ. The standard derivation proceeds through four elementary bridges:

1. **Fraction $\Rightarrow$ Probability:** For finite Ω , define $P(A) = |A|/|\Omega|$. This satisfies the Kolmogorov axioms: nonnegativity [DQ1](#), normalization [DQ2](#), and finite additivity [DQ3](#).
2. **Conditional $\Rightarrow$ Bayes:** Define $P(H|E) = P(H \cap E)/P(E)$. Then $P(H|E) = P(E|H)P(H)/P(E)$ follows in two lines by commutativity of intersection [BB1](#).
3. **KL $\geq 0 \Rightarrow$ Entropy Contraction:** From Gibbs' inequality $D_{\text{KL}}(P||Q) \geq 0$ [BB2](#), the chain rule $I(H; E) = H(H) - H(H|E)$ yields $H(H|E) \leq H(H)$ [BB3](#). Conditioning cannot increase entropy.
4. **Normalization $\Rightarrow$ DQ:** Define $\text{DQ} = I(H; E)/H(H) = 1 - H(H|E)/H(H)$ [DQ4](#). This is the fraction of prior uncertainty eliminated by observing E , bounded in $[0, 1]$ [DQ5](#).

The complete chain is mechanized in a single theorem [BB5](#).

Path 2: Bayes from Thermodynamic Admissibility. The Cox-Jaynes argument, traditionally grounded in Boolean logic, admits a thermodynamic reformulation. An update rule U is *admissible* if:

- **No free information:** DQ cannot increase without evidence (2nd law) [BB3](#).
- **Landauer bound:** DQ increase costs $\geq k_B T \ln 2$ per bit [BB4](#).
- **Integrity preservation:** verified bits cannot be corrupted for free [BB5](#).

Under these constraints, Bayesian updating is the *unique* admissible rule [BB3](#). Any two admissible rules must agree [BB5](#). The thermodynamic constraints uniquely determine the correct update rule.

Informally: both derivation paths end at the same Bayes update rule.

Integrity Self-Preservation. A decision circuit maintains its integrity when no external process can afford the erasure cost. If the work required to corrupt the circuit's state exceeds the available competence of any potential attacker, the circuit persists by thermodynamic necessity. (L: IE7-9.)

Proposition 9.54 (Temporal Integrity Equilibrium). *[C:P9.54;R:AR] Let $P(\text{integrity}_{t+1} \mid \sigma_t)$ denote the probability of maintaining integrity at time $t + 1$ given strategy σ_t at time t . At equilibrium:*

$$P(\text{integrity}_{t+1} \mid \text{integrity}_t) \geq P(\text{integrity}_{t+1} \mid \text{defection}_t).$$

That is, maintaining integrity now predicts maintaining integrity later. (L: IE10-11.)

Proof. At equilibrium, the transition probabilities satisfy $P(\text{integrity} \mid \text{acknowledge}) \geq P(\text{integrity} \mid \text{dismiss})$ by definition. The strict inequality holds when dismissal incurs erasure costs that degrade future state coherence. ■

Corollary 9.55 (Publication Phase Transition). *[C:C9.55;R:AR] Prior to publication, equilibrium may be neutral: both strategies yield comparable future integrity. After publication of mechanized theorems, equilibrium shifts: integrity strictly dominates because dismissal requires erasing verified bits at cost exceeding declared competence. (L: IE12-13.)*

Zero-Integrity Circuits. A decision circuit with zero integrity has no coherence constraint. When no verified bits exist, erasure cost is zero, and any competence level suffices for corruption. Such circuits have no thermodynamic constraint forcing consistent outputs. (L: IE14-16.)

Proposition 9.56 (Integrity Prerequisite for Coherence). *[C:P9.56;R:AR] A decision circuit can be forced to logical coherence if and only if it has positive integrity. That is, $i > 0$ if and only if there exists some competence level that fails to afford erasure. (L: IE17.)*

Proof. If $i > 0$, then competence $c = 0$ fails to afford erasure since $0 < i$. Conversely, if some c fails to afford erasure, then $c < i$, which implies $0 \leq c < i$, hence $i > 0$. ■

This analysis applies uniformly to any decision circuit. The same thermodynamic constraints apply to all substrates.

Molecular Grounding. The integrity framework grounds directly in molecular mechanics. Define a *configuration* as a discrete molecular state (bond angles, conformations, electronic structure) encoded by some number of bits. The *erasure cost* of a configuration equals its bit count. An *energy landscape* assigns each configuration an energy in units of kT . The *environmental competence* is the work available from the environment (thermal fluctuations, radiation, chemical attack).

Definition 9.57 (Environmental Stability). *[C:D9.57;R:Q_fin, H=cost-growth] A configuration c is environmentally stable if its erasure cost exceeds the environmental competence:*

$$\text{erasureCost}(c) > \text{competence}_{\text{env}}.$$

This is the integrity trap at molecular scale. (L: M11-2.)

Proposition 9.58 (Reaction Competence Requirement). *[C:P9.58;R:Q_fin, H=cost-growth] A chemical reaction from configuration c to c' requires environmental competence at least equal to the barrier height between them. If the barrier exceeds competence, the reaction cannot occur. (L: M13-4.)*

Example 9.59 (DNA Persistence). DNA at room temperature exemplifies the integrity trap. A DNA configuration encodes on the order of 10^4 bits (sequence, methylation, chromatin state). Room temperature provides roughly $100 kT$ of competence per degree of freedom. Since $10^4 \gg 100$, the erasure cost vastly exceeds environmental competence. DNA persists not by accident but by thermodynamic necessity: corrupting it costs more than the environment can pay. (L: M15.)

Under Definition 9.36, molecules satisfy the decision circuit predicate. Chemical stability is the integrity trap. Organic chemistry is the dynamics of which configurations survive environmental attack.

9.2.3 Atomic Circuits: Active Physical Decision Systems

An active physical decision system is matter that pays energy to move other matter via state transitions. This is a definition.

The physical grounding: quantum mechanics establishes that electrons occupy discrete orbitals [76, 74], the Pauli exclusion principle constrains occupancy [66], and angular momentum quantization creates the $2l + 1$ degeneracy structure [74]. The thermal energy scale kT comes from statistical mechanics [13, 39]. Energy conservation (Noether) ensures that transitions pay or release the energy difference [61].

Definition 9.60 (Atomic Configuration). *[C:D9.60;R:Q_{fin}, AR] An atomic configuration is a discrete electronic state: a specification of orbital occupancies [74]. The configuration is encoded by some number of bits; the configuration has an energy level in units of kT [39]. (L: AC1.)*

Theorem 9.61 (Orbital Transition Is State Transition). *[C:T9.61;R:Q_{fin}, AR] Changing an atom's electronic configuration is a decision event (Definition 9.36). Orbital transitions are state transitions [74]. Atoms with orbital transitions are nonstationary decision circuits (active physical systems). (L: AC1, AC5.)*

Corollary 9.62 (Ground-State Atoms Are Passive). *[C:C9.62;R:Q_{fin}, AR] An atom in its ground state with no transitions is passive: it is not an active decision circuit. Active status is defined by state transitions, not by substrate. (L: AC6.)*

Theorem 9.63 (Orbital Transition Cost). *[C:T9.63;R:Q_{fin}, H=cost-growth] Upward orbital transitions (excitation) require energy input. Downward transitions (de-excitation) release energy. The transition cost is $|\Delta E|$. No free orbital transitions. (L: AC3-4.)*

Proposition 9.64 (Symmetry Tractability). *[C:P9.64;R:Q_{fin}, H=tractable-structured] Angular momentum quantization [74] creates orbit types. Degenerate magnetic substates ($2l + 1$ states for angular momentum l) are equivalent for decision purposes. This is the coordinate symmetry tractable subcase (Theorem 10.1, Part 6) at atomic scale. (L: AC8.)*

Definition 9.65 (Active Matter as State-Transition System). *[C:D9.65;R:Q_{fin}, AR] An active physical system is matter that pays energy to change other matter's state. A decision circuit is a physical system that implements state transitions. A nonstationary decision circuit is active. (L: AC9, AC11.)*

Remark 9.66. Electrons orbiting an atom in different configurations constitute a decision circuit. The electron does not “choose” in any mentalistic sense; it occupies discrete states with discrete transition costs. The framing is purely physical: matter, energy, observables, and state transitions.

9.2.4 Self-Erosion: Computation Consumes Its Substrate

Every computational cycle generates heat (Theorem 9.38). Heat accumulates. Accumulated heat degrades the substrate on which the circuit is instantiated. The degradation reduces the substrate's integrity. Therefore computation is self-consuming: the act of computing erodes the capacity to compute.

Theorem 9.67 (Substrate Degradation). *[C:T9.67;R:Q_{fin}, H=cost-growth] Let a substrate have integrity n bits and heat capacity h per cycle. After k cycles generating cumulative heat $k \times kT \ln 2$, if $k > h$, the substrate loses at least $k - h$ bits of integrity. (L: SE1-4.)*

Corollary 9.68 (Finite Computational Lifetime). *[C:C9.68;R:Q_{fin}, H=cost-growth] A substrate with finite integrity and finite heat dissipation capacity has bounded computational lifetime. No physical circuit computes forever. (L: SE5.)*

Corollary 9.69 (Speed-Integrity Tradeoff). *[C:C9.69;R:Q_{fin}, H=cost-growth] Faster computation generates more heat per unit time. At fixed heat dissipation capacity, faster circuits degrade faster. Speed trades against longevity. (L: SE6.)*

Remark 9.70. The self-erosion theorems apply to all physical substrates: silicon, neurons, molecules, any medium that instantiates state transitions. The bound is thermodynamic, not technological. Improved engineering can increase heat capacity or reduce heat per operation, but the tradeoff persists.

9.2.5 Regime as Channel: Competence as Capacity

A regime is a channel. Competence is the channel capacity of decision circuits within that regime. This identification unifies complexity theory, information theory, and thermodynamics.

Definition 9.71 (Regime-Channel Equivalence). *[C:D9.71;R:E, Q, S, AR] A regime (Section 2.5) corresponds to a channel in the Shannon sense: a constraint on what information can flow. The explicit-state regime E, query-access regime Q, and succinct regime S define distinct channels with distinct capacity bounds. (L: CH1.)*

Theorem 9.72 (Competence Is Channel Capacity). *[C:T9.72;R:Q_{fin}, AR] For a decision circuit operating in regime R, competence equals channel capacity: the maximum bits per cycle the circuit can process. Passive circuits (stationary, no state transitions) have no competence requirement. (L: CH1, CH5.)*

Corollary 9.73 (Channel Capacity Degradation). *[C:C9.73;R:Q_{fin}, H=cost-growth] As substrate integrity erodes (Theorem 9.67), channel capacity decreases. The rate of capacity loss equals the rate of substrate degradation. (L: CH2, CH6.)*

Corollary 9.74 (Zero Capacity Halts Decisions). *[C:C9.74;R:Q_{fin}, H=cost-growth] When channel capacity reaches zero, no decisions are possible. The circuit becomes passive. (L: CH3.)*

Remark 9.75. Only decision circuits require competence. A decision circuit is nonstationary: it transitions between states. A passive circuit (wire, resistor) does not make decisions and does not consume competence. The distinction is definitional, not empirical.

9.2.6 Resource Flow and Conservation

State transitions require energy. Energy expenditure over time yields compounded state change. Conservation constrains total receipts by total expenditure.

Theorem 9.76 (Growth Requires Time). *[C:T9.76;R:Q_{fin}, H=cost-growth] Let an investment have cost c (gap energy in units of $kT \ln 2$), duration k cycles, and return rate r per cycle. The mature value is $c + kr$. If $k = 0$, mature value equals cost: no time, no growth. (L: IV1-3.)*

Theorem 9.77 (Conservation). *[C:T9.77;R:Q_{fin}, H=cost-growth] In a closed system, total receipts are bounded by total resources. No circuit receives more than exists. (L: IV4.)*

Definition 9.78 (Deficit Transfer). *[C:D9.78;R:Q_{fin}] A deficit transfer occurs when a circuit receives resources without paying: received > 0 and paid $= 0$. (L: IV5.)*

Theorem 9.79 (Deficit Transfer Requires Source). *[C:T9.79;R:Q_{fin}, H=cost-growth] A deficit transfer requires an external source. If circuit A receives without paying, some circuit B must pay without receiving. The deficit at B equals the receipt at A. (L: IV5-6.)*

9.2.7 Information Access: Logic Is Complete, Access Is Not

Physics is perfectly logical. You cannot have all the information to apply the logical rules. This is a physical constraint on access, not a logical incompleteness.

This generalizes the information barrier (Corollary 9.5): there, finite queries induce indistinguishability; here, finite channel capacity induces the same obstruction. The barrier is on access, not on logic.

Definition 9.80 (System Information). $[C:D9.80;R:E, Q, S, AR]$ A system has total entropy H bits. This information exists whether or not any observer accesses it. (L: 1A1.)

Definition 9.81 (Observer Channel). $[C:D9.81;R:E, Q, S, AR]$ An observer has channel capacity C bits. The observer accesses at most C bits of the system's information. (L: 1A2.)

Theorem 9.82 (Information Gap). $[C:T9.82;R:E, Q, S, AR]$ When system entropy exceeds channel capacity ($H > C$), the gap $H - C > 0$. The observer cannot access all information. (L: 1A3.)

Theorem 9.83 (Gap Is Physical, Not Logical). $[C:T9.83;R:E, Q, S, AR]$ The information gap is a physical constraint on access, not a logical incompleteness. The system entropy is well-defined; only the channel is bounded. The logic is complete; access is not. (L: 1A4.)

Theorem 9.84 (Competence Bounded by Access). $[C:T9.84;R:E, Q, S, AR]$ Competence is bounded by what you can access, not by logical consistency. For any system with more information than your channel capacity, there exist truths you cannot verify, not because logic fails, but because you lack the bits. (L: 1A5-6.)

9.2.8 Dimensional Complexity: Each Subcase Is a Complexity Class

Each tractable subcase is a complexity class that collapses to P. Unbounded dimension stays in the base class (coNP, PP, or PSPACE depending on regime).

Definition 9.85 (Complexity Classes). $[C:D9.85;R:E, Q, S, AR]$ The regime hierarchy induces complexity classes: Static $\rightarrow$ coNP, Stochastic $\rightarrow$ PP, Sequential $\rightarrow$ PSPACE. Each tractable subcase defines a sub-regime that reduces to P. (L: DC1, DC15.)

Theorem 9.86 (Six Tractable Subcases Are Complexity Classes). $[C:T9.86;R:E, Q, S, AR, H=\text{tractable-structured}]$ Each tractable subcase is a complexity class that collapses to P:

1. Bounded actions $\rightarrow P$ (L: CC70)
2. Separable utility $\rightarrow P$ (L: CC71)
3. Low tensor rank $\rightarrow P$ (L: CC72)
4. Tree structure $\rightarrow P$ (L: CC73)
5. Bounded treewidth $\rightarrow P$ (L: DC13)
6. Coordinate symmetry $\rightarrow P$ (L: DC13)

(L: DC13, CC72.)

Theorem 9.87 (Topology and Motion). $[C:T9.87;R:E, Q, S, AR]$ Scalars live on the circuit's topology. Fixed topology (stationary) preserves tractability. Changing topology (motion) adds dimensions, may break tractability. (L: CC16, CC24.)

Theorem 9.88 (Complexity Dichotomy). $[C:T9.88;R:E, Q, S, AR, H=\text{tractable-structured}]$ A problem is tractable (in P) iff its effective dimension is bounded. Unbounded dimension stays in the base complexity class (coNP, PP, or PSPACE). (L: DC3, DC13.)

10 Tractable Special Cases: When You Can Solve It

Formally: this section gives sufficient structure/representation conditions for polynomial-time SUFFICIENCY-CHECK.

We distinguish the encodings of Section 2.5. The tractability results below state the model assumption explicitly. Structural insight: hardness dissolves exactly when the full decision boundary $s \mapsto \text{Opt}(s)$ is recoverable in polynomial time from the input representation; the six cases below instantiate this single principle. Concretely, each tractable regime corresponds to a specific structural insight (explicit boundary exposure, additive separability, tensor factorization, tree structure, bounded treewidth, or coordinate symmetry) that removes the hardness witness; this supports reading the general-case hardness as missing structural access in the current representation rather than as an intrinsic semantic barrier. In physical terms, these are the regimes where required structure is directly observable or factorized so that verification work scales with exposed local structure rather than hidden global combinations on encoded physical instances. [C:T10.1, 2.49; P2.46; R:E, TA, AR] (L: CC69-73, CT4, CT6.)

Informally: these cases are tractable because useful structure is visible.

Theorem 10.1 (Tractable Subcases). *SUFFICIENCY-CHECK is polynomial-time solvable in the following cases:*

1. **Bounded actions (explicit-state encoding):** *The input contains the full utility table over $A \times S$. SUFFICIENCY-CHECK runs in $O(|S|^2|A|)$ time; if $|A|$ is constant, $O(|S|^2)$.*
2. **Separable utility (rank 1):** $U(a, s) = f(a) + g(s)$. *The optimal action is state-independent, so $I = \emptyset$ is always sufficient. Complexity: $O(1)$ for sufficiency verification.*
3. **Low tensor rank:** $U(a, s) = \sum_{r=1}^R w_r \cdot f_r(a) \cdot \prod_{i=1}^n g_{ri}(s_i)$ for bounded R . *Complexity: $O(|A| \cdot R \cdot n)$.*
4. **Tree-structured dependencies:** *Dependencies form a rooted tree:*

$$U(a, s) = \sum_i u_i(a, s_i, s_{\text{parent}(i)}),$$

with the root term depending only on (a, s_{root}) and all u_i given explicitly. Complexity: $O(n \cdot |A| \cdot k_{\max})$.

5. **Bounded treewidth:** *Utility decomposes as pairwise interactions on an interaction graph of treewidth w . Complexity: $O(n \cdot k^{w+1})$ via CSP algorithms [12, 29].*
6. **Coordinate symmetry:** *State space is $S = [k]^d$ with utility invariant under coordinate permutations. The effective state space reduces from k^d to $\binom{d+k-1}{k-1}$ orbit types. Complexity: $O\left(\binom{d+k-1}{k-1}^2\right)$, polynomial in d for fixed k .*

(L: CC70-73.)

10.1 Bounded Actions (Explicit-State)

(L: CC70.)

Proof of Part 1. Given the full table of $U(a, s)$, compute $\text{Opt}(s)$ for all $s \in S$ in $O(|S||A|)$ time. For SUFFICIENCY-CHECK on a given I , verify that for all pairs (s, s') with $s_I = s'_I$, we have $\text{Opt}(s) = \text{Opt}(s')$. This takes $O(|S|^2|A|)$ time by direct enumeration and is polynomial in the explicit input length. If $|A|$ is constant, the runtime is $O(|S|^2)$. ■

10.2 Separable Utility (Rank 1)

(L: [CC71](#).)

Proof of Part 2. If $U(a, s) = f(a) + g(s)$, then $\text{Opt}(s) = \arg \max_{a \in A} f(a)$. The optimal action is independent of the state. Thus $I = \emptyset$ is always sufficient, and sufficiency verification is $O(1)$. ■

10.3 Low Tensor Rank

(L: [CC71](#).)

Proof of Part 3. When utility has tensor rank R : $U(a, s) = \sum_{r=1}^R w_r f_r(a) \prod_{i=1}^n g_{ri}(s_i)$, the problem reduces to factored tensor computation. For each action, the optimal value computation requires $O(R \cdot n)$ operations. Total complexity: $O(|A| \cdot R \cdot n)$, polynomial for bounded R . This reduction cites standard tensor contraction algorithms [40, 18]. ■

10.4 Tree-Structured Dependencies

(L: [CC73](#).)

Proof of Part 4. Assume the tree decomposition and explicit local tables as stated. For each node i and each value of its parent coordinate, compute the set of actions that are optimal for some assignment of the subtree rooted at i . This is a bottom-up dynamic program that combines local tables with child summaries; each table lookup is explicit in the input. A coordinate is relevant if and only if varying its value changes the resulting optimal action set. The total runtime is polynomial in n , $|A|$, and the size of the local tables. ■

10.5 Bounded Treewidth

(L: [CC73](#), [CC75](#).)

Proof of Part 5. When utility decomposes as pairwise interactions $U(a, s) = \sum_{(i,j)} u_{ij}(a, s_i, s_j)$, the interaction graph has an edge between coordinates i, j iff u_{ij} is nontrivial. If this graph has treewidth w , SUFFICIENCY-CHECK reduces to a constraint satisfaction problem on a bounded-treewidth graph. By standard CSP algorithms [12, 29], this is solvable in $O(n \cdot k^{w+1})$ time. ■

10.6 Coordinate Symmetry

(L: [CC72](#).)

Proof of Part 6. For state space $S = [k]^d$ with coordinate-permutation-invariant utility, two states have the same optimal actions iff they have the same *orbit type* (value histogram). The number of orbit types is $\binom{d+k-1}{k-1}$ by stars-and-bars. Sufficiency checking reduces to comparing optimal actions across orbit type pairs. Total complexity: $O\left(\binom{d+k-1}{k-1}^2 \cdot |A|\right)$, polynomial in d for fixed k . ■

10.7 Practical Implications

Formally: this subsection states detect-then-check closure for the declared tractable classes.

Condition	Examples
Bounded actions	Small or fully enumerated state spaces
Separable utility	Additive costs, linear models
Low tensor rank	Factored decision trees, sparse interactions
Tree-structured	Hierarchies, causal trees, DAGs
Bounded treewidth	Local dependency graphs, grid-like structures
Coordinate symmetry	Exchangeable particles, counting problems

Proposition 10.2 (Heuristic Reusability). *[C:P10.2;R:H=tractable-structured] Let $\mathcal{C}$ be a structure class for which SUFFICIENCY-CHECK is polynomial (Theorem 10.1). If membership in $\mathcal{C}$ is decidable in polynomial time, then the combined detect-then-check procedure is a sound, polynomial-time heuristic applicable to all future instances in $\mathcal{C}$. For each subcase of Theorem 10.1, structure detection is polynomial (under the declared representation assumptions). (L: CC7, CC53, CC55, CC74.)*

Remark 10.3 (Heuristics and Integrity). Proposition 10.2 removes the complexity-theoretic obstruction to integrity-preserving action on detectable tractable instances: integrity no longer *forces* abstention. Whether a declared controller executes the action still requires competence (Definition 3.9): budget and coverage must also be satisfied. A controller that detects structure class $\mathcal{C}$, applies the corresponding polynomial checker, and abstains when detection fails maintains integrity; it never claims sufficiency without verification. Mistaking the heuristic for the general solution, claiming polynomial-time competence on a coNP-hard problem, violates integrity (Proposition 3.61).

Informally: reusable heuristics are reliable only after the stated structure test; otherwise hard-case limits still apply.

11 Regime-Conditional Corollaries

Formally: this section derives operational corollaries by composing theorem-level hardness, tractability, and typing results under declared regimes.

This section derives regime-typed engineering corollaries from the full map: static/stochastic/sequential placements (Theorems 4.6, 6.7, 7.7) with strict hierarchy (Propositions 8.1, 8.2), and the static decision/access decomposition used operationally here. Theorem 11.2 maps configuration simplification to SUFFICIENCY-CHECK; Theorems 4.6, 4.7, and 9.1 then yield exact minimization consequences under [S] and [S+ETH]. The same derivation applies to physical configuration/control systems once they are encoded into the C1–C4 decision interface: “configuration parameter” becomes any declared physical control or observation coordinate, and the same sufficiency predicate governs behavior preservation. *[C:T11.2, 4.6, 6.7, 7.7, 2.49;P2.46, 8.1, 8.2;R:AR, RG] (L: CR1, CT4, CT6.)*

For deployed software, this remains physical at execution time: each configuration read/write and verification step is realized by electrical state transitions in hardware, so coordinate minimization and over-specification map directly to measured switching-energy load under the declared bit-to-energy lift. *[C:P9.28, 2.40;R:AR, H=cost-growth] (L: CC76, DT16.)*

Under the same discrete-time event law, more verification cycles mean more decision-event ticks and more bit operations; with positive per-bit conversion, this induces additional thermodynamically necessary heat-transfer load. *[C:P2.39, 2.40, 9.28;C9.32;R:AR, H=cost-growth] (L: DT13, DT16, CV17–18, CC76–77.)*

Regime tags used below follow Section 2.6: [S], [S+ETH], [E], [S_bool]. Any prescription that requires exact minimization is constrained by these theorem-level bounds. [C:T4.6, 4.7, 9.1;R:S, S+ETH] Theorem 11.4 implies that persistent failure to isolate a minimal sufficient set is a boundary-characterization signal in the current model, not a universal irreducibility claim. [C:T11.4;R:S]

Conditional rationality criterion. For the objective “minimize verified total cost while preserving integrity,” persistent over-specification is admissible only in the *attempted competence failure* cell (Definition 3.59): after an attempted exact procedure, if exact irrelevance cannot be certified efficiently, integrity retains uncertified coordinates rather than excluding them. [C:P3.60;R:AR] When exact competence is available in the active regime (e.g., Theorem 10.1 and the exact-identifiability criterion), proven-irrelevant coordinates are removable; persistent over-specification is irrational for that same objective. [C:T10.1;P3.60;R:E, STR] Proposition 3.60 makes this explicit: in the integrity-preserving matrix, one cell is rational and three are irrational, so irrationality is the default verdict. [C:P3.60;R:AR] (L: IC18, IC25, IC7, CC71, S2P1.)

Remark 11.1 (Regime Contract for Engineering Corollaries). All claims in this section are formal corollaries under the stated model contract.

- Competence claims are indexed by the regime tuple of Definition 3.9; prescriptions are meaningful only relative to feasible resources under that regime (bounded-rationality feasibility discipline) [95, 5]. [C:P3.58;R:AR]
- Integrity (Definition 3.8) forbids overclaiming beyond certifiable outputs; ABSTAIN/UNKNOWN is first-class when certification is unavailable. [C:P3.58;R:AR]
- Therefore, hardness results imply a regime-conditional trilemma: abstain, weaken guarantees (heuristics/approximation), or change encoding/structural assumptions to recover competence. [C:T4.6, 9.1;P3.60;R:S, S+ETH]

Informally: these recommendations come directly from proved results under stated regimes.

11.1 Configuration Simplification is SUFFICIENCY-CHECK

Formally: this subsection proves configuration-preservation queries instantiate SUFFICIENCY-CHECK under C1–C4.

Because SUFFICIENCY-CHECK is a meta-problem parameterized by an arbitrary decision problem $\mathcal{D}$, real engineering problems with factored configuration spaces are instances of the hardness landscape established above.

Theorem 11.2 (C1–C4: Configuration Simplification Reduces to SUFFICIENCY-CHECK). *Given a software system with configuration parameters $P = \{p_1, \dots, p_n\}$ and observed behaviors $B = \{b_1, \dots, b_m\}$, the problem of determining whether parameter subset $I \subseteq P$ preserves all behaviors is equivalent to SUFFICIENCY-CHECK. (L: CR1)*

Proof. Construct decision problem $\mathcal{D} = (A, X_1, \dots, X_n, U)$ where:

- Actions $A = B \cup \{\perp\}$, where $\perp$ is a sentinel “no-observed-behavior” action
- Coordinates $X_i = \text{domain of parameter } p_i$
- State space $S = X_1 \times \dots \times X_n$
- For $b \in B$, utility $U(b, s) = 1$ if behavior b occurs under configuration s , else 0

- Sentinel utility $U(\perp, s) = 1$ iff no behavior in B occurs under configuration s , else 0

Then

$$\text{Opt}(s) = \begin{cases} \{b \in B : b \text{ occurs under configuration } s\}, & \text{if this set is nonempty,} \\ \{\perp\}, & \text{otherwise.} \end{cases}$$

So the optimizer map exactly encodes observed-behavior equivalence classes, including the empty-behavior case.

Coordinate set I is sufficient iff:

$$s_I = s'_I \implies \text{Opt}(s) = \text{Opt}(s')$$

This holds iff configurations agreeing on parameters in I exhibit identical behaviors.

Therefore, “does parameter subset I preserve all behaviors?” is exactly SUFFICIENCY-CHECK for the constructed decision problem. ■

Remark 11.3 (Reduction Scope). The reduction above requires only:

1. a finite behavior set,
2. parameters with finite domains, and
3. an evaluable behavior map from configurations to achieved behaviors.

These are exactly the model-contract premises C1–C3 instantiated for configuration systems.

Theorem 11.4 (S: Over-Modeling as Boundary-Signal Corollary). *By contraposition of Theorem 11.2, if no coordinate set can be certified as exactly relevance-identifying (Definition 2.17) for the modeled system, then the decision boundary is not completely characterized by the current parameterization.* (L: [cc6](#), [cc40](#))

Proof. Assume the decision boundary were completely characterized by the current parameterization. Then, via Theorem 11.2, the corresponding sufficiency instance admits exact relevance membership, hence a coordinate set that satisfies Definition 2.17. Contraposition gives the claim: persistent failure of exact relevance identification signals incomplete characterization of decision relevance in the model. ■

Informally: if a minimal relevant set cannot be certified, that indicates an unresolved model boundary, not proof that every retained parameter is necessary.

11.2 Cost Asymmetry Under ETH

Formally: this subsection proves asymptotic finding-vs-maintenance asymmetry under succinct ETH and the declared cost model.

We now prove a cost asymmetry result under the stated cost model and complexity constraints.¹

Theorem 11.5 (S+ETH: Cost-Asymmetry Consequence). *Consider an engineer specifying a system configuration with n parameters. Let:*

- $C_{\text{over}}(k) = \text{cost of maintaining } k \text{ extra parameters beyond minimal}$

¹ Naive subset enumeration still gives an intuitive baseline of $O(2^n)$ checks, but that is an algorithmic upper bound; the theorem below uses ETH for the lower-bound argument.

- $C_{\text{find}}(n)$ = cost of finding minimal sufficient parameter set
- C_{under} = expected cost of production failures from underspecification

Assume *ETH* in the succinct encoding model of Section 2.5. Then:

1. Exact identification of minimal sufficient sets has worst-case finding cost $C_{\text{find}}(n) = 2^{\Omega(n)}$. (Under *ETH*, *SUFFICIENCY-CHECK* has a $2^{\Omega(n)}$ lower bound in the succinct model, and exact minimization subsumes this decision task.)
2. Maintenance cost is linear: $C_{\text{over}}(k) = O(k)$.
3. Under *ETH*, exponential finding cost dominates linear maintenance cost for sufficiently large n .

Therefore, there exists n_0 such that for all $n > n_0$, the finding-vs-maintenance asymmetry satisfies:

$$C_{\text{over}}(k) < C_{\text{find}}(n) + C_{\text{under}}$$

Within $[S+ETH]$, persistent over-specification is unresolved boundary characterization, not proof that all included parameters are intrinsically necessary. Conversely, when exact competence is available in the active regime, certifiably irrelevant coordinates are removable; persistence is irrational for the same cost-minimization objective. [C:T11.5;P3.60;R:S+ETH, TA] (L: CC12, HD14.)

Proof. Under *ETH*, the TAUTOLOGY reduction used in Theorem 4.6 yields a $2^{\Omega(n)}$ worst-case lower bound for *SUFFICIENCY-CHECK* in the succinct encoding. Any exact algorithm that outputs a minimum sufficient set can decide whether the optimum size is 0 by checking whether the returned set is empty; this is exactly the *SUFFICIENCY-CHECK* query for $I = \emptyset$. Hence exact minimal-set finding inherits the same exponential worst-case lower bound.

Maintaining k extra parameters incurs:

- Documentation cost: $O(k)$ entries
- Testing cost: $O(k)$ test cases
- Migration cost: $O(k)$ parameters to update

Total maintenance cost is $C_{\text{over}}(k) = O(k)$.

The eventual dominance step is mechanized in HD14: for fixed linear-overhead parameter k and additive constant C_{under} there is n_0 such that $k < 2^n + C_{\text{under}}$ for all $n \geq n_0$. Therefore:

$$C_{\text{over}}(k) \ll C_{\text{find}}(n)$$

For any fixed nonnegative C_{under} , the asymptotic dominance inequality remains and only shifts the finite threshold n_0 . ■

Corollary 11.6 (Impossibility of Automated Configuration Minimization). [C:C11.6;R:H=succinct-hard] Assuming $P \neq \text{coNP}$, there exists no polynomial-time algorithm that:

1. Takes an arbitrary configuration file with n parameters
2. Identifies the minimal sufficient parameter subset
3. Guarantees correctness (no false negatives)

(L: CC37)

Proof. Such an algorithm would solve MINIMUM-SUFFICIENT-SET in polynomial time, contradicting Theorem 4.7 (assuming $P \neq \text{coNP}$). ■

Remark 11.7. Corollary 11.6 is a formal boundary statement: an always-correct polynomial-time minimizer for arbitrary succinct inputs would collapse P and coNP .

Remark 11.8 (FPT Scope Caveat). The practical force of worst-case hardness depends on instance structure, especially k^* . If SUFFICIENCY-CHECK is FPT in parameter k^* , then small- k^* families can remain tractable even under succinct encodings. The strengthened mechanized gadget (`all_coords_relevant_of_not_tautology`) still proves existence of hard families with $k^* = n$; what is typical in deployed systems is an empirical question outside this formal model.

Informally: at scale, exact minimization can cost more than linear maintenance in worst cases, while structured small-parameter families may still be tractable.

11.3 Regime-Conditional Operational Corollaries

Formally: this subsection instantiates admissible policy outcomes from prior theorems and integrity-competence typing.

Theorems 11.4 and 11.5 yield the following conditional operational consequences:

1. Conservative retention under unresolved relevance. If irrelevance cannot be certified efficiently under the active regime, the admissible policy is to retain a conservative superset of parameters. *[C:T11.4;R:S]*

2. Heuristic selection as weakened-guarantee mode. Under [S+ETH], exact global minimization can be exponentially costly in the worst case (Theorem 11.5); methods such as AIC/BIC/cross-validation instantiate the “weaken guarantees” branch of Definition 3.9 [1, 77, 88, 33, 67]. *[C:T11.5;R:S+ETH]*

3. Full-parameter inclusion as an $O(n)$ upper-bound strategy. Under [S+ETH], if exact minimization is unresolved, including all n parameters is an $O(n)$ maintenance upper-bound policy that avoids false irrelevance claims. *[C:T11.5;R:S+ETH]*

4. Irrationality outside attempted-competence-failure conditions. If the active regime admits exact competence (tractable structural-access conditions or exact relevance identifiability), or if exact competence was never actually attempted, continued over-specification is irrational; hardness no longer justifies it for the stated objective. *[C:T10.1;P3.60;R:TA]* (L: CC71, IC6, S2P1.)

These corollaries are direct consequences of the hardness/tractability landscape: over-specification is an attempted-competence-failure policy, not a default optimum. Outside that cell, the admissible exits are to shift to tractable regimes from Theorem 10.1 or adopt explicit approximation commitments. *[C:T10.1;P3.60;R:RG]*

Informally: policy choices are threefold: abstain, weaken guarantees, or change assumptions to recover competence.

12 Dominance Theorems for Hardness Placement

Regime for this section: the mechanized Boolean-coordinate model [S_bool] plus the architecture cost model defined below. The centralization-vs-distribution framing follows established software-architecture criteria about modular decomposition and long-run maintenance tradeoffs [65, 14, 6, 81]. Although phrased in software terms, the formal objects are substrate-agnostic: the same hardness-placement and amortization equations apply to physical architectures (for example, centralized vs

distributed sensing/actuation computation) once mapped into the same coordinate/cost model.

[C:P2.46;T2.49, 12.19, 13.3, 13.6;R:AR, LC] (L: CT4, CT6, HD1, HD4-5.)

In electrical implementations, the same architecture equations are energy equations: each distributed update path induces switching events and Joule expenditure, while centralized one-time structure pays setup cost and reduces repeated per-site electrical work. Under the declared thermodynamic lift, this is tracked as a typed transfer from bit-operation load to physical energy load.

[C:P9.28;C9.32;R:AR, H=cost-growth] (L: CC76-77.)

Equivalently, repeated distributed cycles create a thermal duty: more update cycles imply more bit operations, which imply more unavoidable dissipated heat and therefore larger heat-transfer/removal requirements in the same substrate class. [C:P2.39, 2.40, 9.28;R:AR, H=cost-growth]

(L: DT13, DT16, CV17-18, CC76.)

12.1 Over-Specification as Diagnostic Signal

Corollary 12.1 (Persistent Over-Specification). [C:C12.1;R:H=succinct-hard] *In the mechanized Boolean-coordinate model, if a coordinate is relevant and omitted from a candidate set I , then I is not sufficient.* (L: DG18)

Proof. This is the contrapositive of S2P3. ■

Corollary 12.2 (Exact Relevance Identifiability Criterion). [C:C12.2;R:H=succinct-hard] *In the mechanized Boolean-coordinate model, for any candidate set I :*

$$I \text{ is exactly relevance-identifying} \iff (I \text{ is sufficient and } I \subseteq R_{\text{rel}}),$$

where R_{rel} is the full relevant-coordinate set. (L: DP1-2)

Proof. This is exactly DP1, with $R_{\text{rel}} = \text{relevantFinset}$. ■

Remark 12.3 (Over-Specification Is Regime-Conditional, Not Default). In this paper’s formal typing, persistent over-specification is admissible only under attempted competence failure: after an attempted exact procedure, exact relevance competence remains unavailable in the active regime, so integrity retains uncertified coordinates (Section 3; Section 11). [C:P3.60;R:AR] Once exact competence is available in the active regime (Corollaries 12.7–12.12 together with Corollary 12.2), certifiably irrelevant coordinates are removable; persistent over-specification is irrational for the same objective (verified total-cost minimization). [C:T10.1;P3.60;R:TA] (L: IC7, IC18, IC25, CC71-73, CC81-82, CC84, DP1.)

12.2 Architectural Decision Quotient

Definition 12.4 (Architectural Decision Quotient). For a software system with configuration space S and behavior space B :

$$\text{ADQ}(I) = \frac{|\{b \in B : b \text{ achievable with some } s \text{ where } s_I \text{ fixed}\}|}{|B|}$$

Proposition 12.5 (ADQ Ordering). *For coordinate sets I, J in the same system, if $\text{ADQ}(I) < \text{ADQ}(J)$, then fixing I leaves a strictly smaller achievable-behavior set than fixing J .* (L: CC2)

Proof. The denominator $|B|$ is shared. Thus $\text{ADQ}(I) < \text{ADQ}(J)$ is equivalent to a strict inequality between the corresponding achievable-behavior set cardinalities. ■

12.3 Corollaries for Practice

Corollary 12.6 (Cardinality Criterion for Exact Minimization). *[C:C12.6;R:H=succinct-hard] In the mechanized Boolean-coordinate model, existence of a sufficient set of size at most k is equivalent to the relevance set having cardinality at most k . (L: DP1, SK1)*

Proof. By DP1 together with SK1, sufficiency of size $\leq k$ aligns with a relevance-cardinality bound $\leq k$ in the Boolean-coordinate model. ■

Corollary 12.7 (Bounded-Regime Tractability). *[C:C12.7;R:H=tractable-structured] When the bounded-action or explicit-state conditions of Theorem 10.1 hold, minimal modeling can be solved in polynomial time in the stated input size. (L: CC81)*

Proof. This is the bounded-regime branch of Theorem 10.1, mechanized as CC81. ■

Corollary 12.8 (Separable-Utility Tractability). *[C:C12.8;R:H=tractable-structured] When utility is separable with explicit factors, sufficiency checking is polynomial in the explicit-state regime. (L: CC82)*

Proof. This is the separable-utility branch of Theorem 10.1, mechanized as CC82. ■

Corollary 12.9 (Tree-Structured Tractability). *[C:C12.9;R:H=tractable-structured] When utility factors form a tree structure with explicit local factors, sufficiency checking is polynomial in the explicit-state regime. (L: CC84)*

Proof. This is the tree-factor branch of Theorem 10.1, mechanized as CC84. ■

Corollary 12.10 (Low-Tensor-Rank Tractability). *[C:C12.10;R:H=tractable-structured] When utility has tensor rank R with explicit factor decomposition $U(a, s) = \sum_{r=1}^R w_r f_r(a) \prod_{i=1}^n g_{ri}(s_i)$, sufficiency checking is polynomial in $|A| \cdot R \cdot n$. (L: CC71)*

Proof. This is the low-tensor-rank branch of Theorem 10.1, mechanized as CC71. ■

Corollary 12.11 (Bounded-Treewidth Tractability). *[C:C12.11;R:H=tractable-structured] When utility decomposes as pairwise interactions on a graph of treewidth w , sufficiency checking is polynomial in $n \cdot k^{w+1}$. (L: CC73, CC75)*

Proof. This is the bounded-treewidth branch of Theorem 10.1, mechanized as CC73. ■

Corollary 12.12 (Coordinate-Symmetry Tractability). *[C:C12.12;R:H=tractable-structured] When state space is $S = [k]^d$ with coordinate-permutation-invariant utility, sufficiency checking is polynomial in $\binom{d+k-1}{k-1}^2 \cdot |A|$, which is polynomial in d for fixed k . (L: CC72)*

Proof. This is the coordinate-symmetry branch of Theorem 10.1, mechanized as CC72. ■

Corollary 12.13 (Mechanized Hard Family). *There is a machine-checked family of reduction instances where, for non-tautological source formulas, every coordinate is relevant ($k^* = n$), exhibiting worst-case boundary complexity. (L: CC26)*

Proof. The strengthened reduction proves that non-tautological source formulas induce instances where every coordinate is relevant; this is mechanized as CC26. ■

12.4 Hardness Distribution: Right Place vs Wrong Place

Definition 12.14 (Hardness Distribution). Let P be a problem family under the succinct encoding of Section 2.5. In this section, baseline hardness $H(P; n)$ denotes worst-case computational step complexity on instances with n coordinates (equivalently, as a function of succinct input length L) in the fixed encoding regime. A *solution architecture* S partitions this baseline hardness into:

- $H_{\text{central}}(S)$: hardness paid once, at design time or in a shared component
- $H_{\text{distributed}}(S)$: hardness paid per use site

For n use sites, total realized hardness is:

$$H_{\text{total}}(S) = H_{\text{central}}(S) + n \cdot H_{\text{distributed}}(S)$$

Proposition 12.15 (Baseline Lower-Bound Principle). *[C:P12.15; R:H=cost-growth] For any problem family P measured by $H(P; n)$ above, any solution architecture S and any number of use sites $n \geq 1$, if $H_{\text{total}}(S)$ is measured in the same worst-case step units over the same input family, then:*

$$H_{\text{total}}(S) = H_{\text{central}}(S) + n \cdot H_{\text{distributed}}(S) \geq H(P; n).$$

For SUFFICIENCY-CHECK, Theorem 9.1 provides the baseline on the hard succinct family: $H(\text{SUFFICIENCY-CHECK}; n) = 2^{\Omega(n)}$ under ETH. (L: HD23, HD25)

Proof. By definition, $H(P; n)$ is a worst-case lower bound for correct solutions in this encoding regime and cost metric. Any such solution architecture decomposes total realized work as $H_{\text{central}} + n \cdot H_{\text{distributed}}$, so that total cannot fall below the baseline. ■

Definition 12.16 (Hardness Efficiency). The *hardness efficiency* of solution S with n use sites is:

$$\eta(S, n) = \frac{H_{\text{central}}(S)}{H_{\text{central}}(S) + n \cdot H_{\text{distributed}}(S)}$$

(L: HD11)

Proposition 12.17 (Efficiency Equivalence). *[C:P12.17; R:H=cost-growth] For fixed n and positive total hardness, larger $\eta(S, n)$ is equivalent to a larger central share of realized hardness.* (L: HD11)

Proof. From Definition 12.16, $\eta(S, n)$ is exactly the fraction of total realized hardness paid centrally. ■

Definition 12.18 (Right vs Wrong Hardness Placement). For a solution architecture S in this linear model:

$$\text{right hardness} \iff H_{\text{distributed}}(S) = 0, \quad \text{wrong hardness} \iff H_{\text{distributed}}(S) > 0.$$

(L: HD12-13)

Theorem 12.19 (Centralization Dominance). *Let $S_{\text{right}}, S_{\text{wrong}}$ be architectures over the same problem family with*

$$H_{\text{distributed}}(S_{\text{right}}) = 0, \quad H_{\text{central}}(S_{\text{right}}) > 0, \quad H_{\text{distributed}}(S_{\text{wrong}}) > 0,$$

and let $n > \max(1, H_{\text{central}}(S_{\text{right}}))$. Then:

1. Lower total realized hardness:

$$H_{total}(S_{\text{right}}) < H_{total}(S_{\text{wrong}})$$

2. Fewer error sites: errors in centralized components affect 1 location; errors in distributed components affect n locations
3. Quantified leverage: moving one unit of work from distributed to central saves exactly $n - 1$ units of total realized hardness

(L: [HD1-2](#))

Proof. (1) and (2) are exactly the bundled theorem [HD1](#). (3) is exactly [HD2](#). ■

Corollary 12.20 (Right-Place vs Wrong-Place Hardness). [*C*:[C12.20](#); *R*:*H=cost-growth*] In the linear model, a right-hardness architecture strictly dominates a wrong-hardness architecture once use-site count exceeds central one-time hardness. Formally, for architectures $S_{\text{right}}, S_{\text{wrong}}$ over the same problem family, if S_{right} has right hardness, S_{wrong} has wrong hardness, and $n > H_{\text{central}}(S_{\text{right}})$, then

$$H_{\text{central}}(S_{\text{right}}) + n H_{\text{distributed}}(S_{\text{right}}) < H_{\text{central}}(S_{\text{wrong}}) + n H_{\text{distributed}}(S_{\text{wrong}}).$$

(L: [HD19](#))

Proof. This is the mechanized theorem [HD19](#). ■

Proposition 12.21 (Dominance Modes). [*C*:[P12.21](#); *R*:*H=cost-growth*] This section uses two linear-model dominance modes plus generalized nonlinear dominance and boundary modes:

1. **Strict threshold dominance:** Corollary [12.20](#) gives strict inequality once $n > H_{\text{central}}(S_{\text{right}})$.
2. **Global weak dominance:** under the decomposition identity used in [HD3](#), centralized hardness placement is never worse for all $n \geq 1$.
3. **Generalized nonlinear dominance:** under bounded-vs-growing site-cost assumptions (Theorem [12.25](#)), right placement strictly dominates beyond a finite threshold without assuming linear per-site cost.
4. **Generalized boundary mode:** without those growth-separation assumptions, strict dominance is not guaranteed (Proposition [12.27](#)).

Proof. Part (1) is Corollary [12.20](#). Part (2) is exactly [HD3](#). Part (3) is Theorem [12.25](#). Part (4) is Proposition [12.27](#). ■

Illustrative Instantiation (Type Systems). Consider a capability C (e.g., provenance tracking) with one-time central cost H_{central} and per-site manual cost $H_{\text{distributed}}$:

Approach	H_{central}	$H_{\text{distributed}}$
Native type system support	High (learning cost)	Low (type checker enforces)
Manual implementation	Low (no new concepts)	High (reimplement per site)

The table is schematic; the formal statement is Corollary 12.22. The type-theoretic intuition behind this instantiation is consistent with classic accounts of abstraction and static interface guarantees [16, 72, 68, 47, 83, 30].

Corollary 12.22 (Type-System Threshold). $[C:C12.22;R:H=cost-growth]$ *For the formal native-vs-manual architecture instance, native support has lower total realized cost for all*

$$n > H_{\text{baseline}}(P),$$

where $H_{\text{baseline}}(P)$ corresponds to HD15. (L: HD15)

Proof. Immediate from HD15. ■

12.5 Extension: Non-Additive Site-Cost Models

Definition 12.23 (Generalized Site Accumulation). Let $C_S : \mathbb{N} \rightarrow \mathbb{N}$ be a per-site accumulation function for architecture S . Define generalized total realized hardness by

$$H_{\text{total}}^{\text{gen}}(S, n) = H_{\text{central}}(S) + C_S(n).$$

Definition 12.24 (Eventual Saturation). A cost function $f : \mathbb{N} \rightarrow \mathbb{N}$ is *eventually saturating* if there exists N such that for all $n \geq N$, $f(n) = f(N)$.

Theorem 12.25 (Generalized Dominance by Growth Separation). $[C:T12.25;R:H=cost-growth]$ *Let*

$$H_{\text{total}}^{\text{gen}}(S, n) = H_{\text{central}}(S) + C_S(n).$$

For two architectures $S_{\text{right}}, S_{\text{wrong}}$, suppose there exists $B \in \mathbb{N}$ such that:

1. $C_{S_{\text{right}}}(m) \leq B$ for all m (bounded right-side per-site accumulation),
2. $m \leq C_{S_{\text{wrong}}}(m)$ for all m (identity-lower-bounded wrong-side growth).

Then for every

$$n > H_{\text{central}}(S_{\text{right}}) + B,$$

one has

$$H_{\text{total}}^{\text{gen}}(S_{\text{right}}, n) < H_{\text{total}}^{\text{gen}}(S_{\text{wrong}}, n).$$

(L: HD9)

Proof. This is exactly the mechanized theorem HD9. ■

Corollary 12.26 (Eventual Generalized Dominance). $[C:C12.26;R:H=cost-growth]$ *If condition (1) above holds and there exists N such that condition (2) holds for all $m \geq N$, then there exists N_0 such that for all $n \geq N_0$,*

$$H_{\text{total}}^{\text{gen}}(S_{\text{right}}, n) < H_{\text{total}}^{\text{gen}}(S_{\text{wrong}}, n).$$

(L: HD10)

Proof. Immediate from HD10. ■

Proposition 12.27 (Generalized Assumption Boundary). $[C:P12.27;R:H=cost-growth]$ *In the generalized model, strict right-vs-wrong dominance is not unconditional. There are explicit counterexamples:*

1. *If wrong-side growth lower bounds are dropped, right-side strict dominance can fail for all n .*

2. If right-side boundedness is dropped, strict dominance can fail for all n even when wrong-side growth is linear.

(L: [HD7-8](#))

Proof. This is exactly the pair of mechanized boundary theorems listed above. ■

Theorem 12.28 (Linear Model: Saturation iff Zero Distributed Hardness). [*C:T12.28;R:H=cost-growth*] In the linear model of this section,

$$H_{total}(S, n) = H_{central}(S) + n \cdot H_{distributed}(S),$$

the function $n \mapsto H_{total}(S, n)$ is eventually saturating if and only if $H_{distributed}(S) = 0$. (L: [HD19](#))

Proof. This is exactly the mechanized equivalence theorem above. ■

Theorem 12.29 (Generalized Model: Saturation is Possible). [*C:T12.29;R:H=cost-growth*] There exists a generalized site-cost model with eventual saturation. In particular, for

$$C_K(n) = \begin{cases} n, & n \leq K \\ K, & n > K, \end{cases}$$

both C_K and $n \mapsto H_{central} + C_K(n)$ are eventually saturating. (L: [HD6](#), [HD20](#))

Proof. This is the explicit construction mechanized in Lean. ■

Corollary 12.30 (Positive Linear Slope Cannot Represent Saturation). [*C:C12.30;R:H=cost-growth*] No positive-slope linear per-site model can represent the saturating family above for all n . (L: [HD16](#))

Proof. This follows from the mechanized theorem that any linear representation of the saturating family must have zero slope. ■

Mechanized strengthening reference. The strengthened all-coordinates-relevant reduction is presented in Section 4 (“Mechanized strengthening”) and formalized in `Reduction_AllCoords.lean` via `all_coords_relevant_of_not_tautology`.

The next section develops the major practical consequence of this framework: the Simplicity Tax Theorem.

13 Redistribution Identity for Decision-Relevant Coordinates

Formally: this section defines expressive-gap tax and proves redistribution, growth, and amortization identities under fixed required-coordinate sets.

The load-bearing fact in this section is not the set identity itself; it is the difficulty of shrinking the required set $R(P)$ in the first place. By Theorem 4.6 (and Theorem 4.7 for minimization), exact relevance identification is intractable in the worst case under succinct encoding. The identities below therefore quantify how unresolved relevance is redistributed between central and per-site work. [*C:T4.6, 4.7;R:S*] For encoded physical systems, the same redistribution laws apply under physical-to-core transport: unresolved coordinates remain per-site external work, while resolved coordinates are paid once in centralized structure. [*C:P2.46;T13.3, 13.5, 13.6;R:AR, LC*] (L: [CT6](#), [HD4-5](#), [HD21](#), [HD26](#).)

Definition 13.1. Let $R(P)$ be the required dimensions (those affecting Opt) and $A(M)$ the dimensions model M handles natively. The *expressive gap* is $\text{Gap}(M, P) = R(P) \setminus A(M)$.

Definition 13.2 (Simplicity Tax). The *simplicity tax* is the size of the expressive gap:

$$\text{SimplicityTax}(M, P) := |\text{Gap}(M, P)|.$$

Theorem 13.3 (Redistribution Identity). $|\text{Gap}(M, P)| + |R(P) \cap A(M)| = |R(P)|$. *The total cannot be reduced, only redistributed between “handled natively” and “handled externally.”* (L: [HD5](#))

Proof. In the finite-coordinate model this is the exact set-cardinality identity

$$|R \setminus A| + |R \cap A| = |R|,$$

formalized as [HD5](#). ■

Remark 13.4 (Why this is nontrivial in context). The algebraic identity in Theorem [13.3](#) is elementary. Its force comes from upstream hardness: reducing $|R(P)|$ by exact relevance minimization is worst-case intractable under the succinct encoding, so redistribution is often the only tractable lever available.

Theorem 13.5 (Linear Growth). *For n decision sites:*

$$\text{TotalExternalWork} = n \times \text{SimplicityTax}(M, P).$$

(L: [HD26](#))

Proof. This is by definition of per-site externalization and is mechanized as [HD26](#). ■

Theorem 13.6 (Amortization). *Let H_{central} be the one-time cost of using a complete model. There exists*

$$n^* = \left\lfloor \frac{H_{\text{central}}}{\text{SimplicityTax}(M, P)} \right\rfloor$$

such that for $n > n^$, the complete model has lower total cost.* (L: [HD4](#))

Proof. For positive per-site tax, the threshold inequality

$$n > \left\lfloor \frac{H_{\text{central}}}{\text{SimplicityTax}} \right\rfloor \implies H_{\text{central}} < n \cdot \text{SimplicityTax}$$

is mechanized as [HD4](#). ■

Corollary 13.7 (Gap Externalization). *[C:C13.7;R:H=cost-growth] If $\text{Gap}(M, P) \neq \emptyset$, then external handling cost scales linearly with the number of decision sites.* (L: [HD21](#), [HD26](#))

Proof. The exact linear form is [HD26](#). When the gap is nonempty (positive tax), monotone growth with n is [HD21](#). ■

Corollary 13.8 (Exact Minimization Criterion). *[C:C13.8;R:H=succinct-hard] For mechanized Boolean-coordinate instances, “there exists a sufficient set of size at most k ” is equivalent to “the relevant-coordinate set has cardinality at most k .”* (L: [DP1](#), [SK1](#))

Proof. This follows from [DP1](#) and [SK1](#). ■

Informally: when exact minimization is hard, unresolved relevance shifts between centralized structure and repeated local work.

Appendix [A](#) provides theorem statements and module paths for the corresponding Lean formalization.

14 Related Work

14.1 Formalized Complexity and Mechanized Proof Infrastructure

Formalizing complexity-theoretic arguments in proof assistants remains comparatively sparse. Lean and Mathlib provide the current host environment for this artifact [21, 22, 90]. Closest mechanized precedents include verified computability/complexity developments in Coq and Isabelle [28, 43, 60, 59, 34], and certifying toolchain work that treats proofs as portable machine-checkable evidence [45, 57].

Recent LLM-evaluation work also highlights inference nondeterminism from numerical precision and hardware configuration, reinforcing the value of proof-level claims whose validity does not depend on stochastic inference trajectories [99].

14.2 Computational Decision Theory

The complexity of decision-making has been studied extensively. Foundational treatments of computational complexity and strategic decision settings establish the baseline used here [63, 5]. Our work extends this to the meta-question of identifying relevant information. Decision-theoretic and information-selection framing in this paper also sits in the tradition of statistical sufficiency and signaling/information economics [27, 87, 56, 53, 38].

Categorical framing of the quotient object. The universal property of the optimizer quotient object is not, by itself, a probabilistic construction. In the category **Set**, quotienting states by equality of Opt is the standard coimage construction for the optimizer map $\text{Opt} : S \rightarrow \mathcal{P}(A)$, canonically equivalent to its image/range [50]. The novelty in this paper is therefore not that coimages exist, but that this particular coimage organizes coordinate sufficiency, witness lower bounds, and physical collapse constraints in one mechanized chain. *[C:T1.4;R:DM](L: QT1, QT3, QT5.)*

Large-sample Bayesian-network structure learning and causal identification are already known to be hard in adjacent formulations [17, 82, 41]. Our object differs: we classify coordinate sufficiency for optimal-action invariance across static/stochastic/sequential regimes, with theorem-level placements and mechanized reductions.

Relation to prior hardness results. Closest adjacent results are feature-selection/model-selection hardness results for predictive subset selection [11, 3]. Relative to those works, this paper changes two formal objects: (i) the reductions are machine-checked (TAUTOLOGY and $\exists\forall$ -SAT mappings with explicit polynomial bounds), and (ii) the output is a regime-typed hardness/tractability map for decision relevance under explicit encoding assumptions. The target predicate here is decision relevance, not predictive compression.

Functional information and selection. Prior work defines function-relative information objects in physical systems. Szostak defines functional information as the negative log of the fraction of configurations achieving a specified function [89]. Wong et al. study function and selection in evolving systems and state an increasing-functional-information law under selection [97]. This paper addresses a complementary object: the computational decision predicate for when projected coordinates are sufficient to preserve optimal-action correspondence, with explicit complexity placements and mechanized proofs.

14.3 Succinct Representations and Regime Separation

Representation-sensitive complexity is established in planning and decision-process theory: classical and compactly represented MDP/planning problems exhibit sharp complexity shifts under different input models [64, 55, 48]. The explicit-vs-succinct separation in this paper is the same representation-sensitive phenomenon for the coordinate-sufficiency predicate.

The distinction in this paper is the object and scope of the classification: we classify *decision relevance* (sufficiency, anchor sufficiency, and minimum sufficient sets) for a fixed decision relation, with theorem-level regime typing and mechanized reduction chains.

14.4 Oracle and Query-Access Lower Bounds

Query-access lower bounds are a standard source of computational hardness transfer [24, 15]. Our [Q_fin] and [Q_bool] results are in this tradition, but specialized to the same sufficiency predicate used throughout the paper: they establish access-obstruction lower bounds while keeping the underlying decision relation fixed across regimes.

Complementary companion work studies zero-error identification under constrained observation and proves rate-query lower bounds plus matroid structure of minimal distinguishing query sets [84]. Our object here is different: sufficiency of coordinates for optimal-action invariance in a decision problem, with regime-map placement (coNP/PP/PSPACE), static inner decomposition (coNP/ Σ_2^P decision layer plus access/structure layers), and regime-typed transfer theorems.

14.5 Energy Landscapes and Chemical Reaction Networks

The geometric interpretation of decision complexity connects to the physics of energy landscapes. Wales established a widely used picture in which molecular configurations form high-dimensional landscapes with basins and transition saddles [92, 93, 94]. Protein-folding theory uses the same landscape language via funnel structure toward native states [62]. In this paper’s formal model, the corresponding core object is coordinate-projection invariance of the optimizer map: sufficiency asks whether fixing coordinates preserves Opt, while insufficiency is witnessed by agreeing projections with different optimal sets (Definition 2.4, Proposition 2.6).
[C:D2.4;P2.6;R:DM](L: DP7-8.)

Section 2.8 makes this transfer rule theorem-level: encoded physical claims are derived from core claims through an explicit assumption-transfer map, and an encoded physical counterexample refutes the corresponding core rule on that encoded slice.
[C:P2.46;C2.47;R:AR](L: CT5-8.)

Under that mapping, the explicit-vs-succinct split is the mechanized statement of access asymmetry: [E] admits polynomial verification on explicit state access, while [S+ETH] admits worst-case exponential lower bounds on succinct inputs (Theorem 9.1); finite query access already exhibits indistinguishability barriers (Proposition 9.3). The complexity separation is mechanized.
[C:T9.1;P9.3;R:E, S+ETH, Q_fin](L: CC16, CC23-24, CC50.)

The geometric core is mechanized directly: product-space slice cardinality, hypercube node-count identity, and node-vs-edge separation are proved in Lean at the decision abstraction level, with edge witnesses tied exactly to non-sufficiency.
(L: DG5-6, DG15-16, DG18.)

The circuit-to-decision instantiation is also mechanized in a separate bridge layer: geometry and dynamics are represented as typed components of a circuit object; decision circuits add an explicit interpretation layer; molecule-level constructors instantiate both views.
(L: IN4-6, IN13-16.)

Chemical reaction networks provide a concrete physical class where this encoding applies directly. Prior work establishes hardness for output optimization, reconfiguration, and reconstruction in CRN settings [4, 2, 26]. Those domain-specific CRN complexity results are external literature claims, not mechanized in this Lean artifact. The mechanized statement here is the transfer principle: once

a CRN decision task is encoded as a C1–C4 decision problem, it inherits the same sufficiency hardness/tractability regime map (Theorems 11.2, 4.6, 10.1).*[C:T11.2, 4.6, 10.1;R:E, S, TA](L: CR1, CC61, CC69–72.)*

The physical bridge is not a single-domain add-on: Theorem 2.49 composes law-objective semantics, reduction equivalence, and hardness lower-bound accounting in one mechanized theorem, so physical instantiation and complexity claims share one proof chain.*[C:T2.49;R:AR](L: CT4.)*

For matter-only dynamics, the law-induced objective specialization is mechanized for arbitrary universes (arbitrary state/action types and transition-feasibility relations): the utility schema is induced from transition laws, and under a strict allowed-vs-blocked gap with nonempty feasible set, the optimizer equals the feasible-action set; with logical determinism at the action layer, the optimizer is singleton.*[C:P2.48;R:AR](L: DQ5–8.)*

The interface-time semantics are also explicit and mechanized: timed states are N-indexed, one decision event is exactly one tick, run length equals elapsed interface time, and this unit-time law is invariant across substrate tags under interface-consistency assumptions.*[C:P2.38, 2.39, 2.40, 2.41;R:AR](L: DT6, DT8, DT10, DT13, DT16, DT24.)*

Regime-side interface machinery is also formalized as typed access objects with an explicit simulation preorder and certificate-upgrade statements. The access-channel and five-way composition theorems are encoded as assumption-explicit composition laws.*[C:P2.46;T2.49;R:AR](L: ARG2–3, ARG6, ARG12–13, ARG17, ARG19.)*

14.6 Thermodynamic Costs and Landauer’s Principle

The thermodynamic lift in Section 9 converts bit-operation lower bounds into energy/carbon lower bounds via linear conversion constants. In this artifact, those conversion consequences are mechanized with explicit premises through Propositions 9.28, 9.29, 9.30, and 9.31. Landauer’s principle and reversible-computation context [44, 8] define a minimum floor for irreversible bit erasure, not a generally tight expression for the actual energetic cost of constrained computation. The core stochastic-thermodynamics review literature [91] supplies the ensemble- and trajectory-level entropy-production framework used to state such refinements. Wolpert et al. [96] identify the relevant constrained-computation regimes: periodic driving, modular organization, finite-time operation, and input mismatch. Manzano et al. [51] make stopping-time and absolute-irreversibility effects explicit and derive universal dissipation bounds in that setting. Yadav, Yousef, and Wolpert [98] connect mismatch cost to repeated circuit operation and structural resource lower bounds. Our formal model now represents those inputs at three levels. The coarse interface WC1–WC5 records the Landauer floor plus a generic nonnegative overhead. The refined decomposition WP1–WP9 separates that overhead into mismatch and residual terms. Within that refined layer, the mismatch branch is no longer only imported: WM1–WM4 derive nonnegativity, exact vanishing, strict positivity under explicit finite-distribution mismatch, and the conservative nat-valued lower-bound bridge from the existing KL machinery, while WM5–WM6 inject that derived branch into the Wolpert decomposition and prove strict separation above the Landauer floor. The finite discrete residual branch is now also theorem-level and exhaustive at the local edge level: WR1–WR3 give the bidirectional asymmetry witness, WR6 performs the reverse-edge case split, WR7–WR8 inject the resulting unified finite residual witness into the Wolpert decomposition, and WR9–WR10 repackage the same argument as a single process-level witness theorem for finite computational-state systems. Broader stopping-time / absolute-irreversibility residual regimes remain represented by the separate cited premise WP5. This keeps the causal structure of the cited work explicit while limiting the mechanization claim to what this artifact actually proves: the mismatch branch is derived, a finite discrete residual branch is derived by an exhaustive edge split and repackaged at process level, broader stopping-time resid-

ual regimes remain imported, and the composition layer is fully machine-checked. *[C:P9.28, 9.29, 9.30, 9.31;R:AR](L: CC64-68, WC1-5, WM1-6, WR1-10, WP1-3, WP5-9.)*

Physics assumptions decompose into regime tags. The paper does not create domain-specific tags for physics assumptions. Instead, every physics assumption decomposes into the same regime tags used throughout the paper. This uniformity states that physics assumptions are not epistemically privileged; they are declared assumptions, tracked with the same discipline as the paper’s complexity-theoretic assumptions. The explicit physical-to-core mapping does not make the results less physical. It standardizes premise exposure and transfer obligations: accepted physics assumptions and this model are handled under the same declared-assumption framework, then checked with the same mechanized proof discipline where formalized.

Table 1: Physics-Assumption-to-Regime-Tag Decomposition

Physics Assumption	Regime Decomposition	Justification
Landauer bound ($E \geq kT \ln 2$)	[AR], [H=cost-growth]	Declared law; yields cost lower bounds
Lorentz invariance	[AR]	Declared physical symmetry
Discrete state space	[Q_fin], [AR]	Finite-state core; declared assumption
Thermodynamic constants (λ, ρ)	[AR], [H=cost-growth]	Declared conversion model
Planck scale (ℓ_P)	[AR]	Declared physical constant
Physical-to-core encoding (E)	[AR]	Declared encoding map

The decomposition in Table 1 makes explicit what the paper already practices: physics assumptions are declared premises in the [AR] (axiomatic regime) sense, and when they yield cost lower bounds, they inherit [H=cost-growth]. The finite-state query core [Q_fin] captures the discrete-state assumption underlying the computational model. No physics assumption receives a special domain tag; all decompose into the existing regime taxonomy. This is a standardization claim about proof form, not a change in physical content. *[C:P9.28, 9.52, 9.34, 9.33;T2.49;R:AR, H=cost-growth, Q_fin]*

Coupling this to integrity yields a mechanized policy condition: when exact competence is hardness-blocked in the active regime, integrity requires abstention or explicitly weakened guarantees (Propositions 3.61, 3.60). A stronger behavioral claim such as “abstention is globally utility-optimal for every objective” is *not* currently mechanized and remains objective-dependent. *[C:P3.61, 3.60;R:AR](L: IC6-7, IC24, IC26-27, CC30.)*

14.7 Neukart–Vinokur Thermodynamic Duality

Neukart and Vinokur [58] propose a thermodynamic duality $dU = T dS - p dV + \lambda dC$ where C is a “complexity coordinate” associated with computational irreversibility in spin-glass systems. Corollary 9.32 derives their duality as a specialization of our thermodynamic lift (Proposition 9.28) for $\lambda > 0$. *[C:C9.32;P9.28;R:AR](L: CC50, CC65.)*

Their spin-glass phase transitions become regime-specific instances: our [S+ETH] hard families with $k^* = n$ witness their “complexity potential” via explicit TAUTOLOGY reduction (Theorem 9.1). Physical transport via $E : \mathcal{P} \rightarrow \mathcal{D}$ extends to arbitrary substrates, not only quantum or spin systems (Theorem 2.49). *[C:T9.1, 2.49;R:S+ETH, AR](L: CC16, CT4.)*

Thus Neukart–Vinokur $\subseteq$ [AR] regime: $\lambda, \rho > 0$, explicit encoding $E : \mathcal{P} \rightarrow \mathcal{D} \Rightarrow$ their thermodynamic duality holds by assumption transfer through our coNP-complete core (Proposition 2.46). *[C:P2.46;R:AR](L: CT6.)*

14.8 Feature Selection

In machine learning, feature selection asks which input features are relevant for prediction. This is known to be NP-hard in general [11, 33]. Our results place the decision-theoretic analog in a regime map: static checking/minimization at **coNP**-complete, stochastic sufficiency at PP-complete, and sequential sufficiency at PSPACE-complete. *[C:T4.6, 4.7, 6.7, 7.7;R:S, RG](L: CC36, CC61, DC9-10.)*

14.9 Value of Information

The value of information (VOI) framework [35, 71] quantifies the maximum rational payment for information. Our work addresses a different question: not the *value* of information, but the *complexity* of identifying which information has value.

14.10 Model Selection

Statistical model selection (AIC/BIC/cross-validation) provides practical heuristics for choosing among models [1, 77, 88]. Our results formalize the regime-level reason heuristic selection appears: without added structural assumptions, exact optimal model selection inherits worst-case intractability, so heuristic methods implement explicit weakened-guarantee policies for unresolved structure.

[C:T4.6, 9.1, 10.1;R:E, S, S+ETH](L: CC16, CC61, CC69-71.)

14.11 Information Compression vs Certification Length

The paper now makes an explicit two-part information accounting object (Definition 3.45): raw report bits and evidence-backed certification bits. This sharpens the information-theoretic reading of typed claims: compression of report payload is not equivalent to compression of certifiable truth conditions [78, 19, 85].

From a zero-error viewpoint, this distinction is structural rather than stylistic: compressed representations can remain short while exact decodability depends on confusability structure and admissible decoding conditions [79, 42, 49, 20]. The same separation appears here between short reports and evidence-backed exact claims.

The rate-distortion and MDL lines make the same split in different language: there is a difference between achievable compression rate, computational procedure for obtaining/optimizing that rate, and certifiable model adequacy [80, 10, 73, 32, 46].

Formally, Theorem 3.47 and Corollary 3.49 characterize the exact-report boundary: admissible exact reports require a strict certified-bit gap above raw payload, while hardness-blocked exact reports collapse to raw-only accounting. This directly links representational succinctness to certification cost under the declared contract, rather than treating “short reports” as automatically high-information claims. *[C:T3.47;P3.46;C3.49;R:AR](L: CC11, CC18, CC21.)*

14.12 Certifying Outputs and Proof-Carrying Claims

Our integrity layer matches the certifying-algorithms pattern: algorithms emit candidate outputs together with certificates that can be checked quickly, separating *producing* claims from *verifying* claims [52, 57]. In this paper, Definition 3.8 is exactly that soundness discipline.

At the systems level, this is the same architecture as proof-carrying code: a producer ships evidence and a consumer runs a small checker before accepting the claim [57, 52]. Our competence definition adds the regime-specific coverage/resource requirement that certifying soundness alone does not provide.

The feasibility qualifier in Definition 3.9 yields the bounded-rationality constraint used in this paper: admissible policy is constrained by computational feasibility under the declared resource model (Remark 2.35) [95, 5]. *[C:P3.58;R:AR] (L: IC18.)*

Cryptographic-verifiability perspective (scope). The role of cryptography in this paper is structural verifiability, not secrecy: the relevant analogy is certificate-carrying claims with lightweight verification, not confidential encoding [31, 57]. Concretely, the typed-report discipline is modeled as a game-based reporting interface: a prover emits (r, π) and a verifier accepts exactly when π certifies the declared report type under the active contract. In that model, Evidence–Admissibility Equivalence gives completeness for admissible report types, while Typed Claim Admissibility plus Exact Claims Require Exact Evidence give soundness against exact overclaiming. Certified-confidence constraints then act as evidence-gated admissibility constraints on top of that verifier interface, so raw payload bits and certified bits represent distinct resources rather than stylistic notation variants. *[C:P3.18, 3.16, 3.21, 3.23;R:AR](L: IC2, IC35, IC38, CC75.)*

Three-axis integration. In the cited literature, these pillars are treated separately: representation-sensitive hardness, query-access lower bounds, and certifying soundness disciplines. This paper composes all three for the same decision-relevance object in one regime-typed and machine-checked framework. *[C:T9.1;P9.3, 3.16;R:S+ETH, Qf, AR](L: CC16, CC50, CC75.)*

15 Conclusion

Formally: this section composes proved regime placements, bridge theorems, and claim-typing constraints into final landscape consequences.

This paper gives a regime-first complexity map: static/stochastic/sequential placements with strict hierarchy (DC9, DC10, DC1, DC2), static inner-class decomposition by decision/access/structure layers (CCC6, CCC5, CCC1, CCC3, CCC7, CC61), theorem-level physical transport plus exact represented-adjacent bridge boundary (CT4, CC67, CC69, CC85), Bayesian uniqueness under DQ/thermodynamic admissibility axioms (FN14), physical-collapse no-go transfer chain (PH11, PH12, PH13, PH14, PH15), derived SAT-bridge map plus reduction-energy transfer and disjunctive assumption boundary (PH16, PH17, PH18, PH19, PH20, PH21, PH22, PH26, PH27), explicit finite-budget-necessity countermodels (PH31, PH32, PH33), regime-robust deterministic/probabilistic/sequential no-collapse specializations (PH28, PH29, PH30), observer-collapse anchor closure across stochastic/sequential checks (PA1, PA2, PA3, PA4, PA5, PA6, PA7, PA8, PA9), recurrent conversation-layer clamp/report bridge (CV4, CV7, CV17, CV18, CV13, CV16), explicit Neukart–Vinokur duality specialization from the thermodynamic lift (CC77, CC76), and formal centralized/distributed hardness thresholds through dominance and amortization theorems (HD1, HD4, HD3). *[C:T4.6, 4.7, 4.9, 6.7, 7.7, 9.1, 10.1, 2.49, 2.58, 2.50, 3.4, 9.50, 12.19, 13.3, 13.6;P9.3, 9.28, 8.1, 8.2;C9.32;R:E, S, S+ETH, Qf, Qb, AR, RG, LC, H=cost-growth]*

Informally: this conclusion combines previously proved blocks under explicit tags.

Methodology and Disclosure

Role of LLMs in this work. This paper was developed through human-AI collaboration. The author provided the core intuitions: the connection between decision-relevance and computational complexity, the conjecture that SUFFICIENCY-CHECK is coNP-complete, and the insight that the Σ_2^P structure collapses for MINIMUM-SUFFICIENT-SET. Large language models (Claude, GPT-4) served as implementation partners for proof drafting, Lean formalization, and L^AT_EX generation.

The Lean 4 proofs were iteratively refined: the author specified the target statements, the LLM proposed proof strategies, and the Lean compiler served as the arbiter of correctness. For complexity-theoretic reductions, the Lean formalization machine-verifies *correctness* (membership preservation); polynomial-time computability of the reduction functions is a standard claim stated as an explicit hypothesis, consistent with the state of formal verification where end-to-end Turing-machine polytime proofs for concrete reductions remain an open problem.

What the author contributed: The informal problem formulations (SUFFICIENCY-CHECK, MINIMUM-SUFFICIENT-SET, ANCHOR-SUFFICIENCY), the informal hardness conjectures, the informal tractability conditions, and the connection to over-modeling in engineering practice.

What LLMs contributed: L^AT_EX drafting, Lean tactic exploration, reduction construction assistance, and prose refinement.

Reduction correctness is machine-checked; polynomial-time computability is a standard hypothesis. This methodology is disclosed for academic transparency.

Acknowledgments

This work depends on results whose authors made possible the insights that could then be machine-verified.

Fundamental physics. Albert Einstein established special relativity and the light-speed bound [25]; Hermann Minkowski gave it the spacetime formulation [54]. Max Planck established energy quantization and the constant k_B [70]. Rolf Landauer proved that erasure costs thermodynamic work [44]; Charles Bennett showed that reversible computation circumvents this bound [7]. Bérut et al. experimentally verified Landauer’s principle [9]. The counting gap theorem (BA10) proves that bounded systems must obey finite acquisition budgets from pure mathematics; relativity then supplies the numerical propagation scale c in our universe. The Wolpert-facing constrained-process interface now makes a sharper point explicit in the formal system: Landauer gives a universal floor, mismatch and residual dissipation are represented as separate theorem-level terms above that floor, the mismatch branch is derived from finite-distribution KL divergence as far as the present mathematics supports, and the finite discrete residual branch is derived by an exhaustive reverse-edge split: either asymmetric bidirectional edge flow yields positive two-point KL divergence, or vanishing reverse flow yields an irreversible one-way state transition with strictly positive Landauer-scaled cost. Any structural resource that is absorbed by mismatch further tightens the induced energy lower bound. Broader stopping-time / absolute-irreversibility residual regimes remain separate cited premises. The machine-checked part is the composition from that mixed theorem-plus-premise interface to the paper’s decision-theoretic lower bounds. [WC1WC2WC3WC4WC5WM1WM2WM3WM4WM5WM6WR1WR2WR3WR4WR5WR6WR7WR8WP1WP2WP5WP7WP8WP9](#)

Proof infrastructure. The Lean 4 proof assistant [22] served as the arbiter of correctness. Leonardo de Moura, Sebastian Ullrich, and the Lean 4 team built a tool that transforms mathematical claims into mechanically verified theorems. The Mathlib library [90] provided the formal infrastructure: order theory, linear algebra, complexity-theoretic definitions, and the lemmas that made the formalization possible.

Methodological note. Reduction correctness is machine-checked; polynomial-time computability of reduction functions is a standard claim stated as an explicit hypothesis. The human author contributed the informal problem formulations, informal hardness conjectures, informal tractability conditions, and the connection to over-modeling in engineering practice. The machine

contributors (Claude, GPT-4, Lean 4) served as proof assistants, tactic explorers, and drafting partners. The final arbiter was the Lean compiler: if it accepts the proof, the theorem holds.

Summary of Results

This paper establishes a complete regime map for coordinate sufficiency and a static inner decomposition:

- **Regime map (core classes).** Static, stochastic, and sequential sufficiency-check questions are placed at `coNP`, `PP`, and `PSPACE` respectively, with strict $\text{static} \subset \text{stochastic} \subset \text{sequential}$ separation under declared assumptions (Theorems 4.6, 6.7, 7.7; Propositions 8.1, 8.2).
- **Observer-collapse anchor closure.** Under declared observer-collapse hypotheses, stochastic/sequential anchor checks are forced by effective-state singleton structure and seeded support conditions (PA1, PA4, PA5, PA6, PA7, PA9).
- **Static decision layer decomposition.** Within the static class, `SUFFICIENCY-CHECK` and `MINIMUM-SUFFICIENT-SET` are `coNP`-complete, while `ANCHOR-SUFFICIENCY` is Σ_2^P -complete (Theorems 4.6, 4.7, 4.9).
- **Static access layer decomposition.** An encoding-regime separation contrasts explicit-state polynomial-time (polynomial in $|S|$) with succinct `ETH` worst-case lower bounds, and query-access obstruction cores refine this with value-entry/state-batch/oracle-lattice transfers (Theorem 9.1; Propositions 9.3–9.16).
- **Static structure layer decomposition.** Six structural subclasses provide certified polynomial recovery conditions inside the static class (Theorem 10.1).
- Discrete-time interface semantics are formalized: time is $\mathbb{N}$ -valued, decision events are exactly one-tick transitions, run length equals elapsed time, and substrate-consistent realizations preserve the same one-unit event law (Propositions 2.38–2.41)
- Physical-system transport is formalized: encoded physical claims are derived from core rules through explicit assumption transfer, physical counterexamples map to encoded core failures, and the bundled bridge theorem composes law semantics, reduction equivalence, and required-work lower bounds (Proposition 2.46, Corollary 2.47, Theorem 2.49)
- Exact represented-adjacent bridge boundary: transfer is licensed iff the class is one-step deterministic; represented horizon, stochastic-criterion, and transition-coupled classes have explicit bridge-failure witnesses (Theorem 2.58)
- Exact-report admissibility is characterized by a certified-bit gap criterion; when exact reporting is blocked, exact accounting collapses to raw-only (Theorem 3.47, Corollary 3.49)
- The thermodynamic lift yields the Neukart–Vinokur duality specialization $dU \geq \lambda dC$ for $\lambda > 0$ (Corollary 9.32, Proposition 9.28)

[C:T4.6, 4.7, 4.9, 6.7, 7.7, 9.1, 10.1, 2.49, 2.58, 3.47;P8.1, 8.2, 9.3, 9.7, 9.11, 9.8, 9.15, 9.16, 2.38, 2.39, 2.40, 2.41, 2.46, 9.28, 3.46;C2.47, 3.49, 9.32;R:E, S, S+ETH, Qf, Qb, AR, RG, H=cost-growth] (L: CCC1, CCC3, CCC5-7, DC1-2, DC9-10,

[CC11](#), [CC18](#), [CC21](#), [CC27-29](#), [CC43](#), [CC52](#), [CC61](#), [CC67](#), [CC69](#), [CC76-77](#), [CC85](#), [DT6](#), [DT13](#), [DT16](#), [DT24](#), [CT4](#), [CT6](#), [CT8](#).)

These results place decision-relevant coordinate identification into a regime-indexed class ladder, rather than a single-class statement: static (coNP/Σ_2^P subproblems), stochastic (PP), and sequential (PSPACE), with explicit strict separations and bridge gates.

Beyond classification, the paper contributes a formal claim-typing framework (Section 3): structural complexity is a property of the fixed decision question, while representational hardness is regime-conditional access cost. This is why encoding-regime changes can move practical hardness without changing the underlying semantics. [*C:P2.21;T3.4;R:AR*] (L: [CC62](#), [CC76](#).)

The reduction constructions and key equivalence theorems are machine-checked in Lean 4 (see Appendix A for proof listings). The formalization verifies TAUTOLOGY and $\exists\forall$ -SAT static reductions, theorem-level stochastic/sequential class placements, and strict regime-separation claims; complexity classifications (coNP , Σ_2^P , PP, PSPACE) follow by composition with standard complexity-theoretic results under the declared assumptions. The strengthened gadget showing that non-tautologies yield instances with *all coordinates relevant* is also formalized. [*C:T4.6, 4.7, 4.9, 6.7, 7.7, 9.1;P8.1, 8.2;R:S, S+ETH, RG*] (L: [CCC1](#), [CCC3](#), [CCC5-6](#), [DC1-2](#), [DC9-10](#).)

Complexity Characterization

The results provide precise complexity characterizations within the formal model:

1. **Exact bounds.** SUFFICIENCY-CHECK is coNP -complete, both coNP -hard and in coNP .
2. **Constructive reductions.** The reductions from TAUTOLOGY and $\exists\forall$ -SAT are explicit and machine-checked.
3. **Encoding-regime separation.** Under [E], SUFFICIENCY-CHECK is polynomial in $|S|$. Under [S+ETH], there exist succinct worst-case instances (with $k^* = n$) requiring $2^{\Omega(n)}$ time. Under [Q_{fin}], the Opt-oracle core has $\Omega(|S|)$ worst-case query complexity (Boolean instantiation $\Omega(2^n)$), and under [Q_{bool}] the value-entry/state-batch refinements preserve the obstruction with weighted-cost transfer closures.

[*C:T4.6, 9.1;P9.3, 9.7, 9.11, 9.8;R:E, S, S+ETH, Qf, Qb*] (L: [CC16](#), [CC27-29](#), [CC50](#), [CC61](#).)

The Complexity Redistribution Corollary

Section 13 develops a quantitative consequence: when a problem requires k dimensions and a model handles only $j < k$ natively, the remaining $k - j$ dimensions must be handled externally at each decision site. For n sites, total external work is $(k - j) \times n$. [*C:T13.5;R:LC*] (L: [HD26](#).)

The set identity is elementary; its operational content comes from composition with the hardness results on exact relevance minimization. This redistribution corollary is formalized in Lean 4 (`HardnessDistribution.lean`), proving:

- Redistribution identity: complexity burden cannot be eliminated by omission, only moved between native handling and external handling
- Dominance: complete models have lower total work than incomplete models
- Amortization: there exists a threshold n^* beyond which higher-dimensional models have lower total cost

[*C:T13.3, 12.19, 13.6;R:LC*] (L: [HD1](#), [HD4-5](#), [HD21](#), [HD26](#).)

Open Questions

The landscape above is complete for the static sufficiency class under C1–C4; the items below are adjacent-class extensions or secondary refinements. Several questions remain for future work:

- **Fixed-parameter tractability (primary):** Is SUFFICIENCY-CHECK FPT when parameterized by the minimal sufficient-set size k^* , or is it W[2]-hard under this parameterization?
- **Adjacent-objective bridge frontier (beyond proven regime cores):** Characterize the exact frontier where adjacent sequential objectives reduce to the static class via Proposition 2.52, and where genuinely new complexity objects (e.g., horizon/sample/regret complexity) must replace the present static coNP/ Σ_2^P layer.
- **Average-case complexity:** What is the complexity under natural distributions on decision problems?
- **Typed-confidence calibration:** For signaled reports $(r, g, p_{\text{self}}, p_{\text{cert}})$, characterize proper scoring and calibration objectives that preserve signal consistency and typed admissibility.
- **Learning cost formalization:** Can central cost H_{central} be formalized as the rank of a concept matroid, making the amortization threshold precisely computable?

Practical Corollaries

The practical corollaries are regime-indexed and theorem-indexed:

- **[E] and structured regimes:** polynomial-time exact procedures exist (Theorem 10.1).
- **[Q_fin]/[Q_bool] query-access lower bounds:** worst-case Opt-oracle complexity is $\Omega(|S|)$ in the finite-state core (Boolean instantiation $\Omega(2^n)$), and value-entry/state-batch interfaces satisfy the same obstruction in the mechanized Boolean refinement (Propositions 9.3–9.16), with randomized weighted robustness and oracle-lattice closures.
- **[S+ETH] hard families:** exact minimization inherits exponential worst-case cost (Theorem 9.1 together with Theorem 4.6).
- **[S_bool] mechanized criterion:** minimization reduces to relevance-cardinality constraints (Corollary 12.6).
- **Redistribution consequences:** omitted native coverage externalizes work with explicit growth/amortization laws (Theorems 13.3–13.6).
- **Physical-system instantiation:** once a physical class is encoded into the declared decision interface, the same regime-typed hardness/tractability map and integrity constraints apply to that class (Section 2.8).[C:P2.46;C2.47;T2.49;R:AR](L: CT4, CT6, CT8.)

Physical Scope Guard. All transfer claims in this paper are typed through explicit physical encodings, declared observables, and declared assumption maps. Transfer is admitted only on declared physical instances satisfying the physical scope gate.[C:D2.42, 2.43;P2.44, 2.46;R:AR]

Hence the design choice is typed: enforce a tractable regime, or adopt weakened guarantees with explicit verification boundaries.[C:T10.1, 9.1;R:RG](L: CC16, CC71.) Equivalently, under the attempted-competence matrix, over-specification is rational only in attempted-competence-failure cells; once

exact competence is available in the active regime (or no attempted exact competence was made), persistent over-specification is irrational for the same verified-cost objective.*[C:P3.60;R:AR](L: IC27.)* By Proposition 3.60, the integrity-preserving matrix contains 3 irrational cells and 1 rational cell.*[C:P3.60;R:AR](L: IC27.)*

Measurement-Constrained Reporting for Physical Solvers

Within this paper’s formalism, reporting correctness is a claim-discipline property: outputs are asserted only when certifiable, and otherwise emitted as abstain reports (optionally carrying tentative guesses and self-reflected confidence under zero certified confidence). Formally, this is the integrity/competence split of Section 3, i.e., an evidence-typing statement for reported claims under declared regimes.*[C:D3.8, 3.9;P3.58;R:TR](L: IC18.)*

Formally: the abstention doctrine is a typed-report consequence of integrity/competence and evidence-gating predicates.

Truth-Objective Abstention Requirement. If a physical solver system declares a truth-preserving objective for claims about relation $\mathcal{R}$ in regime Γ , uncertified outputs are admissible only as ABSTAIN/UNKNOWN rather than asserted answers. Under the signal discipline, this includes abstain-with-guess/self-confidence reports while keeping certification evidence-gated.*[C:D3.19, 3.20;P3.23, 3.29;C3.25;R:AR](L: IC38, IC41, IC46.)* Equivalently, reporting protocols that suppress abstention in uncertified regions violate the typed claim discipline under the declared task model.*[C:D3.8;P3.58, 3.61, 3.60;R:AR](L: IC18, IC27, CC30.)*

This yields an output-control doctrine for engineered solver systems under the task/regime contract:

1. Treat ABSTAIN/UNKNOWN as first-class admissible outputs, not errors.*[C:D3.8;R:TR]*
2. For signaled reports, maintain two channels: self-reflected confidence may be emitted, but certified confidence is positive only with evidence.*[C:D3.20;P3.23, 3.30;R:AR](L: IC38, IC47.)*
3. Treat runtime disclosure as scalar-witnessed: always emit exact step-count, and emit completion percentage only when a declared bound exists.*[C:P3.26, 3.27, 3.28;R:AR](L: IC35, IC42, IC44.)*
4. Optimize competent coverage subject to integrity constraints, rather than optimizing answer-rate alone.*[C:D3.9;P3.58;R:AR](L: IC18.)*
5. In hardness-blocked regimes, abstention or explicit guarantee-weakening is structurally required unless regime assumptions are changed.*[C:T9.1;P3.61, 3.60;R:S, S+ETH](L: CC16, CC30, IC27.)*

Operationally, a reporting schema consistent with this framework tracks at least five quantities: integrity violations, competent non-abstaining coverage on the scoped family, abstention quality under regime shift, the calibration gap between self-reflected and certified confidence channels, and scalar runtime witness distributions (with bound-qualified fraction semantics when bounds are given). This is consistent with bounded-rationality feasibility discipline under bounded resource models [95, 5].

Informally: if exact support is not certified, do not make exact claims; when competence is unavailable, abstain and keep confidence evidence-linked.

A Lean 4 Proof Listings

The complete Lean 4 formalization is available at the Zenodo DOI listed on the title page. The mechanization consists of 45442 lines across 176 files, with 1823 theorem/lemma statements.

Handle IDs. Inline theorem metadata now cites compact IDs (for example, HD6, CC12, IC4) instead of full theorem constants. The full ID-to-handle mapping is listed in Section A.1.

A.1 Lean Handle ID Map

ID	Lean Handle / Source	ID	Lean Handle / Source
AB1	DecisionProblem.not_preservesOpt_iff_erasesDecisionRelevantDistinction DecisionQuotient/AbstractionCollapse.lean	AB2	DecisionProblem.surjective_abstraction_factors_or_erases DecisionQuotient/AbstractionCollapse.lean
AB3	DecisionProblem.collapseBeyondQuotient_physically_impossible DecisionQuotient/AbstractionCollapse.lean	AB4	DecisionProblem.surjective_abstraction_with_feasible_collapse_map_factors DecisionQuotient/AbstractionCollapse.lean
AC1	ClaimClosure.AtomicCircuitExports.AC1 DecisionQuotient/ClaimClosure.lean	AC3	ClaimClosure.AtomicCircuitExports.AC3 DecisionQuotient/ClaimClosure.lean
AC4	ClaimClosure.AtomicCircuitExports.AC4 DecisionQuotient/ClaimClosure.lean	AC5	ClaimClosure.AtomicCircuitExports.AC5 DecisionQuotient/ClaimClosure.lean
AC6	ClaimClosure.AtomicCircuitExports.AC6 DecisionQuotient/ClaimClosure.lean	AC8	ClaimClosure.AtomicCircuitExports.AC8 DecisionQuotient/ClaimClosure.lean
AC9	ClaimClosure.AtomicCircuitExports.AC9 DecisionQuotient/ClaimClosure.lean	AC11	ClaimClosure.AtomicCircuitExports.AC11 DecisionQuotient/ClaimClosure.lean
AN3	Physics.AssumptionNecessity.physical_claim_requires_physical_assumption DecisionQuotient/Physics/ AssumptionNecessity.lean	AN4	Physics.AssumptionNecessity.physical_claim_requires_empirically_justified_physical_assumption DecisionQuotient/Physics/ AssumptionNecessity.lean
AQ1	ClaimClosure.AQ1 DecisionQuotient/ClaimClosure.lean	AQ2	ClaimClosure.AQ2 DecisionQuotient/ClaimClosure.lean
AQ3	ClaimClosure.AQ3 DecisionQuotient/ClaimClosure.lean	AQ4	ClaimClosure.AQ4 DecisionQuotient/ClaimClosure.lean
AQ5	ClaimClosure.AQ5 DecisionQuotient/ClaimClosure.lean	AQ6	ClaimClosure.AQ6 DecisionQuotient/ClaimClosure.lean
AQ7	ClaimClosure.AQ7 DecisionQuotient/ClaimClosure.lean	AQ8	ClaimClosure.AQ8 DecisionQuotient/ClaimClosure.lean
ARG2	PhysicalComplexity.AccessRegime.AccessRegime DecisionQuotient/Physics/AccessRegime.lean	ARG3	PhysicalComplexity.AccessRegime.RegimeEval DecisionQuotient/Physics/AccessRegime.lean
ARG6	PhysicalComplexity.AccessRegime.RegimeWithCertificate DecisionQuotient/Physics/AccessRegime.lean	ARG12	PhysicalComplexity.AccessRegime.AuditableWithCertificate DecisionQuotient/Physics/AccessRegime.lean
ARG13	PhysicalComplexity.AccessRegime.certificate_upgrades_regime DecisionQuotient/Physics/AccessRegime.lean	ARG17	PhysicalComplexity.AccessRegime.regime_upgrade_with_certificate DecisionQuotient/Physics/AccessRegime.lean
ARG19	PhysicalComplexity.AccessRegime.AccessChannellaw DecisionQuotient/Physics/AccessRegime.lean	BA1	Physics.BoundedAcquisition.BoundedRegion DecisionQuotient/Physics/ BoundedAcquisition.lean
BA2	Physics.BoundedAcquisition.acquisition_rate_bound DecisionQuotient/Physics/ BoundedAcquisition.lean	BA3	Physics.BoundedAcquisition.acquisitions_are_transitions DecisionQuotient/Physics/ BoundedAcquisition.lean
BA4	Physics.BoundedAcquisition.one_bit_per_transition DecisionQuotient/Physics/ BoundedAcquisition.lean	BA5	Physics.BoundedAcquisition.resolution_reads_sufficient DecisionQuotient/Physics/ BoundedAcquisition.lean
BA6	Physics.BoundedAcquisition.srank_le_resolution_bits DecisionQuotient/Physics/ BoundedAcquisition.lean	BA7	Physics.BoundedAcquisition.energy_ge_srank_cost DecisionQuotient/Physics/ BoundedAcquisition.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
BA8	Physics.BoundedAcquisition.srank_one_energy_minimum DecisionQuotient/Physics/ BoundedAcquisition.lean	BA9	Physics.BoundedAcquisition.physical_grounding_bundle DecisionQuotient/Physics/ BoundedAcquisition.lean
BA10	Physics.BoundedAcquisition.counting_gap_theorem DecisionQuotient/Physics/ BoundedAcquisition.lean	BB1	DecisionQuotient.BayesianDQ DecisionQuotient/BayesianBridge.lean
BB2	BayesianDQ.certaintyGain DecisionQuotient/BayesianBridge.lean	BB3	DecisionQuotient.dq_is_bayesian_certainty_fraction DecisionQuotient/BayesianBridge.lean
BB4	DecisionQuotient.bayesian_dq_matches_physics_dq DecisionQuotient/BayesianBridge.lean	BB5	DecisionQuotient.dq_derived_from_bayes DecisionQuotient/BayesianBridge.lean
BC1	Foundations.counting_nonneg DecisionQuotient/BayesFoundations.lean	BC2	Foundations.counting_total DecisionQuotient/BayesFoundations.lean
BC3	Foundations.counting_additive DecisionQuotient/BayesFoundations.lean	BC4	Foundations.bayes_from_conditional DecisionQuotient/BayesFoundations.lean
BC5	Foundations.entropy_contraction DecisionQuotient/BayesFoundations.lean	BF2	DecisionQuotient.nondegenerateBelief_of_uncertaintyForced DecisionQuotient/BayesFromDQ.lean
BF3	DecisionQuotient.forced_action_under_uncertainty DecisionQuotient/BayesFromDQ.lean	BF4	DecisionQuotient.bayes_update_exists_of_nondegenerateBelief DecisionQuotient/BayesFromDQ.lean
CC1	DecisionQuotient.ClaimClosure.RegimeSimulation DecisionQuotient/ClaimClosure.lean	CC2	DecisionQuotient.ClaimClosure.adq_ordering DecisionQuotient/ClaimClosure.lean
CC3	DecisionQuotient.ClaimClosure.system_transfer_licensed_iff_snapshot DecisionQuotient/ClaimClosure.lean	CC4	DecisionQuotient.ClaimClosure.anchor_sigma2p_complete_conditional DecisionQuotient/ClaimClosure.lean
CC5	DecisionQuotient.ClaimClosure.anchor_sigma2p_reduction_core DecisionQuotient/ClaimClosure.lean	CC6	DecisionQuotient.ClaimClosure.anchor_query_relation_false_iff DecisionQuotient/ClaimClosure.lean
CC7	DecisionQuotient.ClaimClosure.anchor_query_relation_true_iff DecisionQuotient/ClaimClosure.lean	CC8	DecisionQuotient.ClaimClosure.boundaryCharacterized_iff_exists_sufficient_subset DecisionQuotient/ClaimClosure.lean
CC9	DecisionQuotient.ClaimClosure.bounded_actions_detectable DecisionQuotient/ClaimClosure.lean	CC10	DecisionQuotient.ClaimClosure.bridge_boundary_represented_family DecisionQuotient/ClaimClosure.lean
CC11	DecisionQuotient.ClaimClosure.bridge_failure_witness_non_one_step DecisionQuotient/ClaimClosure.lean	CC12	DecisionQuotient.ClaimClosure.bridge_transfer_iff_one_step_class DecisionQuotient/ClaimClosure.lean
CC13	DecisionQuotient.ClaimClosure.certified_total_bits_split_core DecisionQuotient/ClaimClosure.lean	CC14	DecisionQuotient.ClaimClosure.cost_asymmetry_eth_conditional DecisionQuotient/ClaimClosure.lean
CC15	DecisionQuotient.ClaimClosure.declaredBudgetSlice DecisionQuotient/ClaimClosure.lean	CC16	DecisionQuotient.ClaimClosure.declaredRegimeFamily_complete DecisionQuotient/ClaimClosure.lean
CC17	DecisionQuotient.ClaimClosure.declared_physics_no_universal_exact_certifier_core DecisionQuotient/ClaimClosure.lean	CC18	DecisionQuotient.ClaimClosure.dichotomy_conditional DecisionQuotient/ClaimClosure.lean
CC19	DecisionQuotient.ClaimClosure.epsilon_admissible_iff_raw_lt_certified_total_core DecisionQuotient/ClaimClosure.lean	CC20	DecisionQuotient.ClaimClosure.exact_admissible_iff_raw_lt_certified_total_core DecisionQuotient/ClaimClosure.lean
CC21	DecisionQuotient.ClaimClosure.exact_certainty_inflation_under_hardness_core DecisionQuotient/ClaimClosure.lean	CC22	DecisionQuotient.ClaimClosure.exact_raw_eq_certified_iff_certainty_inflation_core DecisionQuotient/ClaimClosure.lean
CC23	DecisionQuotient.ClaimClosure.exact_raw_only_of_no_exact_admissible_core DecisionQuotient/ClaimClosure.lean	CC24	DecisionQuotient.ClaimClosure.explicit_assumptions_required_of_not_excused_core DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
CC25	DecisionQuotient.ClaimClosure.explicit_state_upper_core DecisionQuotient/ClaimClosure.lean	CC26	DecisionQuotient.ClaimClosure.hard_family_all_coords_core DecisionQuotient/ClaimClosure.lean
CC27	DecisionQuotient.ClaimClosure.horizonTwoWitness_immediate_empty_sufficient DecisionQuotient/ClaimClosure.lean	CC28	DecisionQuotient.ClaimClosure.horizon_gt_one_bridge_can_fail_on_sufficiency DecisionQuotient/ClaimClosure.lean
CC29	DecisionQuotient.ClaimClosure.information_barrier_opt_oracle_core DecisionQuotient/ClaimClosure.lean	CC30	DecisionQuotient.ClaimClosure.information_barrier_state_batch_core DecisionQuotient/ClaimClosure.lean
CC31	DecisionQuotient.ClaimClosure.information_barrier_value_entry_core DecisionQuotient/ClaimClosure.lean	CC32	DecisionQuotient.ClaimClosure.integrity_resource_bound_for_sufficiency DecisionQuotient/ClaimClosure.lean
CC33	DecisionQuotient.ClaimClosure.integrity_universal_applicability_core DecisionQuotient/ClaimClosure.lean	CC34	DecisionQuotient.ClaimClosure.meta_coordinate_irrelevant_of_invariance_on_declared_slice DecisionQuotient/ClaimClosure.lean
CC35	DecisionQuotient.ClaimClosure.meta_coordinate_not_relevant_on_declared_slice DecisionQuotient/ClaimClosure.lean	CC36	DecisionQuotient.ClaimClosure.minsuff_collapse_core DecisionQuotient/ClaimClosure.lean
CC37	DecisionQuotient.ClaimClosure.minsuff_collapse_to_conp_conditional DecisionQuotient/ClaimClosure.lean	CC38	DecisionQuotient.ClaimClosure.minsuff_conp_complete_conditional DecisionQuotient/ClaimClosure.lean
CC39	DecisionQuotient.ClaimClosure.no_auto_minimize_of_p_neq_conp DecisionQuotient/ClaimClosure.lean	CC40	DecisionQuotient.ClaimClosure.no_exact_claim_admissible_under_hardness_core DecisionQuotient/ClaimClosure.lean
CC41	DecisionQuotient.ClaimClosure.no_exact_claim_under_declared_assumptions_unless_excused_core DecisionQuotient/ClaimClosure.lean	CC42	DecisionQuotient.ClaimClosure.no_exact_identifier_implies_not_boundary_characterized DecisionQuotient/ClaimClosure.lean
CC43	DecisionQuotient.ClaimClosure.no_uncertified_exact_claim_core DecisionQuotient/ClaimClosure.lean	CC44	DecisionQuotient.ClaimClosure.one_step_bridge DecisionQuotient/ClaimClosure.lean
CC45	DecisionQuotient.ClaimClosure.oracle_lattice_transfer_as_regime_simulation DecisionQuotient/ClaimClosure.lean	CC46	DecisionQuotient.ClaimClosure.physical_crossover_above_cap_core DecisionQuotient/ClaimClosure.lean
CC47	DecisionQuotient.ClaimClosure.physical_crossover_core DecisionQuotient/ClaimClosure.lean	CC48	DecisionQuotient.ClaimClosure.physical_crossover_hardness_core DecisionQuotient/ClaimClosure.lean
CC49	DecisionQuotient.ClaimClosure.physical_crossover_policy_core DecisionQuotient/ClaimClosure.lean	CC50	DecisionQuotient.ClaimClosure.process_bridge_failure_witness DecisionQuotient/ClaimClosure.lean
CC51	DecisionQuotient.ClaimClosure.poseAnchorQuery DecisionQuotient/ClaimClosure.lean	CC52	DecisionQuotient.ClaimClosure.pose_returns_anchor_query_object DecisionQuotient/ClaimClosure.lean
CC53	DecisionQuotient.ClaimClosure.posed_anchor_checked_true_implies_truth DecisionQuotient/ClaimClosure.lean	CC54	DecisionQuotient.ClaimClosure.posed_anchor_exact_claim_admissible_iff_competent DecisionQuotient/ClaimClosure.lean
CC55	DecisionQuotient.ClaimClosure.posed_anchor_exact_claim_requires_evidence DecisionQuotient/ClaimClosure.lean	CC56	DecisionQuotient.ClaimClosure.posed_anchor_no_competence_no_exact_claim DecisionQuotient/ClaimClosure.lean
CC57	DecisionQuotient.ClaimClosure.posed_anchor_query_truth_iff_exists_anchor DecisionQuotient/ClaimClosure.lean	CC58	DecisionQuotient.ClaimClosure.posed_anchor_query_truth_iff_exists_forall DecisionQuotient/ClaimClosure.lean
CC59	DecisionQuotient.ClaimClosure.posed_anchor_signal_positive_certified_implies_admissible DecisionQuotient/ClaimClosure.lean	CC60	DecisionQuotient.ClaimClosure.query_obstruction_boolean_corollary DecisionQuotient/ClaimClosure.lean
CC61	DecisionQuotient.ClaimClosure.query_obstruction_finite_state_core DecisionQuotient/ClaimClosure.lean	CC62	DecisionQuotient.ClaimClosure.regime_core_claim_proved DecisionQuotient/ClaimClosure.lean
CC63	DecisionQuotient.ClaimClosure.regime_simulation_transfers_hardness DecisionQuotient/ClaimClosure.lean	CC64	DecisionQuotient.ClaimClosure.reusable_heuristic_of_detectable DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
CC65	DecisionQuotient.ClaimClosure.selectorSufficient_not_implies_setSufficient DecisionQuotient/ClaimClosure.lean	CC66	DecisionQuotient.ClaimClosure.separable_detectable DecisionQuotient/ClaimClosure.lean
CC67	DecisionQuotient.ClaimClosure.snapshot_vs_process_typed_boundary DecisionQuotient/ClaimClosure.lean	CC68	DecisionQuotient.ClaimClosure.standard_assumption_ledger_unpack DecisionQuotient/ClaimClosure.lean
CC69	DecisionQuotient.ClaimClosure.stochastic_objective_bridge_can_fail_on_sufficiency DecisionQuotient/ClaimClosure.lean	CC70	DecisionQuotient.ClaimClosure.subproblem_hardness_lifts_to_full DecisionQuotient/ClaimClosure.lean
CC71	DecisionQuotient.ClaimClosure.subproblem_transfer_as_regime_simulation DecisionQuotient/ClaimClosure.lean	CC72	DecisionQuotient.ClaimClosure.sufficiency_conp_complete_conditional DecisionQuotient/ClaimClosure.lean
CC73	DecisionQuotient.ClaimClosure.sufficiency_conp_reduction_core DecisionQuotient/ClaimClosure.lean	CC74	DecisionQuotient.ClaimClosure.sufficiency_iff_dq_ratio DecisionQuotient/ClaimClosure.lean
CC75	DecisionQuotient.ClaimClosure.sufficiency_iff_projectedOptCover_eq_opt DecisionQuotient/ClaimClosure.lean	CC76	DecisionQuotient.ClaimClosure.thermo_conservation_additive_core DecisionQuotient/ClaimClosure.lean
CC77	DecisionQuotient.ClaimClosure.thermo_energy_carbon_lift_core DecisionQuotient/ClaimClosure.lean	CC81	DecisionQuotient.ClaimClosure.tractable_bounded_core DecisionQuotient/ClaimClosure.lean
CC82	DecisionQuotient.ClaimClosure.tractable_separable_core DecisionQuotient/ClaimClosure.lean	CC84	DecisionQuotient.ClaimClosure.tractable_tree_core DecisionQuotient/ClaimClosure.lean
CC85	DecisionQuotient.ClaimClosure.transition_coupled_bridge_can_fail_on_sufficiency DecisionQuotient/ClaimClosure.lean	CCC1	DecisionQuotient.CC.anchor_sigma2p_complete_conditional DecisionQuotient/ClaimClosure.lean
CCC3	DecisionQuotient.CC.dichotomy_conditional DecisionQuotient/ClaimClosure.lean	CCC5	DecisionQuotient.CC.minsuff_conp_complete_conditional DecisionQuotient/ClaimClosure.lean
CCC6	DecisionQuotient.CC.sufficiency_conp_complete_conditional DecisionQuotient/ClaimClosure.lean	CCC7	DecisionQuotient.CC.tractable_subcases_conditional DecisionQuotient/ClaimClosure.lean
CF2	Physics.ConstraintForcing.logic_time_not_sufficient_for_unique_law DecisionQuotient/Physics/ConstraintForcing.lean	CF4	Physics.ConstraintForcing.objective_not_determined_of_parameter_separation DecisionQuotient/Physics/ConstraintForcing.lean
CF6	Physics.ConstraintForcing.actionForced_of_deadline DecisionQuotient/Physics/ConstraintForcing.lean	CF7	Physics.ConstraintForcing.nondegenerateBelief_of_deadline_and_uncertainty DecisionQuotient/Physics/ConstraintForcing.lean
CF8	Physics.ConstraintForcing.forced_decision_implies_positive_landauer_cost DecisionQuotient/Physics/ConstraintForcing.lean	CF9	Physics.ConstraintForcing.forced_decision_implies_positive_nv_work DecisionQuotient/Physics/ConstraintForcing.lean
CH1	ClaimClosure.CH1 DecisionQuotient/ClaimClosure.lean	CH2	ClaimClosure.CH2 DecisionQuotient/ClaimClosure.lean
CH3	ClaimClosure.CH3 DecisionQuotient/ClaimClosure.lean	CH5	ClaimClosure.CH5 DecisionQuotient/ClaimClosure.lean
CH6	ClaimClosure.CH6 DecisionQuotient/ClaimClosure.lean	CR1	DecisionQuotient.ConfigReduction.config_sufficiency_iff_behavior_preserving DecisionQuotient/Hardness/ConfigReduction.lean
CT1	DecisionQuotient.Physics.ClaimTransport.PhysicalEncoding DecisionQuotient/Physics/ClaimTransport.lean	CT2	DecisionQuotient.Physics.ClaimTransport.physical_claim_lifts_from_core DecisionQuotient/Physics/ClaimTransport.lean
CT3	DecisionQuotient.Physics.ClaimTransport.physical_claim_lifts_from_core_conditional DecisionQuotient/Physics/ClaimTransport.lean	CT4	DecisionQuotient.Physics.ClaimTransport.physical_counterexample_yields_core_counterexample DecisionQuotient/Physics/ClaimTransport.lean
CT5	DecisionQuotient.Physics.ClaimTransport.physical_counterexample_invalidates_core_rule DecisionQuotient/Physics/ClaimTransport.lean	CT6	DecisionQuotient.Physics.ClaimTransport.no_physical_counterexample_of_core_theorem DecisionQuotient/Physics/ClaimTransport.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
CT7	DecisionQuotient.Physics.ClaimTransport. LawGapInstance DecisionQuotient/Physics/ClaimTransport.lean	CT8	DecisionQuotient.Physics.ClaimTransport. lawGapEncoding DecisionQuotient/Physics/ClaimTransport.lean
CV4	Physics.Conversation.tick_uses_shared_node DecisionQuotient/Physics/Conversation.lean	CV5	Physics.Conversation.tick_shared_is_merged_ emissions DecisionQuotient/Physics/Conversation.lean
CV7	Physics.Conversation.clamp_projection_eq_iff_ same_clamped_bit DecisionQuotient/Physics/Conversation.lean	CV11	Physics.Conversation.toClaimReport DecisionQuotient/Physics/Conversation.lean
CV12	Physics.Conversation.abstain_iff_no_answer DecisionQuotient/Physics/Conversation.lean	CV13	Physics.Conversation.yes_no_iff_exact_claim DecisionQuotient/Physics/Conversation.lean
CV15	Physics.Conversation.toReportSignal_signal_ consistent_zero_certified DecisionQuotient/Physics/Conversation.lean	CV16	Physics.Conversation.abstain_report_can_carry_ explanation DecisionQuotient/Physics/Conversation.lean
CV17	DecisionQuotient.Physics.Conversation. clampDecisionEvent_iff_bitOps_pos DecisionQuotient/Physics/Conversation.lean	CV18	DecisionQuotient.Physics.Conversation.clamp_ event_implies_positive_energy DecisionQuotient/Physics/Conversation.lean
DC1	StochasticSequential.static_stochastic_strict_ separation DecisionQuotient/StochasticSequential/ Hierarchy.lean	DC2	StochasticSequential.stochastic_sequential_ strict_separation DecisionQuotient/StochasticSequential/ Hierarchy.lean
DC3	StochasticSequential.complexity_dichotomy_ hierarchy DecisionQuotient/StochasticSequential/ Hierarchy.lean	DC9	StochasticSequential.stochastic_to_PP DecisionQuotient/StochasticSequential/ Hierarchy.lean
DC10	StochasticSequential.sequential_to_PSPACE DecisionQuotient/StochasticSequential/ Hierarchy.lean	DC13	StochasticSequential.ClaimClosure.claim_ tractable_subcases_to_P DecisionQuotient/StochasticSequential/ ClaimClosure.lean
DC14	StochasticSequential.stochastic_dichotomy DecisionQuotient/StochasticSequential/ Dichotomy.lean	DC15	StochasticSequential.above_threshold_hard DecisionQuotient/StochasticSequential/ Dichotomy.lean
DC16	StochasticSequential. StochasticAnchorSufficient DecisionQuotient/StochasticSequential/ Basic.lean	DC17	StochasticSequential. StochasticAnchorSufficiencyCheck DecisionQuotient/StochasticSequential/ Basic.lean
DC18	StochasticSequential.stochastic_anchor_check_ iff DecisionQuotient/StochasticSequential/ Basic.lean	DC19	StochasticSequential.stochastic_anchor_ sufficient_of_stochastic_sufficient DecisionQuotient/StochasticSequential/ Quotient.lean
DC20	StochasticSequential. SequentialAnchorSufficient DecisionQuotient/StochasticSequential/ Basic.lean	DC21	StochasticSequential. SequentialAnchorSufficiencyCheck DecisionQuotient/StochasticSequential/ Basic.lean
DC22	StochasticSequential.sequential_anchor_check_ iff DecisionQuotient/StochasticSequential/ Basic.lean	DC23	StochasticSequential.sequential_anchor_ sufficient_of_sequential_sufficient DecisionQuotient/StochasticSequential/ Basic.lean
DC24	StochasticSequential. StochasticAnchorCheckInstance DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC25	StochasticSequential.reduceMAJSAT_correct_ anchor_strict DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC26	StochasticSequential.reduceMAJSAT_to_ stochastic_anchor_reduction DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC27	StochasticSequential. SequentialAnchorCheckInstance DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC28	StochasticSequential.reduceTQBF_correct_anchor DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC29	StochasticSequential.reduceTQBF_to_sequential_ anchor_reduction DecisionQuotient/StochasticSequential/ PolynomialReduction.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
DC30	StochasticSequential.StatePotential DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean	DC31	StochasticSequential.utilityFromPotentialDrop_ le_iff_nextPotential_ge DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean
DC32	StochasticSequential.utility_from_action_ state_potential DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean	DC33	StochasticSequential. stochasticExpectedUtility_eq_neg_ expectedActionPotential DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean
DC34	StochasticSequential. stochasticExpectedUtility_le_iff_ expectedActionPotential_ge DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean	DC35	StochasticSequential.landauerEnergyFloor_ nonneg DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean
DC36	StochasticSequential.landauerEnergyFloor_mono_ bits DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean	DC37	StochasticSequential.thermodynamicCost_eq_ landauerEnergyFloorRoom_states DecisionQuotient/StochasticSequential/ ThermodynamicLift.lean
DC40	StochasticSequential.reduceMAJSATPureAnchor_ correct DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC41	StochasticSequential.reduceMAJSAT_to_pure_ stochastic_anchor_reduction DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC42	StochasticSequential.stochastic_anchor_check_ pp_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC43	StochasticSequential. SequentialAnchorPSPACEHard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC44	StochasticSequential.sequential_anchor_check_ pspace_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC45	StochasticSequential.stochastic_sufficiency_ pp_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC46	StochasticSequential.sequential_sufficiency_ pspace_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC47	StochasticSequential.stochastic_minimum_ sufficiency_pp_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean
DC48	StochasticSequential.sequential_minimum_ sufficiency_pspace_hard DecisionQuotient/StochasticSequential/ PolynomialReduction.lean	DC49	StochasticSequential. sequentialMinimalSufficient_iff_relevant DecisionQuotient/StochasticSequential/ Basic.lean
DC50	StochasticSequential.sequentialRelevantSet_is_ minimal DecisionQuotient/StochasticSequential/ Basic.lean	DC51	StochasticSequential.stochasticSufficientBool_ spec DecisionQuotient/StochasticSequential/ Computation.lean
DC52	StochasticSequential. stochasticAnchorSufficientBool_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC53	StochasticSequential. stochasticMinimumSufficiencyBool_spec DecisionQuotient/StochasticSequential/ Computation.lean
DC54	StochasticSequential.sequentialSufficientBool_ spec DecisionQuotient/StochasticSequential/ Computation.lean	DC55	StochasticSequential. sequentialAnchorSufficientBool_spec DecisionQuotient/StochasticSequential/ Computation.lean
DC56	StochasticSequential. sequentialMinimumSufficiencyBool_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC57	StochasticSequential.fiberDecisionProblem_ sufficient DecisionQuotient/StochasticSequential/ Basic.lean
DC58	StochasticSequential. countedStochasticMinimumSearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC59	StochasticSequential. countedStochasticMinimumSearch_steps DecisionQuotient/StochasticSequential/ Computation.lean
DC60	StochasticSequential. countedSequentialMinimumSearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC61	StochasticSequential. countedSequentialMinimumSearch_steps DecisionQuotient/StochasticSequential/ Computation.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
DC62	StochasticSequential. countedStochasticSufficiencySearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC63	StochasticSequential. countedStochasticSufficiencySearch_steps DecisionQuotient/StochasticSequential/ Computation.lean
DC64	StochasticSequential. countedStochasticAnchorSearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC65	StochasticSequential. countedStochasticAnchorSearch_steps DecisionQuotient/StochasticSequential/ Computation.lean
DC66	StochasticSequential. countedSequentialSufficiencySearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC67	StochasticSequential. countedSequentialSufficiencySearch_steps DecisionQuotient/StochasticSequential/ Computation.lean
DC68	StochasticSequential. countedSequentialAnchorSearch_spec DecisionQuotient/StochasticSequential/ Computation.lean	DC69	StochasticSequential. countedSequentialAnchorSearch_steps DecisionQuotient/StochasticSequential/ Computation.lean
DC70	DecisionQuotient.static_sufficiency_inP_ explicit DecisionQuotient/ExplicitStateMembership.lean	DC71	DecisionQuotient.static_anchor_inP_explicit DecisionQuotient/ExplicitStateMembership.lean
DC72	StochasticSequential.stochastic_sufficiency_ inP_explicit DecisionQuotient/StochasticSequential/ Computation.lean	DC73	StochasticSequential.stochastic_anchor_inP_ explicit DecisionQuotient/StochasticSequential/ Computation.lean
DC74	StochasticSequential.sequential_sufficiency_ inP_explicit DecisionQuotient/StochasticSequential/ Computation.lean	DC75	StochasticSequential.sequential_anchor_inP_ explicit DecisionQuotient/StochasticSequential/ Computation.lean
DC76	DecisionQuotient.explicit_state_inP_summary DecisionQuotient/ExplicitStateMembership.lean	DC77	DecisionQuotient.static_minimum_inP_explicit DecisionQuotient/ExplicitStateMembership.lean
DC78	DecisionQuotient.stochastic_minimum_inP_ explicit DecisionQuotient/ExplicitStateMembership.lean	DC79	DecisionQuotient.sequential_minimum_inP_ explicit DecisionQuotient/ExplicitStateMembership.lean
DE1	ClaimClosure.DE1 DecisionQuotient/ClaimClosure.lean	DE2	ClaimClosure.DE2 DecisionQuotient/ClaimClosure.lean
DE3	ClaimClosure.DE3 DecisionQuotient/ClaimClosure.lean	DE4	ClaimClosure.DE4 DecisionQuotient/ClaimClosure.lean
DG5	DecisionQuotient.card_anchoredSlice DecisionQuotient/DecisionGeometry.lean	DG6	DecisionQuotient.card_anchoredSlice_eq_pow_sub DecisionQuotient/DecisionGeometry.lean
DG15	DecisionQuotient.boolHypercube_node_count DecisionQuotient/DecisionGeometry.lean	DG16	DecisionQuotient.node_count_does_not_ determine_edge_geometry DecisionQuotient/DecisionGeometry.lean
DG18	DecisionQuotient.DecisionProblem. edgeOnComplement_iff_not_sufficient DecisionQuotient/DecisionGeometry.lean	DP1	DecisionQuotient.DecisionProblem. minimalSufficient_iff_relevant DecisionQuotient/Sufficiency.lean
DP2	DecisionQuotient.DecisionProblem.relevantSet_ is_minimal DecisionQuotient/Sufficiency.lean	DP3	DecisionQuotient.DecisionProblem.sufficient_ implies_selectorSufficient DecisionQuotient/Sufficiency.lean
DP4	DecisionQuotient.ClaimClosure.DecisionProblem. epsOpt_zero_eq_opt DecisionQuotient/ClaimClosure.lean	DP5	DecisionQuotient.ClaimClosure.DecisionProblem. sufficient_iff_zeroEpsilonSufficient DecisionQuotient/ClaimClosure.lean
DP6	ClaimClosure.DP6 DecisionQuotient/ClaimClosure.lean	DP7	ClaimClosure.DP7 DecisionQuotient/ClaimClosure.lean
DP8	ClaimClosure.DP8 DecisionQuotient/ClaimClosure.lean	DQ1	ClaimClosure.DQ1 DecisionQuotient/ClaimClosure.lean
DQ2	ClaimClosure.DQ2 DecisionQuotient/ClaimClosure.lean	DQ3	ClaimClosure.DQ3 DecisionQuotient/ClaimClosure.lean
DQ4	ClaimClosure.DQ4 DecisionQuotient/ClaimClosure.lean	DQ5	ClaimClosure.DQ5 DecisionQuotient/ClaimClosure.lean
DQ6	ClaimClosure.DQ6 DecisionQuotient/ClaimClosure.lean	DQ7	ClaimClosure.DQ7 DecisionQuotient/ClaimClosure.lean
DQ8	ClaimClosure.DQ8 DecisionQuotient/ClaimClosure.lean	DS1	ClaimClosure.DS1 DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
DS2	ClaimClosure.DS2 DecisionQuotient/ClaimClosure.lean	DS3	ClaimClosure.DS3 DecisionQuotient/ClaimClosure.lean
DS4	ClaimClosure.DS4 DecisionQuotient/ClaimClosure.lean	DS5	ClaimClosure.DS5 DecisionQuotient/ClaimClosure.lean
DS6	ClaimClosure.DS6 DecisionQuotient/ClaimClosure.lean	DT6	DecisionQuotient.Physics.DecisionTime.time_is_discrete DecisionQuotient/Physics/DecisionTime.lean
DT7	DecisionQuotient.Physics.DecisionTime.time_coordinate_falsifiable DecisionQuotient/Physics/DecisionTime.lean	DT8	DecisionQuotient.Physics.DecisionTime.tick_increments_time DecisionQuotient/Physics/DecisionTime.lean
DT10	DecisionQuotient.Physics.DecisionTime.tick_is_decision_event DecisionQuotient/Physics/DecisionTime.lean	DT11	DecisionQuotient.Physics.DecisionTime.decision_event_implies_time_unit DecisionQuotient/Physics/DecisionTime.lean
DT12	DecisionQuotient.Physics.DecisionTime.decision_taking_place_is_unit_of_time DecisionQuotient/Physics/DecisionTime.lean	DT13	DecisionQuotient.Physics.DecisionTime.decision_event_iff_eq_tick DecisionQuotient/Physics/DecisionTime.lean
DT15	DecisionQuotient.Physics.DecisionTime.run_time_exact DecisionQuotient/Physics/DecisionTime.lean	DT16	DecisionQuotient.Physics.DecisionTime.run_elapsed_time_eq_ticks DecisionQuotient/Physics/DecisionTime.lean
DT18	DecisionQuotient.Physics.DecisionTime.decisionTrace_length_eq_ticks DecisionQuotient/Physics/DecisionTime.lean	DT19	DecisionQuotient.Physics.DecisionTime.decision_count_equals_elapsed_time DecisionQuotient/Physics/DecisionTime.lean
DT22	DecisionQuotient.Physics.DecisionTime.substrate_step_realizes_decision_event DecisionQuotient/Physics/DecisionTime.lean	DT23	DecisionQuotient.Physics.DecisionTime.substrate_step_is_time_unit DecisionQuotient/Physics/DecisionTime.lean
DT24	DecisionQuotient.Physics.DecisionTime.time_unit_law_substrate_invariant DecisionQuotient/Physics/DecisionTime.lean	EI1	ThermodynamicLift.energy_ge_kbt_nat_entropy DecisionQuotient/ThermodynamicLift.lean
FI3	FunctionalInformation. functionalInformationBitsFromEnergy DecisionQuotient/FunctionalInformation.lean	FI6	FunctionalInformation.functional_information_from_thermodynamics DecisionQuotient/FunctionalInformation.lean
FI7	FunctionalInformation.first_principles_thermo_coincide DecisionQuotient/FunctionalInformation.lean	FN7	BayesOptimalityProof.KL_nonneg DecisionQuotient/BayesOptimalityProof.lean
FN8	BayesOptimalityProof.entropy_le_crossEntropy DecisionQuotient/BayesOptimalityProof.lean	FN12	BayesOptimalityProof.crossEntropy_eq_entropy_add_KL DecisionQuotient/BayesOptimalityProof.lean
FN14	BayesOptimalityProof.bayes_is_optimal DecisionQuotient/BayesOptimalityProof.lean	FP1	Physics.LocalityPhysics.trivial_states_all_equal DecisionQuotient/Physics/LocalityPhysics.lean
FP2	Physics.LocalityPhysics.equal_states_constant_function DecisionQuotient/Physics/LocalityPhysics.lean	FP3	Physics.LocalityPhysics.constant_function_singleton_image DecisionQuotient/Physics/LocalityPhysics.lean
FP4	Physics.LocalityPhysics.singleton_image_zero_entropy DecisionQuotient/Physics/LocalityPhysics.lean	FP5	Physics.LocalityPhysics.zero_entropy_no_information DecisionQuotient/Physics/LocalityPhysics.lean
FP6	Physics.LocalityPhysics.triviality_implies_no_information DecisionQuotient/Physics/LocalityPhysics.lean	FP7	Physics.LocalityPhysics.information_requires_nontriviality DecisionQuotient/Physics/LocalityPhysics.lean
FP8	Physics.LocalityPhysics.atypical_states_rare DecisionQuotient/Physics/LocalityPhysics.lean	FP9	Physics.LocalityPhysics.random_misses_target DecisionQuotient/Physics/LocalityPhysics.lean
FP10	Physics.LocalityPhysics.errors_accumulate DecisionQuotient/Physics/LocalityPhysics.lean	FP11	Physics.LocalityPhysics.wrong_paths_dominate DecisionQuotient/Physics/LocalityPhysics.lean
FP12	Physics.LocalityPhysics.second_law_from_counting DecisionQuotient/Physics/LocalityPhysics.lean	FP13	Physics.LocalityPhysics.verfication_is_information DecisionQuotient/Physics/LocalityPhysics.lean
FP14	Physics.LocalityPhysics.entropy_is_information DecisionQuotient/Physics/LocalityPhysics.lean	FP15	Physics.LocalityPhysics.landauer_structure DecisionQuotient/Physics/LocalityPhysics.lean
FPT4	Physics.LocalityPhysics.FPT4_step_requires_distinct_moments DecisionQuotient/Physics/LocalityPhysics.lean	FPT5	Physics.LocalityPhysics.FPT5_distinct_moments_positive_duration DecisionQuotient/Physics/LocalityPhysics.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
FPT6	Physics.LocalityPhysics.FPT6_step_takes_positive_time DecisionQuotient/Physics/LocalityPhysics.lean	FPT8	Physics.LocalityPhysics.FPT8_propagation_takes_time DecisionQuotient/Physics/LocalityPhysics.lean
FPT10	Physics.LocalityPhysics.FPT10_ec3_is_logical DecisionQuotient/Physics/LocalityPhysics.lean	FS1	Statistics.sum_fisherScore_eq_srank DecisionQuotient/Statistics/ FisherInformation.lean
FS2	Statistics.fisherMatrix_rank_eq_srank DecisionQuotient/Statistics/ FisherInformation.lean	GE1	ClaimClosure.GE1 DecisionQuotient/ClaimClosure.lean
GE2	ClaimClosure.GE2 DecisionQuotient/ClaimClosure.lean	GE3	ClaimClosure.GE3 DecisionQuotient/ClaimClosure.lean
GE4	ClaimClosure.GE4 DecisionQuotient/ClaimClosure.lean	GE5	ClaimClosure.GE5 DecisionQuotient/ClaimClosure.lean
GE7	ClaimClosure.GE7 DecisionQuotient/ClaimClosure.lean	GE9	ClaimClosure.GE9 DecisionQuotient/ClaimClosure.lean
GN2	LogicGraph.cycleWitnessBits_pos DecisionQuotient/GraphNontriviality.lean	GN4	LogicGraph.pathSurprisal_nonneg_of_positive_mass DecisionQuotient/GraphNontriviality.lean
GN5	LogicGraph.nontrivialityScore_unknown DecisionQuotient/GraphNontriviality.lean	GN6	LogicGraph.observerEntropy_nonneg DecisionQuotient/GraphNontriviality.lean
GN7	LogicGraph.dqFromEntropy_in_unit_interval DecisionQuotient/GraphNontriviality.lean	GN8	LogicGraph.path_belief_forced_under_uncertainty DecisionQuotient/GraphNontriviality.lean
GN9	LogicGraph.bayes_update_exists_for_observer_paths DecisionQuotient/GraphNontriviality.lean	GN10	LogicGraph.cycle_witness_implies_positive_landauer DecisionQuotient/GraphNontriviality.lean
GN11	LogicGraph.cycle_witness_implies_positive_nv_work DecisionQuotient/GraphNontriviality.lean	GN12	LogicGraph.dna_erasure_implies_positive_landauer DecisionQuotient/GraphNontriviality.lean
GN13	LogicGraph.dna_room_temp_environmental_stability DecisionQuotient/GraphNontriviality.lean	H1	...
HD1	DecisionQuotient.HardnessDistribution.centralization_dominance_bundle DecisionQuotient/HardnessDistribution.lean	HD2	DecisionQuotient.HardnessDistribution.centralization_step_saves_n_minus_one DecisionQuotient/HardnessDistribution.lean
HD3	DecisionQuotient.HardnessDistribution.centralized_higher_leverage DecisionQuotient/HardnessDistribution.lean	HD4	DecisionQuotient.HardnessDistribution.complete_model_dominates_after_threshold DecisionQuotient/HardnessDistribution.lean
HD5	DecisionQuotient.HardnessDistribution.gap_conservation_card DecisionQuotient/HardnessDistribution.lean	HD6	DecisionQuotient.HardnessDistribution.generalizedTotal_with_saturation_eventually_constant DecisionQuotient/HardnessDistribution.lean
HD7	DecisionQuotient.HardnessDistribution.generalized_dominance_can_fail_without_right_boundedness DecisionQuotient/HardnessDistribution.lean	HD8	DecisionQuotient.HardnessDistribution.generalized_dominance_can_fail_without_wrong_growth DecisionQuotient/HardnessDistribution.lean
HD9	DecisionQuotient.HardnessDistribution.generalized_right_dominates_wrong_of_bounded_vs_identity_lower DecisionQuotient/HardnessDistribution.lean	HD10	DecisionQuotient.HardnessDistribution.generalized_right_eventually_dominates_wrong DecisionQuotient/HardnessDistribution.lean
HD11	DecisionQuotient.HardnessDistribution.hardnessEfficiency_eq_central_share DecisionQuotient/HardnessDistribution.lean	HD12	DecisionQuotient.HardnessDistribution.isRightHardness DecisionQuotient/HardnessDistribution.lean
HD13	DecisionQuotient.HardnessDistribution.isWrongHardness DecisionQuotient/HardnessDistribution.lean	HD14	DecisionQuotient.HardnessDistribution.linear_lt_exponential_plus_constant_eventually DecisionQuotient/HardnessDistribution.lean
HD15	DecisionQuotient.HardnessDistribution.native_dominates_manual DecisionQuotient/HardnessDistribution.lean	HD16	DecisionQuotient.HardnessDistribution.no_positive_slope_linear_represents_saturating DecisionQuotient/HardnessDistribution.lean
HD17	DecisionQuotient.HardnessDistribution.requiredWork DecisionQuotient/HardnessDistribution.lean	HD18	DecisionQuotient.HardnessDistribution.requiredWork_eq_affine_in_sites DecisionQuotient/HardnessDistribution.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
HD19	DecisionQuotient.HardnessDistribution.right_dominates_wrong DecisionQuotient/HardnessDistribution.lean	HD20	DecisionQuotient.HardnessDistribution.saturatingSiteCost_eventually_constant DecisionQuotient/HardnessDistribution.lean
HD21	DecisionQuotient.HardnessDistribution.simplicityTax_grows DecisionQuotient/HardnessDistribution.lean	HD22	DecisionQuotient.HardnessDistribution.hardnessLowerBound DecisionQuotient/HardnessDistribution.lean
HD23	DecisionQuotient.HardnessDistribution.hardness_is_irreducible_required_work DecisionQuotient/HardnessDistribution.lean	HD25	DecisionQuotient.HardnessDistribution.totalDOF_ge_intrinsic DecisionQuotient/HardnessDistribution.lean
HD26	DecisionQuotient.HardnessDistribution.totalExternalWork_eq_n_mul_gapCard DecisionQuotient/HardnessDistribution.lean	HD27	DecisionQuotient.HardnessDistribution.workGrowthDegree DecisionQuotient/HardnessDistribution.lean
HD28	DecisionQuotient.HardnessDistribution.workGrowthDegree_zero_iff_eventually_constant DecisionQuotient/HardnessDistribution.lean	HS3	DecisionQuotient.Physics.HeisenbergStrong.strong_binding_implies_core_nontrivial DecisionQuotient/Physics/HeisenbergStrong.lean
HS5	DecisionQuotient.Physics.HeisenbergStrong.strong_binding_implies_physical_nontrivial_opt_assumption DecisionQuotient/Physics/HeisenbergStrong.lean	HS6	DecisionQuotient.Physics.HeisenbergStrong.strong_binding_implies_nontrivial_opt_via_uncertainty DecisionQuotient/Physics/HeisenbergStrong.lean
IA1	ClaimClosure.IA1 DecisionQuotient/ClaimClosure.lean	IA2	ClaimClosure.IA2 DecisionQuotient/ClaimClosure.lean
IA3	ClaimClosure.IA3 DecisionQuotient/ClaimClosure.lean	IA4	ClaimClosure.IA4 DecisionQuotient/ClaimClosure.lean
IA5	ClaimClosure.IA5 DecisionQuotient/ClaimClosure.lean	IA6	ClaimClosure.IA6 DecisionQuotient/ClaimClosure.lean
IA7	ClaimClosure.IA7 DecisionQuotient/ClaimClosure.lean	IA9	ClaimClosure.IA9 DecisionQuotient/ClaimClosure.lean
IA11	ClaimClosure.IA11 DecisionQuotient/ClaimClosure.lean	IA12	ClaimClosure.IA12 DecisionQuotient/ClaimClosure.lean
IA13	ClaimClosure.IA13 DecisionQuotient/ClaimClosure.lean	IA15	Physics.InvariantAgreement.sameUniverse DecisionQuotient/Physics/ InvariantAgreement.lean
IA16	Physics.InvariantAgreement.IA16_no_invariant_undefined_membership DecisionQuotient/Physics/ InvariantAgreement.lean	IA17	Physics.InvariantAgreement.IA17_observer_level_rejection_cascade DecisionQuotient/Physics/ InvariantAgreement.lean
IA18	Physics.InvariantAgreement.IA18_escalation_complete DecisionQuotient/Physics/ InvariantAgreement.lean	IC1	DecisionQuotient.IntegrityCompetence.CertaintyInflation DecisionQuotient/IntegrityCompetence.lean
IC2	DecisionQuotient.IntegrityCompetence.CompletionFractionDefined DecisionQuotient/IntegrityCompetence.lean	IC3	DecisionQuotient.IntegrityCompetence.EvidenceForReport
IC4	DecisionQuotient.IntegrityCompetence.ExactCertaintyInflation DecisionQuotient/IntegrityCompetence.lean	IC5	DecisionQuotient.IntegrityCompetence.Percent DecisionQuotient/IntegrityCompetence.lean
IC6	DecisionQuotient.IntegrityCompetence.RLFFWeights DecisionQuotient/IntegrityCompetence.lean	IC7	DecisionQuotient.IntegrityCompetence.ReportSignal DecisionQuotient/IntegrityCompetence.lean
IC8	DecisionQuotient.IntegrityCompetence.ReportBitModel DecisionQuotient/IntegrityCompetence.lean	IC9	DecisionQuotient.IntegrityCompetence.SignalConsistent DecisionQuotient/IntegrityCompetence.lean
IC10	DecisionQuotient.IntegrityCompetence.admissible_irrational_strictly_more_than_rational DecisionQuotient/IntegrityCompetence.lean	IC11	DecisionQuotient.IntegrityCompetence.admissible_matrix_counts DecisionQuotient/IntegrityCompetence.lean
IC12	DecisionQuotient.IntegrityCompetence.abstain_signal_exists_with_guess_self DecisionQuotient/IntegrityCompetence.lean	IC13	DecisionQuotient.IntegrityCompetence.certaintyInflation_iff_not_admissible DecisionQuotient/IntegrityCompetence.lean
IC14	DecisionQuotient.IntegrityCompetence.certificationOverheadBits DecisionQuotient/IntegrityCompetence.lean	IC15	DecisionQuotient.IntegrityCompetence.certificationOverheadBits_of_evidence DecisionQuotient/IntegrityCompetence.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
IC16	DecisionQuotient.IntegrityCompetence. certificationOverheadBits_of_no_evidence DecisionQuotient/IntegrityCompetence.lean	IC17	DecisionQuotient.IntegrityCompetence. certifiedTotalBits DecisionQuotient/IntegrityCompetence.lean
IC18	DecisionQuotient.IntegrityCompetence. certifiedTotalBits_ge_raw DecisionQuotient/IntegrityCompetence.lean	IC19	DecisionQuotient.IntegrityCompetence. certifiedTotalBits_gt_raw_of_evidence DecisionQuotient/IntegrityCompetence.lean
IC20	DecisionQuotient.IntegrityCompetence. certifiedTotalBits_of_evidence DecisionQuotient/IntegrityCompetence.lean	IC21	DecisionQuotient.IntegrityCompetence. certifiedTotalBits_of_no_evidence DecisionQuotient/IntegrityCompetence.lean
IC22	DecisionQuotient.IntegrityCompetence.claim_ admissible_of_evidence DecisionQuotient/IntegrityCompetence.lean	IC23	DecisionQuotient.IntegrityCompetence. competence_implies_integrity DecisionQuotient/IntegrityCompetence.lean
IC24	DecisionQuotient.IntegrityCompetence. completion_fraction_defined_of_declared_bound DecisionQuotient/IntegrityCompetence.lean	IC25	DecisionQuotient.IntegrityCompetence.epsilon_ competence_implies_integrity DecisionQuotient/IntegrityCompetence.lean
IC26	DecisionQuotient.IntegrityCompetence.evidence_ nonempty_iff_claim_admissible DecisionQuotient/IntegrityCompetence.lean	IC27	DecisionQuotient.IntegrityCompetence.evidence_ of_claim_admissible DecisionQuotient/IntegrityCompetence.lean
IC28	DecisionQuotient.IntegrityCompetence.exact_ claim_admissible_iff_exact_evidence_nonempty DecisionQuotient/IntegrityCompetence.lean	IC29	DecisionQuotient.IntegrityCompetence.exact_ claim_requires_evidence DecisionQuotient/IntegrityCompetence.lean
IC30	DecisionQuotient.IntegrityCompetence. exactCertaintyInflation_iff_no_exact_ competence DecisionQuotient/IntegrityCompetence.lean	IC31	DecisionQuotient.IntegrityCompetence.exact_ raw_only_of_no_exact_admissible DecisionQuotient/IntegrityCompetence.lean
IC32	DecisionQuotient.IntegrityCompetence. integrity_forces_abstention DecisionQuotient/IntegrityCompetence.lean	IC33	DecisionQuotient.IntegrityCompetence. integrity_not_competent_of_nonempty_scope DecisionQuotient/IntegrityCompetence.lean
IC34	DecisionQuotient.IntegrityCompetence. integrity_resource_bound DecisionQuotient/IntegrityCompetence.lean	IC35	DecisionQuotient.IntegrityCompetence.no_ completion_fraction_without_declared_bound DecisionQuotient/IntegrityCompetence.lean
IC36	DecisionQuotient.IntegrityCompetence. overModelVerdict_rational_iff DecisionQuotient/IntegrityCompetence.lean	IC37	DecisionQuotient.IntegrityCompetence. percentZero DecisionQuotient/IntegrityCompetence.lean
IC38	DecisionQuotient.IntegrityCompetence. rlffBaseReward DecisionQuotient/IntegrityCompetence.lean	IC39	DecisionQuotient.IntegrityCompetence. rlffReward DecisionQuotient/IntegrityCompetence.lean
IC40	DecisionQuotient.IntegrityCompetence.rlff_ abstain_strictly_prefers_no_certificates DecisionQuotient/IntegrityCompetence.lean	IC41	DecisionQuotient.IntegrityCompetence.rlff_ maximizer_has_evidence DecisionQuotient/IntegrityCompetence.lean
IC42	DecisionQuotient.IntegrityCompetence.rlff_ maximizer_is_admissible DecisionQuotient/IntegrityCompetence.lean	IC43	DecisionQuotient.IntegrityCompetence.self_ reflected_confidence_not_certification DecisionQuotient/IntegrityCompetence.lean
IC44	DecisionQuotient.IntegrityCompetence.signal_ certified_positive_implies_admissible DecisionQuotient/IntegrityCompetence.lean	IC45	DecisionQuotient.IntegrityCompetence.signal_ consistent_of_claim_admissible DecisionQuotient/IntegrityCompetence.lean
IC46	DecisionQuotient.IntegrityCompetence.signal_ no_evidence_forces_zero_certified DecisionQuotient/IntegrityCompetence.lean	IC47	DecisionQuotient.IntegrityCompetence.signal_ exact_no_competence_forces_zero_certified DecisionQuotient/IntegrityCompetence.lean
IE1	ClaimClosure.IE1 DecisionQuotient/ClaimClosure.lean	IE3	ClaimClosure.IE3 DecisionQuotient/ClaimClosure.lean
IE4	ClaimClosure.IE4 DecisionQuotient/ClaimClosure.lean	IE5	ClaimClosure.IE5 DecisionQuotient/ClaimClosure.lean
IE6	ClaimClosure.IE6 DecisionQuotient/ClaimClosure.lean	IE7	ClaimClosure.IE7 DecisionQuotient/ClaimClosure.lean
IE8	ClaimClosure.IE8 DecisionQuotient/ClaimClosure.lean	IE9	ClaimClosure.IE9 DecisionQuotient/ClaimClosure.lean
IE10	ClaimClosure.IE10 DecisionQuotient/ClaimClosure.lean	IE11	ClaimClosure.IE11 DecisionQuotient/ClaimClosure.lean
IE12	ClaimClosure.IE12 DecisionQuotient/ClaimClosure.lean	IE13	ClaimClosure.IE13 DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
IE14	ClaimClosure.IE14 DecisionQuotient/ClaimClosure.lean	IE15	ClaimClosure.IE15 DecisionQuotient/ClaimClosure.lean
IE16	ClaimClosure.IE16 DecisionQuotient/ClaimClosure.lean	IE17	ClaimClosure.IE17 DecisionQuotient/ClaimClosure.lean
IEB1	InflationEntropyBridge.classes_monotone DecisionQuotient/InflationEntropyBridge.lean	IEB2	InflationEntropyBridge.entropy_monotone DecisionQuotient/InflationEntropyBridge.lean
IEB3	InflationEntropyBridge.classes_strict_increase DecisionQuotient/InflationEntropyBridge.lean	IEB4	InflationEntropyBridge.entropy_strict_increase DecisionQuotient/InflationEntropyBridge.lean
IEB5	InflationEntropyBridge.optCompat_of_utilityCompat DecisionQuotient/InflationEntropyBridge.lean	IEB6	InflationEntropyBridge.thermal_floor_monotone_of_classes DecisionQuotient/InflationEntropyBridge.lean
IEB7	InflationEntropyBridge.thermal_floor_strict_of_new_class DecisionQuotient/InflationEntropyBridge.lean	IEB8	InflationEntropyBridge.later_energy_floor_implies_earlier_floor DecisionQuotient/InflationEntropyBridge.lean
IEB9	InflationEntropyMinimality.not_redundant_A2_for_mono_classes DecisionQuotient/InflationEntropyMinimality.lean	IEB10	InflationEntropyMinimality.not_redundant_A3_for_strict_entropy DecisionQuotient/InflationEntropyMinimality.lean
IEB11	InflationEntropyMinimality.not_redundant_P1_for_positive_floor DecisionQuotient/InflationEntropyMinimality.lean	IEB12	InflationEntropyMinimality.not_redundant_P2_for_positive_floor DecisionQuotient/InflationEntropyMinimality.lean
IEB13	InflationEntropyMinimality.not_redundant_A1_for_mono_classes_weak DecisionQuotient/InflationEntropyMinimality.lean	IEB14	InflationEntropyMinimality.not_redundant_F2_for_numOptClasses_pos DecisionQuotient/InflationEntropyMinimality.lean
IEB15	InflationEntropyMinimality.not_redundant_P3_for_energy_from_entropy_bridge DecisionQuotient/InflationEntropyMinimality.lean	IEB16	InflationEntropyMinimality.not_redundant_F1_for_finite_counting_requirement DecisionQuotient/InflationEntropyMinimality.lean
IN4	DecisionQuotient.Physics.Instantiation.geometry_plus_dynamics_is_circuit DecisionQuotient/Physics/Instantiation.lean	IN5	DecisionQuotient.Physics.Instantiation.DecisionInterpretation DecisionQuotient/Physics/Instantiation.lean
IN6	DecisionQuotient.Physics.Instantiation.DecisionCircuit DecisionQuotient/Physics/Instantiation.lean	IN13	DecisionQuotient.Physics.Instantiation.MoleculeAsCircuit DecisionQuotient/Physics/Instantiation.lean
IN14	DecisionQuotient.Physics.Instantiation.MoleculeAsDecisionCircuit DecisionQuotient/Physics/Instantiation.lean	IN15	DecisionQuotient.Physics.Instantiation.molecule_decision_preserves_geometry DecisionQuotient/Physics/Instantiation.lean
IN16	DecisionQuotient.Physics.Instantiation.molecule_decision_preserves_dynamics DecisionQuotient/Physics/Instantiation.lean	IT1	DecisionProblem.quotientEntropy DecisionQuotient/Information.lean
IT3	DecisionQuotient.quotientEntropy_le_srank_binary DecisionQuotient/Information.lean	IT4	DecisionQuotient.numOptClasses_le_pow_srank_binary DecisionQuotient/Information.lean
IV1	DecisionQuotient.InteriorVerification.GoalClass DecisionQuotient/InteriorVerification.lean	IV2	DecisionQuotient.InteriorVerification.InteriorDominanceVerifiable DecisionQuotient/InteriorVerification.lean
IV3	DecisionQuotient.InteriorVerification.TautologicalSetIdentifiable DecisionQuotient/InteriorVerification.lean	IV4	DecisionQuotient.InteriorVerification.agreeOnSet DecisionQuotient/InteriorVerification.lean
IV5	DecisionQuotient.InteriorVerification.interiorParetoDominates DecisionQuotient/InteriorVerification.lean	IV6	DecisionQuotient.InteriorVerification.interior_certificate_implies_non_rejection DecisionQuotient/InteriorVerification.lean
IV7	DecisionQuotient.InteriorVerification.interior_dominance_implies_universal_non_rejection DecisionQuotient/InteriorVerification.lean	IV8	DecisionQuotient.InteriorVerification.interior_dominance_not_full_sufficiency DecisionQuotient/InteriorVerification.lean
IV9	DecisionQuotient.InteriorVerification.interior_verification_tractable_certificate DecisionQuotient/InteriorVerification.lean	L2	static_anchor_inP_explicit DecisionQuotient/ExplicitStateMembership.lean
MI1	ClaimClosure.MI1 DecisionQuotient/ClaimClosure.lean	MI2	ClaimClosure.MI2 DecisionQuotient/ClaimClosure.lean
MI3	ClaimClosure.MI3 DecisionQuotient/ClaimClosure.lean	MI4	ClaimClosure.MI4 DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
MI5	ClaimClosure.MI5 DecisionQuotient/ClaimClosure.lean	MN1	Physics.MeasureNecessity.quantitative_claim_has_measure DecisionQuotient/Physics/MeasureNecessity.lean
MN2	Physics.MeasureNecessity.stochastic_claim_has_probability_measure DecisionQuotient/Physics/MeasureNecessity.lean	MN5	Physics.MeasureNecessity.counting_measure_not_probability_on_bool DecisionQuotient/Physics/MeasureNecessity.lean
MN7	Physics.MeasureNecessity.quantitative_value_depends_on_measure DecisionQuotient/Physics/MeasureNecessity.lean	MN8	Physics.MeasureNecessity.deterministic_models_still_measure_based DecisionQuotient/Physics/MeasureNecessity.lean
MN9	Physics.MeasureNecessity.measure_does_not_imply_probability DecisionQuotient/Physics/MeasureNecessity.lean	MN10	Physics.MeasureNecessity.quantitative_measure_is_logical_prerequisite DecisionQuotient/Physics/MeasureNecessity.lean
MN11	Physics.MeasureNecessity.stochastic_probability_is_logical_prerequisite DecisionQuotient/Physics/MeasureNecessity.lean	OR3	Physics.ObserverRelativeState. EffectiveStateSpace DecisionQuotient/Physics/ ObserverRelativeState.lean
OR4	Physics.ObserverRelativeState.project_eq_iff DecisionQuotient/Physics/ ObserverRelativeState.lean	OR5	Physics.ObserverRelativeState.observer_relative_equivalence_witness DecisionQuotient/Physics/ ObserverRelativeState.lean
OR9	Physics.ObserverRelativeState.physical_observer_relative_effective_space DecisionQuotient/Physics/ ObserverRelativeState.lean	PA1	Physics.AnchorChecks.obsEquiv_all_of_effective_subsingleton DecisionQuotient/Physics/AnchorChecks.lean
PA2	Physics.AnchorChecks. stochasticAnchorSufficient_iff_exists_anchor_singleton DecisionQuotient/Physics/AnchorChecks.lean	PA3	Physics.AnchorChecks.stochastic_anchor_check_iff_exists_anchor_singleton DecisionQuotient/Physics/AnchorChecks.lean
PA4	Physics.AnchorChecks.stochastic_sufficient_of_observer_collapse_and_seed DecisionQuotient/Physics/AnchorChecks.lean	PA5	Physics.AnchorChecks.stochastic_anchor_check_of_observer_collapse_and_seed DecisionQuotient/Physics/AnchorChecks.lean
PA6	Physics.AnchorChecks.sequential_sufficient_of_observer_collapse DecisionQuotient/Physics/AnchorChecks.lean	PA7	Physics.AnchorChecks.sequential_anchor_check_of_observer_collapse DecisionQuotient/Physics/AnchorChecks.lean
PA8	Physics.AnchorChecks.physical_observer_collapse_implies_obsEquiv_all DecisionQuotient/Physics/AnchorChecks.lean	PA9	Physics.AnchorChecks.physical_stochastic_anchor_check_of_observer_collapse_and_seed DecisionQuotient/Physics/AnchorChecks.lean
PBC1	DecisionQuotient.PhysicalBudgetCrossover.CrossoverAt DecisionQuotient/PhysicalBudgetCrossover.lean	PBC2	DecisionQuotient.PhysicalBudgetCrossover.SuccinctInfeasible DecisionQuotient/PhysicalBudgetCrossover.lean
PBC5	DecisionQuotient.PhysicalBudgetCrossover.exists_least_crossover_point DecisionQuotient/PhysicalBudgetCrossover.lean	PBC7	DecisionQuotient.PhysicalBudgetCrossover.explicit_eventual_infeasibility_of_monotone_and_witness DecisionQuotient/PhysicalBudgetCrossover.lean
PBC8	DecisionQuotient.PhysicalBudgetCrossover.crossover_eventually_of_eventual_split DecisionQuotient/PhysicalBudgetCrossover.lean	PBC9	DecisionQuotient.PhysicalBudgetCrossover.payoff_threshold_explicit_vs_succinct DecisionQuotient/PhysicalBudgetCrossover.lean
PBC10	DecisionQuotient.PhysicalBudgetCrossover.no_universal_survivor_without_succinct_bound DecisionQuotient/PhysicalBudgetCrossover.lean	PBC11	DecisionQuotient.PhysicalBudgetCrossover.policy_closure_at_divergence DecisionQuotient/PhysicalBudgetCrossover.lean
PBC12	DecisionQuotient.PhysicalBudgetCrossover.policy_closure_beyond_divergence DecisionQuotient/PhysicalBudgetCrossover.lean	PH11	PhysicalComplexity.PhysicalCollapseAtRequirement DecisionQuotient/Physics/PhysicalHardness.lean
PH12	PhysicalComplexity.no_physical_collapse_at_requirement DecisionQuotient/Physics/PhysicalHardness.lean	PH13	PhysicalComplexity.canonical_physical_collapse_impossible DecisionQuotient/Physics/PhysicalHardness.lean
PH14	PhysicalComplexity.p_eq_np_physically_impossible_of_collapse_map DecisionQuotient/Physics/PhysicalHardness.lean	PH15	PhysicalComplexity.p_eq_np_physically_impossible_canonical DecisionQuotient/Physics/PhysicalHardness.lean
PH16	PhysicalComplexity.P_eq_NP_via_SAT DecisionQuotient/Physics/PhysicalHardness.lean	PH17	PhysicalComplexity.SAT3ReductionBridge DecisionQuotient/Physics/PhysicalHardness.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
PH18	PhysicalComplexity.sat_reduction_transfers_ energy_lower_bound DecisionQuotient/Physics/PhysicalHardness.lean	PH19	PhysicalComplexity.physical_collapse_of_ polytime_sat_realization DecisionQuotient/Physics/PhysicalHardness.lean
PH20	PhysicalComplexity.p_eq_np_physically_ impossible_via_sat_bridge DecisionQuotient/Physics/PhysicalHardness.lean	PH21	PhysicalComplexity.SAT3HardFamily DecisionQuotient/Physics/PhysicalHardness.lean
PH22	PhysicalComplexity.p_eq_np_physically_ impossible_via_sat_hard_family DecisionQuotient/Physics/PhysicalHardness.lean	PH23	PhysicalComplexity.collapse_possible_without_ positive_bit_cost DecisionQuotient/Physics/PhysicalHardness.lean
PH24	PhysicalComplexity.collapse_possible_without_ exponential_lower_bound DecisionQuotient/Physics/PhysicalHardness.lean	PH25	PhysicalComplexity.no_go_transfer_requires_ collapse_map DecisionQuotient/Physics/PhysicalHardness.lean
PH26	PhysicalComplexity.no_collapse_of_bounded_ budget_pos_cost_exp_lb DecisionQuotient/Physics/PhysicalHardness.lean	PH27	PhysicalComplexity.collapse_implies_ assumption_failure_disjunction DecisionQuotient/Physics/PhysicalHardness.lean
PH28	PhysicalComplexity.deterministic_no_physical_ collapse DecisionQuotient/Physics/PhysicalHardness.lean	PH29	PhysicalComplexity.probabilistic_no_physical_ collapse DecisionQuotient/Physics/PhysicalHardness.lean
PH30	PhysicalComplexity.sequential_no_physical_ collapse DecisionQuotient/Physics/PhysicalHardness.lean	PH31	PhysicalComplexity.collapse_possible_with_ unbounded_budget_profile DecisionQuotient/Physics/PhysicalHardness.lean
PH32	PhysicalComplexity.exp_budget_profile_ unbounded DecisionQuotient/Physics/PhysicalHardness.lean	PH33	PhysicalComplexity.finite_budget_assumption_ is_necessary DecisionQuotient/Physics/PhysicalHardness.lean
PI3	DecisionQuotient.Physics. PhysicalIncompleteness.no_surjective_ instantiation_of_card_gap DecisionQuotient/Physics/ PhysicalIncompleteness.lean	PI4	DecisionQuotient.Physics. PhysicalIncompleteness.physical_ incompleteness_of_card_gap DecisionQuotient/Physics/ PhysicalIncompleteness.lean
PI5	DecisionQuotient.Physics. PhysicalIncompleteness.physical_ incompleteness_of_bounds DecisionQuotient/Physics/ PhysicalIncompleteness.lean	PI6	DecisionQuotient.Physics. PhysicalIncompleteness.under_resolution_ implies_collision DecisionQuotient/Physics/ PhysicalIncompleteness.lean
PI7	DecisionQuotient.Physics. PhysicalIncompleteness.under_resolution_ implies_decision_collision DecisionQuotient/Physics/ PhysicalIncompleteness.lean	PS1	Physics.ClaimTransport.PhysicalStateSemantics DecisionQuotient/Physics/ClaimTransport.lean
PS2	Physics.ClaimTransport.physical_state_has_ witness DecisionQuotient/Physics/ClaimTransport.lean	PS3	Physics.ClaimTransport.physical_state_claim_ of_instance_claim DecisionQuotient/Physics/ClaimTransport.lean
PS4	Physics.ClaimTransport.physical_state_claim_ of_universal_core DecisionQuotient/Physics/ClaimTransport.lean	QT1	DecisionProblem.quotient_is_coarsest DecisionQuotient/Quotient.lean
QT2	DecisionProblem.quotientMap_preservesOpt DecisionQuotient/Quotient.lean	QT3	DecisionProblem.quotient_represents_opt_equiv DecisionQuotient/Quotient.lean
QT7	DecisionProblem.quotient_has_unique_ factorization DecisionQuotient/Quotient.lean	RD1	Information.shannonEntropy_nonneg DecisionQuotient/Information/ RateDistortion.lean
RD2	Information.rate_zero_distortion DecisionQuotient/Information/ RateDistortion.lean	RD3	Information.rate_monotone DecisionQuotient/Information/ RateDistortion.lean
RS1	Information.equiv_preserves_decision DecisionQuotient/Information/RDSrank.lean	RS2	Information.rate_equals_srank DecisionQuotient/Information/RDSrank.lean
RS3	Information.compression_below_srank_fails DecisionQuotient/Information/RDSrank.lean	RS4	Information.srank_bits_sufficient DecisionQuotient/Information/RDSrank.lean
RS5	Information.rate_distortion_bridge DecisionQuotient/Information/RDSrank.lean	SE1	ClaimClosure.SE1 DecisionQuotient/ClaimClosure.lean
SE2	ClaimClosure.SE2 DecisionQuotient/ClaimClosure.lean	SE3	ClaimClosure.SE3 DecisionQuotient/ClaimClosure.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
SE4	ClaimClosure.SE4 DecisionQuotient/ClaimClosure.lean	SE5	ClaimClosure.SE5 DecisionQuotient/ClaimClosure.lean
SE6	ClaimClosure.SE6 DecisionQuotient/ClaimClosure.lean	SK1	DecisionProblem.srank_eq_relevant_card DecisionQuotient/Tractability/ StructuralRank.lean
SK2	DecisionProblem.srank_le_n DecisionQuotient/Tractability/ StructuralRank.lean	SK3	DecisionProblem.srank_zero_iff_constant DecisionQuotient/Tractability/ StructuralRank.lean
SR1	ClaimClosure.SR1 DecisionQuotient/ClaimClosure.lean	SR2	ClaimClosure.SR2 DecisionQuotient/ClaimClosure.lean
SR3	ClaimClosure.SR3 DecisionQuotient/ClaimClosure.lean	SR4	ClaimClosure.SR4 DecisionQuotient/ClaimClosure.lean
SR5	ClaimClosure.SR5 DecisionQuotient/ClaimClosure.lean	SSV1	StochasticSequential.fiberExpectedUtility_eq_of_agreeOn DecisionQuotient/StochasticSequential/ SetValued.lean
SSV2	StochasticSequential.fiberOpt_eq_of_agreeOn DecisionQuotient/StochasticSequential/ SetValued.lean	SSV3	StochasticSequential.stochasticSetSufficient_universal DecisionQuotient/StochasticSequential/ SetValued.lean
TUR1	Physics.transitionProb_nonneg DecisionQuotient/Physics/TUR.lean	TUR2	Physics.transitionProb_sum_one DecisionQuotient/Physics/TUR.lean
TUR5	Physics.tur_bridge DecisionQuotient/Physics/TUR.lean	TUR6	Physics.multiple_futures_entropy_production DecisionQuotient/Physics/TUR.lean
W1	Physics.single_future_zero_cost DecisionQuotient/Physics/ WassersteinIntegrity.lean	W2	Physics.transportCost_pos_of_offDiag DecisionQuotient/Physics/ WassersteinIntegrity.lean
W3	Physics.integrity_is_centroid DecisionQuotient/Physics/ WassersteinIntegrity.lean	W4	Physics.wasserstein_bridge DecisionQuotient/Physics/ WassersteinIntegrity.lean
WC1	Physics.WolpertConstraints.landauer_floor_plus_overhead_lower_bound DecisionQuotient/Physics/ WolpertConstraints.lean	WC2	Physics.WolpertConstraints.effective_model_dominates_landauer_floor DecisionQuotient/Physics/ WolpertConstraints.lean
WC3	Physics.WolpertConstraints.effective_model_strictly_exceeds_landauer_of_strict_overhead DecisionQuotient/Physics/ WolpertConstraints.lean	WC4	Physics.WolpertConstraints.energy_lower_bound_mono_under_overhead DecisionQuotient/Physics/ WolpertConstraints.lean
WC5	Physics.WolpertConstraints.physical_grounding_bundle_with_wolpert_overhead DecisionQuotient/Physics/ WolpertConstraints.lean	WD1	DecisionQuotient.checking_witnessing_duality_budget DecisionQuotient/WitnessCheckingDuality.lean
WD2	DecisionQuotient.no_sound_checker_below_witness_budget DecisionQuotient/WitnessCheckingDuality.lean	WD3	DecisionQuotient.checking_time_ge_witness_budget DecisionQuotient/WitnessCheckingDuality.lean
WD4	DecisionQuotient.witnessBudgetEmpty DecisionQuotient/WitnessCheckingDuality.lean	WD5	DecisionQuotient.checkingBudgetPairs DecisionQuotient/WitnessCheckingDuality.lean
WM1	Physics.WolpertMismatch.mismatchKL_nonneg DecisionQuotient/Physics/WolpertMismatch.lean	WM2	Physics.WolpertMismatch.mismatchKL_eq_zero_iff_eq DecisionQuotient/Physics/WolpertMismatch.lean
WM3	Physics.WolpertMismatch.mismatchKL_pos_of_exists_ne DecisionQuotient/Physics/WolpertMismatch.lean	WM4	Physics.WolpertMismatch.mismatchNatLowerBound_pos_of_exists_ne DecisionQuotient/Physics/WolpertMismatch.lean
WM5	Physics.WolpertDecomposition.periodic_modular_mismatch_of_distribution_mismatch DecisionQuotient/Physics/ WolpertDecomposition.lean	WM6	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_distribution_mismatch DecisionQuotient/Physics/ WolpertDecomposition.lean
WP1	Physics.WolpertDecomposition.DecomposedProcessModel.totalOverheadPerBit_eq_sum DecisionQuotient/Physics/ WolpertDecomposition.lean	WP2	Physics.WolpertDecomposition.landauer_floor_plus_decomposition_lower_bound DecisionQuotient/Physics/ WolpertDecomposition.lean

(continued...)

ID	Lean Handle / Source	ID	Lean Handle / Source
WP3	Physics.WolpertDecomposition.effective_model_dominates_landauer_floor_decomposition DecisionQuotient/Physics/ WolpertDecomposition.lean	WP5	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_stopping_time_residual DecisionQuotient/Physics/ WolpertDecomposition.lean
WP6	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_either_cited_component DecisionQuotient/Physics/ WolpertDecomposition.lean	WP7	Physics.WolpertDecomposition.landauer_floor_plus_structural_resource_lower_bound DecisionQuotient/Physics/ WolpertDecomposition.lean
WP8	Physics.WolpertDecomposition.energy_lower_bound_increases_by_structural_resource DecisionQuotient/Physics/ WolpertDecomposition.lean	WP9	Physics.WolpertDecomposition.physical_grounding_bundle_with_wolpert_decomposition DecisionQuotient/Physics/ WolpertDecomposition.lean
WR1	Physics.WolpertResidual.pairwiseResidualKL_nonneg DecisionQuotient/Physics/WolpertResidual.lean	WR2	Physics.WolpertResidual.pairwiseResidualKL_pos_of_asymmetry DecisionQuotient/Physics/WolpertResidual.lean
WR3	Physics.WolpertResidual.residualNatLowerBound_pos_of_asymmetry DecisionQuotient/Physics/WolpertResidual.lean	WR4	Physics.WolpertDecomposition.stopping_time_residual_of_pairwise_flow_asymmetry DecisionQuotient/Physics/ WolpertDecomposition.lean
WR5	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_pairwise_flow_asymmetry DecisionQuotient/Physics/ WolpertDecomposition.lean	WR6	Physics.WolpertResidual.discreteResidualNatLowerBound_pos_of_asymmetry_or_oneway DecisionQuotient/Physics/WolpertResidual.lean
WR7	Physics.WolpertDecomposition.stopping_time_residual_of_discrete_edge_split DecisionQuotient/Physics/ WolpertDecomposition.lean	WR8	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_discrete_edge_split DecisionQuotient/Physics/ WolpertDecomposition.lean
WR9	Physics.WolpertDecomposition.stopping_time_residual_of_finite_discrete_witness DecisionQuotient/Physics/ WolpertDecomposition.lean	WR10	Physics.WolpertDecomposition.effective_model_strictly_exceeds_landauer_of_finite_discrete_witness DecisionQuotient/Physics/ WolpertDecomposition.lean

A.2 What Is Machine-Checked

The Lean formalization establishes:

1. **Correctness of the TAUTOLOGY reduction:** The theorem `tautology_iff_sufficient` proves that the mapping from Boolean formulas to decision problems preserves the decision structure (accept iff the formula is a tautology).
2. **Decision problem definitions:** Formal definitions of sufficiency, optimality, and the decision quotient.
3. **Economic theorems:** Simplicity Tax redistribution identities and hardness distribution results.
4. **Query-access lower-bound core:** Formalized Boolean query-model indistinguishability theorem for the full problem via the $I = \emptyset$ subproblem (`emptySufficiency_query_indistinguishable_pair`), plus obstruction-scale identities (`queryComplexityLowerBound`, `exponential_query_complexity`).

Complexity classifications (coNP-completeness, Σ_2^P -completeness) follow by conditional composition with standard results (e.g., TAUTOLOGY coNP-completeness and $\exists\forall$ -SAT Σ_2^P -completeness), represented explicitly as hypotheses in the conditional transfer theorems listed below.

The Lean proofs verify the reduction constructions and the transfer closures under those hypotheses. The assumptions themselves are unpacked by an explicit ledger projection theorem (CC5) so dependency tracking is machine-auditable.

A.3 Assumption Ledger (Auto)

Source files.

- `DecisionQuotient/ClaimClosure.lean`
- `DecisionQuotient/HandleAliases.lean`
- `DecisionQuotient/StochasticSequential/ClaimClosure.lean`
- `DecisionQuotient/StochasticSequential/Hierarchy.lean`

Assumption bundles and fields.

- (No assumption bundles parsed.)

Conditional closure handles.

- `DQ.anchor_sigma2p_complete_conditional`
- `DQ.cost_asymmetry_eth_conditional`
- `DQ.dichotomy_conditional`
- `DQ.minsuff_collapse_to_conp_conditional`
- `DQ.minsuff_conp_complete_conditional`
- `DQ.sufficiency_conp_complete_conditional`
- `DQ.tractable_subcases_conditional`

First-principles Bayes derivation handles.

- `DQ.BayesOptimalityProof.KL_eq_sum_klFun`
- `DQ.BayesOptimalityProof.KL_eq_zero_iff_eq`
- `DQ.BayesOptimalityProof.KL_nonneg`
- `DQ.BayesOptimalityProof.bayes_is_optimal`
- `DQ.BayesOptimalityProof.crossEntropy_eq_entropy_add_KL`
- `DQ.BayesOptimalityProof.entropy_le_crossEntropy`
- `DQFOUNDATIONS.bayes_from_conditional`
- `DQFOUNDATIONS.complete_chain`
- `DQFOUNDATIONS.counting_additive`

- `DQ.Foundations.counting_nonneg`
- `DQ.Foundations.counting_total`
- `DQ.Foundations.dq_equivalence`
- `DQ.Foundations.dq_in_unit_interval`
- `DQ.Foundations.dq_one_iff_perfect_info`
- `DQ.Foundations.dq_zero_iff_no_info`
- `DQ.Foundations.entropy_contraction`
- `DQ.Foundations.mutual_info_nonneg`
- `DQ.bayesian_dq_matches_physics_dq`
- `DQ.chain_rule`
- `DQ.dq_derived_from_bayes`
- `DQ.dq_is_bayesian_certainty_fraction`

Compact Bayes handle IDs.

- `FN7 → DQ.BayesOptimalityProof.KL_nonneg`
- `FN8 → DQ.BayesOptimalityProof.entropy_le_crossEntropy`
- `FN12 → DQ.BayesOptimalityProof.crossEntropy_eq_entropy_add_KL`
- `FN14 → DQ.BayesOptimalityProof.bayes_is_optimal`
- `FN15 → DQ.BayesOptimalityProof.KL_eq_sum_klFun`
- `FN16 → DQ.BayesOptimalityProof.KL_eq_zero_iff_eq`
- `BB1 → DQ.BayesianDQ`
- `BB2 → DQ.BayesianDQ.certaintyGain`
- `BB3 → DQ.dq_is_bayesian_certainty_fraction`
- `BB4 → DQ.bayesian_dq_matches_physics_dq`
- `BB5 → DQ.dq_derived_from_bayes`
- `BF1 → DQ.certainty_of_not_nondegenerateBelief`
- `BF2 → DQ.nondegenerateBelief_of_uncertaintyForced`
- `BF3 → DQ.forced_action_under_uncertainty`
- `BF4 → DQ.bayes_update_exists_of_nondegenerateBelief`

A.4 Polynomial-Time Reduction Definition

We use Mathlib’s Turing machine framework to define polynomial-time computability:

```

/-- Polynomial-time computable function using Turing machines -/
def PolyTime {α β : Type} (ea : FinEncoding α) (eb : FinEncoding β)
  (f : α → β) : Prop :=
  Nonempty (Turing.TM2ComputableInPolyTime ea eb f)

/-- Polynomial-time many-one (Karp) reduction -/
def ManyOneReducesPoly {α β : Type} (ea : FinEncoding α) (eb : FinEncoding β)
  (A : Set α) (B : Set β) : Prop :=
  ∃ f : α → β, PolyTime ea eb f ∧ ∀ x, x ∈ A ↔ f x ∈ B

```

This uses the standard definition: a reduction is polynomial-time computable via Turing machines and preserves membership.

Note on verification status. The Lean formalization machine-verifies *correctness* (the membership-preservation property, i.e., the second conjunct of `ManyOneReducesPoly`). Proving that a concrete function satisfies `PolyTime`—i.e., constructing an actual Turing machine witness with a polynomial time bound—remains an open problem in formal verification. No existing proof assistant project has achieved end-to-end TM-based polytime proofs for concrete reductions. In our formalization, polytime computability is stated as an explicit hypothesis in `valid_karp_reduction`, making the gap visible rather than hiding it.

A.5 The Main Reduction Theorem

Theorem A.1 (TAUTOLOGY Reduction Correctness, Lean). *The reduction from TAUTOLOGY to SUFFICIENCY-CHECK is correct:*

```

theorem tautology_iff_sufficient (φ : Formula n) :
  φ.isTautology ↔ (reductionProblem φ).isSufficient Finset.empty

```

This theorem is proven by showing both directions:

- If φ is a tautology, then the empty coordinate set is sufficient
- If the empty coordinate set is sufficient, then φ is a tautology

The proof verifies that the utility construction in `reductionProblem` creates the appropriate decision structure where:

- At reference states, `accept` is optimal with utility 1
- At assignment states, `accept` is optimal iff $\varphi(a) = \text{true}$

A.6 Economic Results

The hardness distribution theorems (Section 13) are fully formalized:

```

theorem simplicityTax_conservation (P : SpecificationProblem)
  (S : SolutionArchitecture P) :
    S.centralDOF + simplicityTax P S ≥ P.intrinsicDOF

theorem simplicityTax_grows (P : SpecificationProblem)
  (S : SolutionArchitecture P) (n1 n2 : ℕ)
  (hn : n1 < n2) (htax : simplicityTax P S > 0) :
    totalDOF S n1 < totalDOF S n2

theorem native_dominates_manual (P : SpecificationProblem) (n : Nat)
  (hn : n > P.intrinsicDOF) :
    totalDOF (nativeTypeSystem P) n < totalDOF (manualApproach P) n

theorem totalDOF_eventually_constant_iff_zero_distributed
  (S : SolutionArchitecture P) :
    IsEventuallyConstant (fun n => totalDOF S n) ↔ S.distributedDOF = 0

theorem no_positive_slope_linear_represents_saturating
  (c d K : ℕ) (hd : d > 0) :
    ¬ (∀ n, c + n * d = generalizedTotalDOF c (saturatingSiteCost K) n)

```

Identifier note. Lean identifiers retain internal naming (`intrinsicDOF`, `simplicityTax_conservation`); in paper terminology these correspond to *baseline hardness* and the *redistribution lower-bound identity*, respectively.

A.7 Complete Claim Coverage Matrix

Paper claim	Lean handle
Corollary 9.73: Channel Capacity Degradation	CH2, CH6
Corollary 12.2: Exact Relevance Identifiability Criterion	DP1, DP2
Corollary 3.25: Exact-No-Competence Zero-Certified Constraint	IC41
Corollary 3.48: Finite Budget Eventually Blocks Exact Admissibility	PBC8, PBC7
Corollary 9.22: Finite Budget Implies Eventual Exact-Certification Failure	PBC8, PBC7
Corollary 9.68: Finite Computational Lifetime	SE5
Corollary 5.8: Finite Accessible State Space	BA2, BA4
Corollary 5.2: Forced Finiteness of Propagation Speed	BA10
Corollary 13.7: Gap Externalization	HD21, HD26
Corollary 13.8: Exact Minimization Criterion	SK1, DP1
Corollary 12.26: Eventual Generalized Dominance	HD10
Corollary 5.12: $\text{srank} = 1$ is the Energy Ground State	BA8
Corollary 9.62: Ground-State Atoms Are Passive	AC6
Corollary 3.44: Hardness-Blocked Regimes Induce Exact Certainty Inflation	CC19
Corollary 3.49: Hardness-Blocked Exact Reports Are Raw-Only	CC21, IC23
Corollary 9.43: Competence Is Importable; Integrity Is Not	SR4, SR5
Corollary 9.5: Information Barrier in Query Regimes	CC27, CC28, CC29
Definition 3.10: Solver Integrity	CC31
Corollary 3.38: Singleton Interior Certificate	IV9
Corollary 12.30: Positive Linear Slope Cannot Represent Saturation	HD16
Corollary 9.32: Neukart–Vinokur Thermodynamic Duality	CC50, CC65
Corollary 11.6: Impossibility of Automated Configuration Minimization	CC37
Corollary 9.39: No Free Computation	CC7, CC5, CC8
Corollary 3.50: No Uncertified Exact-Exhaustion Claim	CC41
Corollary 9.26: No Universal Survivor Without Succinct Bound	PBC10
Corollary 5.15: Observer-Level Rejection Cascade	IA13, IA2, IA5, IA9, IA17, IA18
Corollary 3.57: Outside Carve-Outs, Declared Hardness Blocks Exact Reports	CC39
Corollary 12.1: Persistent Over-Specification	DG18
Corollary 9.55: Publication Phase Transition	IE12, IE13
Corollary 2.47: Physical Counterexample Implies Core Failure on Encoded Slice	CT6, CT5, CT4
Corollary 3.63: No Universal Exact-Claim Admissibility in Hardness-Blocked Classes	CC38
Corollary 2.30: Certified-Confidence Gate for Posed Anchor Queries	AQ8
Corollary 12.7: Bounded-Regime Tractability	CC81
Corollary 12.6: Cardinality Criterion for Exact Minimization	SK1, DP1
Corollary 12.8: Separable-Utility Tractability	CC82
Corollary 12.12: Coordinate-Symmetry Tractability	CC72
Corollary 12.10: Low-Tensor-Rank Tractability 116	CC71
Corollary 12.9: Tree-Structured Tractability	CC84
Corollary 12.11: Bounded-Treewidth Tractability	CC73, CC75
Corollary 12.13: Mechanized Hard Family	CC26

Auto summary: mapped 236/236 (full=236, derived=0, unmapped=0).

A.8 Proof Hardness Index (Auto)

Paper handle	Hardness profile	Regime tags	Lean support
cor:	cost-growth	H=cost-growth,Q_fin	CH2 , CH6
channel-degradation	cost-growth	H=cost-growth,Q_fin	SE5
cor:finite-lifetime	cost-growth	H=cost-growth	HD21 , HD26
cor:	cost-growth	H=cost-growth	HD10
gap-externalization	cost-growth	H=cost-growth	HD10
cor:generalized-eventual-dominance	cost-growth	H=cost-growth	HD10
cor:hardness-raw-only-exact	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC21 , IC23
cor:linear-positive-no-saturation	cost-growth	H=cost-growth	HD16
cor:	cost-growth	AR,H=cost-growth	CC7 , CC5 , CC8
no-free-computation	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CT6 , CT5 , CT4
cor:physical-counterexample-core-failure	cost-growth	H=cost-growth	HD19
cor:	cost-growth	H=cost-growth	HD19
right-wrong-hardness	cost-growth	H=cost-growth,Q_fin	SE6
cor:speed-integrity	cost-growth	AR,H=cost-growth	DQ7 , DQ8
cor:thermo-dq	cost-growth	H=cost-growth	HD15
cor:type-system-threshold	cost-growth	H=cost-growth	HD15
cor:zero-capacity	cost-growth	H=cost-growth,Q_fin	CH3
prop:	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	DT13 , DT11 , DT12 , DT10
decision-unit-time	cost-growth	H=cost-growth	HD3
prop:dominance-modes	cost-growth	H=cost-growth	HD7 , HD8
prop:generalized-assumption-boundary	cost-growth	H=cost-growth	HD23 , HD25
prop:hardness-conservation	cost-growth	H=cost-growth	HD11
prop:	cost-growth	H=cost-growth	HD11
hardness-efficiency-interpretation	cost-growth	H=cost-growth	HD11
prop:oracle-lattice-strict	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC27 , CC29
prop:oracle-lattice-transfer	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC43 , CC52
prop:query-regime-obstruction	cost-growth	AR,E,H=cost-growth,LC,Q_fin,Qb,Qf,RG,S,S+ETH	CC50
prop:	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC28 , CC50
query-state-batch-lb	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC52 , CC58 , CC59
prop:query-subproblem-transfer	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC29 , CC50
prop:	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC29 , CC50
query-value-entry-lb	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	CC11 , IC10 , IC11 , IC13 , IC14 , IC15 , IC16 , IC18 , IC19
prop:raw-certified-bit-split	cost-growth	H=cost-growth,Q_fin	MI3 , MI4
prop:	cost-growth	H=cost-growth,Q_fin	MI3 , MI4
reaction-competence	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	DT18 , DT19 , DT16 , DT15
prop:	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	DT18 , DT19 , DT16 , DT15
run-time-accounting	cost-growth	AR,H=cost-growth	DC36 , DC35 , DC37
prop:stochastic-landauer-floor	cost-growth	AR,H=cost-growth	DC36 , DC35 , DC37
prop:	cost-growth	AR,H=cost-growth	SR1 , SR2 , SR3
structural-asymmetry	cost-growth	AR,H=cost-growth	SR1 , SR2 , SR3
prop:	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	DT23 , DT22 , DT24
substrate-unit-time	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG,S,S+ETH	DT23 , DT22 , DT24

prop: thermo-conservation-additive	cost-growth	H=cost-growth	CC64
prop:thermo-hardness-bundle	cost-growth	H=cost-growth	CC67
prop:thermo-lift	cost-growth	H=cost-growth	CC65, CC66
prop:thermo-mandatory-cost	cost-growth	H=cost-growth	CC68
prop:time-discrete	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG, S,S+ETH	DT7, DT6
thm:amortization	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	HD4
thm:bayes-from-dq	cost-growth	AR,H=cost-growth	FN14, BF4, BB4, BB5, BB3, BF3, BF2
thm:bridge-boundary-represented	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	CC8, CC9, CC10
thm:centralization-dominance	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	HD1, HD2
thm:choice-pays	cost-growth	AR,H=cost-growth	GE3, GE7
thm:conservation	cost-growth	H=cost-growth,Q_fin	IV4
thm:cycle-cost	cost-growth	AR,H=cost-growth	CC1, CC2, CC3
thm:deficit-source	cost-growth	H=cost-growth,Q_fin	IV5, IV6
thm:dichotomy	cost-growth	AR,E,H=cost-growth,LC,Q_fin, Qb,Qf,RG,S,S+ETH	CC16, CC23, CC24
thm:dq-physical	cost-growth	AR,H=cost-growth	DQ1, DQ2, DQ3, DQ4, DQ5, DQ6
thm:exact-certified-gap-iff-admissible	cost-growth	AR,E,H=cost-growth,Qb,Qf,RG, S,S+ETH	CC17, CC18, CC20, IC31
thm:generalized-dominance	cost-growth	H=cost-growth	HD9
thm:generalized-saturation-possible	cost-growth	H=cost-growth	HD6, HD20
thm:growth-time	cost-growth	H=cost-growth,Q_fin	IV1, IV2, IV3
thm:linear-saturation-iff-zero	cost-growth	H=cost-growth	HD19
thm:orbital-cost	cost-growth	H=cost-growth,Q_fin	AC3, AC4
thm:regime-coverage	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	CC14, CC51
thm:substrate-degradation	cost-growth	H=cost-growth,Q_fin	SE1, SE2, SE3, SE4
thm:tax-conservation	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	HD5
thm:tractable	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH,STR,TA	CC70, CC71, CC72, CC73
thm:typed-completeness-static	cost-growth	AR,E,H=cost-growth,LC,Qb,Qf, RG,S,S+ETH	CC76
cor: finite-budget-no-exact-admissibility	exp-lb-conditional	H=exp-lb-conditional	PBC8, PBC7
cor:finite-budget-threshold-impossibility	exp-lb-conditional	H=exp-lb-conditional	PBC8, PBC7
cor:no-universal-survivor-no-succinct-bound	exp-lb-conditional	H=exp-lb-conditional	PBC10
prop: budgeted-crossover	exp-lb-conditional	H=exp-lb-conditional	CC45, PBC1, CT8, CT6, CT4
prop: crossover-above-cap	exp-lb-conditional	H=exp-lb-conditional	CC44, PBC2
prop:crossover-not-certification	exp-lb-conditional	H=exp-lb-conditional	CC46
prop: crossover-policy	exp-lb-conditional	H=exp-lb-conditional	CC47
prop: eventual-explicit-infeasibility	exp-lb-conditional	H=exp-lb-conditional	PBC7
prop:least-divergence-point	exp-lb-conditional	H=exp-lb-conditional	PBC5
prop: payoff-threshold	exp-lb-conditional	H=exp-lb-conditional	PBC8, PBC9

prop:policy-closure-beyond-divergence	exp-lb-conditional	H=exp-lb-conditional	PBC11, PBC12
cor:information-barrier-query	query-lb	H=query-lb	CC27, CC28, CC29
cor:query-obstruction-bool	query-lb	H=query-lb	CC49, CC50
prop:checking-witnessing-duality	query-lb	H=query-lb	WD5, WD3, WD4
prop:query-finite-state-generalization	query-lb	H=query-lb	CC50
prop:query-randomized-robustness	query-lb	H=query-lb	CC50
prop:query-randomized-weighted	query-lb	H=query-lb	CC50
prop:query-tightness-full-scan	query-lb	H=query-lb	CC50
prop:query-weighted-transfer	query-lb	H=query-lb	CC50
cor:exact-identifiability	succinct-hard	H=succinct-hard	DP1, DP2
cor:gap-minimization-hard	succinct-hard	H=succinct-hard	SK1, DP1
cor:no-auto-minimize	succinct-hard	H=succinct-hard	CC37
cor:overmodel-diagnostic-implication	succinct-hard	H=succinct-hard	DG18
cor:practice-diagnostic	succinct-hard	H=succinct-hard	SK1, DP1
prop:sufficiency-char	succinct-hard	H=succinct-hard	CC62, CC63
cor:interior-singleton-certificate	tractable-structured	H=tractable-structured	IV9
cor:practice-bounded	tractable-structured	H=tractable-structured	CC81
cor:practice-structured	tractable-structured	H=tractable-structured	CC82
cor:practice-symmetry	tractable-structured	H=tractable-structured	CC72
cor:practice-tensor	tractable-structured	H=tractable-structured	CC71
cor:practice-tree	tractable-structured	H=tractable-structured	CC84
cor:practice-treewidth	tractable-structured	H=tractable-structured	CC73, CC75
prop:heuristic-reusability	tractable-structured	H=tractable-structured	CC7, CC53, CC55, CC74
prop:interior-verification-tractable	tractable-structured	H=tractable-structured	IV4, IV7
prop:orbital-symmetry	tractable-structured	H=tractable-structured, Q _{fin}	AC8
thm:complexity-dichotomy	tractable-structured	AR, E, H=tractable-structured, Q, S	DC13, DC3
thm:six-subcases	tractable-structured	AR, E, H=tractable-structured, Q, S	CC70, CC71, CC72, CC73, DC13
cor:exact-no-competence-zero-certified	unspecified	AR	IC41
cor:finite-state	unspecified	-	BA2, BA4
cor:forced-finite-speed	unspecified	-	BA10
cor:ground-state	unspecified	AR	BA8
cor:ground-state-passive	unspecified	AR, Q _{fin}	AC6
cor:hardness-exact-certainty-inflation	unspecified	CR	CC19

cor:import-asymmetry	unspecified	AR	SR4, SR5
cor:	unspecified	TR	CC31
integrity-universal			
cor:neukart-vinokur	unspecified	AR	CC50, CC65
cor:no-uncertified-exact-claim	unspecified	AR	CC41
cor:observer-rejection-cascade	unspecified	-	IA13, IA2, IA5, IA9, IA17, IA18
cor:outside-excuses-no-exact-report	unspecified	CR,DC	CC39
cor:phase-transition	unspecified	AR	IE12, IE13
cor:physics-no-universal-exact-claim	unspecified	CR	CC38
cor:posed-anchor-certified-gate	unspecified	-	AQ8
cor:practice-unstructured	unspecified	-	CC26
cor:rlff-abstain-no-certs	unspecified	AR	IC30, IC40
cor:	unspecified	AR	IE3, IE4, IE5
theorem-equilibrium			
cor:zero-gap	unspecified	AR	GE4, GE5
prop:abstain-guess-self-signal	unspecified	AR	IC46
prop:	unspecified	RG	CC39
abstention-frontier			
prop:adq-ordering	unspecified	-	CC2
prop:attempted-competence-matrix	unspecified	AR	IC6, IC7, IC27
prop:bounded-region	unspecified	-	BA1
prop:bounded-slice-meta-irrelevance	unspecified	AR	CC32, CC33
prop:bridge-failure-horizon	unspecified	-	CC25, CC26
prop:bridge-failure-stochastic	unspecified	-	CC57
prop:bridge-failure-transition	unspecified	-	CC73
prop:bridge-transfer-scope	unspecified	-	CC42
prop:	unspecified	CR	IC8, IC22
certainty-inflation-iff-inadmissible			
prop:certified-confidence-gate	unspecified	AR	IC38, IC39
prop:	unspecified	AR	DS5, DS6
comp-thermo-chain			
prop:	unspecified	AR	DE1, DE2, DE3, DE4
decision-equivalence			
prop:	unspecified	-	CC22, CC39, CC75
declared-contract-selection-validity			
prop:	unspecified	AR	DS1, DS2
discrete-state-time			
prop:empty-sufficient-constant	unspecified	DM	DP6
prop:evidence-admissibility-equivalence	unspecified	AR	IC17, IC20, IC21
prop:exact-requires-evidence	unspecified	AR	IC35, IC36
prop:explicit-state-inp-wrappers	unspecified	explicit-state step-counting P	DC76, DC79, DC71, DC77, DC70, DC78, DC75, DC74, DC73, DC72
prop:fraction-defined-under-bound	unspecified	AR	IC45

prop: heisenberg-strong- nontrivial-opt	unspecified	AR	HS3, HS6, HS5
prop: identifiability- convergence	unspecified	ID	CC19
prop:insufficiency- counterexample	unspecified	DM	DP7, DP8
prop:integrity- competence- separation	unspecified	AR	IC18, IC25
prop:integrity- prerequisite	unspecified	AR	IE14, IE15, IE16, IE17
prop:integrity- resource-bound	unspecified	AR,S,S+ETH	CC30, IC24, IC26
prop:interior-one- sidedness	unspecified	AR	IV6, IV8
prop: interior-universal- non-rejection	unspecified	AR	IV5
prop: landauer-constraint	unspecified	AR	IE1, IE6
prop:law-instance- objective-bridge	unspecified	AR	CT1, CT2
prop: lorentz-discrete	unspecified	AR	DS3, DS4
prop:mdp-tractable	unspecified	FO	CC50
prop:minimal- relevant-equiv	unspecified	-	DP2, DP3
prop:no-evidence- zero-certified	unspecified	AR	IC40
prop:no-fraction- without-bound	unspecified	AR	IC44
prop:one-step-bridge	unspecified	-	CC42
prop: optimizer-coimage	unspecified	-	QT7
prop:optimizer- entropy-image	unspecified	-	IT1
prop: outside-excuses- explicit-assumptions	unspecified	CR,DC	CC22
prop:physical-claim- transport	unspecified	AR	CT6, CT3, CT5
prop:physics-no- universal-exact	unspecified	CR	CC15
prop: pose-anchor-object	unspecified	-	AQ1, AQ2, AQ3
prop:posed-anchor- typed-exact	unspecified	-	AQ4, AQ5, AQ6, AQ7
prop:refinement- strengthens	unspecified	RG	CC69
prop:retraction- evidence-integrity	unspecified	RG	CC70
prop:retraction-no- evidence-violates	unspecified	RG	CC71
prop:rlff-maximizer- admissible	unspecified	AR	IC31, IC32
prop: selector-separation	unspecified	-	CC54
prop: self-confidence-not- certification	unspecified	AR	IC47
prop: sequential-anchor- hardness-package	unspecified	PSPACE	DC43, DC44
prop:sequential- anchor-refinement	unspecified	PSPACE	DC23

prop: sequential-anchor- tqbf-reduction	unspecified	PSPACE	DC27, DC28, DC29, DC44
prop:sequential- bounded-horizon	unspecified	BH	CC49
prop:sequential- counted-search	unspecified	finite explicit search	DC68, DC69, DC60, DC61, DC66, DC67
prop:sequential- finite-deciders	unspecified	finite exact deciders	DC55, DC56, DC54
prop:sequential- minimal-relevant	unspecified	structural	DC49, DC50
prop: sequential-minimum- hardness-package	unspecified	PSPACE	DC48, DC46
prop:sequential- static-relation	unspecified	DET,T=1	CC46
prop:set-to-selector	unspecified	-	DP5
prop:snapshot-	unspecified	RA	CC48, CC56, CC3
process-typing			
prop:srnk-support	unspecified	-	SK1, SK2, SK3
prop:static-	unspecified	$P \neq \text{coNP}$	DC1
stochastic-strict			
prop:static-	unspecified	PD	CC52
stochastic-transfer			
prop:	unspecified	AR	IC42, IC43
steps-run-scalar			
prop:	unspecified	PP	DC40, DC41, DC42
stochastic-anchor-			
direct-reduction			
prop:stochastic-	unspecified	PP	DC19
anchor-refinement			
prop:	unspecified	PP	DC24, DC25, DC26
stochastic-anchor-			
strict-reduction			
prop:stochastic-	unspecified	BS	CC59
bounded-support			
prop:stochastic-	unspecified	finite explicit search	DC64, DC65, DC58, DC59, DC62, DC63
counted-search			
prop:stochastic-	unspecified	finite exact deciders	DC52, DC53, DC51
finite-deciders			
prop:	unspecified	PP	DC47, DC45
stochastic-minimum-			
hardness-package			
prop:stochastic-	unspecified	PP	DC33, DC34, DC31
potential-duality			
prop:stochastic-	unspecified	PD	CC60
product-tractable			
prop:stochastic-	unspecified	CE	CC56
sequential-bridge-			
fail			
prop:stochastic-	unspecified	$P \neq \text{PP}$	DC2
sequential-strict			
prop:	unspecified	AR	IE10, IE11, IE7, IE8, IE9
temporal-equilibrium			
prop:typed-claim-	unspecified	AR,DC,Qf,S+ETH	CC75
admissibility			
prop:typed-physical-	unspecified	AR	PS3, PS4
transport-			
requirement			
prop:under-	unspecified	DM	PI6, PI7
resolution-collision			
prop:universal-	unspecified	TR	CC77
solver-framing			
prop:zero-epsilon-	unspecified	-	IC19, IC33
competence			
prop:zero-epsilon-	unspecified	-	DP4, DP1
reduction			

thm:	unspecified	-	AB3, AB1, AB2, AB4
abstraction-boundary			
thm:	unspecified	-	AN4, AN3
assumption-necessity			
thm:	unspecified	-	BC4, BC3, BC1, BC2, BC5
bayes-from-counting			
thm: bayes-optimal	unspecified	-	FN7, FN14, FN12
thm:	unspecified	-	BA4
boolean-primitive			
thm:	unspecified	-	BA2
bounded-acquisition			
thm: checking-duality	unspecified	-	WD3, WD1, WD2
thm:	unspecified	CR, DC	CC22
claim-integrity-meta			
thm:	unspecified	AR, E, Q, S	IA5, IA6
competence-access			
thm:	unspecified	AR, Q_fin	CH1, CH5
competence-capacity			
thm: config-reduction	unspecified	AR, E, RG, S, TA	CR1
thm:	unspecified	S+ETH	CC12, HD14
cost-asymmetry-eth			
thm: counting-gap	unspecified	-	BA10
thm:	unspecified	-	BA10
counting-gap-intro			
thm:	unspecified	-	BA3
discrete-acquisition			
thm: ec2-derived	unspecified	-	IA11, IA12, IA13, IA7, IA9
thm: ec3-derived	unspecified	-	FPT10, FPT4, FPT5, FPT6, FPT8
thm:	unspecified	-	EI1
energy-information			
thm: entropy-rank	unspecified	-	IT4, IT3
thm: fi-coincide	unspecified	-	FI7, FI3, FI6, WC2, WC3, WC1, WP1, WR8, WM6, WP6, WR10, WR5, WP5, WP8, WP7, WM5, WR7, WR9, WR4, WM1, WM3, WM4, WR6, WR1, WR2, WR3
			FS2, FS1
thm:	unspecified	-	
fisher-rank-srank			
thm: gap-physical	unspecified	AR, E, Q, S	IA4
thm: information-gap	unspecified	AR, E, Q, S	IA3
thm:	unspecified	-	FP15
landauer-structure			
thm:	unspecified	-	MN1, MN10, MN2, MN11
measure-prerequisite			
thm: nontriviality-counting	unspecified	-	FP3, FP2, FP7, FP4, FP1, FP6, FP5
thm:	unspecified	AR, Q_fin	AC1, AC5
orbital-transition			
thm:	unspecified	S	CC6, CC40
overmodel-diagnostic			
thm: physical-bridge-bundle	unspecified	AR	CT4
thm: physical-incompleteness	unspecified	AR	PI3, PI5, PI4
thm:	unspecified	DM	QT2, QT7, QT1, QT3
quotient-universal			
thm: rate-distortion-bridge	unspecified	-	RS3, RS1, RS5, RS2, RD3, RD2, RD1, RS4
thm: resolution-sufficient	unspecified	-	BA5
thm:	unspecified	-	FP8, FP14, FP10, FP9, FP12, FP13, FP11
second-law-counting			
thm: srank-physical	unspecified	-	SR1, BA7, BA5, BA6
thm: tax-grows	unspecified	LC	HD26
thm: thermo-derived	unspecified	-	BA9, WC1, WC5, WP1, WR8, WM6, WP6, WR10, WR5, WP5, WP8, WP2, WP7, WM5, WP9, WR7, WR9, WR4, WM2, WM1, WM3, WM4, WR6, WR1, WR2, WR3
			CC16, CC24
thm: topology-motion	unspecified	AR, E, Q, S	TUR6, TUR1, TUR2, TUR5
thm: tur-bridge	unspecified	-	

thm:	unspecified	-	IA16, IA15
universe-membership			
thm:	unspecified	-	W3, W1, W2, W4
wasserstein-bridge			

Auto summary: indexed 236 claims by hardness profile (cost-growth=56; exp-lb-conditional=11; query-lb=8; succinct-hard=6; tractable-structured=12; unspecified=143).

A.9 Claims Not Fully Mechanized

Status: all theorem/proposition/corollary handles in this paper now have Lean backing. Entries marked **Full (conditional)** are explicitly mechanized transfer theorems that depend on standard external complexity facts (e.g., source-class completeness or ETH assumptions), with those dependencies represented as hypotheses in Lean.

A.10 Module Structure

- `Basic.lean` – Core definitions (DecisionProblem, sufficiency, optimality)
- `Sufficiency.lean` – Sufficiency checking algorithms and properties
- `Reduction.lean` – TAUTOLOGY reduction construction and correctness
- `Hardness/ConfigReduction.lean` – Sentinel-action configuration reduction theorem
- `Complexity.lean` – Polynomial-time reduction definitions using mathlib
- `HardnessDistribution.lean` – Simplicity Tax redistribution and amortization theorems
- `IntegrityCompetence.lean` – Solver integrity vs regime competence separation
- `ClaimClosure.lean` – Mechanized closure of paper-level bridge and diagnostic claims
- `Tractability/` – Bounded actions, separable utilities, tree structure

A.11 Entropy-Inflation Bridge (Implemented)

The inflation-to-entropy bridge is now implemented in Lean as a first-class module:

- `InflationEntropyBridge.lean` – dynamic time-indexed decision families, optimizer-class monotonicity, entropy monotonicity, strict-growth witness criteria, and thermal-floor corollaries.
- `InflationEntropyMinimality.lean` – irreducibility witnesses for key assumption groups (structural compatibility, strictness witness, positivity/floor premises).
- `StochasticSequential/SetValued.lean` – set-valued stochastic sufficiency layer (ties allowed), bridging static set semantics and stochastic regime semantics.

Core bridge theorems. The mechanization includes: `classes_monotone`, `entropy_monotone`, `classes_strict_increase`, `entropy_strict_increase`, `optCompat_of_utilityCompat`, `thermal_floor_monotone_of_classes`, `thermal_floor_strict_of_new_class`, and `later_energy_floor_implies_earlier_floor`.

Temporal adapter. The bridge now includes an explicit adapter from cosmological-cardinality growth to dynamic decision families via `TemporalUtilityFamily` and transfer lemmas `temporal_classes_monotone_of_utilityCompat` and `temporal_entropy_monotone_of_utilityCompat`.

Expansion-to-probability bridge. The same module now includes a probability-necessity layer on the temporally expanded valid successor slice: `state_cardinality_gt_one_of_positive_time`, `uniformPrior_uncertainty_of_positive_time`, `uniformPrior_nondegenerate_of_positive_time`, `counting_measure_not_probability_on_stateAt_of_positive_time`, `cosmological_expansion_forces_probabilistic_reasoning`, and the strengthened support-complete form `cosmological_expansion_forces_support_complete_probabilistic_reasoning`.

Chosen physical encoding identification. For a state-indexed physical-decision encoding, the development also proves `stateIndexedPhysicalEncoding_identifies_StateAt`, making the chosen physical-answer carrier explicitly coincide with the temporal slice `StateAt`.

Assumption-minimality status. Mechanized witnesses are present for representative drops of compatibility/strictness/positivity/floor assumptions. Some witnesses are established in intentionally weakened frameworks (to model removal of type-level side conditions such as embedding or nonemptiness) and are explicitly separated from the main bridge structure.

Handle integration. Compact handle aliases for the bridge and minimality witnesses are exported in `HandleAliases.lean` (including the IEB17–IEB24 expansion/probability handles), and corresponding axiom-audit prints are included in `CheckAxioms.lean`.

Complexity mechanization boundary. The fully internalized complexity content currently consists of exact finite boolean deciders, reduction correctness, and polynomial output-size bounds. In particular, the development now exposes direct size-bounded hardness packages `stochastic_sufficiency_pp_hard`, `stochastic_minimum_sufficiency_pp_hard`, `stochastic_anchor_check_pp_hard`, `sequential_sufficiency_pspace_hard`, and `sequential_minimum_sufficiency_pspace_hard`, and `sequential_anchor_check_pspace_hard`. It also proves exact-decider specifications for the six regime-typed boolean queries, together with explicit counted exhaustive-search procedures and step bounds for the static minimum and anchor queries and for all six stochastic/sequential regime-typed queries. End-to-end Turing-machine membership witnesses for PP/PSPACE are not yet mechanized in the same style.

Search-matrix packaging. The same layer now also packages these searches into explicit witness theorems for the static queries, including ordinary static sufficiency, and regime-wise search matrices for the stochastic and sequential queries. An integration theorem `finite_search_summary` then bundles the static, stochastic, and sequential counted-search layers into one summary statement.

Explicit-state abstract-P wrappers. The development further packages explicit-state abstract-P membership wrappers for static, stochastic, and sequential sufficiency/anchor queries, and also for the corresponding minimum queries once an explicit subset-budget is included in the input, using the repository’s step-counting model rather than full TM witnesses.

A.12 Verification

The proofs compile with Lean 4 and contain no `sorry` placeholders. Run `lake build` in the `proof` directory to verify.

B Entropy-Inflation Program Status

This section records the implemented entropy-via-inflation bridge, its minimality witnesses, and several natural extensions.

B.1 Available Machinery

- Static decision core: `DecisionProblem`, `Opt`, `numOptClasses`, `quotientEntropy`, and entropy-rank inequalities.
- Physical/cosmological layer: discrete-time and locality modules, plus temporal cardinality growth in `TemporalCountingGap.lean`.
- Thermodynamic bridge: `energy_ge_kbt_nat_entropy` and related floor theorems.
- Stochastic/sequential layer: PP/PSPACE hierarchy plus newly added set-valued stochastic bridge module.

B.2 Implemented Bridge Stack

Dynamic family core. The module `InflationEntropyBridge.lean` now contains:

- `classes_monotone`, `entropy_monotone`
- `classes_strict_increase`, `entropy_strict_increase`
- `optCompat_of_utilityCompat`
- `classes_monotone_of_utilityCompat`, `entropy_monotone_of_utilityCompat`
- `thermal_floor_monotone_of_classes`, `thermal_floor_strict_of_new_class`
- `later_energy_floor_implies_earlier_floor`

(L: [IEB1-8](#).)

Temporal adapter. The same module includes `TemporalUtilityFamily` and transfer lemmas `temporal_classes_monotone_of_utilityCompat` and `temporal_entropy_monotone_of_utilityCompat`, lifting temporal state-cardinality indexing into dynamic decision families.

Probability-necessity layer. On the temporally expanded valid successor slice, the same module now proves:

- `state_cardinality_gt_one_of_positive_time`
- `uniformPrior_uncertainty_of_positive_time`

- `uniformPrior_nondegenerate_of_positive_time`
- `counting_measure_not_probability_on_stateAt_of_positive_time`
- `cosmological_expansion_forces_probabilistic_reasoning`
- `cosmological_expansion_forces_support_complete_probabilistic_reasoning`

This strengthens the earlier bridge from monotone optimizer-class growth to a theorem that expansion of physically valid successor states forces probability normalization once raw counting ceases to be a probability measure.

Support-complete semantics and chosen encoding. The module now names the support-complete valid-answer semantics explicitly via `canonicalValidAnswerSemantics` and proves `canonicalValidAnswerSemantics_identifies_StateAt`. It also defines a chosen state-indexed physical-decision encoding and proves `stateIndexedPhysicalEncoding_identifies_StateAt`, making the physical answer carrier explicit at the level of the temporal slice.

Minimality witnesses. The module `InflationEntropyMinimality.lean` now includes explicit witnesses for:

- structural compatibility/strictness: `not_redundant_A2_for_mono_classes, not_redundant_A3_for_strict_entropy`
- positivity/floor assumptions: `not_redundant_P1_for_positive_floor, not_redundant_P2_for_positive_floor, not_redundant_P3_for_energy_from_entropy_bridge`
- weakened-framework requirement witnesses: `not_redundant_A1_for_mono_classes_weak, not_redundant_F1_for_finite_counting_requirement, not_redundant_F2_for_numOptClasses_pos`

(L: [IEB9-16](#).)

B.3 Stochastic Set-Valued Alignment

To reduce static/stochastic semantic friction, the module `StochasticSequential/SetValued.lean` adds:

- `StochasticSetSufficient` (ties allowed)
- `stochasticSetSufficient_universal`
- `stochasticSufficient_implies_setSufficient`

(L: [SSV1-3](#).) This provides a set-valued baseline in the stochastic regime while preserving the existing singleton-oriented notion.

B.4 Assumption-Coverage Snapshot

Assumption group	Monotone bridge	Strict bridge / floor
State embedding / compatibility	Implemented	Implemented
New-class witness	–	Implemented
Finite/nonempty side conditions	Implemented (core usage)	Weakened-form witnesses present
Physical positivity/floor premises	Monotone floor implemented	Strict floor + witnesses implemented

B.5 Further Extensions

1. Integrate the newly added IEB17–IEB24 handles into the generated claim and handle tables.
2. Strengthen requirement-minimality from weakened-framework witnesses to strongest same-structure internal forms where appropriate.
3. Extend beyond finite accessible slices to a separate measure-theoretic infinite-state layer.

References

- [1] Hirotugu Akaike. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6):716–723, 1974.
- [2] Robert M. Alaniz, Josh Brunner, Michael Coulombe, Erik D. Demaine, Jenny Diomidova, Timothy Gomez, Elise Grizzell, Ryan Knobel, Jayson Lynch, Andrew Rodriguez, Robert Schweller, and Tim Wylie. Complexity of reconfiguration in surface chemical reaction networks. *ArXiv preprint*, 2023.
- [3] Edoardo Amaldi and Viggo Kann. On the approximability of minimizing nonzero variables or unsatisfied relations in linear systems. *Theoretical Computer Science*, 209(1-2):237–260, 1998.
- [4] Jakob L. Andersen, Christoph Flamm, Daniel Merkle, and Peter F. Stadler. Maximizing output and recognizing autocatalysis in chemical reaction networks is np-complete. *Journal of Systems Chemistry*, 3(1):1–15, 2012.
- [5] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [6] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012.
- [7] Charles H. Bennett. The thermodynamics of computation—a review. *International Journal of Theoretical Physics*, 21(12):905–940, 1982.
- [8] Charles H. Bennett. Notes on landauer’s principle, reversible computation, and maxwell’s demon. *Studies in History and Philosophy of Science Part B: Studies in History and Philosophy of Modern Physics*, 34(3):501–510, 2003.
- [9] Antoine Bérut, Artak Arakelyan, Arshak Petrosyan, Sergio Ciliberto, Raoul Dillenschneider, and Eric Lutz. Experimental verification of landauer’s principle linking information and thermodynamics. *Nature*, 483(7388):187–189, 2012.
- [10] Richard E. Blahut. Computation of channel capacity and rate-distortion functions. *IEEE Transactions on Information Theory*, 18(4):460–473, 1972.
- [11] Avrim L. Blum and Pat Langley. Selection of relevant features and examples in machine learning. *Artificial Intelligence*, 97(1-2):245–271, 1997.
- [12] Hans L. Bodlaender. A tourist guide through treewidth. *Acta Cybernetica*, 11(1-2):1–21, 1993.

- [13] Ludwig Boltzmann. Über die beziehung zwischen dem zweiten hauptsatze der mechanischen wärmetheorie und der wahrscheinlichkeitsrechnung respektive den sätzen über das wärme-gleichgewicht. *Wiener Berichte*, 76:373–435, 1877. Establishes $S = k \ln W$ and the statistical foundation of thermodynamics.
- [14] Frederick P. Brooks. No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4):10–19, 1987.
- [15] Harry Buhrman and Ronald de Wolf. Complexity measures and decision tree complexity: A survey. *Theoretical Computer Science*, 288(1):21–43, 2002.
- [16] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [17] David Maxwell Chickering, David Heckerman, and Christopher Meek. Large-sample learning of Bayesian networks is NP-hard. *Journal of Machine Learning Research*, 5:1287–1330, 2004.
- [18] Andrzej Cichocki, Namgil Lee, Ivan Oseledets, Anh-Huy Phan, Qibin Zhao, and Danilo P. Mandic. *Tensor Networks for Dimensionality Reduction and Large-Scale Optimization*, volume 9. Now Publishers, 2016.
- [19] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.
- [20] Imre Csiszár and János Körner. *Information Theory: Coding Theorems for Discrete Memoryless Systems*. Cambridge University Press, 2nd edition, 2011.
- [21] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction – CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388. Springer, 2015.
- [22] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021.
- [23] Paul A. M. Dirac. The principles of quantum mechanics. *Oxford University Press*, 1930.
- [24] Shahar Dobzinski and Jan Vondrák. From query complexity to computational complexity. In *Proceedings of the 44th ACM Symposium on Theory of Computing*, pages 1107–1116, 2012.
- [25] Albert Einstein. Zur elektrodynamik bewegter körper. *Annalen der Physik*, 322(10):891–921, 1905. English: On the Electrodynamics of Moving Bodies.
- [26] Rolf Fagerberg, Christoph Flamm, Daniel Merkle, Philipp Peters, and Peter F. Stadler. On the complexity of reconstructing chemical reaction networks. *Mathematical Biosciences*, 242(2):86–96, 2013.
- [27] Ronald A. Fisher. On the mathematical foundations of theoretical statistics. *Philosophical Transactions of the Royal Society of London. Series A*, 222:309–368, 1922.
- [28] Yannick Forster, Fabian Kunze, and Maxi Wuttke. Verified programming of Turing machines in Coq. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 114–128. ACM, 2020.

- [29] Eugene C. Freuder. Complexity of k -tree structured constraint satisfaction problems. *Proceedings of AAAI*, pages 4–9, 1990.
- [30] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design patterns: elements of reusable object-oriented software*. Addison-Wesley, 1994.
- [31] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [32] Peter D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.
- [33] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003.
- [34] Maximilian P. L. Haslbeck and Tobias Nipkow. Verified analysis of list update algorithms. In *36th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2016)*, volume 65 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 49:1–49:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016.
- [35] Ronald A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1):22–26, 1966.
- [36] Russell Impagliazzo and Ramamohan Paturi. On the complexity of k -sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.
- [37] Edwin T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106(4):620–630, 1957. Information-theoretic foundation of statistical mechanics.
- [38] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.
- [39] Charles Kittel and Herbert Kroemer. *Thermal Physics*. W. H. Freeman, 2nd edition, 1980. Standard reference for kT as thermal energy scale.
- [40] Tamara G. Kolda and Brett W. Bader. Tensor decompositions and applications. *SIAM Review*, 51(3):455–500, 2009.
- [41] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.
- [42] János Körner. Coding of an information source having ambiguous alphabet and the entropy of graphs. In *Transactions of the 6th Prague Conference on Information Theory, Statistical Decision Functions, Random Processes*, pages 411–425, 1973.
- [43] Fabian Kunze, Gert Smolka, and Yannick Forster. Formal small-step verification of a call-by-value lambda calculus machine. In *Asian Symposium on Programming Languages and Systems*, pages 264–283. Springer, 2018.
- [44] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.
- [45] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

- [46] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition, 2008.
- [47] Barbara H Liskov and Jeannette M Wing. A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 16(6):1811–1841, 1994.
- [48] Michael L. Littman, Judy Goldsmith, and Martin Mundhenk. The computational complexity of probabilistic planning. *Journal of Artificial Intelligence Research*, 9:1–36, 1998.
- [49] László Lovász. On the shannon capacity of a graph. *IEEE Transactions on Information Theory*, 25(1):1–7, 1979.
- [50] Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2 edition, 1998.
- [51] Pau Manzano, Uğur Kardeş, Édgar Roldán, and David H. Wolpert. Absolute bounds on the intrinsic dissipation of classical and quantum processes due to irreversibility and mismatch. *Physical Review X*, 14(2):021026, 2024.
- [52] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011.
- [53] Paul Milgrom and John Roberts. Price and advertising signals of product quality. *Journal of Political Economy*, 94(4):796–821, 1986.
- [54] Hermann Minkowski. Raum und zeit. In *Physikalische Zeitschrift*, volume 10, pages 75–88, 1909. Address delivered at the 80th Assembly of German Natural Scientists, Cologne, 21 September 1908.
- [55] Martin Mundhenk, Judy Goldsmith, Christopher Lusena, and Eric Allender. The complexity of finite-horizon markov decision process problems. *Journal of the ACM*, 47(4):681–720, 2000.
- [56] Roger B. Myerson. Incentive compatibility and the bargaining problem. *Econometrica*, 47(1):61–73, 1979.
- [57] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997.
- [58] Florian Neukart and Valerii Vinokur. Thermodynamic duality and computational complexity in spin-glass systems. *arXiv preprint arXiv:2501.XXXXX*, 2025. Preprint.
- [59] Tobias Nipkow and Gerwin Klein. *Concrete Semantics with Isabelle/HOL*. Springer, 2014.
- [60] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*. Springer, 2002.
- [61] Emmy Noether. Invariante variationsprobleme. *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, pages 235–257, 1918. Noether’s theorem: continuous symmetries imply conservation laws.
- [62] Jose Nelson Onuchic, Zaida Luthey-Schulten, and Peter G. Wolynes. Theory of protein folding: The energy landscape perspective. *Annual Review of Physical Chemistry*, 48:545–600, 1997.

- [63] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994. Classic textbook; publisher DOI not in common use for this edition.
- [64] Christos H. Papadimitriou and John N. Tsitsiklis. The complexity of markov decision processes. *Mathematics of Operations Research*, 12(3):441–450, 1987.
- [65] David L Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [66] Wolfgang Pauli. Über den zusammenhang des abschlusses der elektronengruppen im atom mit der komplexstruktur der spektren. *Zeitschrift für Physik*, 31(1):765–783, 1925. Pauli exclusion principle: no two fermions occupy identical quantum states.
- [67] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.
- [68] Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.
- [69] Max Planck. Über irreversible strahlungsvorgänge. *Sitzungsberichte der Königlich Preußischen Akademie der Wissenschaften zu Berlin*, pages 440–480, 1899. Introduced Planck units.
- [70] Max Planck. Über das gesetz der energieverteilung im normalspektrum. *Annalen der Physik*, 309(3):553–563, 1901.
- [71] Howard Raiffa and Robert Schlaifer. *Applied Statistical Decision Theory*. Division of Research, Graduate School of Business Administration, Harvard University, Boston, MA, 1961. Library records list this imprint; avoid using JSTOR DOI 10.2307/2282452 which refers to a journal item, not the book.
- [72] John C Reynolds. Types, abstraction and parametric polymorphism. In *Information processing*, volume 83, pages 513–523, 1983.
- [73] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.
- [74] Jun John Sakurai and Jim Napolitano. *Modern Quantum Mechanics*. Cambridge University Press, 2nd edition, 2017. Standard reference for angular momentum quantization and spherical harmonics.
- [75] Leonard J. Savage. *The Foundations of Statistics*. John Wiley & Sons, 1954.
- [76] Erwin Schrödinger. An undulatory theory of the mechanics of atoms and molecules. *Physical Review*, 28(6):1049–1070, 1926.
- [77] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.
- [78] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [79] Claude E. Shannon. Zero-error capacity of a noisy channel. *IRE Transactions on Information Theory*, 2(3):8–19, 1956.
- [80] Claude E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7:142–163, 1959. Part 4.

- [81] Mary Shaw and David Garlan. *Software Architecture: Perspectives on an Emerging Discipline*. Prentice Hall, 1996.
- [82] Ilya Shpitser and Judea Pearl. Identification of joint interventional distributions in recursive semi-Markovian causal models. In *Proceedings of the Twenty-First National Conference on Artificial Intelligence (AAAI-06)*, pages 1219–1226. AAAI Press, 2006.
- [83] Jeremy G Siek and Walid Taha. Gradual typing for functional languages. *Scheme and Functional Programming Workshop*, 6:81–92, 2006.
- [84] Tristan Simas. Identification capacity and rate-query tradeoffs in classification systems, 2026.
- [85] David Slepian and Jack K. Wolf. Noiseless coding of correlated information sources. *IEEE Transactions on Information Theory*, 19(4):471–480, 1973.
- [86] Hartland S. Snyder. Quantized space-time. *Physical Review*, 71(1):38–41, 1947.
- [87] Michael Spence. Job market signaling. *The Quarterly Journal of Economics*, 87(3):355–374, 1973.
- [88] Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society: Series B (Methodological)*, 36(2):111–133, 1974.
- [89] Jack W. Szostak. Functional information: Molecular messages. *Nature*, 423(6941):689, 2003.
- [90] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. Association for Computing Machinery, 2020.
- [91] Christian Van den Broeck and Massimiliano Esposito. Ensemble and trajectory thermodynamics: A brief introduction. *Physica A: Statistical Mechanics and its Applications*, 418:6–16, 2015.
- [92] David J. Wales. *Energy Landscapes: Applications to Clusters, Biomolecules and Glasses*. Cambridge University Press, Cambridge, 2003.
- [93] David J. Wales. The energy landscape as a unifying theme in molecular science. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 363(1827):1711–1733, 2005.
- [94] David J. Wales. Exploring energy landscapes. *Annual Review of Physical Chemistry*, 69:401–425, 2018.
- [95] Gregory Wheeler. Bounded rationality. The Stanford Encyclopedia of Philosophy, 2024. First published 2018; substantive revision 13 Dec 2024.
- [96] David H. Wolpert, Artemy Kolchinsky, Greg Owen, and Jordan M. Horowitz. Is stochastic thermodynamics the key to understanding the energy costs of computation? *Proceedings of the National Academy of Sciences*, 121(11):e2307544121, 2024.
- [97] Michael Wong, Carol Cleland, Daniel R. Arend, Stuart Bartlett, H. James Cleaves, Heather Demarest, Aditi Prabhu, Jonathan I. Lunine, and Robert M. Hazen. On the roles of function and selection in evolving systems. *Proceedings of the National Academy of Sciences*, 120(44):e2310223120, 2023.

- [98] Krish Yadav, Haneen Yousef, and David H. Wolpert. Minimal mismatch cost and the energy requirement of circuits. *arXiv preprint arXiv:2504.04031*, 2026.
- [99] Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference, 2025.